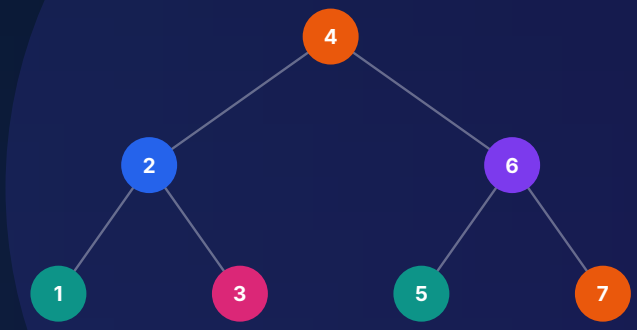


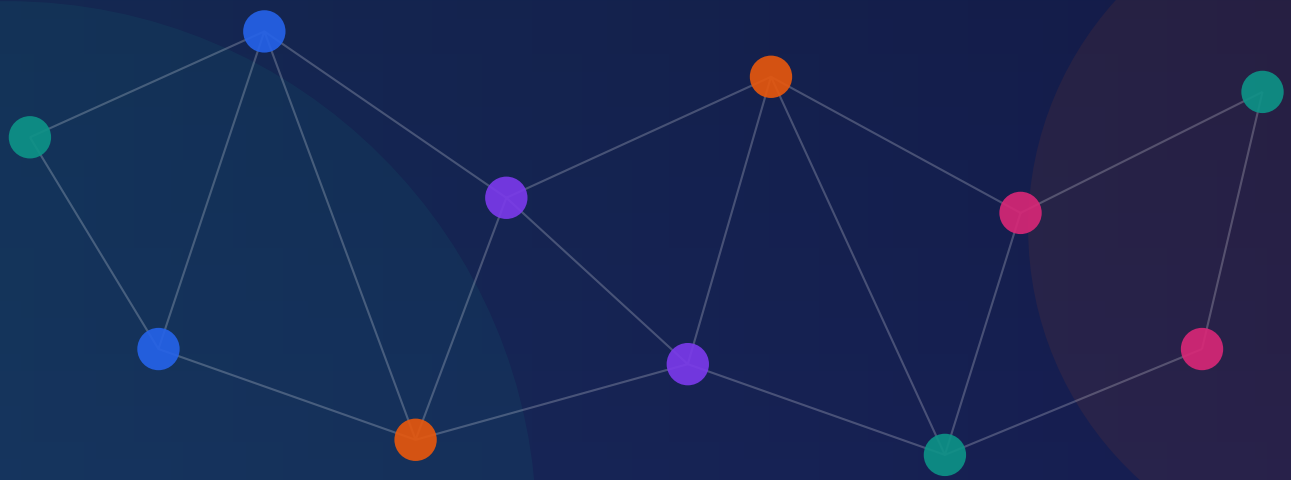
A PATTERN-BASED, MASTERY-DRIVEN GUIDE

Mastering Data Structures & Algorithms

with C++

Second Edition

Learn to Understand, Recognize, Derive, Implement and Retain





Understand Recognize Derive Implement Retain

A PATTERN-BASED, MASTERY-DRIVEN GUIDE

Mastering Data Structures & Algorithms with C++

Second Edition

Learn to Understand, Recognize, Derive, Implement and Retain

A pattern-based, mastery-driven guide for a beginner with basic C++ knowledge. By the end you should be able to solve problems you have never seen, not just follow solutions you have read.

TABLE OF CONTENTS

Contents

What Is New in This Edition ix

PART 0 Getting Started

How to Use This Book 2

The Big Idea . . . 2
The Teaching Spine . . . 2
The Scaffolding Ladder (Learn to Code Alone) . . . 3
The Learning Loop . . . 3
The Hint System (Struggle Productively) . . . 4
The Problem Ladder . . . 4
The Interview Framework (use for every problem) . . . 4
Conventions . . . 4

1 C++ Primer for DSA 6

1.1 Program Skeleton . . . 7
1.2 Data Types and the Overflow Trap . . . 7
1.3 Vectors: Your Default Array . . . 7
1.4 Strings . . . 8
1.5 Pairs, Tuples, Structs . . . 8
1.6 The Containers You Will Use . . . 8
1.7 Lambdas and Custom Sorting . . . 9
1.8 Pointers and Dynamic Memory . . . 9
1.9 Recursion Basics . . . 10
1.10 Common C++ Pitfalls . . . 10
Chapter 1 Toolkit: Practice, Review and Mastery . . . 11

2 Complexity Analysis 15

2.1 Why It Matters . . . 15
2.2 Big-O in One Page . . . 16
2.3 Reading the Constraints . . . 16
2.4 Space Complexity . . . 17
2.5 Amortized Analysis . . . 17

2.6 Complexity of Recursion . . . 17
2.7 Summary Checklist . . . 17
Chapter 2 Toolkit: Practice, Review and Mastery . . . 18

PART I Linear Structures

3 Arrays & Strings 22
3.0 Concept . . . 23
3.1 Pattern: In-Place Rearrangement . . . 23
3.2 Pattern: Two Pointers . . . 26
3.3 Pattern: Kadane's Algorithm Sliding Window . . . 28
3.4 Pattern: Prefix Sums Difference Arrays . . . 30
3.5 Pattern: Majority Counting Tricks . . . 31
3.6 Pattern: Chunks, Cycles Cyclic Placement . . . 33
3.7 Pattern: Greedy on Arrays . . . 35
3.8 Pattern: Math Number Theory . . . 36
3.9 Multi-Technique Problems . . . 38
Chapter 3 Toolkit: Practice, Review and Mastery . . . 39

4 Matrix 43
4.0 Concept . . . 43
4.1 Pattern: Traversal . . . 44
4.2 Pattern: Sorted Matrix Tricks . . . 45
Chapter 4 Toolkit: Practice, Review and Mastery . . . 48

5 Sorting & Searching 52
5.0 Concept . . . 52
5.1 Pattern: Binary Search Fundamentals . . . 53
5.2 Pattern: Binary Search on the Answer . . . 55
5.3 Pattern: Classic Sorting Merge Sort Counting . . . 58
Chapter 5 Toolkit: Practice, Review and Mastery . . . 60

6 Linked List 64
6.0 Concept . . . 65
6.1 Pattern: Traversal Reversal . . . 65
6.2 Pattern: Fast Slow Pointers . . . 67
6.3 Pattern: Merging Sorting Lists . . . 69
6.4 Pattern: Design with Lists . . . 70
Chapter 6 Toolkit: Practice, Review and Mastery . . . 72

7 Stacks & Queues 76
7.0 Concept . . . 77
7.1 Pattern: Monotonic Stack . . . 77
7.2 Pattern: Parentheses Bracket Matching . . . 81

7.3 Pattern: Stack Queue Design. 83
7.4 Pattern: Sliding Window with Deque. 84
Chapter 7 Toolkit: Practice, Review and Mastery. 87

PART II Non-Linear Structures

8 Trees 92

8.0 Concept 93
8.1 Pattern: Traversals. 93
8.2 Pattern: Views of a Binary Tree 95
8.3 Pattern: BST Properties Operations 97
8.4 Pattern: Lowest Common Ancestor 99
8.5 Pattern: Construct Trees from Traversals 100
8.6 Pattern: Path Problems (Postorder DP) 101
8.7 Pattern: Advanced Design 102
Chapter 8 Toolkit: Practice, Review and Mastery. 106

9 Graphs 110

9.0 Concept 110
9.1 Pattern: BFS DFS Fundamentals. 111
9.2 Pattern: Grid-Based BFS/DFS 113
9.3 Pattern: Topological Sort 117
9.4 Pattern: Shortest Paths 118
9.5 Pattern: Minimum Spanning Tree. 121
9.6 Pattern: Union-Find (Disjoint Set Union) 122
9.7 Pattern: SCC, Articulation Points Euler Circuits 124
9.8 Pattern: Max Flow Bipartite Matching 126
9.9 Advanced / Miscellaneous Graph Problems. 128
Chapter 9 Toolkit: Practice, Review and Mastery. 129

10 HashMap & Heap 134

10.0 Concept 134
10.1 Pattern: HashMap + Prefix Sum for Subarrays. 135
10.2 Pattern: Frequency Counting Anagrams 137
10.3 Pattern: Sliding Window with HashMap 139
10.4 Pattern: Top-K Heap Basics 140
10.5 Pattern: Two Heaps Median Techniques 142
10.6 Pattern: Miscellaneous HashMap Applications 143
Chapter 10 Toolkit: Practice, Review and Mastery 145

PART III Problem-Solving Paradigms

11 Greedy 150

11.0 Concept . . . 150
11.1 Pattern: Interval Scheduling Greedy . . . 151
11.2 Pattern: Classic Greedy Algorithms . . . 153
11.3 Pattern: Array Greedy Exchange Arguments . . . 155
11.4 Pattern: Resource Allocation Scheduling . . . 157
11.5 SPOJ Classics Advanced Greedy . . . 158
Chapter 11 Toolkit: Practice, Review and Mastery . . . 159

12 Dynamic Programming 163

12.0 Concept . . . 163
12.1 Pattern: 1D DP Foundations . . . 166
12.2 Pattern: Knapsack Family . . . 169
12.3 Pattern: Longest Increasing Subsequence (LIS) . . . 171
12.4 Pattern: Stock Trading (State Machine DP) . . . 173
12.5 Pattern: Grid Path DP . . . 175
12.6 Pattern: Partition Interval DP . . . 177
12.7 Pattern: String DP, LCS Family . . . 180
12.8 Pattern: Palindromes in Strings . . . 183
12.9 Pattern: String DP, Pattern Matching . . . 185
12.10 Pattern: Game Theory DP Miscellaneous . . . 186
Chapter 12 Toolkit: Practice, Review and Mastery . . . 189

13 String Matching Algorithms 193

13.0 Concept . . . 193
13.1 KMP Algorithm . . . 194
13.2 Z Algorithm . . . 195
13.3 Rabin-Karp Algorithm . . . 196
13.4 Manacher's Algorithm . . . 197
Chapter 13 Toolkit: Practice, Review and Mastery . . . 199

14 Backtracking 203

14.0 Concept . . . 203
14.1 Pattern: Combinatorial Generation . . . 204
14.2 Pattern: Grid / Path Backtracking . . . 206
14.3 Pattern: Constraint Satisfaction . . . 207
14.4 Pattern: String Backtracking . . . 210
Chapter 14 Toolkit: Practice, Review and Mastery . . . 212

15 Bit Manipulation 216

15.0 Concept . . . 216
15.1 Pattern: Bit Basics . . . 217
15.2 Pattern: XOR Tricks . . . 218
15.3 Pattern: Advanced Bit Manipulation . . . 220
Chapter 15 Toolkit: Practice, Review and Mastery . . . 223

16 Tries 227

16.0 Concept . . . 228
16.1 Pattern: Trie Fundamentals . . . 228
16.2 Pattern: Advanced Trie Applications . . . 230
Chapter 16 Toolkit: Practice, Review and Mastery . . . 233

APPENDICES

A C++ STL Quick Reference 239

A.1 Containers at a Glance . . . 239
A.2 Algorithms (<algorithm>, <numeric>) . . . 240
A.3 Idioms You Will Use Constantly . . . 240
A.4 Complexity Cheat Sheet of Common Operations . . . 242

B Pattern Recognition Cheat Sheet 243

When Stuck, Ask: . . . 244
B.2 Look-alike Patterns: How to Tell Them Apart . . . 244

C Mastery-Based Study Plan 246

The Mastery Gate (applies to every chapter) . . . 246
Stages and Indicative Pace . . . 246
Weekly Rhythm . . . 247
Fast-Track Reference: the Original 69-Day Pace . . . 247

D Interview Checklist 248

Before You Code . . . 248
While Coding . . . 248
After Coding . . . 248
Common Bug Checklist . . . 248

E Sheet Coverage Index (all 425 problems) 250

Array String (60 problems) . . . 250
Matrix (10 problems) . . . 253
Sorting and Searching (15 problems) . . . 253
Linked List (19 problems) . . . 254
Stacks and Queue (28 problems) . . . 255
Tree (47 problems) . . . 257
Graph (61 problems) . . . 259

HashMap and Heap (39 problems) 262
Greedy (35 problems) 263
Dynamic Programming (66 problems) 265
Bonus: String Matching Algorithms (6 problems) 268
Backtracking (19 problems) 268
Bit Manipulation (14 problems) 269
Tries (6 problems) 270

F Mistake and Learning Log 271

G Spaced Revision and Cumulative Self-Tests 272

G.1 Spaced Revision Schedule 272
G.2 Cumulative Self-Tests 272

H Quality Assurance Report 274

H.1 What Was Run 274
H.2 Defects Found and Fixed 275
H.3 What Was Not Verified (Limitations) 276
H.4 Reproduce the Checks 277

I C++ Implementation Lab 278

I.1 Linked Lists: Creating, Changing the Head, Freeing 278
I.2 Trees: Building from a Level-Order List 278
I.3 Graph Representations 279
I.4 Comparators and Lambdas 279
I.5 Iterators and Erasing 280
I.6 References, Copies and Pointers: Quick Rules 280
I.7 Recursion Checklist 280
I.8 Pitfall Table 280

J Companion Solutions for Hint-Only Sheet Problems 282

J.1 Search in Rotated Sorted Array II (duplicates allowed) 282
J.2 Find Minimum in Rotated Sorted Array II (duplicates allowed) 283
J.3 Next Greater Element in a Circular Array 283
J.4 Stock Span Problem 284
J.5 Minimum Bracket Reversals to Balance an Expression 284
J.6 Preorder and Postorder Traversal (iterative) 285
J.7 Zigzag Level Order, Bottom View and Diagonal Traversal 286
J.8 Insert Delete GetRandom O(1), Duplicates Allowed 287
J.9 Substrings with Equal Numbers of 0s, 1s and 2s 288
J.10 Queue Using Two Stacks 289

Closing Words 289

SECOND EDITION

What Is New in This Edition

This edition keeps the original chapters, solved examples and practice sets, and wraps them in a stronger learning system. Every chapter now opens with intuition and a picture, every pattern has a recognition card, and every chapter closes with a toolkit for practice, debugging, revision and a mastery checkpoint.

#	Improvement	Where to find it
1	Consistent teaching spine	Preface, "The Teaching Spine"; chapter opener, pattern cards, toolkit
2	Pattern recognition (clues, prerequisites, limits, variations, failures)	**Pattern Card** under every pattern heading; Appendix B (incl. B.2 look-alikes)
3	Brute force to optimal reasoning	Solved examples; "Why It Works" in each toolkit
4	Progressive independent coding	Preface, "The Scaffolding Ladder" (worked example included)
5	Visual explanations	"Picture It" in every chapter opener
6	Problem ladder (foundation to challenge)	"Problem Ladder" in every toolkit
7	Self-contained problems	**Problems nA / nB** in every toolkit (statement, example, constraints, complexity, difficulty, prerequisites)
8	Staged hints	Level 1 / 2 / 3 hints on every new problem; practice tables relabelled
9	Code explained at the right level	Inline comments on key lines; editorials explain invariants, not every statement
10	C++ implementation skills	Chapter 1, Appendix A, **Appendix I (Implementation Lab)**
11	Debugging and testing exercises	"Debug Lab" in every toolkit; Appendix D bug table
12	Compare similar algorithms	"Compare and Contrast" in every toolkit; Appendix B.2
13	Correctness reasoning	"Why It Works" in every toolkit
14	Difficult topics deepened	Extra worked reasoning on Kadane, binary search, monotonic stack, Floyd, Dijkstra, DP, KMP; hand-traced dry runs added to most solved examples in the graph, DP, string, backtracking, bit and trie chapters
15	Chapter endings	"Revision Sheet" in every toolkit
16	Mixed practice and revision	"Mixed Practice"; **Appendix G** (spaced revision and cumulative tests); Appendix F (mistake log)
17	Engagement	Chapter hooks, analogies, "After this chapter you can..."
18	Mastery-based study plan	**Appendix C** (gates, not days)
19	Transfer learning	"Transfer Check" in every toolkit; Appendix G tests
20	Quality assurance	**Appendix H** (what was compiled, run and tested, the bugs that testing found and fixed, and what was not verified)
21	Mastery checkpoint per topic	"Mastery Checkpoint" in every toolkit

#	Improvement	Where to find it
22	Outcomes over quantity	Fewer promises about problem counts; depth added per pattern
23	Complete sheet coverage index	**Appendix E**: all 425 problems of the sheet, each with its location in the book and whether it has a worked solution or a hint only
24	Companion solutions for hint-only problems	**Appendix J**: nine tested solutions (covering twelve sheet rows) for problems that the chapters only hint at, plus a note on the two-stack queue of Section 7.3

PART 0

Getting Started

Preface

1 C++ Primer for DSA
2 Complexity Analysis

How to Use This Book

The Big Idea

Most people learn DSA by grinding hundreds of random problems. It works poorly because **problems are not random; they are variations on a small set of patterns.** This book is organised around 65 pattern sections (plus the four algorithm sections of Chapter 13). Once you own a pattern, a "new" problem is an old friend in disguise.

But recognising a pattern is not enough. The goal of this book is **five abilities**:

Ability	You can…	Trained by
Understand	explain a concept in plain words	chapter opener, concept sections, recall questions
Recognize	see which pattern a problem needs	pattern cards, Transfer Check, Appendix B
Derive	reach the algorithm from the brute force	solved examples, "Why It Works", staged hints
Implement	write correct C++ alone	scaffolding ladder, Problems nA/nB, Debug Lab
Retain	still do all of the above a month later	revision sheets, Appendices F and G

The Teaching Spine

Every important idea in this book is taught in the same order. When you study a pattern, look for these ten steps and, if one is missing, supply it yourself:

1. **Intuition**: a story or picture of why the idea exists.
2. **Simple explanation**: the idea in two sentences.
3. **Visual example**: a diagram of pointer, stack, window or table states.
4. **Brute force**: the simplest correct solution and its cost.
5. **Optimization**: what is wasted, and why removing it is safe.
6. **Implementation**: clean C++17.
7. **Dry run**: trace a small input by hand.
8. **Complexity**: time and space, with the reason.
9. **Correctness**: the invariant or proof sketch that makes it work.
10. **Practice**: variations, a debugging exercise and a disguised problem.

The **Chapter Toolkit** at the end of every chapter and the new **Problems nA/nB** follow the full spine. The solved examples inherited from the first edition cover steps 1 to 6 and 8 in compact form; in this edition most of them also carry a hand-traced dry run (step 7), and steps 9 and 10 are supplied by the toolkit.

The Scaffolding Ladder (Learn to Code Alone)

Reading a solution and writing one are different skills. For every pattern, climb these rungs in order and only move up when the current rung is easy:

Rung	Task	Example with Container With Most Water (Ex 3.4)
1. **Study**	Read the full solution and trace it	read Ex 3.4 and trace `h = [1,8,6,2,5,4,8,3,7]`
2. **Fill the blanks**	Complete missing code	see below
3. **Pseudocode to code**	Translate given steps	see below
4. **Blank page**	Code from the problem statement only	statement only
5. **Unfamiliar problem**	Solve a disguised or new problem	Transfer Check, Appendix G

Rung 2: fill the blanks.

```
int maxArea(vector<int>& h) {
    int l = 0, r = ______, best = 0;
    while (l ___ r) {
        best = max(best, min(h[l], h[r]) * (______));
        if (h[l] < h[r]) ______; else ______;
    }
    return best;
}
```

ANSWER

`r = (int)h.size() - 1`; `l < r`; `r - l`; `l++`; `r--`.

Rung 3: pseudocode to code.

Sketch

```
l = first index, r = last index, best = 0
while l is left of r:
    area = (smaller of the two heights) * (distance between them)
    best = max(best, area)
    move the pointer sitting on the SHORTER line one step inward
return best
```

The Learning Loop

For every pattern:

1. Read the opener, the pattern card and the concept.
2. **Cover the code** and try the solved example yourself for 15 minutes (rung 1 to 4).
3. Read the approach, then the code. Type it; do not copy-paste.
4. Do the Practice Set using the **hint levels** below.
5. In the toolkit: do the Debug Lab, Problems nA/nB, then the Transfer Check.
6. Pass the **Mastery Checkpoint**. Log mistakes in Appendix F. Revise on the schedule in Appendix G.

The Hint System (Struggle Productively)

Never jump straight to the answer. Spend at least 15 to 20 minutes, then open hints **one level at a time**:

Level	What it gives	In this book
1	Which concept or pattern is relevant	one line above each practice table; first hint on Problems nA/nB
2	The approach in outline	the "Hint" column; second hint on Problems nA/nB
3	The key step or invariant	third hint on Problems nA/nB; the closest solved example
4	Full solution and editorial	collapsed "Solution" blocks

The Problem Ladder

Each toolkit organises practice into five tiers so difficulty rises gradually: **foundation**, **pattern variations**, **combined patterns**, **independent recognition** (you must spot the pattern yourself), and **challenge**.

The Interview Framework (use for every problem)

Step	Question to ask yourself
1. **Understand**	Inputs, outputs, constraints, edge cases (empty, one element, duplicates, negatives)?
2. **Examples**	Work a small example by hand.
3. **Brute force**	What is the simplest correct solution? Its complexity?
4. **Optimize**	What is repeated or wasted? Which pattern removes it?
5. **Code**	Clean code, meaningful names.
6. **Test**	Dry run on your example and on edge cases. State time and space.

Conventions

- All code targets **C++17**. Compile with `g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined file.cpp` while learning; sanitizers catch many bugs for free.
- Difficulty tags: E easy, M medium, H hard. [solved] = solved example; [practice] = practice row.
- Code favors **clarity over cleverness**. Comments mark the key invariant or the tricky line, not every statement.
- **Problems nA/nB** are self-contained: statement, example, constraints, expected complexity, prerequisites, three staged hints and a collapsed solution.
- Counts in this book (produced by script from the headings and tables): **158 numbered solved examples**, **201 practice rows** under *Practice Set* headings (plus the hint tables in Sections 9.9, 11.4 and 11.5), **30 self-contained problems** (nA/nB) and **9 companion solutions** for hint-only sheet problems (Appendix J.1 to J.9, covering 12 sheet rows). The sheet lists **425** problems (the first edition's title said 450, which was wrong). Some solved examples cover several named problems; **Appendix E** maps every sheet problem to its place in the book. See **Appendix H** for what was and was not verified.

```cpp
#include <bits/stdc++.h>
using namespace std;
```

NOTE

`<bits/stdc++.h>` works on GCC/Clang and most online judges. In a real project include specific headers.

CHAPTER 1

C++ Primer for DSA

1

START HERE

Your program reads 100,000 numbers and prints the wrong answer, but only when the input is empty. The culprit is one innocent-looking expression, `v.size() - 1`. This chapter is about the small C++ facts that quietly decide whether your DSA code works.

ANALOGY

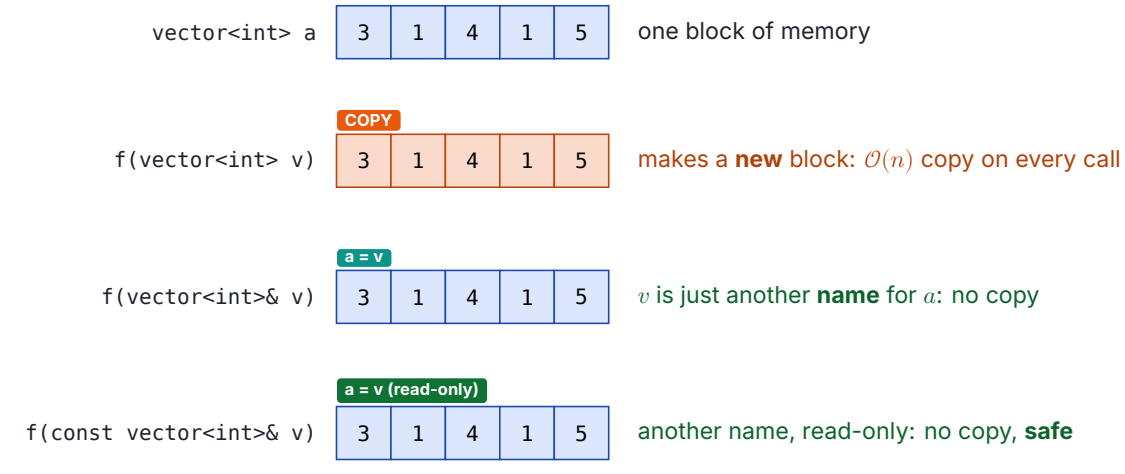
Think of C++ containers as labelled storage boxes: `vector` is a row of numbered lockers, `unordered_map` is a coat-check desk (hand in a ticket, get your coat back), `priority_queue` is a pile where the biggest item is always on top.

AFTER THIS CHAPTER YOU CAN

- ✓ Choose the right STL container for a task and state its main costs.
- ✓ Write a comparator or lambda for custom sorting.
- ✓ Pass containers by reference and explain why.
- ✓ Spot unsigned wrap-around and overflow before they bite.

PICTURE IT

Passing a vector by value vs by reference



If you are learning C++ alongside DSA, read this chapter once now, then keep **Appendix A** open as a reference. You do not need all of C++ for DSA, only a focused subset.

1.1 Program Skeleton

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);   // faster I/O
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;     // read n numbers

    cout << "Sum = " << accumulate(a.begin(), a.end(), 0LL) << "\n";
    return 0;
}
```

1.2 Data Types and the Overflow Trap

Type	Size	Range (approx.)
`int`	4 bytes	-2.1×10^9 to 2.1×10^9
`long long`	8 bytes	-9.2×10^{18} to 9.2×10^{18}
`char`	1 byte	a character or small integer
`bool`	1 byte	`true` / `false`
`double`	8 bytes	decimal numbers
`size_t`	8 bytes	unsigned, used by `.size()`

Overflow: if values or their sums can pass about 2×10^9, use `long long`. The constants `INT_MAX` and `INT_MIN` live in `<climits>`.

Unsigned trap: `v.size()` is unsigned. `v.size() - 1` on an empty vector wraps around to a huge number. Write `(int)v.size() - 1` when it could be zero.

```cpp
vector<int> v;
for (int i = 0; i < (int)v.size() - 1; i++) { /* safe */ }
```

1.3 Vectors: Your Default Array

```cpp
vector<int> a;                      // empty
vector<int> b(5, 0);                // five zeros
vector<vector<int>> grid(3, vector<int>(4, 0));  // 3x4 matrix of 0s

a.push_back(10);                    // add at end   O(1) amortized
a.pop_back();                       // remove last  O(1)
```

```
a.size(); a.empty(); a.back(); a.front();
a.insert(a.begin() + 1, 99);         // O(n)
a.erase(a.begin());                  // O(n)
sort(a.begin(), a.end());            // O(n log n)
reverse(a.begin(), a.end());
```

Pass big containers by reference, otherwise C++ copies them:

```
int sumAll(const vector<int>& a);    // no copy, read-only
void modify(vector<int>& a);         // no copy, can modify
```

1.4 Strings

```
string s = "hello";
s.size(); s[0]; s += " world"; s.substr(1, 3);   // "ell" (start, length)
s.find("lo");                          // index, or string::npos if absent
to_string(123);                        // "123"
stoi("456");                           // 456
sort(s.begin(), s.end());              // sort characters
```

Characters are small integers: `'c' - 'a'` gives 2. This is the key trick for frequency arrays: `int freq[26]; freq[ch - 'a']++;`

1.5 Pairs, Tuples, Structs

```
pair<int, string> p = {1, "one"};
cout << p.first << " " << p.second;

vector<pair<int,int>> v = {{3,1}, {1,2}, {2,0}};
sort(v.begin(), v.end());                        // sorts by first, then second

auto [x, y] = v[0];                              // structured binding (C++17)

struct Edge { int to, weight; };
Edge e{5, 10};
```

1.6 The Containers You Will Use

Container	What it is	Typical operations	Cost
`vector<T>`	dynamic array	index, push_back	$\mathcal{O}(1)$
`stack<T>`	LIFO	push, pop, top	$\mathcal{O}(1)$
`queue<T>`	FIFO	push, pop, front	$\mathcal{O}(1)$
`deque<T>`	double-ended queue	push/pop both ends	$\mathcal{O}(1)$

Container	What it is	Typical operations	Cost
`priority_queue<T>`	max-heap	push, pop, top	$\mathcal{O}(\log n)$
`set<T>` / `map<K,V>`	sorted, balanced tree	insert, find, erase	$\mathcal{O}(\log n)$
`unordered_set` / `unordered_map`	hash table	insert, find, erase	$\mathcal{O}(1)$ avg
`multiset<T>`	sorted, allows duplicates	insert, erase one	$\mathcal{O}(\log n)$
`list<T>`	doubly linked list	splice, insert at iterator	$\mathcal{O}(1)$

```
unordered_map<string, int> freq;
for (auto& w : words) freq[w]++;            // missing key starts at 0
if (freq.count("apple")) { /* exists */ }

priority_queue<int> maxHeap;                                  // largest on top
priority_queue<int, vector<int>, greater<int>> minHeap;       // smallest on top
```

1.7 Lambdas and Custom Sorting

```
vector<pair<int,int>> v = {{1,5}, {2,3}, {3,4}};
sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
    return a.second < b.second;             // sort by second ascending
});
```

A lambda is an inline, nameless function. The `[]` is the *capture list*. Use `[&]` to capture surrounding variables by reference.

1.8 Pointers and Dynamic Memory

Linked lists and trees use pointers:

```
struct Node {
    int val;
    Node* next;
    Node(int v) : val(v), next(nullptr) {}   // constructor
};

Node* head = new Node(1);
head->next = new Node(2);
// Every `new` needs exactly one matching `delete` (Appendix I.1 shows how to free a whole
↪  list).
```

`p->val` is shorthand for `(*p).val`. Always check `if (p != nullptr)` before dereferencing.

Memory rule. Every `new` must be matched by exactly one `delete`; forgetting it is a *memory leak*, and deleting twice or using a pointer after `delete` is undefined behaviour. In a short interview answer people often skip the clean-up and say so out loud, but the code in this book that builds nodes (linked lists, trees, tries) frees them, and the `-fsanitize=address` flag from the Conventions section reports leaks, so you can check yourself. A deleted pointer is still holding the old address; set it to `nullptr` if you keep it around.

1.9 Recursion Basics

Every recursive function needs (1) a **base case** and (2) a **smaller subproblem**.

```
int factorial(int n) {
    if (n <= 1) return 1;          // base case
    return n * factorial(n - 1);   // smaller problem
}
```

1.10 Common C++ Pitfalls

Pitfall	Fix
`int` overflow in sums/products	use `long long`
`v.size() - 1` on empty vector	cast to `int`
Modifying a container while iterating	use index loops or save changes for later
`map[key]` creates the key if missing	use `.count(key)` or `.find(key)` to check only
Copying large vectors into functions	pass by reference
Uninitialized variables	always initialize
Negative `%` gives negative result	`((a % m) + m) % m`
`vector<bool>` is special	prefer `vector<char>` if you need references

CHAPTER 1 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Read input into a `vector`, print sum and max (1.1, 1.3).
2. **Pattern variations:** Sort pairs by two keys with a lambda (1.7).
3. **Combined patterns:** Use `unordered_map` to count frequencies (1.6).
4. **Independent recognition:** Without help: dedupe a vector preserving order (Problem 1B).
5. **Challenge:** Write a function that returns the k-th largest using `priority_queue` (see 10.4 later).

Compare and Contrast

Container	Pick it when	Avoid it when
`vector`	default; random access, push at end	frequent insert/erase in the middle
`deque`	push/pop at both ends	you need contiguous memory
`set` / `map`	sorted order, lower_bound, range queries	you only need membership (hash is faster)
`unordered_set` / `unordered_map`	$\mathcal{O}(1)$ average lookup	you need sorted order, or adversarial inputs
`priority_queue`	repeatedly take the max/min	you need to search or remove arbitrary items

Why It Works (Correctness Corner)

Loop invariants. An invariant is a sentence that is true before and after every iteration. For `for (read...) if (a[read] != 0) swap(a[write++], a[read]);` the invariant is: *$a[0..write)$ holds the non-zeros seen so far, in order.* If you can state the invariant, you can nearly always prove the loop right. Build this habit now; every later chapter relies on it.

Debug Lab: Off-by-one with unsigned size

```cpp
// Sum of absolute differences of neighbours. Crashes or prints garbage on an EMPTY vector.
long long adjacentDiffSum(const vector<int>& v) {
    long long s = 0;
    for (int i = 0; i < v.size() - 1; i++)
        s += abs(v[i + 1] - v[i]);
    return s;
}
```

Your task: Find the bug and fix it.

ANSWER

`v.size()` is unsigned. For an empty vector `v.size() - 1` wraps to a huge number, so the loop reads far outside the vector. Fix: `for (int i = 0; i + 1 < (int)v.size(); i++)`. Always test the empty input.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 1A E

Rank the Students

Statement. Given a list of (name, score) pairs, return it sorted by score from high to low; students with equal scores are ordered by name A to Z.

Example. `[("bo",90),("al",90),("cy",95)]` → `[("cy",95),("al",90),("bo",90)]`

Constraints. $1 \le n \le 10^5$; names are distinct.

Expected complexity. $\mathcal{O}(n \log n)$ time.

Prerequisites. 1.7 Lambdas and Custom Sorting

HINT 1 which idea is relevant?

Which STL function sorts? What extra thing can you give it?

HINT 2 outline of the approach

A comparator returns true when the first argument must come BEFORE the second.

HINT 3 the key step

Compare scores first; only when they are equal compare names. Be careful: for scores you want `>`, for names `<`.

SOLUTION and editorial (try first, then compare)

```
vector<pair<string,int>> rankStudents(vector<pair<string,int>> s) {
    sort(s.begin(), s.end(), [](const auto& a, const auto& b) {
        if (a.second != b.second) return a.second > b.second;   // higher score first
        return a.first < b.first;                               // tie: name A-Z
    });
    return s;
}
```

The comparator defines a strict ordering with two keys. A common bug is to return `a.second >= b.second`; comparators must be *strict* (never true for equal items) or `sort` may crash.

PROBLEM 1B E

Remove Duplicates, Keep Order

Statement. Return the array with repeated values removed, keeping only the first occurrence of each value and preserving order.

Example. `[3,1,3,2,1]` → `[3,1,2]`

Constraints. $0 \le n \le 10^5$; values fit in `int`.

Expected complexity. $\mathcal{O}(n)$ average time, $\mathcal{O}(n)$ space.

Prerequisites. 1.6 The Containers You Will Use

HINT 1 which idea is relevant?

You need to remember what you have already seen. Which container answers 'seen it?' fastest?

HINT 2 outline of the approach

`unordered_set::insert` tells you whether the value was new.

HINT 3 the key step

Check the `.second` of the pair returned by `insert`.

SOLUTION and editorial (try first, then compare)

```
vector<int> dedupe(const vector<int>& a) {
    unordered_set<int> seen;
    vector<int> out;
    for (int x : a)
        if (seen.insert(x).second) out.push_back(x);   // .second is true only for a new
        ↪  value
    return out;
}
```

`insert(x).second` is true exactly when `x` was not present, so one call both tests and records. Sorting would destroy the original order, which is why a set of seen values is the right tool.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T1.1 A cashier app lists customers by total spent, highest first, alphabetical on ties.

PATTERN

Custom comparator with two keys (Problem 1A in disguise).

T1.2 A log has event ids and you must print each id once, in the order first seen.

PATTERN

Seen-set filter (Problem 1B in disguise).

Mixed Practice

Preview of Chapter 10: read n numbers, print the k most frequent values. Which two containers from 1.6 do you need?

Revision Sheet

Key takeaways

- Default to `vector`; pass big containers by `const&`.
- `size()` is unsigned: cast before subtracting.
- Comparators must be strict (`<`, never `<=`).
- Use `long long` whenever sums or products can pass ~2×10^9.

Pattern-recognition clues

- 'In sorted order by two keys' → comparator/lambda.
- 'Seen before?' → hash set.
- 'Largest/smallest repeatedly' → priority_queue.

Common mistakes

- `v.size() - 1` on an empty vector.

- Missing `&` so a huge vector is copied on every call.
- `int` overflow in sums and products.

Complexity summary

Operation	Cost
`vector` push_back / index	$\mathcal{O}(1)$ amortized / $\mathcal{O}(1)$
`vector` insert/erase in middle	$\mathcal{O}(n)$
`sort`	$\mathcal{O}(n \log n)$
`unordered_map` access	$\mathcal{O}(1)$ average, $\mathcal{O}(n)$ worst
`map` access	$\mathcal{O}(\log n)$

Recall questions (answer aloud, then check the chapter)

1. Why is `v.size() - 1` dangerous?
2. What does a comparator return?
3. When is `unordered_map` a bad choice?
4. What is passed when you write `vector<int> a` (no `&`) as a parameter?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Write, without looking, a program that reads `n` and `n` integers, removes duplicates keeping order, sorts the result descending, and prints it. Then test with `n = 0`.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 2

Complexity Analysis

START HERE

Two programs solve the same problem. One finishes in 0.02 seconds on 10^5 numbers; the other is still running after a minute. Complexity analysis is how you predict that gap before writing a single line.

ANALOGY

Big-O is like estimating travel time by *distance class*: walking, driving or flying. You do not care about minutes; you care whether the trip is hours or days.

AFTER THIS CHAPTER YOU CAN

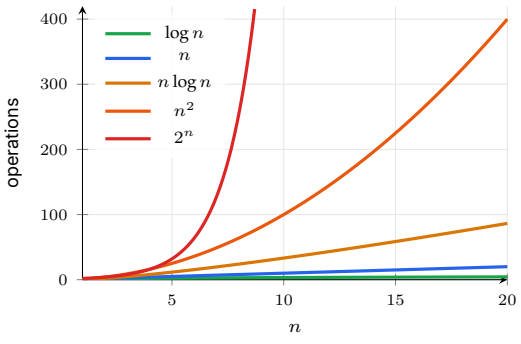
- ✓ Read constraints and decide which complexity is allowed.
- ✓ Compute time and space complexity of loops, nested loops and simple recursion.
- ✓ Explain amortized cost with an example.
- ✓ Choose between two algorithms using constraints rather than guesswork.

PICTURE IT

How fast does each growth rate blow up?

n	$\log n$	$n \log n$	n^2
10^3	10	10^4	10^6
10^5	17	1.7×10^6	10^{10}
10^6	20	2×10^7	10^{12}

10^{10} operations: too slow for 1 second



2.1 Why It Matters

Complexity tells you how a solution's running time grows as input grows. Two correct solutions can differ by "finishes instantly" versus "runs for a year."

2.2 Big-O in One Page

Big-O describes the **upper bound** of growth, ignoring constants.

```cpp
// O(1): constant
int first(vector<int>& a) { return a[0]; }

// O(n): one pass
int sum(vector<int>& a) { int s = 0; for (int x : a) s += x; return s; }

// O(n^2): nested loops
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) { /* ... */ }

// O(log n): halve the problem each step
while (n > 1) n /= 2;

// O(n log n): do O(log n) work n times, e.g. sorting
```

Four rules that cover most of what you will need.

1. **Drop constants.** `3n` and `n/2 + 7` are both $\mathcal{O}(n)$.
2. **Keep only the fastest-growing term.** `n^2 + 5n + 100` is $\mathcal{O}(n^2)$, because for large `n` the `n^2` part dwarfs the rest.
3. **Steps one after another add; steps inside steps multiply.** Two separate loops of `n` cost $\mathcal{O}(n+n) = \mathcal{O}(n)$. A loop of `n` containing a loop of `n` costs $\mathcal{O}(n \cdot n)$.
4. **Different inputs get different letters.** Looping over an array of length `n` and then over a string of length `m` is $\mathcal{O}(n+m)$, not $\mathcal{O}(n^2)$.

Where does `log n` come from? If every step throws away half of what is left, you need about log2(n) steps to reach 1: for n = 10^9 that is only about 30 steps. Binary search, halving loops and balanced trees are all "halving" stories.

Big-O is an **upper bound**. When we quote it without a qualifier we mean the worst case; if we mean something else we say *average* or *amortized*.

2.3 Reading the Constraints

The constraint on `n` hints at the intended complexity. Assume about 10^8 simple operations per second.

n up to	Likely intended complexity	Typical techniques
10 - 12	$\mathcal{O}(n!)$	permutations, brute force
20 - 25	$\mathcal{O}(2^n)$	subsets, backtracking, bitmask
500	$\mathcal{O}(n^3)$	Floyd-Warshall, interval DP
5,000	$\mathcal{O}(n^2)$	2D DP, nested loops
10^5 - 10^6	$\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$	sorting, heap, binary search, hashing, two pointers
10^9 and above	$\mathcal{O}(\log n)$ or $\mathcal{O}(1)$	binary search, math, fast power

2.4 Space Complexity

Count the *extra* memory your algorithm uses, including the **recursion stack**. A recursive DFS on a path-shaped tree of n nodes uses $\mathcal{O}(n)$ stack space.

2.5 Amortized Analysis

`vector::push_back` is $\mathcal{O}(1)$ *amortized*. Occasionally it reallocates ($\mathcal{O}(n)$), but averaged over many calls the cost per call is constant. The same idea explains why a monotonic stack is $\mathcal{O}(n)$: each element is pushed once and popped at most once.

2.6 Complexity of Recursion

Recurrence	Result	Example
T(n) = T(n-1) + $\mathcal{O}(1)$	$\mathcal{O}(n)$	linear recursion
T(n) = 2T(n/2) + $\mathcal{O}(n)$	$\mathcal{O}(n \log n)$	merge sort
T(n) = T(n/2) + $\mathcal{O}(1)$	$\mathcal{O}(\log n)$	binary search
T(n) = 2T(n-1) + $\mathcal{O}(1)$	$\mathcal{O}(2^n)$	subsets, Tower of Hanoi (naive Fibonacci follows T(n) = T(n-1) + T(n-2): also exponential, about 1.6^n)

> **RULE OF THUMB**
> a recursion tree's cost is (number of nodes) x (work per node).

Why T(n) = 2T(n/2) + $\mathcal{O}(n)$ is $\mathcal{O}(n \log n)$. Draw the recursion tree. The top call does about `n` work. Its two children each do `n/2`, together `n`. The four grandchildren do `n/4` each, together `n`. Every level costs about `n` in total, and halving `n` until it reaches 1 gives about log2(n) levels. So the total is `n` work per level times log n levels.

2.7 Summary Checklist

- Say complexity for **both time and space** at the end of every solution.
- If your solution is too slow, ask: *"What am I recomputing?"* The answer points to hashing, prefix sums, DP, or sorting.

CHAPTER 2 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Classify single and nested loops.
2. **Pattern variations:** Analyse a loop where the variable doubles.
3. **Combined patterns:** Analyse recursion trees ($T(n) = 2T(n/2) + n$).
4. **Independent recognition:** Given $n = 2 \times 10^5$ pick the largest allowed complexity.
5. **Challenge:** Explain why `vector::push_back` is $\mathcal{O}(1)$ amortized.

Compare and Contrast

Growth	$n = 10^5$ operations	Verdict
$\mathcal{O}(n)$	10^5	trivial
$\mathcal{O}(n \log n)$	$\sim 1.7 \times 10^6$	easy
$\mathcal{O}(n\sqrt{n})$	$\sim 3 \times 10^7$	fine
$\mathcal{O}(n^2)$	10^{10}	too slow in 1 s

Why It Works (Correctness Corner)

Why doubling gives $\mathcal{O}(1)$ amortized. Pushing n items into a vector that doubles when full copies $1 + 2 + 4 + \ldots + n/2 < n$ elements in total. So n pushes cost less than 2n element moves: under 2 per push on average.

Debug Lab: Hidden O(n^2)

```
string s;
for (int i = 0; i < n; i++) s = s + "x";     // looks O(n)
```

Your task: What is the real complexity and how do you fix it?

ANSWER

`s = s + "x"` builds a new string and copies the old one each time: $\mathcal{O}(n^2)$ total. Use `s += "x"` (amortized $\mathcal{O}(1)$ per append) or `s.push_back('x')`.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T2.1 Code A: `for i in 1..n: for j in 1..n: ...`. Code B: `for i in 1..n: for j = 1; j <= n; j *= 2: ...`. Which is faster as n grows?

PATTERN

A is $\mathcal{O}(n^2)$; B is $\mathcal{O}(n \log n)$, because j doubles each time. B is much faster.

T2.2 `while (n > 1) n /= 2;` runs how many times for $n = 10^9$?

PATTERN

About 30 times: $\mathcal{O}(\log\ n)$.

Mixed Practice

Before every future problem, say aloud: 'n is ___, so I may use at most $\mathcal{O}$(___)'. Practise this on Problems 3A, 5B and 10B.

Revision Sheet

Key takeaways

- Constraints tell you the target complexity: n $\leq 10^5 \rightarrow \mathcal{O}(n\ \log\ n)$; n $\leq$ 5000 $\rightarrow \mathcal{O}(n^2)$; n $\leq$ 20 $\rightarrow \mathcal{O}(2^n)$.
- Drop constants and lower-order terms but never forget hidden costs (string copies, `erase` in a vector).
- Recursion cost = (number of calls) x (work per call).
- Space complexity includes the recursion stack.

Pattern-recognition clues

- Loop variable doubles/halves $\rightarrow$ log n.
- Nested loops over the same range $\rightarrow n^2$.
- Two halves + linear merge $\rightarrow$ n log n.

Common mistakes

- Ignoring that `substr`, copies, and `erase` are $\mathcal{O}(n)$.
- Counting only time, not recursion-stack space.
- Using $\mathcal{O}(n^2)$ when n = 10^5.

Complexity summary

n up to	Typical allowed
20-25	$\mathcal{O}(2^n)$, backtracking, bitmask
500	$\mathcal{O}(n^3)$
5000	$\mathcal{O}(n^2)$
10^5-10^6	$\mathcal{O}(n\ \log\ n)$, $\mathcal{O}(n)$
10^9+	$\mathcal{O}(\log\ n)$ or $\mathcal{O}(1)$

Recall questions (answer aloud, then check the chapter)

1. What is the complexity of binary search and why?
2. Why is `push_back` $\mathcal{O}(1)$ amortized?
3. What is the recursion-stack space of DFS on a path graph with n nodes?
4. Which operations on `vector` cost $\mathcal{O}(n)$?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐

Ability	Evidence	Done
Code	Pick five solved examples from later chapters. For each, state time and space complexity *before* reading the book's analysis, then check yourself.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

PART I

Linear Structures

3 Arrays & Strings

4 Matrix

5 Sorting & Searching

6 Linked List

7 Stacks & Queues

CHAPTER 3

Arrays & Strings

3

START HERE

Given a week of daily profit/loss numbers, which consecutive stretch earned the most? Checking every stretch is slow; one clever pass (Kadane) answers in a single sweep. Arrays are the backbone of almost every other topic, and a handful of patterns solve most of them.

ANALOGY

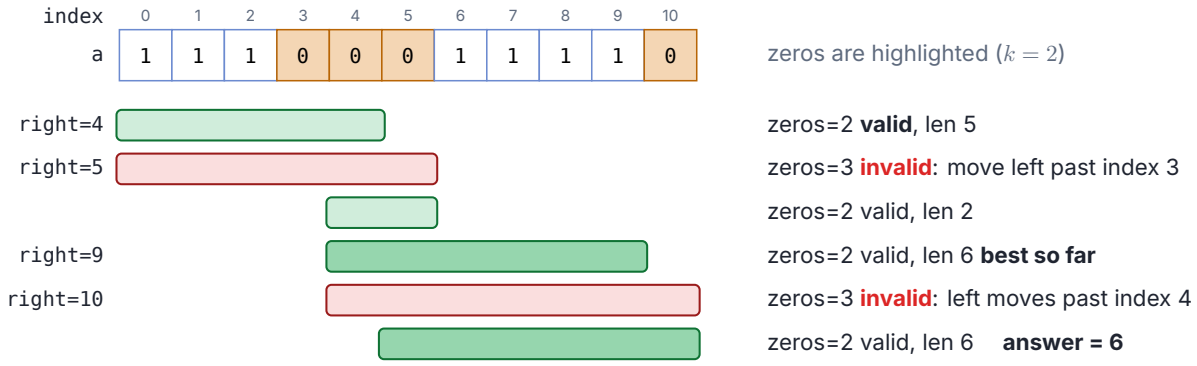
A **sliding window** is a caterpillar: the head moves forward, and the tail catches up only when the caterpillar has grown too long. **Prefix sums** are a running odometer: the distance between two mileposts is just the difference of two readings.

AFTER THIS CHAPTER YOU CAN

- ✓ Choose between two pointers, sliding window and prefix sums from the problem statement.
- ✓ Derive Kadane's recurrence and explain why it is correct.
- ✓ Rearrange arrays in place using read/write pointers and reversals.
- ✓ Detect when a window technique is invalid (negative numbers!).

PICTURE IT

Sliding window: longest run with at most k = 2 zeros



3.0 Concept

An **array** stores elements in contiguous memory, so `a[i]` is $\mathcal{O}(1)$. Strings are arrays of characters. Nearly every interview starts here, and most array problems reduce to one of a handful of patterns:

Pattern	Recognition clue
In-place rearrangement	"rotate", "segregate", "partition", "$\mathcal{O}(1)$ extra space"
Two pointers	sorted array, pair/triplet with a condition, palindromes
Sliding window / Kadane	contiguous subarray, "longest/shortest/max sum"
Prefix sum / difference array	many range queries or range updates
Majority / counting tricks	"more than n/2 times"
Cyclic placement	values in range 1..n, "missing" or "duplicate"
Greedy	"minimum number of...", local choice is safe
Math / number theory	primes, gcd, powers, modular arithmetic

3.1 Pattern: In-Place Rearrangement

PATTERN CARD

SPOT IT WHEN	Rotate, partition, segregate, rearrange or 'do it in place / $\mathcal{O}(1)$ extra space'.
KNOW FIRST	Swapping, array indices, reversing a range.
BREAKS WHEN	Relative order must be preserved AND you need $\mathcal{O}(1)$ space (then a stable method like write-pointer is needed, not swap-with-end).
COMMON VARIATIONS	Reversal trick (rotate), Dutch flag (3 classes), next permutation (find pivot, swap, reverse).

Idea: reshape the array using swaps and reversals, with $\mathcal{O}(1)$ extra memory.

SOLVED EXAMPLE 3.1 — E

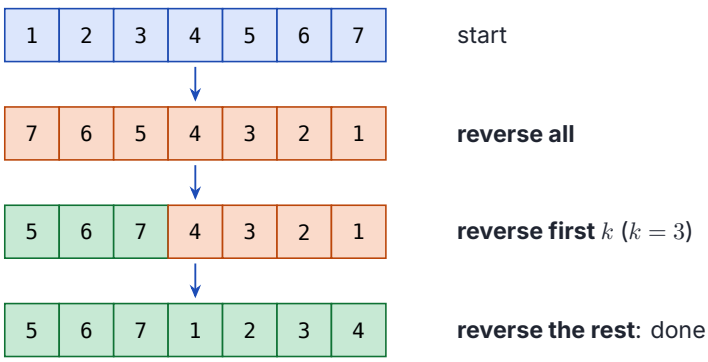
Rotate Array

Problem: Rotate `a` to the right by `k` steps. `[1,2,3,4,5,6,7], k=3` gives `[5,6,7,1,2,3,4]`.

Brute force: rotate by one position k times: $\mathcal{O}(n \cdot k)$.

Insight (the reversal trick): reversing the whole array moves the last k elements to the front, but backwards. Then reverse each part to fix the order.

PICTURE IT



```cpp
void rotateArray(vector<int>& a, int k) {      // k >= 0
    int n = a.size();
    if (n == 0) return;                        // k %= 0 would divide by zero
    k %= n;                                    // k may exceed n
    reverse(a.begin(), a.end());
    reverse(a.begin(), a.begin() + k);
    reverse(a.begin() + k, a.end());
}
```

Complexity: $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space. (`k` must be non-negative; an empty array is returned unchanged.)

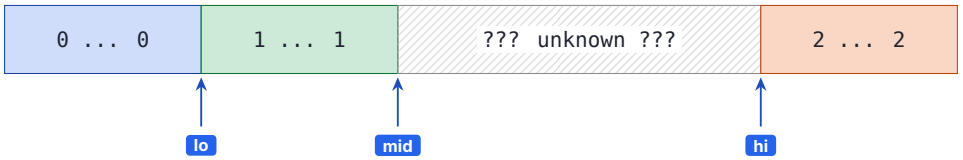
SOLVED EXAMPLE 3.2 E

Segregate 0s, 1s and 2s (Dutch National Flag)

Problem: Sort an array containing only 0, 1, 2 in one pass.

Approach: keep three regions with three pointers.

PICTURE IT



- `a[mid] == 0`: swap with `lo`, advance both.
- `a[mid] == 1`: just advance `mid`.
- `a[mid] == 2`: swap with `hi`, shrink `hi` (do **not** advance `mid`; the swapped-in element is unchecked).

```cpp
void sort012(vector<int>& a) {
    int lo = 0, mid = 0, hi = (int)a.size() - 1;
    while (mid <= hi) {
        if (a[mid] == 0)      swap(a[lo++], a[mid++]);
        else if (a[mid] == 1) mid++;
        else                  swap(a[mid], a[hi--]);
    }
}
```

The same partition idea solves **Segregate 0s and 1s** (two pointers) and **Sort Array By Parity**.

Trace on `[2,0,2,1,0]` (`lo = mid = 0`, `hi = 4`):

step	a[mid]	action	array after	lo, mid, hi
1	2	swap with `hi`, `hi--` (do NOT move `mid`)	`[0,0,2,1,2]`	0, 0, 3
2	0	swap with `lo`, both advance	`[0,0,2,1,2]`	1, 1, 3
3	0	swap with `lo`, both advance	`[0,0,2,1,2]`	2, 2, 3
4	2	swap with `hi`, `hi--`	`[0,0,1,2,2]`	2, 2, 2
5	1	`mid++`	`[0,0,1,2,2]`	2, 3, 2

`mid > hi`, so the unknown region is empty and the array is sorted. Step 1 shows why `mid` stays put after a swap with `hi`: the value that arrives from the right end has not been looked at yet.

SOLVED EXAMPLE 3.3 M

Next Greater Element III (Next Permutation)

Problem: Given `n`, find the smallest integer with the same digits that is greater than `n`; return -1 if none or it overflows 32-bit.

Approach (next permutation): for `1 2 4 3`:

1. Scan from the right to find the first digit smaller than its right neighbor (`2`, the *pivot*).
2. Find the smallest digit to the right of the pivot that is larger than it (`3`) and swap: `1 3 4 2`.
3. Reverse the suffix after the pivot (it was descending, now make it ascending): `1 3 2 4`.

```cpp
int nextGreaterElement(int n) {
    string s = to_string(n);
    int i = (int)s.size() - 2;
    while (i >= 0 && s[i] >= s[i + 1]) i--;       // find pivot
    if (i < 0) return -1;                         // fully descending: no answer
    int j = (int)s.size() - 1;
    while (s[j] <= s[i]) j--;                     // smallest larger digit on right
    swap(s[i], s[j]);
    reverse(s.begin() + i + 1, s.end());
    long long ans = stoll(s);
    return ans > INT_MAX ? -1 : (int)ans;
}
```

Complexity: $\mathcal{O}(d)$ for d digits. In practice the STL's `next_permutation` does steps 1-3 for you.

Practice Set 3.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *In-Place Rearrangement* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Squares of a Sorted Array E	Two pointers from both ends; the largest square is at one end. Fill result from the back.
Reverse Vowels of a String E	Two pointers; swap only when both point to vowels.
Sort Array By Parity E	Same partition idea as Dutch flag with two regions.
Segregate 0s and 1s E	Two pointers: swap when left sees 1 and right sees 0.
Maximum Swap M	Record last index of each digit 0-9; for each position try the largest digit that appears later.
Wiggle Sort M	One pass: for index i, enforce `a[i-1] <= a[i]` when i is odd, `>=` when even; swap if violated.
Push Dominoes M	Compute the force from the nearest `R` on the left and `L` on the right (two passes).
Minimum Domino Rotations For Equal Row M	Only `tops[0]` or `bottoms[0]` can be the target value; test each candidate.
Multiply Strings M	Grade-school multiplication: `res[i+j+1] += a[i]*b[j]` then propagate carries.

3.2 Pattern: Two Pointers

PATTERN CARD

SPOT IT WHEN	Sorted input (or sortable) + pair/triplet, or you work from both ends of a sequence.
KNOW FIRST	Sorting; why a comparison lets you discard one end safely.
BREAKS WHEN	Array is unsorted and order must be kept (use a hash map); the discard rule has no monotonic reason behind it.
COMMON VARIATIONS	Opposite-direction (pair sum, water), same-direction (read/write, gap), 3Sum = fix one + two pointers.

Use when: the array is sorted (or can be), you need a pair/triplet, or you work from both ends. Two indices move based on a comparison, so each element is visited once: $\mathcal{O}(n)$ instead of $\mathcal{O}(n^2)$.

PICTURE IT



each step moves one pointer inward, so every element is visited once

SOLVED EXAMPLE 3.4 M

Container With Most Water

Problem: `h[i]` are vertical lines. Pick two lines that, with the x-axis, hold the most water.

Brute force: try every pair, $\mathcal{O}(n^2)$.

Insight: area = `min(h[l], h[r]) * (r - l)`. Start with the widest container. Moving the *taller* line inward can only shrink width without raising the min, so it can never help. Move the **shorter** line.

```
int maxArea(vector<int>& h) {
    int l = 0, r = (int)h.size() - 1, best = 0;
    while (l < r) {
        best = max(best, min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;
    }
    return best;
}
```

Complexity: $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

SOLVED EXAMPLE 3.5 M

Trapping Rain Water

Problem: Given bar heights, compute how much rain water is trapped.

Key fact: water above bar `i` is `min(maxLeft, maxRight) - h[i]`.

Approach: with two pointers, whichever side has the *smaller* bar is bounded by its own side's maximum, so we can settle that side immediately.

```cpp
int trap(vector<int>& h) {
    int l = 0, r = (int)h.size() - 1, lmax = 0, rmax = 0, water = 0;
    while (l < r) {
        if (h[l] < h[r]) {
            lmax = max(lmax, h[l]);
            water += lmax - h[l];
            l++;
        } else {
            rmax = max(rmax, h[r]);
            water += rmax - h[r];
            r--;
        }
    }
    return water;
}
```

Complexity: $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

SOLVED EXAMPLE 3.6 E/M

Two Sum / Pair with Given Difference

Sorted-array two sum with two pointers:

```cpp
// returns true if a pair sums to target (a is sorted)
bool hasPairSum(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return true;
        if (s < target) l++; else r--;
    }
    return false;
}
```

For the **unsorted** Two Sum, use a hash map (Chapter 10). For **Pair with Given Difference** (sorted, `d > 0`), keep `i < j` both moving right: if `a[j] - a[i] < d` move `j`; if `> d` move `i`.

Practice Set 3.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Two Pointers* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Valid Palindrome II E	Two pointers inward; on first mismatch try skipping the left or right char and check the rest is a palindrome.
Boats to Save People M	Sort; pair heaviest with lightest if they fit, otherwise heaviest goes alone.

3.3 Pattern: Kadane's Algorithm & Sliding Window

PATTERN CARD

SPOT IT WHEN	'Contiguous subarray / substring', max sum, or longest/shortest window with a condition.
KNOW FIRST	Prefix idea; what 'window valid' means for this problem.
BREAKS WHEN	Values can be negative AND the condition is not monotonic (shrinking does not restore validity) → use prefix sums + hash map.
COMMON VARIATIONS	Kadane (best ending here), fixed window, variable window, 'at most K' minus 'at most K-1' for 'exactly K'.

SOLVED EXAMPLE 3.7 E

Maximum Subarray (Kadane's)

Problem: Find the contiguous subarray with the largest sum.

Insight: at each index, either **extend** the best subarray ending at the previous index, or **start fresh** here. If the running sum goes negative it only hurts, so drop it.

```
int maxSubArray(vector<int>& a) {
    int cur = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max(a[i], cur + a[i]);     // extend or restart
        best = max(best, cur);
    }
    return best;
}
```

Complexity: $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Trace on `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`:

i	a[i]	cur = max(a[i], cur + a[i])	best
0	-2	-2 (start)	-2
1	1	max(1, -1) = **1** (restart)	1
2	-3	max(-3, -2) = -2	1
3	4	max(4, 2) = **4** (restart)	4
4	-1	max(-1, 3) = 3	4
5	2	max(2, 5) = 5	5
6	1	max(1, 6) = 6	**6**
7	-5	max(-5, 1) = 1	6
8	4	max(4, 5) = 5	6

The best subarray is `[4,-1,2,1]` with sum 6. Notice `cur` is the best sum of a subarray that must END at `i`; `best` remembers the largest `cur` seen. An array of all negatives still works: `cur` and `best` start at `a[0]` rather than 0, so the answer is the largest single element.

SOLVED EXAMPLE 3.8 M

Maximum Product Subarray

Twist: a negative number can flip the smallest product into the largest. Track both max and min ending here. (Assume every subarray product fits in `int`; otherwise use `long long`.)

```
int maxProduct(vector<int>& a) {
    int curMax = a[0], curMin = a[0], best = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        if (a[i] < 0) swap(curMax, curMin);    // sign flip swaps roles
        curMax = max(a[i], curMax * a[i]);
        curMin = min(a[i], curMin * a[i]);
        best = max(best, curMax);
    }
    return best;
}
```

SOLVED EXAMPLE 3.9 M

Number of Subarrays with Bounded Maximum

Problem: Count subarrays whose maximum lies in `[L, R]`.

Insight (count by right endpoint): for each `i` as right end, valid left ends `l` must satisfy: no element `> R` in `[l..i]`, and at least one element in `[L..R]`. Let `lastBig` = last index with value `> R`, `lastIn` = last index with value in `[L, R]`. Valid left ends are `(lastBig, lastIn]`, so there are `max(0, lastIn - lastBig)` of them.

```
int numSubarrayBoundedMax(vector<int>& a, int L, int R) {
    int lastBig = -1, lastIn = -1, ans = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        if (a[i] > R) lastBig = i;
        if (a[i] >= L && a[i] <= R) lastIn = i;
        ans += max(0, lastIn - lastBig);
    }
    return ans;
}
```

The Sliding Window Template

Use it for "longest/shortest subarray satisfying a condition" when elements are non-negative or conditions are monotonic:

```
int l = 0;
for (int r = 0; r < n; r++) {
    // 1. add a[r] to window state
    while (/* window invalid */) {
        // 2. remove a[l] from window state
        l++;
    }
    // 3. window [l..r] is valid: update answer
}
```

You will use this exact skeleton again in Chapter 10 (strings with hash maps).

Practice Set 3.3

BEFORE OPENING THE HINTS
try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Kadane's Algorithm & Sliding Window* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
K-Concatenation Maximum Sum M	If `k == 1` plain Kadane. For `k >= 2`: Kadane on two copies; if total sum `> 0` add `(k-2) * total`.
Max Sum of Two Non-Overlapping Subarrays H	Prefix sums; for each split point track the best window of length `L` on the left and `M` on the right (try both orders).
Smallest Range Covering Elements from K Lists H	Min-heap holding one element from each list; track current max; advance the list that owns the min.

3.4 Pattern: Prefix Sums & Difference Arrays

PATTERN CARD

SPOT IT WHEN	Many range-sum queries, or many range updates followed by one read.
KNOW FIRST	Cumulative sums; 1-indexed vs 0-indexed bookkeeping.
BREAKS WHEN	Array changes between queries (use a Fenwick/segment tree); the operation is not invertible (e.g. max has no subtraction).
COMMON VARIATIONS	1D/2D prefix sums, difference array, prefix product (with care for zeros), prefix XOR.

Prefix sum: `pre[i] = a[0] + ... + a[i-1]`, so `sum(l..r) = pre[r+1] - pre[l]` in $\mathcal{O}(1)$.

Difference array: the reverse trick. To add `v` to all of `a[l..r]`, do `d[l] += v; d[r+1] -= v`; one prefix sum at the end rebuilds the result. Many range updates cost $\mathcal{O}(1)$ each.

SOLVED EXAMPLE 3.10 M

Product of Array Except Self

Problem: `res[i]` = product of all elements except `a[i]`, **without division**.

Insight: `res[i] = (product of everything left of i) * (product of everything right of i)`: a prefix product and a suffix product.

```
vector<int> productExceptSelf(vector<int>& a) {
    int n = a.size();
    vector<int> res(n, 1);
    int left = 1;
    for (int i = 0; i < n; i++) { res[i] = left; left *= a[i]; }
    int right = 1;
    for (int i = n - 1; i >= 0; i--) { res[i] *= right; right *= a[i]; }
    return res;
}
```

SOLVED EXAMPLE 3.11 M

Range Addition

Problem: Array of length `n` starts at 0. Apply updates `[start, end, inc]`. Return the final array.

```
vector<int> getModifiedArray(int n, vector<vector<int>>& updates) {
    vector<int> diff(n + 1, 0);
    for (auto& u : updates) {
        diff[u[0]] += u[2];
        diff[u[1] + 1] -= u[2];
    }
    vector<int> res(n);
    int run = 0;
    for (int i = 0; i < n; i++) { run += diff[i]; res[i] = run; }
    return res;
}
```

Complexity: $\mathcal{O}(n + q)$ instead of $\mathcal{O}(n \cdot q)$.

Practice Set 3.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Prefix Sums & Difference Arrays* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Max Range Query M	Difference array over all ranges, then prefix sum and take the argmax.
Napoleon Cake M	Sweep from the last layer downward carrying how far the cream from above reaches; mark layers drenched with a difference array or running counter.
Consecutive Numbers Sum H	Math: `N = k*x + k(k-1)/2`; loop `k` while `k(k-1)/2 < N` and count integer `x`.
Sum of Subsequence Widths H	Sort. Element `a[i]` is max in `2^i` subsequences and min in `2^(n-1-i)`. Sum `a[i] * (2^i - 2^(n-1-i))` mod p.

3.5 Pattern: Majority & Counting Tricks

PATTERN CARD

SPOT IT WHEN	'Appears more than n/2 (n/k) times', 'find the element that...' with $\mathcal{O}(1)$ space.
KNOW FIRST	Counting with a map (the baseline you are improving on).
BREAKS WHEN	No guarantee that a majority exists: the voting result is only a CANDIDATE; always verify with a second pass.
COMMON VARIATIONS	Boyer-Moore (k=2), Misra-Gries (general k), bit-by-bit counting.

SOLVED EXAMPLE 3.12 E

Majority Element (Boyer-Moore Voting)

Problem: Find the element that appears more than `n/2` times.

Insight: pair off different elements and cancel them. The majority survives because it outnumbers everyone else combined.

```
int majorityElement(vector<int>& a) {
    int cand = 0, count = 0;
    for (int x : a) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    return cand;          // guaranteed to exist; otherwise verify with a second pass
}
```

Trace on `[2,2,1,1,1,2,2]` (majority is 2, 4 of 7):

x	candidate before	count before	after
2	-	0	candidate 2, count 1
2	2	1	count 2
1	2	2	count 1
1	2	1	count 0
1	2	0	count was 0 so candidate = 1, count 1
2	1	1	count 0
2	1	0	count was 0 so candidate = 2, count 1

The final candidate is 2. Each decrement cancels one majority element against one different element; since the majority has more than half of all elements, it cannot be fully cancelled.

SOLVED EXAMPLE 3.13 H

Majority Element (General, more than n/k)

Generalization (Misra-Gries): keep at most `k-1` candidates with counters. A new element not in the table and with the table full decrements *every* counter (removing zeros). Finally verify survivors by counting.

```
vector<int> majorityK(vector<int>& a, int k) {
    unordered_map<int,int> cnt;
    for (int x : a) {
        cnt[x]++;
        if ((int)cnt.size() == k) {                // k distinct: cancel one of each
            for (auto it = cnt.begin(); it != cnt.end();) {
                if (--(it->second) == 0) it = cnt.erase(it);
                else ++it;
            }
        }
    }
    vector<int> res;
    for (auto& [val, _] : cnt) {                   // verify candidates
        if (count(a.begin(), a.end(), val) > (int)a.size() / k) res.push_back(val);
    }
    return res;
```

```
}
```

With `k = 3` this solves **Majority Element II** (elements more than n/3 times). With `k = 2` it is Boyer-Moore.

Complexity: each time the table reaches `k` distinct values we remove one copy of each, so there are at most `n/k` such rounds of $\mathcal{O}(k)$ work: $\mathcal{O}(n)$ amortized for the counting pass (plus $\mathcal{O}(k)$ per verification count, so $\mathcal{O}(nk)$ overall in the simple code above), and $\mathcal{O}(k)$ space.

Practice Set 3.5

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Majority & Counting Tricks* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Majority Element II M	`majorityK` with `k = 3` (at most 2 answers).
Largest Number At Least Twice the Others E	Find max and second max in one pass; compare.
Check Whether One String Is a Rotation of Another E	`s2` is a rotation of `s1` iff `s1.size()==s2.size()` and `s2` appears in `s1 + s1`.

3.6 Pattern: Chunks, Cycles & Cyclic Placement

PATTERN CARD

SPOT IT WHEN	Values lie in 1..n (or 0..n-1), find missing/duplicate/misplaced, or 'minimum swaps'.
KNOW FIRST	Array as a permutation; cycles of a permutation.
BREAKS WHEN	Values fall outside the index range (clamp or ignore them first), or the array must stay unmodified.
COMMON VARIATIONS	Cyclic placement (swap into home), index-as-hash (negate), permutation-cycle counting, chunk counting via running max.

SOLVED EXAMPLE 3.14 M

Max Chunks To Make Sorted

Problem: `a` is a permutation of `0..n-1`. Split into the most chunks such that sorting each chunk individually sorts the whole array.

Insight: a chunk can end at index `i` exactly when the maximum of the prefix equals `i` (all of `0..i` are already inside).

```
int maxChunksToSorted(vector<int>& a) {
    int chunks = 0, mx = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        mx = max(mx, a[i]);
        if (mx == i) chunks++;
    }
    return chunks;
}
```

SOLVED EXAMPLE 3.15 H

Max Chunks To Make Sorted II

Twist: values may repeat and be arbitrary. Keep a **stack of chunk maximums**. When a new value is smaller than the top, it must merge into earlier chunks; pop and remember the largest.

```
int maxChunksToSortedII(vector<int>& a) {
    vector<int> st;                                  // chunk maxima, increasing
    for (int x : a) {
        int mx = x;
        while (!st.empty() && st.back() > x) {       // x forces a merge
            mx = max(mx, st.back());
            st.pop_back();
        }
        st.push_back(mx);
    }
    return st.size();
}
```

SOLVED EXAMPLE 3.16 M

Minimum Swaps to Sort an Array (Permutation Cycles)

Insight: map each element to its sorted position. This forms **cycles**. A cycle of length `L` needs exactly `L - 1` swaps. (The elements are assumed **distinct**; with duplicates there are several valid target positions and the cycle count below is no longer guaranteed to be minimal.)

Example. `[4,3,2,1]`: sorted order is `[1,2,3,4]`. Element 4 belongs at index 3, element 1 at index 0, so positions 0 and 3 form a cycle of length 2; positions 1 and 2 form another. Swaps = (2 - 1) + (2 - 1) = 2.

```
int minSwaps(vector<int>& a) {
    int n = a.size();
    vector<pair<int,int>> v(n);                      // (value, original index)
    for (int i = 0; i < n; i++) v[i] = {a[i], i};
    sort(v.begin(), v.end());
    vector<bool> vis(n, false);
    int swaps = 0;
    for (int i = 0; i < n; i++) {
        if (vis[i] || v[i].second == i) continue;    // already placed
        int len = 0, j = i;
        while (!vis[j]) { vis[j] = true; j = v[j].second; len++; }
        swaps += len - 1;
    }
    return swaps;
}
```

SOLVED EXAMPLE 3.17 H

First Missing Positive

Problem: Find the smallest missing positive integer in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space.

Insight (cyclic placement): among `n` numbers, the smallest missing positive must be one of `1..n+1` (if `1..n` are all present, the answer is `n+1`). So we only care about values in `1..n`. Use the array itself as a hash table: put each value `v` (if `1 <= v <= n`) at index `v-1`. Then the first index `i` where `a[i] != i+1` reveals the answer `i+1`.

```cpp
int firstMissingPositive(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++) {
        while (a[i] > 0 && a[i] <= n && a[a[i] - 1] != a[i])
            swap(a[i], a[a[i] - 1]);                // send a[i] to its home
    }
    for (int i = 0; i < n; i++)
        if (a[i] != i + 1) return i + 1;
    return n + 1;
}
```

Trace on `[3,4,-1,1]`:

1. `i = 0`: `a[0] = 3` belongs at index 2, and `a[2] = -1` is not 3, so swap: `[-1,4,3,1]`. Now `a[0] = -1` is out of range, so move on.
2. `i = 1`: `a[1] = 4` belongs at index 3, and `a[3] = 1` is not 4, so swap: `[-1,1,3,4]`. Now `a[1] = 1` belongs at index 0, and `a[0] = -1` is not 1, so swap: `[1,-1,3,4]`. Now `a[1] = -1` is out of range, so move on.
3. `i = 2`: `a[2] = 3` is already home (`a[a[2]-1] == a[2]`), skip. `i = 3`: `a[3] = 4` is home, skip.

The scan finds `a[1] = -1 != 2`, so the answer is **2**. The `a[a[i]-1] != a[i]` test also stops the loop on duplicates, which would otherwise swap forever.

Complexity: $\mathcal{O}(n)$: each swap places one value permanently.

3.7 Pattern: Greedy on Arrays

PATTERN CARD

SPOT IT WHEN	'Minimum / maximum' with a local choice that seems obviously safe, or partitioning into groups.
KNOW FIRST	An exchange-argument mindset (Ch 11 intro).
BREAKS WHEN	A locally best choice can hurt later (coin systems like {1,3,4}); then it is DP.
COMMON VARIATIONS	Last-occurrence partitioning, jump-reach, buy-low-sell-high.

SOLVED EXAMPLE 3.18 — M

Partition Labels

Problem: Split a string so each letter appears in at most one part; maximize the number of parts.

Insight: record the **last index** of each letter. Walk the string, extending the current part's end to cover the last occurrence of every letter seen. When `i` reaches that end, cut.

```cpp
vector<int> partitionLabels(string s) {
    int last[26];
    for (int i = 0; i < (int)s.size(); i++) last[s[i] - 'a'] = i;
    vector<int> parts;
    int start = 0, end = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        end = max(end, last[s[i] - 'a']);
        if (i == end) { parts.push_back(end - start + 1); start = i + 1; }
    }
    return parts;
}
```

Practice Set 3.7

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Greedy on Arrays* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Maximize Distance to Closest Person M	Three cases: leading empty seats, trailing empty seats, and gaps (`gap/2`).
Minimise the Maximum Difference Between Heights M	Each height must change by exactly `+k` or `-k`. Sort; the shorter towers (a prefix) should go UP by `k` and the taller towers (the rest) DOWN by `k`. Try every split point; for split `i` the new minimum is `min(a[0]+k, a[i+1]-k)` and the new maximum is `max(a[i]+k, a[n-1]-k)`. Keep the smallest difference.
Long Pressed Name M	Two pointers: advance both on match; if mismatch, current typed char must equal previous typed char.
Buddy Nim M	Game theory: check the parity/XOR condition that decides the winner; simulate small cases to spot it.
Best Meeting Point H	Manhattan distance separates into rows and columns; the **median** of each coordinate list minimizes the sum.

3.8 Pattern: Math & Number Theory

PATTERN CARD

SPOT IT WHEN	Primes, divisors, powers modulo m, GCD/LCM, counting by formula.
KNOW FIRST	Modulo arithmetic; integer overflow limits.
BREAKS WHEN	n up to ~1e12 with sieve (memory) or exponent so huge you must reduce it first (Euler / Fermat).
COMMON VARIATIONS	Sieve, segmented sieve, fast power, Euclid, modular inverse.

SOLVED EXAMPLE 3.19 E

Sieve of Eratosthenes

Problem: List all primes up to `n`.

Insight: for each prime `p`, cross out its multiples starting at `p*p` (smaller multiples such as `2p, 3p, ...` were already crossed out by the smaller primes 2, 3, ...). We only need primes `p` with `p*p <= n`, because every composite number up to `n` has a prime factor at most `sqrt(n)`.

```
vector<int> sieve(int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = false;
    if (n >= 1) isPrime[1] = false;
    for (long long p = 2; p * p <= n; p++)
        if (isPrime[p])
            for (long long m = p * p; m <= n; m += p) isPrime[m] = false;
    vector<int> primes;
    for (int i = 2; i <= n; i++) if (isPrime[i]) primes.push_back(i);
```

```
    return primes;
}
```

Complexity: $\mathcal{O}(n \log \log n)$.

SOLVED EXAMPLE 3.20 E

Fast Exponentiation, GCD, Coprime

```
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = res * base % mod;      // use this bit
        base = base * base % mod;                 // square
        exp >>= 1;
    }
    return res;                                   // O(log exp)
}

int gcdOf(int a, int b) { return b == 0 ? a : gcdOf(b, a % b); }   // or std::gcd
bool coprime(int a, int b) { return gcdOf(a, b) == 1; }            // Relatively Prime
```

Why fast power works. Write the exponent in binary: `13 = 1101b`, so `x^13 = x^8 * x^4 * x^1`. The loop squares `base` each round (`x, x^2, x^4, x^8, ...`) and multiplies it into `res` only when the current binary digit of `exp` is 1. That is $\mathcal{O}(\log \text{exp})$ multiplications instead of `exp`. The `% mod` after every multiplication keeps numbers small; the code assumes `mod` is below about 3×10^9 so that a product of two residues fits in `long long`.

Fibonacci Number: iterate with two variables (`prev`, `cur`) for $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space.

Practice Set 3.8

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Math & Number Theory* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Fibonacci Number E	Two variables. For huge `n` use matrix exponentiation.
Segmented Sieve M	Sieve primes up to `sqrt(R)`; use them to mark composites in `[L, R]`.
K-LCM I / K-LCM II M	Split `n` into `k` positive numbers whose LCM is at most `n/2`. For `k = 3` build by cases: `n` odd gives `(1, (n-1)/2, (n-1)/2)`; `n % 4 == 0` gives `(n/2, n/4, n/4)`; otherwise `(2, (n-2)/2, (n-2)/2)`. For larger `k`, pad with `k-3` ones and reduce to the `k = 3` case.
Meximization H	Greedy with sorting and counts: build the answer prefix so MEX is maximized at each step.

3.9 Multi-Technique Problems

Problem	Level 2 hint (approach)
Bulb Switcher M	A bulb ends ON iff it has an odd number of divisors, i.e. is a perfect square. Answer: `floor(sqrt(n))`.
Kth Largest and Smallest Element in an Array M	`nth_element` ($\mathcal{O}(n)$ average) or a heap of size `k`; or Quickselect.
Max Distance Between Two Elements ($a[i] \leq a[j], i < j$) M	Prefix-minimum array and suffix-maximum array, then two pointers.
Minimum Jump Required to Reach a Number M	Add `1,2,3,...` until sum `>= target` and `(sum - target)` is even.
Bulb Switcher III H	A moment is "all on so far" when the max index turned on equals the count turned on.
Orderly Queue H	If `k == 1`, try all rotations and take the smallest. If `k >= 2`, any permutation is reachable, so return the sorted string.
Interesting Subarrays H	Read the exact condition; most variants reduce to prefix aggregates with a hash map or monotonic structure.

CHAPTER COVERAGE

all 60 Array & String problems from your sheet appear as Solved Examples or in the practice tables of Sections 3.1-3.9.

CHAPTER 3 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 3.1 Rotate Array, 3.7 Kadane, 3.12 Majority (Boyer-Moore).
2. **Pattern variations:** 3.2 Dutch Flag, 3.8 Max Product Subarray, 3.10 Product Except Self.
3. **Combined patterns:** 3.9 Bounded Maximum, 3.16 Min Swaps, 3.18 Partition Labels.
4. **Independent recognition:** Practice Sets 3.2 and 3.4 *without* reading the section titles first.
5. **Challenge:** Ex 3.13 (n/k majority), 3.15 Chunks II, 3.17 First Missing Positive.

Compare and Contrast

	Sliding window	Prefix sums
Idea	grow right, shrink left	cumulative totals, subtract
Needs	condition restorable by shrinking (usually non-negative values)	any values, including negatives
Answers	one best window	many queries, or 'count windows with sum K' (with a hash map)
Space	$\mathcal{O}(1)$	$\mathcal{O}(n)$

Rule of thumb: negative numbers + 'sum equals K' → prefix sums + hash map (Chapter 10). Positive numbers + 'longest/shortest window' → sliding window.

Why It Works (Correctness Corner)

Kadane by induction. Let `end[i]` be the best sum of a subarray *ending exactly at i*. Either it is just `a[i]`, or it extends the best subarray ending at `i-1`: `end[i] = max(a[i], end[i-1] + a[i])`. The answer is `max_i end[i]`, since every optimal subarray ends somewhere.

Two-pointer container proof. When `h[l] < h[r]`, any container using `l` with a right end `r' < r` is both narrower and no taller than the current one (limited by `h[l]`). So `l` can never be part of a better pair with anything left of `r`; discarding it loses nothing.

Debug Lab: Off-by-one in window length

```
int longestOnes(vector<int>& a, int k) {          // longest run with at most k zeros
    int left = 0, zeros = 0, best = 0;
    for (int right = 0; right < a.size(); right++) {
        if (a[right] == 0) zeros++;
        while (zeros > k) { if (a[left] == 0) zeros--; left++; }
        best = max(best, right - left);
    }
    return best;
}
```

Your task: Input `[1,1,1]`, `k = 0` should return 3. What does this return, and why?

ANSWER

It returns 2. A window from `left` to `right` inclusive has `right - left + 1` elements. Fix: `best = max(best, right - left + 1)`. Also compare `int` with `a.size()` safely by casting.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 3A M

Longest Run with At Most K Zeros

Statement. Given a binary array and an integer `k`, return the length of the longest contiguous subarray that contains at most `k` zeros (you may think of flipping up to `k` zeros to ones).

Example. `a = [1,1,1,0,0,0,1,1,1,1,0], k = 2` → `6`

Constraints. $0 \le n \le 10^5$; a[i] in {0,1}; $0 \le k \le n$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 3.3 Sliding Window

HINT 1 which idea is relevant?

The answer is a contiguous stretch with a condition on a count. Which pattern is built for that?

HINT 2 outline of the approach

Keep a window [left, right] and count zeros inside it. When is the window invalid?

HINT 3 the key step

Extend `right` every step. While zeros > k, move `left` forward, updating the zero count.

SOLUTION and editorial (try first, then compare)

```cpp
int longestOnes(const vector<int>& a, int k) {
    int left = 0, zeros = 0, best = 0;
    for (int right = 0; right < (int)a.size(); right++) {
        if (a[right] == 0) zeros++;
        while (zeros > k) {                // window invalid: shrink until it is valid again
            if (a[left] == 0) zeros--;
            left++;
        }
        best = max(best, right - left + 1);
    }
    return best;
}
```

Every index is added once (`right`) and removed at most once (`left`), so the total work is $\mathcal{O}(n)$ even though there is a `while` inside a `for`. The window is always valid after the `while`, so its size is a candidate answer.

PROBLEM 3B E

Move Zeroes (Stable)

Statement. Move all zeros to the end of the array in place, keeping the relative order of the non-zero elements. Use $\mathcal{O}(1)$ extra space.

Example. `[0,1,0,3,12]` → `[1,3,12,0,0]`

Constraints. $0 \leq n \leq 10^5$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space, one pass.

Prerequisites. 3.1 In-Place Rearrangement

HINT 1 which idea is relevant?

Think of two roles: a pointer that *reads* every element and one that marks where the next non-zero belongs.

HINT 2 outline of the approach

Everything before `write` is final: non-zeros in the original order.

HINT 3 the key step

When `a[read] != 0`, swap it into position `write` and advance `write`.

SOLUTION and editorial (try first, then compare)

```
void moveZeroes(vector<int>& a) {
    int write = 0;                          // everything before 'write' is non-zero, in
    ↪  original order
    for (int read = 0; read < (int)a.size(); read++)
        if (a[read] != 0) swap(a[write++], a[read]);
}
```

Invariant: `a[0..write)` holds the non-zeros seen so far in order, and `a[write..read)` is all zeros. The swap keeps both facts true. Swapping with itself when `write == read` is harmless.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T3.1 A bakery logs units sold each hour (all positive). Find the longest stretch of hours whose total is at most S.

PATTERN

Variable sliding window (all values positive, so shrinking restores validity).

T3.2 10^5 updates say 'add d to shelves l..r'; afterwards print every shelf.

PATTERN

Difference array: add at l, subtract at r+1, take one prefix sum.

Mixed Practice

Combine 3.4 and 10.1 (later): count subarrays whose sum equals K when values may be negative. Which of the two techniques in this chapter's comparison table fails, and what replaces it?

Revision Sheet

Key takeaways

- Window = non-negative values + monotone condition; otherwise prefix sums + hash map.
- Kadane: extend or restart.
- Two pointers need a *reason* each move discards something safely.
- Read/write pointers give stable in-place filters.

Pattern-recognition clues

- 'Contiguous' + 'longest/shortest' → window.
- 'Many range queries' → prefix sums.
- 'Sorted + pair' → two pointers.
- '1..n' values → cyclic placement.

Common mistakes

- Using a window when negatives are present.
- `right - left` instead of `right - left + 1`.
- Trusting a majority candidate without verifying.

Complexity summary

Technique	Time	Space
Two pointers	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Sliding window	$\mathcal{O}(n)$	$\mathcal{O}(1)$-$\mathcal{O}(k)$
Kadane	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Prefix sums	$\mathcal{O}(n)$ build, $\mathcal{O}(1)$ query	$\mathcal{O}(n)$

Recall questions (answer aloud, then check the chapter)

1. Why can't a plain window solve 'subarray sum = K' with negatives?
2. State Kadane's recurrence.
3. What is the invariant of the Dutch flag algorithm?
4. When does Boyer-Moore need a second pass?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Without notes: (1) explain Kadane in two sentences, (2) given a new subarray problem decide window vs prefix sums and justify, (3) code Problem 3A from a blank file, (4) test with `k = 0`, all zeros, empty array, (5) state complexity.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 4

Matrix

4

START HERE

A bouquet-wrapping robot must visit every cell of a grid in a spiral. Simple to describe, surprisingly easy to get wrong: one extra row printed twice and the test fails. Matrices reward careful boundaries and invariants more than clever tricks.

ANALOGY

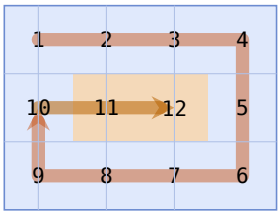
Peeling an onion: remove the outer layer, then repeat on what is left. The four boundaries (`top, bottom, left, right`) are the thickness of the remaining onion.

AFTER THIS CHAPTER YOU CAN

- ✓ Traverse a matrix in spiral/diagonal order without duplicates.
- ✓ Rotate a matrix in place using transpose + reverse.
- ✓ Search a row/column-sorted matrix in $\mathcal{O}(m+n)$.
- ✓ Decide between staircase search and binary search.

PICTURE IT

Spiral order and the shrinking boundaries



`read top row → top++ (top=1)`

`read right col → right-- (right=2)`

`read bottom row → bottom-- (bottom=1), only if top ≤ bottom`

`read left col → left++ (left=1), only if left ≤ right`

After layer 1 only the single row `[11 12]` remains:
the guards stop it from being read twice.

4.0 Concept

A matrix is a 2D array: `vector<vector<int>> m(rows, vector<int>(cols))`. Access `m[i][j]` is row `i`, column `j`. Matrix problems are array problems with **geometry**: directions, boundaries, and index arithmetic.

Direction arrays make grid movement clean:

```
int dr[4] = {-1, 1, 0, 0};
int dc[4] = {0, 0, -1, 1};
for (int d = 0; d < 4; d++) {
```

```
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;   // bounds check
    /* use m[nr][nc] */
}
```

Pattern	Recognition clue
Boundary shrinking	spiral, layer-by-layer traversal
Transpose and reflect	rotation, in-place transform
Staircase search	rows and columns both sorted
Binary search on answer over matrix	k-th smallest, median in sorted matrix

4.1 Pattern: Traversal

PATTERN CARD

SPOT IT WHEN	Visit a grid in a fixed geometric order (spiral, diagonal, layers).
KNOW FIRST	Row/column indexing; four boundary variables.
BREAKS WHEN	Boundaries are updated in the wrong order or not re-checked after the first half of a layer (duplicates on single row/column).
COMMON VARIATIONS	Spiral, layer-by-layer rotate, diagonal sweep, in-place transpose.

SOLVED EXAMPLE 4.1 M

Spiral Traversal of a Matrix

Approach: maintain four boundaries `top, bottom, left, right`. Walk the top row, right column, bottom row, left column; shrink each boundary after use. The `if` checks prevent double-visiting when one dimension runs out.

```
vector<int> spiralOrder(vector<vector<int>>& m) {
    vector<int> res;
    if (m.empty()) return res;
    int top = 0, bot = m.size() - 1, left = 0, right = m[0].size() - 1;
    while (top <= bot && left <= right) {
        for (int j = left; j <= right; j++) res.push_back(m[top][j]);
        top++;
        for (int i = top; i <= bot; i++) res.push_back(m[i][right]);
        right--;
        if (top <= bot) {
            for (int j = right; j >= left; j--) res.push_back(m[bot][j]);
            bot--;
        }
        if (left <= right) {
            for (int i = bot; i >= top; i--) res.push_back(m[i][left]);
            left++;
        }
    }
    return res;
}
```

SOLVED EXAMPLE 4.2 M

Rotate a Matrix by 90 Degrees

Insight: clockwise rotation = **transpose** (swap `m[i][j]` with `m[j][i]`) then **reverse each row**.

```
void rotate90(vector<vector<int>>& m) {
    int n = m.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) swap(m[i][j], m[j][i]);
    for (auto& row : m) reverse(row.begin(), row.end());
}
```

Trace on `[[1,2,3],[4,5,6],[7,8,9]]`. After the transpose the rows become the old columns: `[[1,4,7],[2,5,8],[3,6,9]]`. Reversing each row gives `[[7,4,1],[8,5,2],[9,6,3]]`: the old first column `1,4,7`, read bottom to top, is now the first row. This version works for square `n x n` matrices only (a non-square matrix changes shape when rotated, so it needs a new matrix).

For counter-clockwise, reverse each row first and then transpose (or transpose, then flip top-to-bottom, as in Problem 4B).

4.2 Pattern: Sorted Matrix Tricks

PATTERN CARD

SPOT IT WHEN	Rows AND/OR columns are sorted; 'search / kth / median in a matrix'.
KNOW FIRST	Binary search; staircase intuition.
BREAKS WHEN	Only rows are sorted (columns are arbitrary): staircase fails, binary-search each row instead.
COMMON VARIATIONS	Staircase from a corner, binary search on value range with row-wise upper_bound, heap merge of rows.

SOLVED EXAMPLE 4.3 E

Search an Element in a Row-Column Sorted Matrix

Insight (staircase search): start at the **top-right** corner. If the value is too big, move left (kills a column); if too small, move down (kills a row). Each step removes a row or column: $\mathcal{O}(R + C)$. (The code assumes the matrix has at least one row and one column.)

```
bool searchMatrix(vector<vector<int>>& m, int target) {
    int r = 0, c = (int)m[0].size() - 1;
    while (r < (int)m.size() && c >= 0) {
        if (m[r][c] == target) return true;
        if (m[r][c] > target) c--; else r++;
    }
    return false;
}
```

SOLVED EXAMPLE 4.4 M

Kth Smallest Element in a Row-Column Sorted Matrix

Approach: binary search on the answer value (matrix is `n x n`, rows and columns sorted ascending). For a candidate `x`, count how many elements are `<= x` using a staircase walk ($\mathcal{O}(n)$): start at the bottom-left; if the cell is `<= x`, then the whole column above it is `<= x` too, so add `r + 1` and move right; otherwise move up. The answer is the smallest `x` with count `>= k`. That `x` is always a value that really occurs in the matrix: if it did not, `x - 1` would have the same count and would be an even smaller valid candidate.

```
int kthSmallest(vector<vector<int>>& m, int k) {
    int n = m.size();
    int lo = m[0][0], hi = m[n - 1][n - 1];
    auto countLE = [&](int x) {
        int cnt = 0, r = n - 1, c = 0;          // start bottom-left
        while (r >= 0 && c < n) {
            if (m[r][c] <= x) { cnt += r + 1; c++; }
            else r--;
        }
        return cnt;
    };
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (countLE(mid) < k) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Complexity: $\mathcal{O}(\text{n log(maxValue - minValue)})$.

SOLVED EXAMPLE 4.5 M

Median in a Row-Wise Sorted Matrix

Same idea: rows sorted, columns not (so the staircase walk is unavailable, but each row can still be binary searched). Count elements `<= mid` by `upper_bound` in every row. Median is the smallest value with count `>= (R*C + 1) / 2` (R*C is odd). Each probe costs $\mathcal{O}(R \log C)$ and there are about log(maxValue - minValue) probes.

```
int matrixMedian(vector<vector<int>>& m) {
    int R = m.size(), C = m[0].size();
    int lo = INT_MAX, hi = INT_MIN;
    for (int i = 0; i < R; i++) { lo = min(lo, m[i][0]); hi = max(hi, m[i][C - 1]); }
    int need = (R * C + 1) / 2;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2, cnt = 0;
        for (int i = 0; i < R; i++)
            cnt += upper_bound(m[i].begin(), m[i].end(), mid) - m[i].begin();
        if (cnt < need) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

Practice Set 4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Sorted Matrix Tricks* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Row with Maximum Number of 1's E	Rows are sorted: start from top-right, move left while you see 1, else move down.
Print Elements in Sorted Order (Row-Column Sorted Matrix) M	Min-heap seeded with the first column or first row, like merging K sorted lists.
Common Elements in All Rows of a Matrix M	Hash map counting appearances per row (count once per row); keys seen in all rows.
Find a Specific Pair in a Matrix H	Find `max(c - a)` with `c` below-right of `a`: DP from the bottom-right keeping the max seen.
Trapping Rain Water II H	Min-heap of boundary cells; pop the lowest, flood neighbors (like Dijkstra), water = `max(0, level - height)`.

CHAPTER COVERAGE

all 10 Matrix problems appear above.

CHAPTER 4 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 4.1 Spiral Traversal, 4.2 Rotate 90 degrees.
2. **Variations:** Problem 4B (counter-clockwise), Practice Set 4.
3. **Combined:** Ex 4.4 Kth Smallest (matrix + binary search on answer).
4. **Independent recognition:** Problem 4A (looks like a search, is a staircase).
5. **Challenge:** Ex 4.5 Median in a Row-Wise Sorted Matrix.

Compare and Contrast

	Staircase search	Binary search per row
Needs	rows AND columns sorted	each row sorted
Time	$\mathcal{O}(m+n)$	$\mathcal{O}(m \log n)$
Idea	start at a corner; each step removes a row or column	independent search per row

Why It Works (Correctness Corner)

Staircase invariant. Standing at the top-right cell `(r, c)` of the remaining rectangle: if `g[r][c]` is too big, everything below it in column `c` is bigger still, so column `c` is eliminated; if too small, everything left of it in row `r` is smaller still, so row `r` is eliminated. Each step removes a full row or column, so at most `m + n` steps.

Debug Lab: The transpose that does nothing

```
void transposeInPlace(vector<vector<int>>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)       // <- bug
            swap(a[i][j], a[j][i]);
}
```

Your task: For `[[1,2],[3,4]]` the output is unchanged. Why?

ANSWER

Every off-diagonal pair is swapped twice (once as (i,j), once as (j,i)), which restores the original. Start the inner loop at `j = i + 1` so each pair is swapped exactly once.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 4A

E

Count Negatives in a Sorted Grid

Statement. A `m x n` grid is sorted in non-increasing order along every row and every column. Count the negative numbers.

Example. `[[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]]` → `8`

Constraints. $1 \le$ m, n ≤ 100; |value| ≤ 100.

Expected complexity. $\mathcal{O}(m+n)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 4.2 Sorted Matrix Tricks

HINT 1 which idea is relevant?

Both rows and columns are sorted. What does that let you throw away in one comparison?

HINT 2 outline of the approach

Start at the top-right corner.

HINT 3 the key step

If the cell is negative, the whole column below it is negative: add `m - r` and move left; otherwise move down.

SOLUTION and editorial (try first, then compare)

```cpp
int countNegatives(const vector<vector<int>>& g) {
    int m = g.size(), n = g[0].size(), r = 0, c = n - 1, cnt = 0;
    while (r < m && c >= 0) {
        if (g[r][c] < 0) { cnt += m - r; c--; }   // whole rest of this column is negative
        else r++;
    }
    return cnt;
}
```

Each step eliminates either a row or a column, so there are at most `m + n` steps. The top-right corner is special: it is the smallest in its row and the largest in its column, so each comparison decides in one direction.

PROBLEM 4B

M

Rotate Counter-Clockwise

Statement. Rotate an `n x n` matrix 90 degrees counter-clockwise in place.

Example. `[[1,2],[3,4]]` → `[[2,4],[1,3]]`

Constraints. $1 \le$ n ≤ 500.

Expected complexity. $\mathcal{O}(n^2)$ time, $\mathcal{O}(1)$ extra space.

Prerequisites. 4.1 Traversal, Example 4.2

HINT 1 which idea is relevant?

You already know a clockwise rotation (Example 4.2). Which two simple operations compose to a rotation?

HINT 2 outline of the approach

A rotation = transpose followed by a flip.

HINT 3 the key step

Counter-clockwise: transpose, then reverse the order of the rows (flip top to bottom).

SOLUTION and editorial (try first, then compare)

```cpp
void rotateCCW(vector<vector<int>>& a) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) swap(a[i][j], a[j][i]);   // transpose
    reverse(a.begin(), a.end());                                  // flip top-to-bottom
}
```

Transpose swaps rows and columns. Flipping vertically afterwards sends the top row to the bottom, which gives a counter-clockwise turn. For clockwise, flip each row horizontally instead.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T4.1 In a cinema each row's seat numbers rise left to right and each column's rise top to bottom. Is ticket 57 in the hall?

PATTERN

Staircase search from a corner, $\mathcal{O}(\text{rows} + \text{cols})$.

T4.2 Peel a rectangular cake clockwise and list the sprinkle sizes in order.

PATTERN

Spiral traversal with four shrinking boundaries.

Mixed Practice

Combine 4.2 and 5.2: a row/column-sorted matrix, find the k-th smallest. Which of the two ideas (staircase counting or heap) can serve as the `check(x)` in a binary search on values?

Revision Sheet

Key takeaways

- Name the four boundaries and update them in a fixed order.
- Rotate = transpose + flip.
- Sorted rows+columns: start at a corner.
- Re-check boundaries mid-layer to avoid duplicates on single rows/columns.

Pattern-recognition clues

- 'Spiral', 'layers' → boundaries.
- 'Rotate' → transpose + reverse.
- 'Sorted in both directions' → staircase.

Common mistakes

- Looping `j` from 0 in a transpose.
- Printing the last row/column twice in a spiral.
- Forgetting non-square matrices.

Complexity summary

Task	Time	Space
Spiral	$\mathcal{O}(mn)$	$\mathcal{O}(1)$ extra

Task	Time	Space
Rotate	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Staircase search	$\mathcal{O}(m+n)$	$\mathcal{O}(1)$

Recall questions (answer aloud, then check the chapter)

1. Why start at the top-right (or bottom-left) corner?
2. What two operations rotate a matrix?
3. How do you avoid duplicate cells in a spiral?
4. When does staircase search fail?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Code a spiral traversal for a `3 x 5` matrix from scratch, then test with a single row, single column and `1 x 1`.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 5

Sorting & Searching

5

START HERE

Guess a number from 1 to 1,000,000 with answers 'higher' or 'lower'. You need at most 20 guesses. Binary search turns *any* yes/no question with a monotone answer into a 20-step procedure, including questions that are not about arrays at all.

ANALOGY

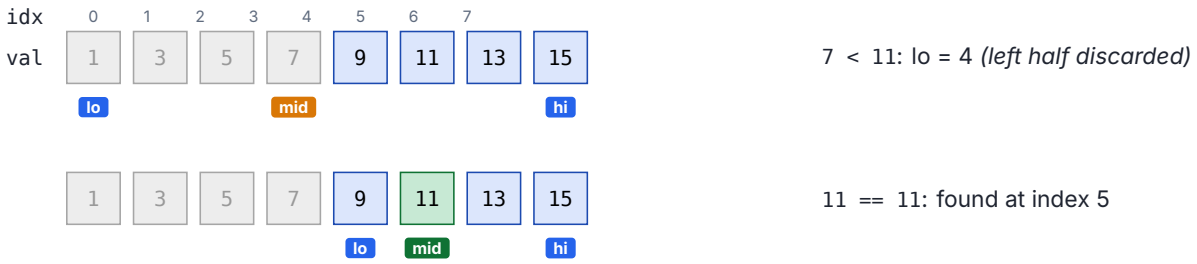
A phone book: open in the middle, decide which half the name is in, repeat. 'Binary search on the answer' does the same with a *question* instead of a book: 'is a speed of 5 fast enough?'

AFTER THIS CHAPTER YOU CAN

- ✓ Write a bug-free binary search by stating the invariant first.
- ✓ Search rotated or duplicated data.
- ✓ Spot a 'minimize the maximum' problem and write its `check(x)`.
- ✓ Count inversions with merge sort.

PICTURE IT

Binary search halves the interval every step



target = 11. 8 items ⇒ at most 4 probes; 10^6 items ⇒ at most 20.

5.0 Concept

Searching finds a target. **Sorting** puts data in order so searching, grouping and comparing become easy. The biggest idea in this chapter: **binary search works whenever there is a yes/no question that flips exactly once.**

Pattern	Recognition clue
Binary search on index	sorted array, rotated sorted array
Binary search on answer	"minimize the maximum", "smallest value such that feasible"
Merge-sort based counting	"count pairs where i < j and a[i] > a[j]"
Counting sort	small value range

5.1 Pattern: Binary Search Fundamentals

PATTERN CARD

SPOT IT WHEN	Sorted or rotated-sorted input; 'find position / first / last / insert'.
KNOW FIRST	Invariants: what does [lo, hi] promise to contain?
BREAKS WHEN	Data has no sorted/monotonic structure; duplicates break the 'which half is sorted' test.
COMMON VARIATIONS	Exact match, lower/upper bound, rotated array, peak finding, first-true boundary.

SOLVED EXAMPLE 5.1 E

Binary Search

```
int binarySearch(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;              // avoids overflow
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Always write `mid = lo + (hi - lo) / 2`. The STL gives you `lower_bound` (first element `>= x`) and `upper_bound` (first element `> x`), both $\mathcal{O}(\log\ n)$.

Two templates, one idea. Binary search keeps an interval that must still contain the answer, and it has two common shapes. Learn which goes with which:

	Exact match (the code above)	First index where a condition is true
Loop	`while (lo <= hi)`	`while (lo < hi)`
Interval means	answer is in `[lo, hi]` or absent	answer is in `[lo, hi]` and exists
Move up	`lo = mid + 1`	`lo = mid + 1`
Move down	`hi = mid - 1` (mid was already rejected)	`hi = mid` (mid might be the answer)
Ends when	interval is empty (`lo > hi`)	one candidate is left (`lo == hi`)

Mixing the two (for example `while (lo < hi)` with `hi = mid - 1`) is the classic source of off-by-one bugs and infinite loops.

SOLVED EXAMPLE 5.2 M

Search in Rotated Sorted Array

Insight: after splitting at `mid`, **at least one half is sorted**. Decide which half is sorted, check whether the target lies inside it, and discard the other half.

```
int searchRotated(vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                        // left half sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                      // right half sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Trace on `a = [4,5,6,7,0,1,2]`, target `0`:

lo	hi	mid	a[mid]	which half is sorted?	decision
0	6	3	7	left `[4..7]` (`a[lo]=4 <= 7`)	0 is not in `[4,7)`, so `lo = 4`
4	6	5	1	left `[0,1]` (`a[lo]=0 <= 1`)	0 is in `[0,1)`, so `hi = 4`
4	4	4	0	-	`a[mid] == target`, return 4

With duplicates (Search in Rotated Sorted Array II): when `a[lo] == a[mid] == a[hi]` you cannot tell which half is sorted (think `[1,1,1,0,1]`); shrink both ends (`lo++, hi--`) and go on. Worst case $\mathcal{O}(n)$. Complete, tested code is in Appendix J.1.

SOLVED EXAMPLE 5.3 M

Find Minimum in Rotated Sorted Array

Compare `a[mid]` with `a[hi]`: if `a[mid] > a[hi]` the minimum is to the right; otherwise it is at `mid` or to the left.

```
int findMin(vector<int>& a) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[hi]) lo = mid + 1;
        else hi = mid;
    }
    return a[lo];
}
```

Trace on `[4,5,6,7,0,1,2]`: `mid = 3` ($7 > a[hi] = 2$) so `lo = 4`; `mid = 5` ($1 \leq 2$) so `hi = 5`; `mid = 4` ($0 \leq 1$) so `hi = 4`; `lo == hi`, answer `a[4] = 0`. Why compare with `a[hi]` and not `a[lo]`? The right end tells you unambiguously which side the "drop" (the minimum) is on: if `a[mid] > a[hi]`, the array must wrap somewhere between `mid` and `hi`.

Find Minimum II (with duplicates): when `a[mid] == a[hi]` you cannot decide a side, but `a[hi]` is safe to drop (another copy of that value sits at `mid`), so the three cases are `a[mid] > a[hi]` → `lo = mid + 1`; `a[mid] < a[hi]` → `hi = mid`; equal → `hi--`. Worst case $\mathcal{O}(n)$. Complete, tested code is in Appendix J.2.

5.2 Pattern: Binary Search on the Answer

PATTERN CARD

SPOT IT WHEN	'Minimize the maximum', 'smallest X such that...', a feasibility check that is monotonic in X.
KNOW FIRST	A check(x) function that you can write in $\mathcal{O}(n)$.
BREAKS WHEN	Feasibility is NOT monotonic (a bigger x can fail again after succeeding).
COMMON VARIATIONS	Min-feasible (shrink hi), max-feasible (shrink lo), real-valued search with iterations.

Recipe:

1. Identify the quantity to optimize (speed, capacity, divisor...).
2. Define `feasible(x)`: can we succeed with value `x`? It must be **monotonic**: if `x` works, every larger (or smaller) value works too.
3. Binary search the smallest/largest feasible `x`.

PICTURE IT



SOLVED EXAMPLE 5.4 M

Koko Eating Bananas

Problem: `piles[i]` bananas, `h` hours. Each hour Koko eats up to `k` bananas from one pile. Find the minimum `k` that finishes in `h` hours.

Feasibility: with speed `k`, hours needed = `sum(ceil(pile / k))`. Larger `k` means fewer hours, so it is monotonic.

```
int minEatingSpeed(vector<int>& piles, int h) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    auto hoursNeeded = [&](int k) {
        long long hrs = 0;
        for (int p : piles) hrs += (p + k - 1) / k;      // ceil division
        return hrs;
    };
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (hoursNeeded(mid) <= h) hi = mid;             // feasible: try smaller
        else lo = mid + 1;
    }
    return lo;
}
```

Trace on `piles = [3,6,7,11]`, `h = 8`: the search range is `lo = 1`, `hi = 11`.

lo	hi	mid = k	hours needed = sum of ceil(pile/k)	feasible (≤ 8)?	next
1	11	6	1+1+2+2 = 6	yes	`hi = 6`
1	6	3	1+2+3+4 = 10	no	`lo = 4`
4	6	5	1+2+2+3 = 8	yes	`hi = 5`
4	5	4	1+2+2+3 = 8	yes	`hi = 4`

`lo == hi == 4`: the slowest speed that still finishes in 8 hours is 4.

SOLVED EXAMPLE 5.5 M

Capacity to Ship Packages Within D Days

Packages must ship **in order**. Feasibility of capacity `cap`: greedily load each day and count days.

```
int shipWithinDays(vector<int>& w, int days) {
    int lo = *max_element(w.begin(), w.end());           // must fit the heaviest
    int hi = accumulate(w.begin(), w.end(), 0);          // everything in one day
    auto daysNeeded = [&](int cap) {
        int d = 1, load = 0;
        for (int x : w) {
            if (load + x > cap) { d++; load = 0; }
            load += x;
        }
        return d;
    };
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (daysNeeded(mid) <= days) hi = mid;
```

```
        else lo = mid + 1;
    }
    return lo;
}
```

This **exact skeleton** also solves **Painter's Partition Problem** and **Split Array Largest Sum**: replace "days" with "painters/subarrays" and "capacity" with "max sum per part".

SOLVED EXAMPLE 5.6 H

Median of Two Sorted Arrays

Goal: $\mathcal{O}(\log(\min(n, m)))$. **Idea:** binary search how many elements `i` to take from the smaller array `A` into the **left half** of the merged order; then `j = (n+m+1)/2 - i` come from `B`. The split is correct when `A[i-1] <= B[j]` and `B[j-1] <= A[i]`.

```
double findMedianSortedArrays(vector<int>& A, vector<int>& B) {      // A and B sorted, not
↪  both empty
    if (A.size() > B.size()) return findMedianSortedArrays(B, A);
    int n = A.size(), m = B.size();
    int lo = 0, hi = n;
    while (lo <= hi) {
        int i = (lo + hi) / 2;
        int j = (n + m + 1) / 2 - i;
        int aL = (i == 0) ? INT_MIN : A[i - 1];
        int aR = (i == n) ? INT_MAX : A[i];
        int bL = (j == 0) ? INT_MIN : B[j - 1];
        int bR = (j == m) ? INT_MAX : B[j];
        if (aL <= bR && bL <= aR) {                          // correct partition
            if ((n + m) % 2) return max(aL, bL);
            return ((double)max(aL, bL) + min(aR, bR)) / 2.0;     // cast first: int + int could
            ↪  overflow
        }
        if (aL > bR) hi = i - 1;                             // took too many from A
        else lo = i + 1;
    }
    return 0.0;
}
```

Why `j = (n+m+1)/2 - i`? The "left half" of the merged order should hold `(n+m+1)/2` elements (for an odd total, one more than the right half, so its largest element is the median). If we take `i` of them from `A`, the rest, `j`, come from `B`. The four numbers `aL, aR, bL, bR` are the elements just left and just right of the two cut points; using `INT_MIN`/`INT_MAX` for "no element" lets the same comparison work at the ends. If `aL > bR` we took too many from `A`, so cut earlier in `A`; if `bL > aR` we took too few, so cut later.

Trace on `A = [1,3]`, `B = [2]`: `A` is longer, so the function swaps them: `A = [2]` (n = 1), `B = [1,3]` (m = 2). Try `i = 0`, `j = (3+1)/2 - 0 = 2`: `aL = -inf`, `aR = 2`, `bL = B[1] = 3`, `bR = +inf`. We need `bL <= aR` but `3 > 2`, so we took too few from `A`: `lo = 1`. Try `i = 1`, `j = 1`: `aL = 2`, `aR = +inf`, `bL = 1`, `bR = 3`. Both conditions hold. The total (3) is odd, so the median is `max(aL, bL) = 2`, which is right for `[1,2,3]`.

Practice Set 5.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Binary Search on the Answer* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key**

step): the closest solved example above.

Problem	Level 2 hint (approach)
Smallest Divisor Given a Threshold M	Feasible(d): `sum(ceil(a[i]/d)) <= threshold`; smallest feasible `d`.
Painter's Partition Problem M	Shipping skeleton: feasible(maxLength) by greedily assigning boards.
Split Array Largest Sum H	Same skeleton with `k` subarrays: minimize the largest sum.

5.3 Pattern: Classic Sorting & Merge Sort Counting

PATTERN CARD

SPOT IT WHEN	Sort needed, or 'count pairs i<j with a[i] > a[j]' style relations.
KNOW FIRST	Recursion; the merge step of two sorted arrays.
BREAKS WHEN	Counting needs a relation that is NOT preserved by sorting the halves (e.g. depends on original positions of equal items).
COMMON VARIATIONS	Merge sort, inversion count, reverse pairs, counting sort when value range is small.

SOLVED EXAMPLE 5.7 E

Counting Sort

When values are small integers (0..K), count then rebuild: $\mathcal{O}(n + K)$.

```
void countingSort(vector<int>& a, int K) {
    vector<int> cnt(K + 1, 0);
    for (int x : a) cnt[x]++;
    int idx = 0;
    for (int v = 0; v <= K; v++)
        while (cnt[v]--) a[idx++] = v;
}
```

SOLVED EXAMPLE 5.8 M

Merge Sort, Sort an Array, and Count Inversions

Merge sort: split in half, sort halves, merge. Always $\mathcal{O}(n \log n)$.

Inversion count: while merging, when we take an element from the *right* half before elements remaining in the left half, each remaining left element forms an inversion with it: add `mid - i`.

Example. `[2,4,1,3,5]` has 3 inversions: `(2,1)`, `(4,1)`, `(4,3)`. Both halves `[2,4]` and `[1,3,5]` are already sorted (0 inversions inside). While merging, `1` (right) is taken while `2, 4` (two elements) are still waiting on the left: `+2`. Then `2` is taken from the left; then `3` (right) is taken while only `4` waits: `+1`. Total `2 + 1 = 3`. Every inversion is counted exactly once, at the moment its larger (left) element is overtaken by its smaller (right) partner.

```
long long mergeSortCount(vector<int>& a, vector<int>& tmp, int l, int r) {   // [l, r)
    if (r - l <= 1) return 0;
    int m = (l + r) / 2;
```

```
    long long inv = mergeSortCount(a, tmp, l, m) + mergeSortCount(a, tmp, m, r);
    int i = l, j = m, k = l;
    while (i < m && j < r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { tmp[k++] = a[j++]; inv += m - i; }     // a[i..m-1] all > a[j]
    }
    while (i < m) tmp[k++] = a[i++];
    while (j < r) tmp[k++] = a[j++];
    copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
    return inv;
}

// Usage: sorts `a` AND returns the inversion count.
long long sortAndCount(vector<int>& a) {
    vector<int> tmp(a.size());
    return mergeSortCount(a, tmp, 0, a.size());
}
```

Complexity: $\mathcal{O}(n \log n)$ time, $\mathcal{O}(n)$ space.

CHAPTER COVERAGE

all 15 Sorting & Searching problems appear above.

CHAPTER 5 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 5.1 Binary Search, 5.7 Counting Sort.
2. **Variations:** Ex 5.2 Rotated Search, 5.3 Rotated Minimum.
3. **Combined:** Ex 5.4 Koko, 5.5 Shipping.
4. **Independent recognition:** Problems 5A and 5B (can you see the hidden binary search?).
5. **Challenge:** Ex 5.6 Median of Two Sorted Arrays.

Compare and Contrast

	Binary search on index	Binary search on answer
Searches	positions in sorted data	values of an unknown quantity
Needs	sorted/monotone data	monotone `check(x)`
Cost	$\mathcal{O}(\log n)$	$\mathcal{O}$(log range x cost of check)

Why It Works (Correctness Corner)

State the invariant, then everything follows. For 'first index where `check` is true': *the answer always lies in `[lo, hi]`*. Loop `while (lo < hi)`, `mid = lo + (hi - lo) / 2`. If `check(mid)` then the answer is `<= mid`, so `hi = mid`; else it is `> mid`, so `lo = mid + 1`. The interval strictly shrinks each time (because `mid < hi`), so the loop ends with `lo == hi`, the answer. Using `mid = (lo + hi) / 2` can overflow `int`.

Debug Lab: Overflow in square-root search

```
int floorSqrt(int x) {
    int lo = 0, hi = x, ans = 0;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (mid * mid <= x) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}
```

Your task: It fails for `x = 2147483647`. Why, and what is the fix?

ANSWER

`lo + hi` can exceed `INT_MAX`, and `mid * mid` certainly overflows `int` when `mid` is about 10^9. Use `long long` for `lo`, `hi`, `mid`, and `mid = lo + (hi - lo) / 2`.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 5A

Integer Square Root

E

Statement. Given `x >= 0`, return `floor(sqrt(x))` without using `sqrt`.

Example. `8` → `2`, `9` → `3`

Constraints. $0 \leq x \leq 2^{31} - 1$.

Expected complexity. $\mathcal{O}(\log x)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 5.1 Binary Search Fundamentals

HINT 1 which idea is relevant?

Think of the candidates 0..x. For which candidates `m` is `m*m <= x`?

HINT 2 outline of the approach

If `m` works, so does every smaller number: the answer is the *largest* working value.

HINT 3 the key step

Binary search; when `mid*mid <= x` remember `mid` and search right. Use `long long`.

SOLUTION and editorial (try first, then compare)

```
int floorSqrt(int x) {
    long long lo = 0, hi = x, ans = 0;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (mid * mid <= x) { ans = mid; lo = mid + 1; }   // mid works, try bigger
        else hi = mid - 1;
    }
    return (int)ans;
}
```

This is the 'last true' variant: `ok(m) = m*m <= x` is true up to some boundary and false after. Store the best working `mid` and keep searching right.

PROBLEM 5B

Smallest Divisor Given a Threshold

M

Statement. Given positive integers `a` and a `threshold`, find the smallest positive integer `d` such that `sum(ceil(a[i] / d)) <= threshold`.

Example. `a = [1,2,5,9], threshold = 6` → `5`

Constraints. $1 \leq n \leq 5 \times 10^4$; $1 \leq a[i] \leq 10^6$; $n \leq \text{threshold} \leq 10^6$.

Expected complexity. $\mathcal{O}(n \log(\max a))$ time.

Prerequisites. 5.2 Binary Search on the Answer

HINT 1 which idea is relevant?

You are asked for the 'smallest value such that...'. What kind of pattern is that?

HINT 2 outline of the approach

If divisor `d` works, does `d + 1` work? (Yes: sums only get smaller.)

HINT 3 the key step

Write `total(d)` in $\mathcal{O}(n)$; binary search `d` in `[1, max(a)]`, shrinking `hi` when it is feasible.

SOLUTION and editorial (try first, then compare)

```cpp
int smallestDivisor(const vector<int>& a, int threshold) {
    auto total = [&](int d) {
        long long s = 0;
        for (int x : a) s += (x + d - 1) / d;              // ceil(x / d)
        return s;
    };
    int lo = 1, hi = *max_element(a.begin(), a.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
       if (total(mid) <= threshold) hi = mid;             // mid is feasible: answer <= mid
        else lo = mid + 1;
    }
    return lo;
}
```

`total(d)` is non-increasing in `d`, so feasibility is monotone. When `d = max(a)` every term is 1, so the sum is `n <= threshold`: the upper bound is always feasible, which is why `hi = max(a)` is a safe starting point.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T5.1 A train must reach a city within H hours; find the minimum constant speed.

PATTERN

Binary search on the answer with `check(speed)`.

T5.2 A sorted list of day numbers was rotated at an unknown pivot; find the earliest day.

PATTERN

Binary search on a rotated array (compare mid with right end).

Mixed Practice

Combine 3.3 and 5.2: 'Find the smallest window size w such that some window of size w has sum $\geq$ S' (positive values). Is it binary-searchable? Is there a faster way?

Revision Sheet

Key takeaways

- Write the invariant before the code.
- `lo + (hi - lo) / 2`.
- Monotone yes/no $\Rightarrow$ binary search on the answer.
- Merge sort counts cross pairs while merging.

Pattern-recognition clues

- 'Sorted', 'rotated', 'first/last position' → index search.
- 'Minimize the maximum' / 'smallest x such that' → answer search.
- 'Count pairs i<j with a[i]>a[j]' → merge sort.

Common mistakes

- `lo <= hi` vs `lo < hi` mixed up with `hi = mid` vs `hi = mid - 1`.
- `mid` overflow.
- `check` not monotone.

Complexity summary

Task	Time
Binary search	$\mathcal{O}(\log\ n)$
Answer search	$\mathcal{O}(\log\ R\,x$ check$)$
Merge sort / inversions	$\mathcal{O}(n\ \log\ n)$
Counting sort	$\mathcal{O}(n +$ range$)$

Recall questions (answer aloud, then check the chapter)

1. What invariant does `while (lo < hi)` maintain?
2. Why `mid = lo + (hi - lo)/2`?
3. What property must `check` have?
4. How does merge sort count inversions?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	From a blank file, write a generic `firstTrue(lo, hi, check)` and use it to solve Problems 5A and 5B. Test boundaries: `lo == hi`, answer at either end.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 6

Linked List

6

START HERE

A music app must delete the 2nd-from-last song of a playlist stored as a chain of nodes, without knowing its length and without a second pass. Linked lists are less about memory and more about *pointer discipline*: draw it, then code it.

ANALOGY

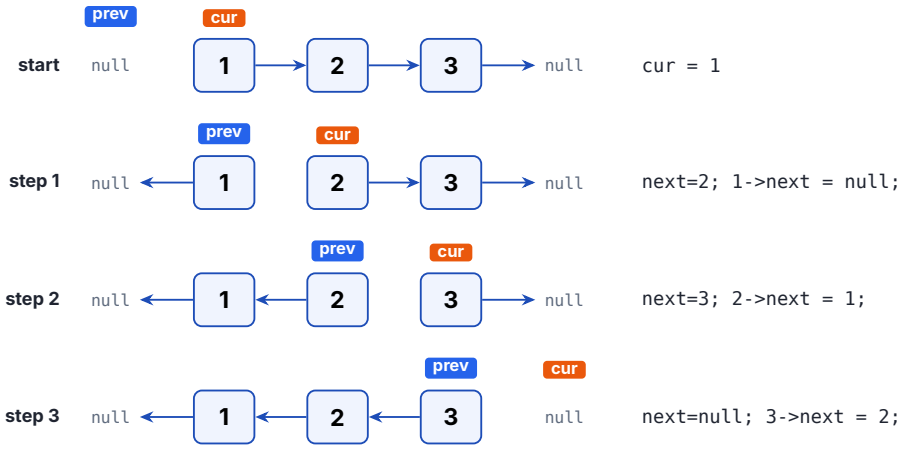
A scavenger hunt: each clue only tells you where the next one is. To remove a clue from the chain you must re-point the clue *before* it, which is why you always need a handle on the previous node.

AFTER THIS CHAPTER YOU CAN

- ✓ Reverse a list or part of a list without losing nodes.
- ✓ Use fast/slow pointers for middle, cycles and k-th from end.
- ✓ Use a dummy node to remove head special cases.
- ✓ Combine a hash map with a doubly linked list (LRU).

PICTURE IT

Reversing 1 → 2 → 3 pointer by pointer



head = prev, giving 3 → 2 → 1 → null. RULE: save `next` **before** you overwrite `cur->next`.

6.0 Concept

A linked list is a chain of nodes, each holding data and a pointer to the next. Insert/delete at a known node is $\mathcal{O}(1)$; random access is $\mathcal{O}(n)$.

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};
```

Three golden tools:

1. **Dummy node:** `ListNode dummy(0); dummy.next = head;` removes head special cases.
2. **Two pointers (fast/slow):** middle, cycle, k-th from end.
3. **Draw the pointers.** Before writing any pointer code, sketch the before and after states.

Pattern	Recognition clue
Fast & slow pointers	middle, cycle, palindrome
In-place reversal	reverse whole list, in groups, partial
Merge	merge sorted lists, merge sort
Hash map + list	LRU cache, copy with random pointers

6.1 Pattern: Traversal & Reversal

PATTERN CARD

SPOT IT WHEN	'Reverse', 'reorder', 'rotate' a list; pointer rewiring.
KNOW FIRST	Pointer assignment order; drawing before/after.
BREAKS WHEN	Losing the rest of the list: you overwrote `next` before saving it.
COMMON VARIATIONS	Iterative reverse, reverse in groups, reverse a segment, recursion.

SOLVED EXAMPLE 6.1 — E

Reverse a Linked List

Approach: walk with three pointers `prev, cur, next`; flip each `cur->next` to point backward.

Sketch

```
Before:  1 -> 2 -> 3 -> null
Step 1:  null <- 1    2 -> 3
Step 2:  null <- 1 <- 2    3
Step 3:  null <- 1 <- 2 <- 3
```

```
ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *cur = head;
    while (cur) {
        ListNode* nxt = cur->next;   // save
        cur->next = prev;            // flip
```

```
        prev = cur;                 // advance
        cur = nxt;
    }
    return prev;
}
```

SOLVED EXAMPLE 6.2 M

Reverse a Linked List in Groups of Given Size

Reverse each block of k; leave a final shorter block alone. Recursion makes it elegant: reverse the first k, then attach the result of solving the rest.

```
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* node = head;
    for (int i = 0; i < k; i++) {              // is there a full group?
        if (!node) return head;
        node = node->next;
    }
    ListNode* prev = reverseKGroup(node, k);    // rest is already solved
    ListNode* cur = head;
    for (int i = 0; i < k; i++) {              // reverse k nodes onto `prev`
        ListNode* nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}
```

Trace on `1 -> 2 -> 3 -> 4 -> 5`, `k = 2`. The recursion first walks k nodes to check that a full group exists, then solves the REST before touching the current group:

1. `reverseKGroup(1)`: nodes `1,2` form a full group; the rest starts at `3`, so we first call `reverseKGroup(3)`.
2. `reverseKGroup(3)`: nodes `3,4` are a full group; the rest starts at `5`, so we call `reverseKGroup(5)`.
3. `reverseKGroup(5)`: only one node is left, the walk runs out of nodes, so it returns `5` unchanged.
4. Back in step 2: `prev = 5`. Reverse `3,4` onto it: `4 -> 3 -> 5`; return `4`.
5. Back in step 1: `prev = 4`. Reverse `1,2` onto it: `2 -> 1 -> 4 -> 3 -> 5`; return `2`.

The variable `prev` starts as "the already-solved rest" instead of `nullptr`; that is the only difference from the plain reversal in Example 6.1.

Practice Set 6.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Traversal & Reversal* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Remove Duplicates from a Sorted Linked List E	One pointer: while `cur->next` has equal value, skip it.
Split a Linked List into Two Parts E	Find the middle with slow/fast, cut the link.
Rotate a Doubly Linked List in Groups of Given Size M	Same as k-group reversal but fix both `prev` and `next` pointers.

Problem	Level 2 hint (approach)
Delete Nodes with a Greater Value on the Right Side M	Reverse the list, keep a running max, delete smaller nodes, reverse back.

6.2 Pattern: Fast & Slow Pointers

PATTERN CARD

SPOT IT WHEN	Middle, cycle, k-th from end, palindrome check on a list.
KNOW FIRST	Pointers advance at different speeds.
BREAKS WHEN	Even/odd length off-by-one decides which middle you get; null checks on `fast->next`.
COMMON VARIATIONS	Floyd cycle (find entry), middle, nth from end, split into halves.

SOLVED EXAMPLE 6.3 E

Middle, Cycle Detection (Floyd's)

Middle: `slow` moves 1 step, `fast` moves 2. When `fast` ends, `slow` is at the middle (for an even number of nodes, `middleNode` below returns the *second* of the two middle nodes).

Cycle: if there is a loop, `fast` eventually laps `slow` and they meet.

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}

bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

// Start of the cycle: after they meet, restart one pointer at head;
// moving both one step at a time makes them meet at the cycle entrance.
ListNode* cycleStart(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}
```

Why restarting at `head` finds the cycle entrance. Let `a` be the number of nodes from `head` to the cycle entrance, `b` the distance from the entrance to the meeting point (going around the cycle), and `c` the cycle length. When they meet, `slow` has walked `a + b` and `fast` has walked twice that, `2(a + b)`. Since `fast` has simply gone around the cycle some whole number `k` of extra times, `2(a + b) = (a + b) + k*c`, so `a + b = k*c`, i.e. `a = k*c - b`. Walking `a` steps from the meeting point therefore goes the rest of the way around the cycle (`c - b`, plus whole laps) and lands exactly on the entrance, while a pointer restarted from `head` also needs `a` steps to reach it. So the two pointers arrive at the entrance together.

SOLVED EXAMPLE 6.4 M

Check if a Linked List Is a Palindrome

Approach: find the middle, reverse the second half, compare both halves. Here the loop condition is `fast->next && fast->next->next`, which stops `slow` on the *first* of the two middle nodes for an even length; `slow->next` is then the start of the second half in both the odd and the even case.

```
bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;
    ListNode *slow = head, *fast = head;
    while (fast->next && fast->next->next) { slow = slow->next; fast = fast->next->next; }
    ListNode *second = reverseList(slow->next);          // from Example 6.1
    ListNode *p1 = head, *p2 = second;
    bool ok = true;
    while (p2) {
        if (p1->val != p2->val) { ok = false; break; }
        p1 = p1->next; p2 = p2->next;
    }
    slow->next = reverseList(second);                    // restore the list (good manners)
    return ok;
}
```

SOLVED EXAMPLE 6.5 M

Intersection Point of Two Linked Lists

Insight: let pointer `a` walk list A then list B; pointer `b` walk list B then list A. Both travel `lenA + lenB` steps in total, so after the switch they are the same distance from the shared tail and they meet at the intersection. If the lists never meet, both pointers reach `nullptr` on the same step and the loop ends with `a == b == nullptr`.

```
ListNode* getIntersection(ListNode* headA, ListNode* headB) {
    ListNode *a = headA, *b = headB;
    while (a != b) {
        a = a ? a->next : headB;
        b = b ? b->next : headA;
    }
    return a;
}
```

6.3 Pattern: Merging & Sorting Lists

PATTERN CARD

SPOT IT WHEN	Merge sorted lists, sort a list, add numbers stored as lists.
KNOW FIRST	Dummy-node technique.
BREAKS WHEN	Using array merge sort habits (random access) on lists; forgetting to attach the leftover tail.
COMMON VARIATIONS	Merge 2, merge k (heap), merge sort on lists, add with carry.

SOLVED EXAMPLE 6.6 — M

Merge Two Sorted Lists, then Merge Sort for Linked Lists

```
ListNode* mergeTwo(ListNode* a, ListNode* b) {
    ListNode dummy(0), *tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else                  { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode *slow = head, *fast = head->next;          // slow ends at end of first half
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = slow->next;
    slow->next = nullptr;                               // cut
    return mergeTwo(sortList(head), sortList(second));
}
```

Complexity: $\mathcal{O}(n \log n)$ time, $\mathcal{O}(\log n)$ recursion stack. Merge sort suits linked lists because merging needs no extra array.

SOLVED EXAMPLE 6.7 — M

Add Two Numbers Represented by Linked Lists

Digits are stored in reverse order (least significant first), so we add left to right with a carry.

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0), *tail = &dummy;
    int carry = 0;
    while (l1 || l2 || carry) {
        int sum = carry;
        if (l1) { sum += l1->val; l1 = l1->next; }
        if (l2) { sum += l2->val; l2 = l2->next; }
        tail->next = new ListNode(sum % 10);
        tail = tail->next;
        carry = sum / 10;
    }
```

```
    return dummy.next;
}
```

Practice Set 6.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Merging & Sorting Lists* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Sort a Linked List of 0s, 1s and 2s E	Count each value and rewrite, or build three sublists and join.
Quicksort for Linked Lists M	Partition around the tail/head pivot into three lists (less, equal, greater), recurse, concatenate.
Sort a 'K'-Sorted Doubly Linked List M	Min-heap of size `k+1` over nodes; pop to build the result list.
Split a Circular Linked List into Two Halves M	Slow/fast to find the middle; fix both tails to point to their heads.

6.4 Pattern: Design with Lists

PATTERN CARD

SPOT IT WHEN	'$\mathcal{O}(1)$ get / put / delete' with ordering or deep copy with extra pointers.
KNOW FIRST	Hash map + doubly linked list combination.
BREAKS WHEN	Hash map stores values instead of node pointers (so you cannot unlink in $\mathcal{O}(1)$).
COMMON VARIATIONS	LRU, LFU, copy random list (interleave or map), flatten multilevel.

SOLVED EXAMPLE 6.8 M

LRU Cache

Requirements: `get` and `put` in $\mathcal{O}(1)$; evict the **least recently used** key when full. (The capacity is at least 1.)

Idea: a **doubly linked list** keeps usage order (front = most recent). A **hash map** maps key to its list node so we can move it in $\mathcal{O}(1)$. C++'s `std::list` plus `splice` does the work.

```
class LRUCache {
    int cap;
    list<pair<int,int>> order;                              // (key, value), front = newest
    unordered_map<int, list<pair<int,int>>::iterator> pos;
public:
    LRUCache(int capacity) : cap(capacity) {}

    int get(int key) {
        auto it = pos.find(key);
        if (it == pos.end()) return -1;
        order.splice(order.begin(), order, it->second);          // move to front
        return it->second->second;
    }
```

```
    void put(int key, int value) {
        auto it = pos.find(key);
        if (it != pos.end()) {
            it->second->second = value;
            order.splice(order.begin(), order, it->second);
            return;
        }
        if ((int)order.size() == cap) {                          // evict LRU (back)
            pos.erase(order.back().first);
            order.pop_back();
        }
        order.emplace_front(key, value);
        pos[key] = order.begin();
    }
};
```

SOLVED EXAMPLE 6.9 M

Copy a Linked List with Random Pointers

Approach (hash map): pass 1: create a copy of every node, storing `old -> new`. Pass 2: wire `next` and `random` through the map. ($\mathcal{O}(1)$-space variant: interleave copies after each original, set random pointers, then split the lists.)

```
struct RNode { int val; RNode *next, *random; RNode(int v) : val(v), next(nullptr),
↪   random(nullptr) {} };

RNode* copyRandomList(RNode* head) {
    unordered_map<RNode*, RNode*> mp;
    for (RNode* p = head; p; p = p->next) mp[p] = new RNode(p->val);
    for (RNode* p = head; p; p = p->next) {
        mp[p]->next   = p->next   ? mp[p->next]   : nullptr;
        mp[p]->random = p->random ? mp[p->random] : nullptr;
    }
    return head ? mp[head] : nullptr;
}
```

Practice Set 6.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Design with Lists* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Flatten a Multilevel Linked List M	Each node has `next` and `bottom`. Recursively flatten the right side, then `mergeTwo` it with the current column using `bottom` pointers.

CHAPTER COVERAGE

all 19 Linked List problems appear above.

CHAPTER 6 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 6.1 Reverse a List.
2. **Variations:** Ex 6.2 Reverse in Groups, Problem 6B Odd-Even.
3. **Combined:** Ex 6.4 Palindrome List (fast/slow + reverse).
4. **Independent recognition:** Problem 6A (where should the pointer *stop*?).
5. **Challenge:** Ex 6.8 LRU Cache, 6.9 Copy with Random Pointers.

Compare and Contrast

	Array / vector	Linked list
Access i-th	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Insert/delete at known node	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Memory locality	excellent	poor (cache misses)
Use in interviews	indexing, sorting	pointer manipulation

In real programs `vector` wins almost always; lists appear in DSA because they train pointer reasoning and in LRU-style designs.

Why It Works (Correctness Corner)

Floyd's cycle detection. If a cycle of length `c` exists, once both pointers are inside it, `fast` gains exactly 1 position per step on `slow`. The gap (mod `c`) therefore decreases by 1 each step and reaches 0 within `c` steps, so they meet. If no cycle exists, `fast` reaches `null` after at most `n/2` steps.

Debug Lab: Null dereference when deleting the head

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode *fast = head, *slow = head;
    for (int i = 0; i < n; i++) fast = fast->next;
    while (fast->next) { fast = fast->next; slow = slow->next; }
    slow->next = slow->next->next;
    return head;
}
```

Your task: Which input crashes, and how does a dummy node fix it?

ANSWER

`[1]` with `n = 1` (or any list where the head must be removed): after the first loop `fast` is `nullptr`, so `fast->next` crashes. Even if it did not crash, the head has no predecessor. With `ListNode dummy; dummy.next = head;` and both pointers starting at `&dummy`, the node before the target always exists.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 6A — M

Remove the N-th Node from the End

Statement. Given the head of a singly linked list, remove the `n`-th node from the end and return the new head. Do it in one pass.

Example. `[1,2,3,4,5], n = 2` → `[1,2,3,5]`

Constraints. $1 \leq \text{length} \leq 10^5$; $1 \leq n \leq \text{length}$.

Expected complexity. $\mathcal{O}(\text{length})$ time, $\mathcal{O}(1)$ space, a single traversal.

Prerequisites. 6.2 Fast & Slow Pointers; dummy node (6.0) (uses the `ListNode` struct from 6.0)

HINT 1 which idea is relevant?

You do not know the length. How can two pointers encode a distance?

HINT 2 outline of the approach

Move `fast` n steps ahead first; then move both until `fast` reaches the end.

HINT 3 the key step

To delete a node you must stop at its *predecessor*. Start both pointers at a dummy node before the head.

SOLUTION and editorial (try first, then compare)

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0); dummy.next = head;
    ListNode *fast = &dummy, *slow = &dummy;
    for (int i = 0; i < n; i++) fast = fast->next;      // open a gap of n nodes
    while (fast->next) { fast = fast->next; slow = slow->next; }
    ListNode* dead = slow->next;                        // slow stops just BEFORE the node
↪   to delete
    slow->next = dead->next;
    delete dead;
    return dummy.next;
}
```

The gap between `fast` and `slow` stays n. When `fast->next` is null, `slow` is exactly one node before the target. The dummy node makes deleting the head the same as deleting any other node.

PROBLEM 6B — M

Odd-Even Linked List

Statement. Group all nodes at odd positions (1st, 3rd, ...) together followed by the nodes at even positions, preserving relative order inside each group. Positions, not values.

Example. `[1,2,3,4,5]` → `[1,3,5,2,4]`

Constraints. $0 \leq \text{length} \leq 10^5$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 6.1 Traversal & Reversal (uses the `ListNode` struct from 6.0)

HINT 1 which idea is relevant?

You need two chains built at the same time. How many pointers do you need to keep?

HINT 2 outline of the approach

Remember the head of the even chain before you start rewiring.

HINT 3 the key step

Loop while `even && even->next`: link odd to even→next, advance, then link even to odd→next, advance. Finally `odd->next = evenHead`.

SOLUTION and editorial (try first, then compare)

```cpp
ListNode* oddEvenList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode *odd = head, *even = head->next, *evenHead = even;
    while (even && even->next) {
        odd->next = even->next;  odd = odd->next;       // link odd positions together
        even->next = odd->next;  even = even->next;     // link even positions together
    }
    odd->next = evenHead;                               // odd chain, then even chain
    return head;
}
```

Trace on `1 -> 2 -> 3 -> 4 -> 5`. Start: `odd = 1`, `even = 2`, `evenHead = 2`.

- Round 1: `odd->next = 3` (so `1 -> 3`), `odd = 3`; `even->next = odd->next = 4` (so `2 -> 4`), `even = 4`.
- Round 2: `even->next` is 5, so continue: `odd->next = 5` (`3 -> 5`), `odd = 5`; `even->next = odd->next = null` (`4 -> null`), `even = null`. The loop stops.
- Finally `odd->next = evenHead`: `5 -> 2`. The list reads `1 -> 3 -> 5 -> 2 -> 4`. The loop condition checks `even->next` because that is the next odd node. Both chains end correctly: the last even node's `next` is already null once we reorder, and the odd tail points to `evenHead`.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T6.1 Two playlists are linked lists that share the same tail; find the first shared song.

PATTERN

Intersection of lists: two pointers that switch heads after reaching the end.

T6.2 A chain of URL redirects may loop forever; detect it using $\mathcal{O}(1)$ memory.

PATTERN

Floyd cycle detection.

Mixed Practice

Combine 6.2 and 6.1: check if a list is a palindrome using $\mathcal{O}(1)$ extra space. Which three steps (find, reverse, compare) and which pointer pattern does each use?

Revision Sheet

Key takeaways

- Draw before/after pointer states.
- Use a dummy node when the head may change.
- Save `next` before overwriting it.
- Fast/slow converts 'distance' into 'position'.

Pattern-recognition clues

- 'Middle', 'cycle', 'k-th from end' → fast & slow.
- 'Reverse' → three pointers (prev, cur, next).
- '$\mathcal{O}(1)$ get/put with eviction' → map + doubly linked list.

Common mistakes

- Overwriting `cur->next` before saving it.
- Forgetting `fast && fast->next` in the loop condition.
- Not setting the new tail's `next` to null (creates a cycle).

Complexity summary

Task	Time	Space
Reverse	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Middle / cycle	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Merge two sorted	$\mathcal{O}(m+n)$	$\mathcal{O}(1)$
Merge sort list	$\mathcal{O}(n \log n)$	$\mathcal{O}(\log n)$ stack

Recall questions (answer aloud, then check the chapter)

1. Why a dummy node?
2. Why does Floyd's algorithm terminate with a meeting point?
3. What do you save before changing `cur->next` in a reversal?
4. Why is LRU built from a map and a doubly linked list?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Draw, then code from scratch: reverse a list, find its middle, and detect a cycle. Test with empty, single-node and two-node lists.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 7

Stacks & Queues

START HERE

'How many days until a warmer day?' for 10^5 days. Checking forward from each day is $\mathcal{O}(n^2)$. A stack answers it in one pass because it remembers only the days still waiting for an answer.

ANALOGY

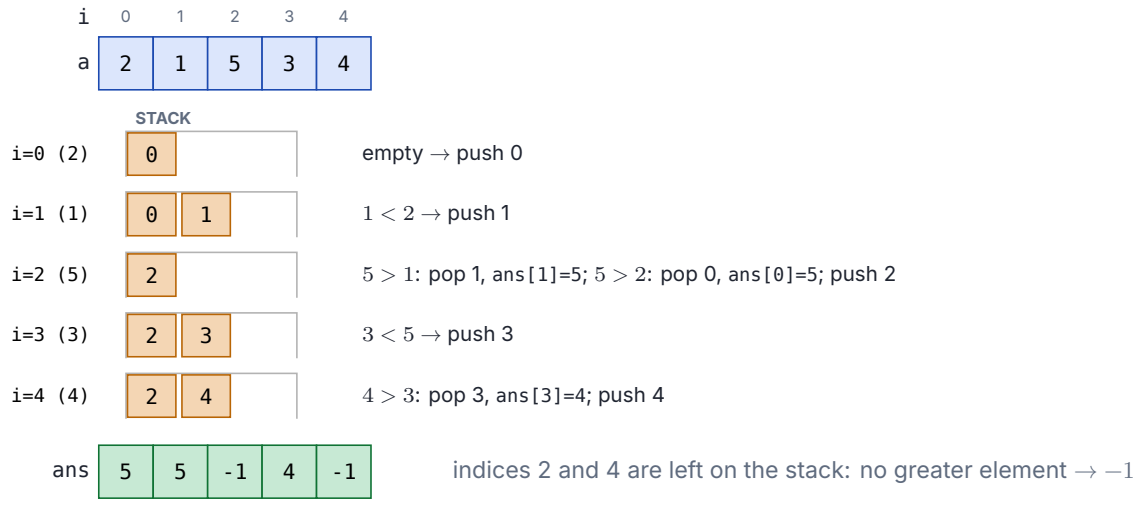
A stack is a pile of plates (last in, first out); a queue is a line at the bank (first in, first out). A **monotonic** stack is a pile where a heavy plate on top throws out every lighter plate below it.

AFTER THIS CHAPTER YOU CAN

- ✓ Recognise 'next greater/smaller' problems and solve them in $\mathcal{O}(n)$.
- ✓ Validate and repair bracket sequences.
- ✓ Implement a stack/queue with extra abilities (min, via two stacks).
- ✓ Use a deque for sliding-window maximum.

PICTURE IT

Monotonic stack: next greater element of [2, 1, 5, 3, 4]



7.0 Concept

- **Stack (LIFO):** last in, first out. Think of a pile of plates. `push`, `pop`, `top`: all $\mathcal{O}(1)$.
- **Queue (FIFO):** first in, first out. Think of a line at a ticket counter. `push`, `pop`, `front`: all $\mathcal{O}(1)$.
- **Deque:** insert/remove at both ends in $\mathcal{O}(1)$.

```
stack<int> st;  st.push(1); st.top(); st.pop(); st.empty();
queue<int> q;   q.push(1);  q.front(); q.pop(); q.size();
deque<int> dq;  dq.push_back(1); dq.push_front(2); dq.pop_back(); dq.pop_front();
```

Pattern	Recognition clue
Monotonic stack	"next greater/smaller", "span", "largest rectangle"
Parentheses matching	brackets, nested structure, "valid", "balanced"
Stack/queue design	"implement X using Y", $\mathcal{O}(1)$ min/max
Monotonic deque	sliding window maximum/minimum
Queue simulation	process in arrival order, level by level (BFS in Chapters 8-9)

7.1 Pattern: Monotonic Stack

PATTERN CARD

SPOT IT WHEN	'Next greater/smaller element', 'span', 'largest rectangle', 'remove digits to minimize'.
KNOW FIRST	Stack basics; what 'monotonic' keeps.
BREAKS WHEN	You need elements in the middle, not just the nearest greater/smaller.
COMMON VARIATIONS	Next greater (right/left), circular array (iterate 2n), histogram, greedy digit removal.

Idea: keep the stack in sorted order (increasing or decreasing). When a new element violates the order, pop until it fits. The popped elements have just found their "next greater/smaller." Each element is pushed and popped **once**, so it is $\mathcal{O}(n)$.

SOLVED EXAMPLE 7.1 E

Next Greater Element (Right Side)

Problem: for every element, find the first larger element to its right (or -1).

Approach: scan from the **right**; pop all stack elements <= current (they can never be anyone's answer to the left, since current is bigger and closer). The top of what remains is the answer.

```
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;                                  // holds values
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && st.top() <= a[i]) st.pop();
        if (!st.empty()) res[i] = st.top();
        st.push(a[i]);
    }
    return res;
```

```
}
```

Trace on `[2,1,5,3,4]` (scanning from the right; the stack is written bottom to top):

i	a[i]	popped (value <= a[i])	stack before push	answer	stack after push
4	4	-	empty	-1	`[4]`
3	3	-	`[4]`	4	`[4,3]`
2	5	3, 4	empty	-1	`[5]`
1	1	-	`[5]`	5	`[5,1]`
0	2	1	`[5]`	5	`[5,2]`

The result is `[5,5,-1,4,-1]`, the same as the picture at the top of the chapter. That picture scans left to right with a stack of *indices still waiting for an answer*; this example scans right to left with a stack of *values that could still be someone's answer*. Both are $\mathcal{O}(n)$; Example 7.2 uses the left-to-right shape.

For **Next Greater Element II (Circular)**, loop `i` from `2n-1` down to `0` using `a[i % n]`, but write an answer only when `i < n` (the second pass over the array exists only to pre-fill the stack). Complete tested code is in Appendix J.3.

SOLVED EXAMPLE 7.2 M

Daily Temperatures (and Stock Span)

Problem: for each day, how many days until a warmer temperature?

Approach (left to right, store indices): the stack holds days still waiting for a warmer day. When today is warmer than the top, that day's wait is `today - topIndex`.

```
vector<int> dailyTemperatures(vector<int>& t) {
    int n = t.size();
    vector<int> res(n, 0);
    stack<int> st;                                  // indices of unresolved days
    for (int i = 0; i < n; i++) {
        while (!st.empty() && t[st.top()] < t[i]) {
            res[st.top()] = i - st.top();
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

Stock Span is the mirror: pop while `price[top] <= price[i]`; span = `i - (index left on top, or -1)`. Complete tested code is in Appendix J.4. **Nearest Smaller to Left and Right** uses the same loop with `>` comparisons, once from each side.

SOLVED EXAMPLE 7.3 H

Largest Rectangular Area in a Histogram

Brute force: for each bar, expand left and right while bars are `>=` it: $\mathcal{O}(n^2)$.

Insight: for bar `i` as the *shortest* bar in the rectangle, we need the nearest smaller bar on the left and on the right. A monotonic increasing stack finds both: when bar `i` is smaller than the top, the top's right boundary is `i`, and its left boundary is the new top below it.

```cpp
int largestRectangle(vector<int> h) {
    h.push_back(0);                                  // sentinel flushes the stack
    stack<int> st;                                   // indices, heights increasing
    int best = 0;
    for (int i = 0; i < (int)h.size(); i++) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int height = h[st.top()]; st.pop();
            int leftBound = st.empty() ? -1 : st.top();
            best = max(best, height * (i - leftBound - 1));
        }
        st.push(i);
    }
    return best;
}
```

Trace on `h = [2,1,5,6,2,3]` (a `0` is appended, so we scan `[2,1,5,6,2,3,0]`; the stack holds indices):

i	h[i]	what is popped (height, left boundary, width)	area	best
0	2	nothing; push 0		0
1	1	pop bar 0: height 2, left boundary -1, width `1 - (-1) - 1 = 1`	2	2
2	5	nothing; stack is `[1,2]`		2
3	6	nothing; stack is `[1,2,3]`		2
4	2	pop bar 3: height 6, left boundary 2, width `4-2-1 = 1`; then pop bar 2: height 5, left boundary 1, width `4-1-1 = 2`	6, **10**	10
5	3	nothing; stack is `[1,4,5]`		10
6	0	pop bar 5: 3 x 1 = 3; pop bar 4: height 2, left boundary 1, width `6-1-1 = 4` gives 8; pop bar 1: height 1, left boundary -1, width `6-(-1)-1 = 6` gives 6	3, 8, 6	**10**

The answer is 10: the bars of height 5 and 6 together form a `5 x 2` rectangle. A popped bar's right boundary is the bar that popped it (`i`), and its left boundary is the index now on top of the stack, because everything between them is at least as tall.

Maximum Size Rectangle of 1's in a Binary Matrix: process row by row, treat each row as a histogram (height of consecutive 1s ending at that row), and call `largestRectangle` per row.

```cpp
int maximalRectangle(vector<vector<int>>& m) {
    if (m.empty()) return 0;
    vector<int> heights(m[0].size(), 0);
    int best = 0;
    for (auto& row : m) {
        for (int j = 0; j < (int)row.size(); j++)
            heights[j] = row[j] ? heights[j] + 1 : 0;
        best = max(best, largestRectangle(heights));
    }
    return best;
}
```

SOLVED EXAMPLE 7.4 M

Remove K Digits

Problem: remove `k` digits from a number string to make it as small as possible.

Greedy insight: a digit larger than the one after it should be removed first. A monotonic *increasing* stack does exactly that.

Trace on `num = "1432219"`, `k = 3`: push `1`; push `4`; `3 < 4` so pop `4` (k = 2) and push `3`: `13`; `2 < 3` so pop `3` (k = 1) and push `2`: `12`; push `2`: `122`; `1 < 2` so pop `2` (k = 0) and push `1`: `121`; push `9`: `1219`. Answer `"1219"`. When `k` runs out the remaining digits are simply appended; if the digits were already increasing (like `"12345"`), nothing gets popped and the extra removals come off the tail.

```
string removeKdigits(string num, int k) {
    string st;                                      // string used as a stack
    for (char c : num) {
        while (k > 0 && !st.empty() && st.back() > c) { st.pop_back(); k--; }
        st.push_back(c);
    }
    while (k-- > 0) st.pop_back();                 // still need removals: drop the largest tail
    int i = 0;
    while (i < (int)st.size() - 1 && st[i] == '0') i++;   // strip leading zeros
    st = st.substr(i);
    return st.empty() ? "0" : st;
}
```

Practice Set 7.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Monotonic Stack* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Next Greater Element II (Circular Array) M	Iterate over the array twice (`2n`) with `i % n`.
Stock Span Problem M	Stack of indices; pop while price `<=` current.
Nearest Smaller to Left and Right M	Two monotonic passes (or one pass using the popped element's info).
Asteroid Collision M	Stack of survivors; a right-moving top collides with a left-moving new asteroid; compare sizes.
Remove Duplicate Letters M	Stack + `last[c]` + `inStack[c]`: pop larger letters that appear again later.
Min Cost Tree From Leaf Values M	Monotonic decreasing stack: when popping a middle value, cost `+= mid * min(neighbors)`.

7.2 Pattern: Parentheses & Bracket Matching

PATTERN CARD

SPOT IT WHEN	Brackets, nesting, 'valid', 'minimum to fix'.
KNOW FIRST	LIFO matching.
BREAKS WHEN	Multiple bracket types with a counter only (a counter cannot tell '(]' from '()').
COMMON VARIATIONS	Validity, longest valid substring, min reversals/insertions, decode string.

SOLVED EXAMPLE 7.5 — E

Valid Parentheses

Approach: push openers; on a closer, the top must be its matching opener.

```
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') st.push(c);
        else {
            if (st.empty()) return false;
            char o = st.top(); st.pop();
            if ((c == ')' && o != '(') || (c == '}' && o != '{') || (c == ']' && o != '['))
            ↪  return false;
        }
    }
    return st.empty();
}
```

SOLVED EXAMPLE 7.6 — H

Length of Longest Valid Parentheses Substring

Approach: keep a stack of **indices**, initially `-1` as a base marker. On `(` push the index. On `)` pop; if the stack becomes empty push the current index as a new base; otherwise `i - top` is a valid length.

Trace on `")()())"`: stack starts `[-1]`.

i	char	action	stack	best
0	)	pop -1, stack empty so push 0 (new base)	[0]	0
1	(	push 1	[0,1]	0
2	)	pop 1; length 2 - 0 = 2	[0]	2
3	(	push 3	[0,3]	2
4	)	pop 3; length 4 - 0 = 4	[0]	**4**
5	)	pop 0, stack empty so push 5	[5]	4

The index on top of the stack is always "the last position that cannot be part of the current valid run", so `i - top` is the length of the valid stretch ending at `i`.

```
int longestValidParentheses(const string& s) {
    stack<int> st;
    st.push(-1);
```

```
    int best = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '(') st.push(i);
        else {
            st.pop();
            if (st.empty()) st.push(i);
            else best = max(best, i - st.top());
        }
    }
    return best;
}
```

SOLVED EXAMPLE 7.7 M

Minimum Add to Make Parentheses Valid (and Minimum Bracket Reversals)

Insight: you do not need a stack, just a counter. `open` counts unmatched `(`; every unmatched `)` costs one insertion.

```
int minAddToMakeValid(const string& s) {
    int open = 0, add = 0;
    for (char c : s) {
        if (c == '(') open++;
        else if (open > 0) open--;
        else add++;                        // unmatched ')'
    }
    return open + add;
}
```

Minimum Bracket Reversals to Balance: if length is odd return -1. After cancelling matched pairs, you are left with `)))...(((`. With `a` unmatched `)` and `b` unmatched `(`: answer = `ceil(a/2) + ceil(b/2)`. Example: `}}{{` has `a = 2`, `b = 2`, so the answer is `1 + 1 = 2` (flip the first `}` and the last `{` to get `{}{}`). Complete tested code (with `{` and `}`) is in Appendix J.5.

Practice Set 7.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Parentheses & Bracket Matching* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Count / Remove Duplicate Parentheses M	Stack of `(` and letters; if after `)` the stack top is `(` immediately, the bracket pair is redundant.

7.3 Pattern: Stack & Queue Design

PATTERN CARD

SPOT IT WHEN	'Design a stack/queue that also supports X in $\mathcal{O}(1)$'.
KNOW FIRST	Amortized analysis.
BREAKS WHEN	Two stacks as a queue: forgetting that pop-from-output must only refill when output is EMPTY.
COMMON VARIATIONS	Min stack, queue via two stacks, stack via queues, deque design.

SOLVED EXAMPLE 7.8 M

Min Stack

Requirement: `push`, `pop`, `top`, `getMin` all $\mathcal{O}(1)$.

Idea: store each element together with the **minimum so far**.

```
class MinStack {
    stack<pair<int,int>> st;                        // (value, min up to here)
public:
    void push(int x) {
        int mn = st.empty() ? x : min(x, st.top().second);
        st.push({x, mn});
    }
    void pop()      { st.pop(); }
    int top()       { return st.top().first; }
    int getMin()    { return st.top().second; }
};
```

Companion: A Queue Built from Two Stacks

A stack returns items in reverse order, and reversing twice restores the original order. Keep an `in` stack for pushes and an `out` stack for pops and peeks. When `out` is empty, pour everything from `in` into `out`: the oldest element ends up on top. Pour **only when `out` is empty**; pouring earlier would put newer elements in front of older ones still waiting in `out`. Each element moves from `in` to `out` at most once, so every operation is $\mathcal{O}(1)$ amortized.

```
class MyQueue {
    stack<int> in, out;                                  // push onto `in`; pop/peek from `out`
    void refill() {                                        // only when `out` is EMPTY: this
    ↪  keeps the order right
        if (out.empty())
            while (!in.empty()) { out.push(in.top()); in.pop(); }   // reversing a stack turns
            ↪  LIFO into FIFO
    }
public:
    void push(int x) { in.push(x); }
    int pop()  { refill(); int v = out.top(); out.pop(); return v; }   // precondition: not empty
    int peek() { refill(); return out.top(); }                         // precondition: not empty
    bool empty() const { return in.empty() && out.empty(); }
};
```

Trace. `push 1, push 2, push 3`: `in = [1,2,3]`. `pop`: `out` is empty, so pour; `out = [3,2,1]` (top is 1); return 1. `push 4`: `in = [4]`. `pop`: `out` is not empty, so return its top, 2 (not 4). The next two pops return 3 and then, after another pour, 4.

Practice Set 7.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Stack & Queue Design* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Validate Stack Sequences M	Simulate: push from `pushed`; pop while top equals the next expected in `popped`.
K Stacks in a Single Array M	Arrays `top[k]` and `next[n]`; a free-list index `free`; push uses `free`, pop returns the slot to `free`.
K Queues in a Single Array M	Same trick with `front[k]`, `rear[k]`, `next[n]` and a free list.
Reverse First K Elements of a Queue M	Pop `k` items into a stack, push them back, then rotate the remaining `n-k` items.
Adapters (Stack Simulation) M	Read the statement carefully; simulate the transformation using a stack.
Max Frequency Stack H	`freq[x]` map plus `group[f]` = stack of elements with frequency `f`; track `maxFreq`.

7.4 Pattern: Sliding Window with Deque

PATTERN CARD

SPOT IT WHEN	Max/min of EVERY window of size k, or first negative in each window.
KNOW FIRST	Sliding window; deque as a double-ended buffer.
BREAKS WHEN	Window size varies per query (use a segment tree / sparse table).
COMMON VARIATIONS	Monotonic deque of indices, deque of candidates, fixed-window first-negative.

Idea: keep indices in a deque ordered so that the **front is always the answer** for the current window. Remove from the front when it leaves the window; remove from the back anything that can never be the best.

SOLVED EXAMPLE 7.9 H

Sliding Window Maximum

```cpp
vector<int> maxSlidingWindow(const vector<int>& a, int k) {
    deque<int> dq;                                  // indices, values decreasing
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();       // out of window
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();    // useless smaller ones
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

Trace on `a = [1,3,-1,-3,5,3,6,7]`, `k = 3` (the deque shows values, front on the left; the real code stores their indices):

i	a[i]	steps	deque	output
0	1	push	`[1]`	
1	3	`1 <= 3` is useless, pop it; push	`[3]`	
2	-1	push (smaller, may matter later)	`[3,-1]`	**3**
3	-3	push	`[3,-1,-3]`	**3**
4	5	front (3, index 1) left the window, pop front; then pop `-1`, `-3` (all `<= 5`); push	`[5]`	**5**
5	3	push	`[5,3]`	**5**
6	6	pop 3, pop 5; push	`[6]`	**6**
7	7	pop 6; push	`[7]`	**7**

Output: `[3,3,5,5,6,7]`. Each index enters and leaves the deque once, so the whole scan is $\mathcal{O}(n)$. The values in the deque are decreasing from front to back, which is why the front is always the maximum of the current window.

SOLVED EXAMPLE 7.10 E

First Negative Integer in Every Window of Size K

Keep a queue of indices of negative numbers inside the window.

```
vector<int> firstNegative(const vector<int>& a, int k) {
    queue<int> neg;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        if (a[i] < 0) neg.push(i);
        if (i >= k - 1) {
            while (!neg.empty() && neg.front() <= i - k) neg.pop();   // expired
            res.push_back(neg.empty() ? 0 : a[neg.front()]);
        }
    }
    return res;
}
```

SOLVED EXAMPLE 7.11 E

Print Binary Numbers from 1 to N Using a Queue

Start with `"1"`. Each time you pop `s`, output it and push `s+"0"` and `s+"1"`. The queue keeps numbers in increasing order.

```
vector<string> generateBinary(int n) {
    queue<string> q;
    q.push("1");
    vector<string> res;
    for (int i = 0; i < n; i++) {
        string s = q.front(); q.pop();
        res.push_back(s);
        q.push(s + "0");
        q.push(s + "1");
```

```
    }
    return res;
}
```

Practice Set 7.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Sliding Window with Deque* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Backspace String Compare E	Process from the end with a skip counter ($\mathcal{O}(1)$ space) or build both strings with a stack.
Maximum Sum of Smallest and Second-Smallest in a Window M	Only adjacent pairs matter: answer is `max(a[i] + a[i+1])` (reason: in any window, take the adjacent pair that contains the window's smallest element; its other member is at least the second smallest, so that pair's sum is at least `smallest + second smallest`; and an adjacent pair is itself a window of size 2).
Car Fleet M	Sort cars by position descending; compute arrival time; a car slower than the one ahead joins its fleet (monotonic stack of times).

CHAPTER COVERAGE

all 28 Stack & Queue problems appear above.

CHAPTER 7 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 7.1 Next Greater, 7.5 Valid Parentheses.
2. **Variations:** Ex 7.2 Daily Temperatures, Problem 7A (RPN).
3. **Combined:** Ex 7.4 Remove K Digits (stack + greedy).
4. **Independent recognition:** Problem 7B (simulation hiding a stack).
5. **Challenge:** Ex 7.3 Histogram, 7.6 Longest Valid Parentheses, 7.9 Sliding Window Maximum.

Compare and Contrast

	Stack	Queue	Deque
Order	LIFO	FIFO	both ends
Typical use	undo, brackets, DFS, monotonic stack	BFS, scheduling	sliding window max
Hallmark clue	'most recent unmatched'	'level by level'	'window moves, drop old, keep best'

Why It Works (Correctness Corner)

Monotonic stack invariant. After processing index `i`, the stack holds indices whose values never increase from bottom to top (equal values may stay, as in Example 7.2), and each is still waiting for a strictly greater element. When `a[i]` arrives, every index on top with a smaller value has just found its answer (`a[i]`), so we pop them. Each index is pushed once and popped at most once, hence $\mathcal{O}(n)$ total even though there is a `while` inside the `for`.

Debug Lab: Operands in the wrong order

```
// RPN: "3 1 -" should give 3 - 1 = 2
int a = st.top(); st.pop();
int b = st.top(); st.pop();
if (op == "-") st.push(a - b);
```

Your task: What does it compute for `3 1 -`?

ANSWER

It computes `1 - 3 = -2`. The first value popped is the RIGHT operand. Pop `b` first, then `a`, and push `a - b`. This matters for `-` and `/` only, which is why `+`/`*` tests pass by accident.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 7A M

Evaluate Reverse Polish Notation

Statement. Evaluate an expression in postfix form. Tokens are integers or the operators `+ - * /`. Division truncates toward zero. The expression is always valid.

Example. `["2","1","+","3","*"]` → `9`

Constraints. $1 \le$ tokens $\le 10^4$; intermediate values fit in 32 bits.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ space.

Prerequisites. 7.0 Concept, 7.2 Parentheses & Matching

HINT 1 which idea is relevant?

Which structure remembers 'the most recent two numbers'?

HINT 2 outline of the approach

Numbers go on the stack; an operator consumes the top two.

HINT 3 the key step

Pop the right operand first, then the left; push the result back.

SOLUTION and editorial (try first, then compare)

```cpp
int evalRPN(const vector<string>& t) {
    stack<long long> st;
    for (const string& s : t) {
        if (s == "+" || s == "-" || s == "*" || s == "/") {
            long long b = st.top(); st.pop();            // right operand comes off first
            long long a = st.top(); st.pop();
            if (s == "+") st.push(a + b);
            else if (s == "-") st.push(a - b);
            else if (s == "*") st.push(a * b);
            else st.push(a / b);                         // C++ truncates toward zero, as
            ↪ required
        } else st.push(stoll(s));
    }
    return (int)st.top();
}
```

Postfix notation is designed for stacks: when an operator appears, its two operands are exactly the two most recent unconsumed values. C++ integer division already truncates toward zero.

PROBLEM 7B M

Asteroid Collision

Statement. Each integer is an asteroid: its sign is the direction (positive = right, negative = left), its absolute value is the size. All move at the same speed. When two meet, the smaller explodes; equal sizes both explode. Return the asteroids that remain.

Example. `[5,10,-5]` → `[5,10]`; `[8,-8]` → `[]`

Constraints. $2 \le$ n $\le 10^4$; values non-zero.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ space.

Prerequisites. 7.1 Monotonic Stack

HINT 1 which idea is relevant?

Collisions only happen between a right-mover on the left and a left-mover on the right. What structure keeps the right-movers waiting?

HINT 2 outline of the approach

Keep surviving asteroids on a stack; process a new left-mover against the top.

HINT 3 the key step

While the new asteroid is negative and the top is positive: compare sizes; pop smaller tops; stop when the new one dies.

SOLUTION and editorial (try first, then compare)

```
vector<int> asteroidCollision(const vector<int>& ast) {
    vector<int> st;                                    // the stack of survivors
    for (int a : ast) {
        bool alive = true;
        while (alive && a < 0 && !st.empty() && st.back() > 0) {   // right-mover meets
        ↪  left-mover
            if (st.back() < -a) st.pop_back();         // top explodes, a keeps going
            else {
                if (st.back() == -a) st.pop_back();    // both explode
                alive = false;                         // a is destroyed
            }
        }
        if (alive) st.push_back(a);
    }
    return st;
}
```

Each asteroid is pushed at most once and popped at most once, so the loop is $\mathcal{O}(n)$. Left-movers that find an empty stack or a left-moving top simply survive and are pushed.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T7.1 For each day report how many days until a warmer one.

PATTERN

Monotonic stack of indices (Example 7.2).

T7.2 A text editor shows tags like `<b><i></b></i>`; is the nesting valid?

PATTERN

Stack matching, as with brackets.

Mixed Practice

Combine 7.1 and 3.2: Trapping Rain Water can be solved with two pointers (3.2) or a monotonic stack. Describe what the stack stores and when water is added.

Revision Sheet

Key takeaways

- Monotonic stack: pop while the new element beats the top.
- Every element enters and leaves once: $\mathcal{O}(n)$.
- Postfix = stack.
- Deque holds candidates in decreasing order for window max.

Pattern-recognition clues

- 'Next greater/smaller', 'span' $\rightarrow$ monotonic stack.
- 'Balanced', 'nested' $\rightarrow$ stack.
- 'Max of each window' $\rightarrow$ monotonic deque.

Common mistakes

- Calling `top()` on an empty stack.
- Wrong operand order for `-` and `/`.
- Storing values when you need indices (distances!).

Complexity summary

Task	Time	Space
Next greater	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Brackets	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Min stack ops	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Sliding window max	$\mathcal{O}(n)$	$\mathcal{O}(k)$

Recall questions (answer aloud, then check the chapter)

1. Why is the monotonic stack $\mathcal{O}(n)$ overall?
2. When do you store indices instead of values?
3. How can two stacks make a queue?
4. What does the deque keep in sliding window maximum?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Implement Next Greater Element (right) from scratch, then adapt it to 'next smaller on the left'. Test descending, ascending and all-equal arrays.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

PART II

Non-Linear Structures

8 Trees

9 Graphs

10 HashMap & Heap

CHAPTER 8

Trees

8

START HERE

A company's org chart is a tree. 'Who is the lowest manager above both Asha and Ben?' is a question you can answer by asking each subtree 'did you see either of them?'. Trees turn big problems into the same problem on smaller subtrees.

ANALOGY

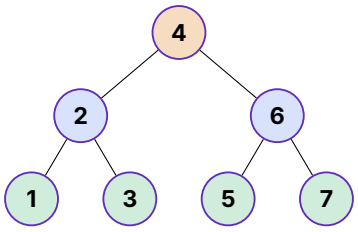
Think of a tree as a family tree with a rule: to solve a question for a person, ask each child, then combine their answers. That is postorder recursion in one sentence.

AFTER THIS CHAPTER YOU CAN

- ✓ Choose pre/in/post/level order for a task.
- ✓ Return information up the tree and keep a separate global answer.
- ✓ Use BST properties (inorder is sorted) and ranges for validation.
- ✓ Explain Fenwick and segment trees at a high level.

PICTURE IT

One tree, four traversal orders



inorder gives sorted order: this is a BST

inorder	1	2	3	4	5	6	7	
preorder	4	2	1	3	6	5	7	root first
postorder	1	3	2	5	7	6	4	children first
level order	4	2	6	1	3	5	7	

8.0 Concept

A **tree** is a hierarchy: a root node, and each node has children. A **binary tree** has at most two (left, right). Trees are naturally **recursive**: every subtree is itself a tree.

Vocabulary you will see in every problem. The **root** is the top node. A node's **children** hang below it and it is their **parent**; a node with no children is a **leaf**. The **subtree** of a node is that node plus everything below it. The **depth** of a node is the number of edges from the root to it (root = 0); the **height** of a tree is the length of the longest downward path (counted here in nodes unless a problem says edges). A tree is **balanced** if no node has two subtrees whose heights differ by more than 1.

How problems write a tree. Level order, left to right, with `null` for a missing child: `[3,9,20,null,null,15,7]` is the tree with root 3, children 9 and 20, and 20's children 15 and 7 (9 is a leaf, so the two `null`s after it stand for its absent children). Appendix I.2 shows how to build such a tree in C++.

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

Binary Search Tree (BST): for every node, all values in the left subtree are **smaller** and all values in the right subtree are **larger**. Inorder traversal of a BST yields sorted order.

The Master Recipe for Tree Recursion

For almost every tree problem, ask:

1. **Base case:** what is the answer for `nullptr`?
2. **Trust recursion:** assume the function already works for `left` and `right`.
3. **Combine:** how do I build my answer from the children's answers?

Pattern	Recognition clue
DFS traversals	inorder/preorder/postorder
BFS (level order)	levels, views, shortest depth, "zigzag"
BST property	search/insert/delete/validate, sorted order
LCA	"common ancestor", distance between nodes
Build from traversals	"construct tree from..."
Path problems (postorder DP)	max path, diameter, per-subtree values
Serialize	save/restore structure

8.1 Pattern: Traversals

PATTERN CARD

SPOT IT WHEN	Any 'visit every node' or 'compute from children' task.
KNOW FIRST	Recursion; call stack picture.
BREAKS WHEN	Deep (skewed) trees: recursion depth may overflow → use an iterative traversal.
COMMON VARIATIONS	Pre/in/post-order, iterative with stack, Morris $\mathcal{O}(1)$ space.

DFS orders: preorder (root, L, R), inorder (L, root, R), postorder (L, R, root). **BFS:** level by level with a queue.

```
void inorder(TreeNode* r, vector<int>& out) {
    if (!r) return;
    inorder(r->left, out);
    out.push_back(r->val);
    inorder(r->right, out);
}
// preorder: push before recursing; postorder: push after both recursions.
```

What the recursion really does. Here is `inorder` on the subtree `2` with children `1` and `3`; each indented line is a deeper call on the call stack:

Trace

```
inorder(2)
  inorder(1)
    inorder(null)  -> returns at once
    visit 1                       out = [1]
    inorder(null)  -> returns at once
  visit 2                         out = [1,2]
  inorder(3)
    inorder(null)
    visit 3                       out = [1,2,3]
    inorder(null)
```

The stack of unfinished calls is as deep as the tree is tall. On a skewed tree with 10^5 nodes that is 10^5 nested calls, which can overflow the stack; an iterative version with your own `stack` avoids that.

SOLVED EXAMPLE 8.1 E

Iterative Inorder Traversal

Recursion uses the call stack; we can use our own. Go left as far as possible, pop, visit, then go right.

```
vector<int> inorderIter(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        res.push_back(cur->val);
        cur = cur->right;
    }
    return res;
}
```

Trace on the subtree `2` with children `1` and `3`. Dive left, pushing as you go: `st = [2,1]`, `cur = null`. Pop 1 and visit it (`out = [1]`), then go right (null). Pop 2 and visit (`out = [1,2]`), then go right to 3. Dive left from 3: `st = [3]`, `cur = null`. Pop 3 and visit (`out = [1,2,3]`); its right is null. The stack is empty and `cur` is null, so we stop. The explicit stack holds exactly the nodes whose left side has been started but whose own visit is still pending.

Iterative preorder: push root; pop, visit, push right then left. **Iterative postorder:** do a modified preorder (root, right, left) and reverse the result. Complete tested code for both is in Appendix J.6.

SOLVED EXAMPLE 8.2 E/M

Level Order, Reverse Level Order, Zigzag

Key trick: at the start of each level, record `size = q.size()`; process exactly that many nodes. On the picture tree the queue goes `[4]` (size 1, level `[4]`), then `[2,6]` (size 2, level `[2,6]`), then `[1,3,5,7]` (size 4, level `[1,3,5,7]`). Without the size snapshot there would be no way to tell where one level ends and the next begins, because children of the current level are pushed into the same queue.

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left)  q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
// Reverse level order: reverse(res.begin(), res.end()) at the end.
// Zigzag: reverse(level.begin(), level.end()) on every odd-numbered level.
```

8.2 Pattern: Views of a Binary Tree

PATTERN CARD

SPOT IT WHEN	'Left/right/top/bottom view', 'level order', 'zigzag', 'vertical order'.
KNOW FIRST	BFS with a queue; (row, column) coordinates.
BREAKS WHEN	Ties in vertical order need a defined rule (sort by row, then value).
COMMON VARIATIONS	Level-size loop, hashmap by column, deque flip for zigzag.

SOLVED EXAMPLE 8.3 E/M

Left/Right View, Top/Bottom View, Vertical Order

Left/Right view: from the level-order loop, take the **first** (left) or **last** (right) node of each level.

```
vector<int> rightView(TreeNode* root) {
    vector<int> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
```

```
            TreeNode* node = q.front(); q.pop();
            if (i == sz - 1) res.push_back(node->val);   // last node of the level
            if (node->left)  q.push(node->left);
            if (node->right) q.push(node->right);
        }
    }
    return res;
}
```

Top/Bottom/Vertical: assign each node a **horizontal distance** `hd` (root = 0, left child `hd-1`, right child `hd+1`). A `map<int, ...>` keyed by `hd` stores the answer per column.

```
vector<int> topView(TreeNode* root) {
    map<int,int> firstAtColumn;                          // hd -> value
    queue<pair<TreeNode*, int>> q;
    if (root) q.push({root, 0});
    while (!q.empty()) {
        auto [node, hd] = q.front(); q.pop();
        if (!firstAtColumn.count(hd)) firstAtColumn[hd] = node->val;   // first seen = topmost
        if (node->left)  q.push({node->left,  hd - 1});
        if (node->right) q.push({node->right, hd + 1});
    }
    vector<int> res;
    for (auto& [hd, v] : firstAtColumn) res.push_back(v);
    return res;
}
```

- **Bottom View:** overwrite on every visit (`mp[hd] = node->val`) so the last (lowest) node wins. Tested code is in Appendix J.7.
- **Vertical Order:** `map<int, vector<int>>`, append each node to its column (BFS keeps top-to-bottom order).
- **Diagonal Traversal:** `d` stays the same for right child, `d+1` for left child; group by `d`.

Worked example on the picture tree (`4`; `2, 6`; `1, 3, 5, 7`). Horizontal distances: `4` is 0, `2` is -1, `6` is +1, `1` is -2, `3` is 0, `5` is 0, `7` is +2. Column 0 holds `4, 3, 5` (in BFS order). The **top view** takes the first node of each column: `1 2 4 6 7`. The **bottom view** takes the last: `1 2 5 6 7` (when two nodes share both column and level, the one visited later, the right one, wins here; some judges prefer the left, so read the statement). The **diagonals** (`d` = number of left steps taken from the root) are `4 6 7 | 2 3 5 | 1`. Tested code for bottom view, diagonal traversal and zigzag is in Appendix J.7.

Practice Set 8.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Views of a Binary Tree* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Boundary Traversal M	Three parts: left boundary (top-down, no leaves), all leaves left to right, right boundary (bottom-up, no leaves).

8.3 Pattern: BST Properties & Operations

PATTERN CARD

SPOT IT WHEN	'BST', 'sorted', 'kth smallest', 'validate', 'insert/delete'.
KNOW FIRST	Inorder of a BST is sorted.
BREAKS WHEN	Validating by comparing only with the parent (must carry a (min, max) range).
COMMON VARIATIONS	Validate, delete (3 cases), recover swapped nodes, BST to sum tree.

SOLVED EXAMPLE 8.4 — M

Validate a BST

Common mistake: only comparing a node with its direct children. Every node must lie within `(low, high)` bounds inherited from ancestors. The bounds are `long long` so that a node that holds `INT_MIN` or `INT_MAX` still compares correctly against the starting "no limit" values.

```
bool isValid(TreeNode* r, long long lo, long long hi) {
    if (!r) return true;
    if (r->val <= lo || r->val >= hi) return false;
    return isValid(r->left, lo, r->val) && isValid(r->right, r->val, hi);
}
bool isValidBST(TreeNode* root) { return isValid(root, LLONG_MIN, LLONG_MAX); }
```

SOLVED EXAMPLE 8.5 — M

Delete a Node in a BST

Three cases: **leaf** (just remove), **one child** (replace with the child), **two children** (copy the inorder successor, i.e. smallest in the right subtree, into this node and delete the successor).

Trace: delete the root 4 of the picture tree (two children). The inorder successor is the leftmost node of the right subtree: from 6 go left to 5. Copy 5 into the root (`root->val = 5`), then delete the node 5 from the right subtree (a leaf, so it is simply removed). The tree is now 5 with children `2 (1, 3)` and `6 (-, 7)`; its inorder sequence `1 2 3 5 6 7` is still sorted. The successor is the right choice because it is the smallest value larger than everything in the left subtree, so it can sit at the root without breaking the BST rule, and having no left child it is easy to remove. Each recursive call returns the new root of its subtree, and the parent re-links it (`root->left = deleteNode(root->left, key)`). The code `delete`s the node that is physically removed, so it assumes the nodes were allocated with `new`; deleting a key that is not in the tree changes nothing.

```
TreeNode* deleteNode(TreeNode* root, int key) {   // nodes were created with new, so we delete
↪  the removed one
    if (!root) return nullptr;
    if (key < root->val)      root->left  = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
       if (!root->left)  { TreeNode* r = root->right; delete root; return r; }   // 0 or 1 child
        if (!root->right) { TreeNode* l = root->left;  delete root; return l; }
        TreeNode* succ = root->right;                // two children
        while (succ->left) succ = succ->left;
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);   // this removes (and frees) the
↪  successor node
```

```
    }
    return root;
}
```

SOLVED EXAMPLE 8.6 M

Convert BST to Greater Sum Tree

Insight: *reverse inorder* (right, node, left) visits values from largest to smallest. Keep a running sum.

```
void convert(TreeNode* r, int& running) {
    if (!r) return;
    convert(r->right, running);
    running += r->val;
    r->val = running;
    convert(r->left, running);
}
```

SOLVED EXAMPLE 8.7 M

Recover a Binary Search Tree

Insight: two nodes were swapped. In the inorder sequence you will see one or two "drops" (`prev > cur`). The **first** violation's `prev` and the **last** violation's `cur` are the swapped nodes.

Trace. If inorder reads `1 3 2 4` (the adjacent nodes 3 and 2 were swapped) there is one drop, `3 > 2`: `first = 3`, `second = 2`; swapping their values fixes it. If inorder reads `3 2 1` (3 and 1 were swapped, far apart) there are two drops: `3 > 2` sets `first = 3`, `second = 2`; then `2 > 1` keeps `first` and moves `second` to 1. Swapping 3 and 1 gives `1 2 3`.

```
void recover(TreeNode* r, TreeNode*& prev, TreeNode*& first, TreeNode*& second) {
    if (!r) return;
    recover(r->left, prev, first, second);
    if (prev && prev->val > r->val) {
        if (!first) first = prev;
        second = r;
    }
    prev = r;
    recover(r->right, prev, first, second);
}
void recoverTree(TreeNode* root) {
    TreeNode *prev = nullptr, *first = nullptr, *second = nullptr;
    recover(root, prev, first, second);
    swap(first->val, second->val);
}
```

Practice Set 8.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *BST Properties & Operations* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Closest BST Value E	Walk down like a search; update the best candidate by `abs(val - target)`.
Check if Preorder Traversal Can Represent a BST M	Stack + `lowerBound`: when you see a value smaller than `lowerBound`, answer is false.
Construct BST from Preorder Traversal M	Recursion with an upper bound: take the next value if it is `< bound`.
Construct BST from Postorder Traversal M	Same, but read the array from the end (root, right, left) with bounds.
Convert a Sorted Doubly Linked List to a Balanced BST M	Pick the middle as root; recurse on halves (or use inorder simulation).
Closest BST Value II H	Inorder traversal with a deque of size `k`, or two stacks (predecessors and successors).
Merge Two BSTs H	Inorder of both to sorted arrays, merge, build a balanced BST. (Or two iterators.)

8.4 Pattern: Lowest Common Ancestor

PATTERN CARD

SPOT IT WHEN	'Lowest common ancestor', 'distance between two nodes'.
KNOW FIRST	Postorder return values.
BREAKS WHEN	Node may not exist in the tree (then 'found both' flags are needed).
COMMON VARIATIONS	General tree LCA, BST LCA (walk down), parent-pointer / binary lifting.

SOLVED EXAMPLE 8.8 M/E

LCA in a Binary Tree and in a BST

Binary tree: if the current node is `p` or `q`, return it. Search both sides. If both sides return non-null, the current node is the LCA; otherwise return whichever side is non-null.

This assumes both `p` and `q` are in the tree. **Trace on the picture tree.** For `p = 1, q = 3`: at `2` the left call returns `1` (found `p`) and the right call returns `3` (found `q`); both are non-null, so `2` is the LCA, and `4` simply passes it up (its right call returns null). For `p = 1, q = 5`: from `4` the left call returns `1` and the right call returns `5`, so `4` itself is the LCA. A node can be its own ancestor: for `p = 2, q = 3` the answer is `2`, because the call at `2` returns immediately when it sees `p`.

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* L = lca(root->left, p, q);
    TreeNode* R = lca(root->right, p, q);
    if (L && R) return root;
    return L ? L : R;
}
```

BST: use the ordering. If both values are smaller go left, if both larger go right, else the current node is the LCA. This is also the $\mathcal{O}(\text{Height})$ **time,** $\mathcal{O}(1)$ **extra space** version (iterative):

```
TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
```

```
        if (p->val < root->val && q->val < root->val) root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;
    }
    return nullptr;
}
```

8.5 Pattern: Construct Trees from Traversals

PATTERN CARD

SPOT IT WHEN	'Construct tree from two traversals'.
KNOW FIRST	Which traversal gives the root; inorder splits left/right.
BREAKS WHEN	Duplicates (the tree is no longer unique), or only preorder + postorder given for non-full trees.
COMMON VARIATIONS	Pre+in, post+in, level+in, serialize/deserialize.

SOLVED EXAMPLE 8.9 M

Construct Tree from Inorder and Preorder

Insight: the first preorder element is the root. Find it in inorder: everything left of it is the left subtree, everything right is the right subtree. A hash map makes the lookup $\mathcal{O}(1)$.

Trace with `pre = [3,9,20,15,7]`, `in = [9,3,15,20,7]`. The counter `idx` walks along `pre`; `lo..hi` is the slice of `in` that belongs to the current subtree. `build(0,4)`: root `3` (`pos[3] = 1`), so the left subtree is `in[0..0]` and the right subtree is `in[2..4]`. Left: root `9`, both of its ranges are empty, so it is a leaf. Right: root `20` (`pos[20] = 3`); its left range `in[2..2]` gives the leaf `15` and its right range `in[4..4]` gives the leaf `7`. The left subtree must be built first, because preorder lists the entire left subtree before the right one. The values must be distinct, otherwise `pos` cannot tell which copy is meant.

```
TreeNode* build(vector<int>& pre, int& idx, int lo, int hi, unordered_map<int,int>& pos) {
    if (lo > hi) return nullptr;
    int v = pre[idx++];
    TreeNode* root = new TreeNode(v);
    int m = pos[v];
    root->left  = build(pre, idx, lo, m - 1, pos);
    root->right = build(pre, idx, m + 1, hi, pos);
    return root;
}
TreeNode* buildTree(vector<int>& pre, vector<int>& in) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
    int idx = 0;
    return build(pre, idx, 0, (int)in.size() - 1, pos);
}
```

For **Inorder + Postorder**: the **last** postorder element is the root; read postorder from the end and build the **right** subtree before the left.

Practice Set 8.5

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Construct Trees from Traversals* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Construct Tree from Preorder and Postorder M	Root = `pre[0]`; `pre[1]` is the left subtree's root; find it in `post` to get the left subtree's size.
Verify Inorder and Level-Order Belong to the Same BST M	BST inorder must be sorted. Recursively split by the level-order root, checking the level-order elements fall in the left/right ranges.

8.6 Pattern: Path Problems (Postorder DP)

PATTERN CARD

SPOT IT WHEN	Max path, diameter, 'sum along root-to-leaf', 'longest path'.
KNOW FIRST	Postorder DFS; global answer vs returned value.
BREAKS WHEN	Returning the wrong quantity: the value passed UP must be a single downward path, while the answer may bend at the node.
COMMON VARIATIONS	Max path sum, diameter, path sum, sum root-to-leaf numbers.

Pattern: a function returns *one* thing to its parent (the best path going **down** through me), while updating a *global answer* with a value that may use **both** children (a path that bends at me).

SOLVED EXAMPLE 8.10 H

Maximum Path Sum in a Binary Tree

```
int bestSum;
int gain(TreeNode* n) {
    if (!n) return 0;
    int l = max(0, gain(n->left));              // ignore negative branches
    int r = max(0, gain(n->right));
    bestSum = max(bestSum, n->val + l + r);     // path bending at n
    return n->val + max(l, r);                  // path extending upward must pick one side
}
int maxPathSum(TreeNode* root) {
    bestSum = INT_MIN;
    gain(root);
    return bestSum;
}
```

Trace on `[-10, 9, 20, null, null, 15, 7]`. `gain(15) = 15` and `gain(7) = 7`. At `20`: `l = 15`, `r = 7`, so the path that bends at 20 is `20 + 15 + 7 = 42` (new best); 20 returns `20 + max(15, 7) = 35` to its parent, because a path continuing upward can use only one child. `gain(9) = 9`. At `-10`: `l = 9`, `r = 35`; the bent path `-10 + 9 + 35 = 34` is not better than 42, and the node returns `-10 + 35 = 25`. The answer is **42**, the path `15 -> 20 -> 7`, which was found at node 20, not at the root. This is why the answer lives in a separate variable instead of the return value.

The **same template** gives: Diameter of a Tree (`l + r` = number of edges bent at node), Longest Zigzag Path, Binary Tree Cameras, Distribute Coins.

SOLVED EXAMPLE 8.11 M

Sum Root to Leaf Numbers

Top-down version: pass the number formed so far downward.

```
int sumNumbers(TreeNode* r, int cur = 0) {
    if (!r) return 0;
    cur = cur * 10 + r->val;
    if (!r->left && !r->right) return cur;     // leaf
    return sumNumbers(r->left, cur) + sumNumbers(r->right, cur);
}
```

Practice Set 8.6

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Path Problems* (*Postorder DP*) idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Populate Next Right Pointers in Each Node M	Level-order, or use already-built `next` pointers of the upper level to connect children.
Maximum Product of Split Binary Tree M	Compute the total sum, then every subtree sum `s`: maximize `s * (total - s)`.
Distribute Coins in a Binary Tree M	Postorder returns `coins - 1` surplus of each subtree; answer sums `abs(surplus)`.
Longest Zigzag Path in a Binary Tree M	Return `(longest path starting with left step, with right step)` from each node.
Binary Tree Coloring Game M	Count nodes in three regions around `x` (left, right, parent side); player 2 wins if any region is more than `n/2`.
Sum of Distances in Tree H	Rerooting: compute subtree sizes/dists from root, then `ans[child] = ans[parent] - size[child] + (n - size[child])`.

8.7 Pattern: Advanced & Design

PATTERN CARD

SPOT IT WHEN	'Range query + update on an array', 'serialize', 'design a tree structure'.
KNOW FIRST	Binary representation of indices; recursion on halves.
BREAKS WHEN	Only static queries (prefix sums or sparse table are simpler).
COMMON VARIATIONS	Fenwick (point update, prefix query), segment tree (any associative op), lazy propagation.

SOLVED EXAMPLE 8.12 H

Serialize and Deserialize a Binary Tree

Idea: preorder traversal with a marker `#` for null. The sequence uniquely identifies the tree.

Example. The tree `1` with left child `2` and right child `3` (which has children `4` and `5`) becomes `1,2,#,#,3,4,#,#,5,#,#,`. Reading it back: `1` is the root; the next token `2` is its left child; the two `#` tokens say that 2 has no children; then `3` is the right child, and so on. Because every missing child is written out, a given string describes only one tree.

```
string serialize(TreeNode* root) {
    if (!root) return "#,";
    return to_string(root->val) + "," + serialize(root->left) + serialize(root->right);
}

TreeNode* deserializeHelper(istringstream& in) {
    string token;
    getline(in, token, ',');
    if (token == "#") return nullptr;
    TreeNode* node = new TreeNode(stoi(token));
    node->left  = deserializeHelper(in);
    node->right = deserializeHelper(in);
    return node;
}
TreeNode* deserialize(const string& data) {
    istringstream in(data);
    return deserializeHelper(in);
}
```

SOLVED EXAMPLE 8.13 M

Fenwick Tree (Binary Indexed Tree)

Purpose: point update and prefix-sum query, both in $\mathcal{O}(\log n)$, using less memory and code than a segment tree.

Trick: `i & -i` extracts the lowest set bit (for `i = 6 = 110b` it is `2`); index `i` stores the sum of a range of that length ending at `i`. For `n = 8`:

i	binary	lowest set bit	t[i] stores the sum of
1	001	1	[1,1]
2	010	2	[1,2]
3	011	1	[3,3]
4	100	4	[1,4]
5	101	1	[5,5]
6	110	2	[5,6]
7	111	1	[7,7]
8	1000	8	[1,8]

`query(7)`: add `t[7]` (`[7,7]`), then `i -= 1` gives 6: add `t[6]` (`[5,6]`), then `i -= 2` gives 4: add `t[4]` (`[1,4]`), then `i = 0`. The three pieces cover `[1,7]` exactly. `update(5, d)`: add `d` to `t[5]`, then `i += 1` gives 6 (`t[6]`), `i += 2` gives 8 (`t[8]`), `i += 8` gives 16 > n, stop: exactly the cells whose range contains position 5. Both loops take at most about log2(n) steps because every move changes the lowest set bit. Indices start at 1.

```cpp
struct Fenwick {
    int n;
    vector<long long> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void update(int i, long long delta) {          // 1-indexed
        for (; i <= n; i += i & -i) t[i] += delta;
    }
    long long query(int i) {                       // sum of [1..i]
        long long s = 0;
        for (; i > 0; i -= i & -i) s += t[i];
        return s;
    }
    long long range(int l, int r) { return query(r) - query(l - 1); }
};
```

SOLVED EXAMPLE 8.14 M

Segment Tree

Purpose: range queries (sum/min/max) and point updates in $\mathcal{O}(\log n)$. This bottom-up version uses an array of size `2n`.

Layout. The `n` leaves live in `t[n..2n-1]`; node `i` has children `2i` and `2i+1` and parent `i/2`; `t[1]` holds the answer for the whole array. For `a = [1,2,3,4]` the array is `t = [_,10,3,7,1,2,3,4]` (`t[2] = 1+2`, `t[3] = 3+4`, `t[1] = 3+7`). **Trace of `query(1,3)`** (sum of `a[1], a[2]` = 5): `l = 5`, `r = 7`. `l` is odd, so take `t[5] = 2` and move `l` to 6. `r` is odd, so step back to `r = 6` and take `t[6] = 3`. Now `l = r = 6`; both halve to 3 and the loop ends: total 5. The query range is half-open, `[l, r)`, and the structure works for any `n` as long as the operation is commutative (sum, min, max).

```cpp
struct SegTree {
    int n;
    vector<int> t;
    SegTree(const vector<int>& a) : n(a.size()), t(2 * a.size()) {
        for (int i = 0; i < n; i++) t[n + i] = a[i];            // leaves
        for (int i = n - 1; i > 0; i--) t[i] = t[2*i] + t[2*i+1];
    }
    void update(int pos, int val) {
        for (t[pos += n] = val; pos > 1; pos >>= 1)
            t[pos >> 1] = t[pos] + t[pos ^ 1];
    }
    int query(int l, int r) {                                   // sum over [l, r)
        int res = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += t[l++];
            if (r & 1) res += t[--r];
        }
        return res;
    }
};
```

To get a min/max tree, replace `+` with `min`/`max` and the identity `0` with `INT_MAX`/`INT_MIN`.

Practice Set 8.7

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Advanced & Design* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Inorder Successor in a BST M	If node has a right child, go leftmost there; else the successor is the last ancestor from which you went left.
Flatten Binary Tree to Linked List M	Reverse postorder (right, left, node) with a `prev` pointer; or Morris-style rewiring.
Convert Binary Tree to Circular Doubly Linked List M	Inorder with `prev`: link `prev->right = cur; cur->left = prev`; finally join head and tail.
Clone a Binary Tree with Random Pointer M	Hash map `old -> new`; first create copies, then set `left`, `right`, `random`.
Count Complete Tree Nodes M	If left height equals right height the subtree is perfect: `2^h - 1`; else recurse. $\mathcal{O}(\log^2 n)$.
LCA in $\mathcal{O}(\text{Height})$ Time, $\mathcal{O}(1)$ Extra Space M	Use parent pointers: align depths, then climb together (or the BST iterative walk above).
Binary Tree Cameras H	Postorder with three states: 0 = needs camera, 1 = has camera, 2 = covered; place a camera when any child needs one.

CHAPTER COVERAGE

all 47 Tree problems appear above.

CHAPTER 8 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 8.1 Iterative Inorder, 8.2 Level Order.
2. **Variations:** Ex 8.3 Views, Problem 8B (inorder on a BST).
3. **Combined:** Ex 8.8 LCA, 8.9 Construct Tree.
4. **Independent recognition:** Problem 8A (return height *and* a failure flag).
5. **Challenge:** Ex 8.10 Max Path Sum, 8.12 Serialize, 8.13-8.14 Fenwick/Segment.

Compare and Contrast

	DFS orders	BFS (level order)
Tool	recursion / stack	queue
Preorder	root first: copy / serialize	
Inorder	BST sorted order	
Postorder	children first: heights, paths, delete	
Level order		shortest depth, views, zigzag

Why It Works (Correctness Corner)

Postorder induction. Claim: `heightOrFail(r)` returns the correct height if the subtree at `r` is balanced, and -1 otherwise. Base: empty tree is balanced with height 0. Step: assume the claim for both children. If either returns -1 the subtree is unbalanced. Otherwise both heights are correct, and the subtree is balanced exactly when they differ by at most 1, in which case its height is `1 + max`.

BST validation. A node's value must lie in an open interval `(lo, hi)` inherited from all its ancestors, not just its parent.

Debug Lab: Validating a BST with only the parent

```
bool isValidBST(TreeNode* r) {
    if (!r) return true;
    if (r->left  && r->left->val  >= r->val) return false;
    if (r->right && r->right->val <= r->val) return false;
    return isValidBST(r->left) && isValidBST(r->right);
}
```

Your task: Find a tree it wrongly accepts.

ANSWER

Root 5, left 1, right 6, and 6's left child 3. Each node is consistent with its parent, but 3 lies in the right subtree of 5 and is smaller than 5. Fix: pass `(lo, hi)` bounds down, using `long long` bounds so `INT_MIN`/`INT_MAX` values are handled.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 8A E

Balanced Binary Tree

Statement. A binary tree is balanced if for every node the heights of its left and right subtrees differ by at most 1. Decide whether a tree is balanced.

Example. `[3,9,20,null,null,15,7]` → `true`; `[1,2,2,3,3,null,null,4,4]` → `false`

Constraints. $0 \leq n \leq 5000$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(h)$ space.

Prerequisites. 8.6 Path Problems (postorder) (uses the `TreeNode` struct from 8.0)

HINT 1 which idea is relevant?

Computing the height of every node separately is $\mathcal{O}(n^2)$ on a skewed tree. How can you check balance *while* computing height?

HINT 2 outline of the approach

Let the function return a height, with a special value meaning 'already unbalanced'.

HINT 3 the key step

Return -1 if either child returns -1 or the heights differ by more than 1.

SOLUTION and editorial (try first, then compare)

```
// Returns the height of the subtree, or -1 as soon as any subtree is unbalanced.
int heightOrFail(TreeNode* r) {
    if (!r) return 0;
    int l = heightOrFail(r->left);   if (l < 0) return -1;
    int h = heightOrFail(r->right);  if (h < 0) return -1;
    if (abs(l - h) > 1) return -1;
    return 1 + max(l, h);
}
bool isBalanced(TreeNode* root) { return heightOrFail(root) >= 0; }
```

A single postorder pass: each node is visited once and the first violation propagates upward as -1, so the answer is `height >= 0`.

PROBLEM 8B M

K-th Smallest in a BST

Statement. Given the root of a BST and `k`, return the k-th smallest value (1-indexed).

Example. BST `[5,3,6,2,4,null,null,1]`, `k = 3` → `3`

Constraints. $1 \leq k \leq n \leq 10^4$.

Expected complexity. $\mathcal{O}(h + k)$ time, $\mathcal{O}(h)$ space.

Prerequisites. Example 8.1 Iterative Inorder, 8.3 BST Properties (uses the `TreeNode` struct from 8.0)

HINT 1 which idea is relevant?

What does an inorder traversal of a BST produce?

HINT 2 outline of the approach

You do not need the whole traversal, only the k-th visited node.

HINT 3 the key step

Use the iterative inorder with a stack and stop when the counter reaches k.

SOLUTION and editorial (try first, then compare)

```cpp
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }   // dive left
        cur = st.top(); st.pop();                        // visit in sorted order
        if (--k == 0) return cur->val;
        cur = cur->right;
    }
    return -1;                                           // k larger than the tree
}
```

Inorder visits BST nodes in ascending order, so the k-th visited node is the answer. Stopping early gives $\mathcal{O}(h + k)$. The function returns -1 if `k > n`; decide in an interview what the contract is.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T8.1 In an org chart find the lowest manager above two given employees.

PATTERN

LCA by postorder return values.

T8.2 Find the chain of managers going downward with the largest total bonus.

PATTERN

Max path sum: return the best single downward chain, update a global answer with the bent path.

Mixed Practice

Combine 8.2 and 9.1 (later): a binary tree is a graph. Explain how level order is BFS with the parent as the only edge direction.

Revision Sheet

Key takeaways

- Recursion on trees = trust the children.
- Returned value vs global answer are different.
- Inorder of a BST is sorted.
- Recursion depth can hit n on skewed trees.

Pattern-recognition clues

- 'Height', 'diameter', 'path' → postorder.
- 'Level', 'view' → BFS.
- 'BST' → inorder / ranges.

- 'Ancestor' → LCA.

Common mistakes

- Dereferencing a null child.
- Validating BST with parent-only checks.
- Mixing up what a function returns and what it records.

Complexity summary

Task	Time	Space
Any traversal	$\mathcal{O}(n)$	$\mathcal{O}(h)$
BST search/insert	$\mathcal{O}(h)$	$\mathcal{O}(1)$-$\mathcal{O}(h)$
Fenwick update/query	$\mathcal{O}(\log\ n)$	$\mathcal{O}(n)$
Segment tree	$\mathcal{O}(\log\ n)$	$\mathcal{O}(n)$

Recall questions (answer aloud, then check the chapter)

1. Which traversal sorts a BST?
2. What does a postorder function return vs record?
3. Why are bounds needed in BST validation?
4. When is level order the natural choice?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Code iterative inorder, then use it to answer both 'k-th smallest' and 'is this a valid BST?'. Test with a single node, skewed trees and duplicate values.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 9

Graphs

9

START HERE

A rumor starts at several houses at once; how many hours until the whole grid hears it? Flight routes, course prerequisites, social networks and mazes are all the same object: nodes and edges. Learn four tools (BFS, DFS, shortest paths, union-find) and most graph problems fall.

ANALOGY

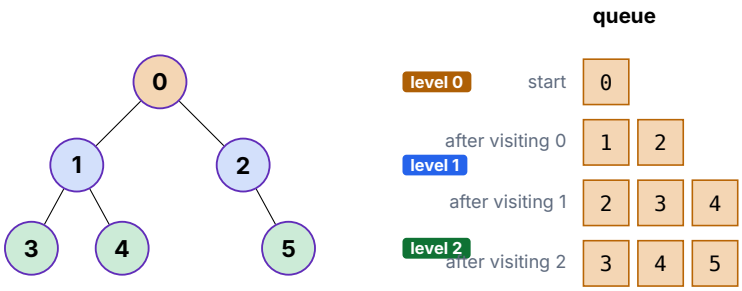
BFS is a ripple spreading from a dropped pebble: closer nodes first. DFS is exploring a maze with a ball of string: go as deep as possible, backtrack at dead ends.

AFTER THIS CHAPTER YOU CAN

- ✓ Model a problem as a graph and pick BFS/DFS.
- ✓ Run Dijkstra, Bellman-Ford and Floyd-Warshall and say when each applies.
- ✓ Order tasks with topological sort and detect cycles.
- ✓ Use union-find for connectivity.

PICTURE IT

BFS spreads in rings



dist: $0 \to 0,\ 1,2 \to 1,\ 3,4,5 \to 2$ (the first time a node is seen is its shortest distance)

9.0 Concept

A **graph** is a set of **vertices** (nodes) connected by **edges**. Edges may be **directed** or **undirected**, and may carry **weights**. Social networks, maps, course prerequisites, and grids are all graphs.

Vocabulary. A **path** is a sequence of vertices joined by edges; a **cycle** is a path that returns to its start. A graph is **connected** if every vertex can reach every other (for directed graphs this is called *strongly* connected). A

DAG is a directed graph with no cycle. The **degree** of a vertex is its number of edges; in a directed graph we separate **in-degree** (edges arriving) and **out-degree** (edges leaving). A **tree** is a connected undirected graph with no cycle, so with `V` vertices it has exactly `V-1` edges. Problems usually give vertices as `0..n-1` (or `1..n`; check) and edges as a list of pairs `[u, v]` or triples `[u, v, w]`.

Representation: Adjacency List

```cpp
int n = 5;                                  // vertices 0..n-1
vector<vector<int>> adj(n);                 // unweighted
adj[0].push_back(1);  adj[1].push_back(0);  // undirected edge 0 - 1

vector<vector<pair<int,int>>> wadj(n);      // weighted: (neighbor, weight)
wadj[0].push_back({1, 7});
```

Representation	Space	Check edge	Iterate neighbors
Adjacency list	$\mathcal{O}(V+E)$	$\mathcal{O}(\text{degree})$	$\mathcal{O}(\text{degree})$
Adjacency matrix	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	$\mathcal{O}(V)$

Use the **list** unless the graph is dense or V is small. Remember that an undirected edge must be stored in **both** directions (as in the code above); forgetting one `push_back` is the most common graph bug (see Appendix I.3).

Pattern Map

Pattern	Recognition clue
BFS	shortest path in **unweighted** graph/grid, level-by-level spread
DFS	connected components, cycle detection, exploring all paths
Topological sort	dependencies, ordering, "prerequisites", DAG
Dijkstra	shortest path, **non-negative** weights
Bellman-Ford	negative weights, "at most K edges/stops"
Floyd-Warshall	all-pairs shortest path, n $\leq$ ~400
MST (Prim/Kruskal)	connect everything at minimum total cost
Union-Find (DSU)	dynamic connectivity, grouping, cycle in undirected graph
SCC / bridges / articulation	critical structure, strongly connected groups
Max flow / matching	capacities, assignments

9.1 Pattern: BFS & DFS Fundamentals

PATTERN CARD

SPOT IT WHEN	'Connected', 'reachable', 'components', 'level by level distance'.
KNOW FIRST	Adjacency list; visited array.
BREAKS WHEN	Marking visited AFTER popping instead of when pushing (duplicates and TLE).
COMMON VARIATIONS	BFS, DFS recursive/iterative, bipartite check, cycle detection.

SOLVED EXAMPLE 9.1 E

BFS and DFS of a Graph

BFS explores in rings of distance from the start; use a queue and a `visited` array. **Mark visited when you push**, not when you pop, to avoid duplicates.

```
vector<int> bfs(int start, const vector<vector<int>>& adj) {
    vector<int> order;
    vector<bool> vis(adj.size(), false);
    queue<int> q;
    q.push(start); vis[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (!vis[v]) { vis[v] = true; q.push(v); }
    }
    return order;
}

void dfs(int u, const vector<vector<int>>& adj, vector<bool>& vis, vector<int>& order) {
    vis[u] = true;
    order.push_back(u);
    for (int v : adj[u])
        if (!vis[v]) dfs(v, adj, vis, order);
}
```

Disconnected graphs: loop over all vertices and start a traversal from each unvisited one. The number of starts is the **number of connected components**.

Trace on the picture graph (edges `0-1, 0-2, 1-3, 1-4, 2-5`, neighbors listed in that order). *BFS from 0:* queue `[0]`; pop 0, push 1 and 2; pop 1, push 3 and 4 (0 is already visited); pop 2, push 5; pop 3, 4, 5. Order: `0 1 2 3 4 5`. To get **distances** instead of an order, store `dist[v] = dist[u] + 1` at the moment you push `v`; that first time is always the shortest, because BFS reaches nodes in rings.

DFS from 0 (each indented call is deeper on the call stack):

Trace

```
dfs(0)  order=[0]
  dfs(1)  order=[0,1]
    (0 already visited)
    dfs(3)  order=[0,1,3]      3 has no unvisited neighbor, return
    dfs(4)  order=[0,1,3,4]    return
  dfs(2)  order=[0,1,3,4,2]
    dfs(5)  order=[0,1,3,4,2,5]
```

BFS goes wide (all neighbors first); DFS goes deep (finish a branch before trying the next one). Recursive DFS needs stack space proportional to the longest path, which can reach `V` on a long chain; for very large inputs use an explicit stack.

SOLVED EXAMPLE 9.2 M

Bipartite Graph Check

Idea: try to 2-color the graph so that neighbors always differ. If BFS finds an edge between two same-colored nodes, it is not bipartite (it contains an odd cycle).

```cpp
bool isBipartite(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> color(n, -1);
    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        queue<int> q; q.push(s); color[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
                else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}
```

Trace. On a triangle `0-1, 1-2, 2-0`: color 0 gets 0; its neighbors 1 and 2 get color 1; when we process 1 we see its neighbor 2 already has color 1, the same as ours, so we return `false`. On a square `0-1-2-3-0` the colors alternate `0,1,0,1` and every edge joins different colors, so the answer is `true`. Any graph with an odd cycle fails and any graph without one succeeds, which is the whole theorem.

Practice Set 9.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *BFS & DFS Fundamentals* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Evaluate Division M	Build a weighted graph: edge `a->b` weight `value`, `b->a` weight `1/value`; answer a query by DFS/BFS multiplying weights.
Mother Vertex M	Run DFS from every unvisited vertex, remember the last one finished; verify it reaches all vertices.

9.2 Pattern: Grid-Based BFS/DFS

PATTERN CARD

SPOT IT WHEN	Grid with cells as nodes: islands, flood, spreading, shortest steps.
KNOW FIRST	BFS/DFS; direction arrays.
BREAKS WHEN	Weighted cells (use Dijkstra/0-1 BFS), or the state needs extra info (keys, remaining breaks).
COMMON VARIATIONS	Flood fill, multi-source BFS, 4 vs 8 directions, state-augmented BFS.

Treat each cell as a node and its 4 neighbors as edges (use the `dr/dc` arrays from Chapter 4). No adjacency list is needed.

SOLVED EXAMPLE 9.3 E

Number of Islands

Idea: every time we find an unvisited land cell, we found a new island; flood-fill it so it is never counted again. (This version destroys the input grid by sinking the land and recurses once per cell, so a huge grid of all land would need an explicit stack or queue instead; copy the grid first if you must keep it.)

```
void flood(vector<vector<char>>& g, int r, int c) {
    if (r < 0 || c < 0 || r >= (int)g.size() || c >= (int)g[0].size() || g[r][c] != '1') return;
    g[r][c] = '0';                                          // mark visited by sinking the land
    flood(g, r + 1, c); flood(g, r - 1, c);
    flood(g, r, c + 1); flood(g, r, c - 1);
}
int numIslands(vector<vector<char>>& g) {
    int count = 0;
    for (int i = 0; i < (int)g.size(); i++)
        for (int j = 0; j < (int)g[0].size(); j++)
            if (g[i][j] == '1') { count++; flood(g, i, j); }
    return count;
}
```

SOLVED EXAMPLE 9.4 M

Rotting Oranges (Multi-Source BFS)

Idea: start BFS from **all rotten oranges simultaneously** (put them all in the queue first). Each BFS level equals one minute.

```
int orangesRotting(vector<vector<int>>& g) {
    int R = g.size(), C = g[0].size(), fresh = 0, minutes = 0;
    queue<pair<int,int>> q;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) {
            if (g[i][j] == 2) q.push({i, j});
            else if (g[i][j] == 1) fresh++;
        }
    int dr[] = {-1, 1, 0, 0}, dc[] = {0, 0, -1, 1};
    while (!q.empty() && fresh > 0) {
        int sz = q.size();
        for (int s = 0; s < sz; s++) {
            auto [r, c] = q.front(); q.pop();
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nc < 0 || nr >= R || nc >= C || g[nr][nc] != 1) continue;
                g[nr][nc] = 2; fresh--;
                q.push({nr, nc});
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

Trace on `[[2,1,1],[1,1,0],[0,1,1]]`. There are 6 fresh oranges and one rotten one at `(0,0)`. Minute 1: it rots `(0,1)` and `(1,0)` (fresh 4). Minute 2: those rot `(0,2)` and `(1,1)` (fresh 2). Minute 3: `(1,1)` rots `(2,1)` (fresh 1). Minute 4: `(2,1)` rots `(2,2)` (fresh 0). Answer `4`. Taking `sz = q.size()` at the top of each round is what makes one pass equal one minute. If fresh oranges remain when the queue empties, some orange is unreachable and the answer is `-1`.

The same multi-source BFS solves **0-1 Matrix** (sources = all zeros, distance = BFS level) and **As Far from Land as Possible** (sources = all land cells).

SOLVED EXAMPLE 9.5 H

Word Ladder

Problem: transform `begin` to `end` changing one letter at a time, each intermediate word being in the dictionary. Find the shortest sequence length.

Idea: words are nodes; edges connect words differing by one letter. BFS gives the shortest path. Generate neighbors by trying all 26 letters at every position.

```
int ladderLength(string begin, string end, vector<string>& wordList) {
    unordered_set<string> dict(wordList.begin(), wordList.end());
    if (!dict.count(end)) return 0;
    queue<string> q; q.push(begin);
    int steps = 1;
    while (!q.empty()) {
        int sz = q.size();
```

```
        while (sz--) {
            string w = q.front(); q.pop();
            if (w == end) return steps;
            for (int i = 0; i < (int)w.size(); i++) {
                char orig = w[i];
                for (char c = 'a'; c <= 'z'; c++) {
                    w[i] = c;
                    if (dict.count(w)) { q.push(w); dict.erase(w); }   // erase = visited
                }
                w[i] = orig;
            }
        }
        steps++;
    }
    return 0;
}
```

Trace for `begin = "hit"`, `end = "cog"`, dictionary `{hot, dot, dog, lot, log, cog}`. Level 1 (steps = 1): `hit`. Its one-letter neighbors in the dictionary: `hot` (steps 2). From `hot`: `dot`, `lot` (steps 3). From those: `dog`, `log` (steps 4). From `dog`: `cog` (steps 5), and `cog == end`, so the answer is **5** (`hit -> hot -> dot -> dog -> cog`, five words). Erasing a word from the dictionary the moment we queue it is the "visited" marker, so every word enters the queue at most once. Cost: about `N * L * 26` neighbor strings, each costing $\mathcal{O}(L)$ to hash, where `N` is the number of words and `L` their length.

Practice Set 9.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Grid-Based BFS/DFS* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Number of Enclaves M	Flood-fill from border land cells; count the land cells left.
Number of Distinct Islands M	DFS recording path shape relative to the starting cell (e.g. direction string); insert into a set.
0-1 Matrix (Multi-Source BFS Distances) M	Queue all zeros with distance 0; BFS fills the rest.
Shortest Bridge M	DFS to find and mark one island, then multi-source BFS from it until touching the other.
As Far from Land as Possible M	Multi-source BFS from all land cells; answer is the last level reached.
Coloring a Border M	DFS the component; a cell is a border if on the grid edge or has a neighbor of a different color.
Sliding Puzzle H	BFS over board states (as strings) with a visited set; neighbors = swap `0` with adjacent tile.
Bus Routes H	BFS over **routes** (not stops); map stop to routes that serve it.
Number of Islands II H	Union-Find (Section 9.6), adding land cells one by one.

9.3 Pattern: Topological Sort

PATTERN CARD

SPOT IT WHEN	'Prerequisites', 'build order', 'detect a cycle in a directed graph'.
KNOW FIRST	Directed graph, in-degree.
BREAKS WHEN	Graph has a cycle (no valid order: detect it by processed $< n$).
COMMON VARIATIONS	Kahn (BFS), DFS finish order, alien dictionary, course schedule.

Applies to DAGs (directed acyclic graphs). A topological order lists nodes so that every edge `u -> v` has `u` before `v`. If the graph has a cycle, no such order exists, which is how we detect impossible schedules.

SOLVED EXAMPLE 9.6 M

Kahn's Algorithm (BFS Topological Sort)

Idea: repeatedly take nodes with **in-degree 0** (no unmet prerequisites), output them, and decrement their neighbors' in-degrees.

```
vector<int> topoSort(int n, const vector<vector<int>>& adj) {
    vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};          // cycle detected
    return order;
}
```

Trace on edges `0->1, 0->2, 1->3, 2->3, 3->4`. In-degrees: `[0,1,1,2,1]`; the queue starts as `[0]`. Pop 0: in-degree of 1 and 2 drop to 0, push both (`order = [0]`). Pop 1: in-degree of 3 drops to 1 (not yet ready). Pop 2: in-degree of 3 drops to 0, push 3. Pop 3: push 4. Pop 4. Order `[0,1,2,3,4]`. With a cycle such as `0->1, 1->0` every in-degree is 1, the queue starts empty, `order.size() = 0 != 2`, and we report the cycle.

Course Schedule II: build edges `prereq -> course`, run Kahn, return the order (or empty if a cycle).

DFS-based topological sort: run DFS, push each node to a list **after** visiting all neighbors, then reverse the list. On the same graph: finish order is `4, 3, 1, 2, 0`; reversed it is `0, 2, 1, 3, 4`, a valid order (a node is finished only after everything that depends on it, so reversing puts it before them). Call `topoDFS` from every unvisited vertex. Unlike Kahn, this version does not notice cycles by itself; for that you need three states (unvisited, in progress, done) and a cycle exists when you reach an in-progress vertex.

```
void topoDFS(int u, const vector<vector<int>>& adj, vector<bool>& vis, vector<int>& out) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) topoDFS(v, adj, vis, out);
    out.push_back(u);                                  // post-order; reverse at the end
}
```

Practice Set 9.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Topological Sort* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Find Eventual Safe States M	Reverse the graph and run Kahn from terminal nodes; or DFS with 3 colors (unvisited, visiting, safe).
Alien Dictionary H	Compare adjacent words; first differing letters give edge `a -> b`. Kahn over letters. Watch for prefix conflicts ("abc" before "ab" is invalid).
Sort Items by Groups Respecting Dependencies H	Two topological sorts: one among groups, one among items inside each group; stitch them.

9.4 Pattern: Shortest Paths

PATTERN CARD

SPOT IT WHEN	'Cheapest/shortest route' on a weighted graph.
KNOW FIRST	Priority queue; relaxation.
BREAKS WHEN	Dijkstra with negative edges (use Bellman-Ford); BFS on weighted graphs.
COMMON VARIATIONS	Dijkstra, Bellman-Ford (K edges / negatives), Floyd-Warshall (all pairs), 0-1 BFS.

Algorithm	Handles	Complexity
BFS	unweighted	$\mathcal{O}(V+E)$
0-1 BFS	weights 0/1	$\mathcal{O}(V+E)$
Dijkstra	non-negative weights	$\mathcal{O}((V+E)\log V)$
Bellman-Ford	negative weights, detects negative cycles	$\mathcal{O}(V \cdot E)$
Floyd-Warshall	all pairs	$\mathcal{O}(V^3)$

SOLVED EXAMPLE 9.7 M

Dijkstra's Algorithm

Idea: always settle the closest unsettled vertex (a min-heap gives it). Once settled, its distance is final because weights are non-negative.

Trace on edges `0->1 (4)`, `0->2 (1)`, `2->1 (2)`, `1->3 (1)`, `2->3 (5)`, source 0:

pop `(d, u)`	what happens	`dist` after (0,1,2,3)	heap gets
`(0, 0)`	relax 1 to 4 and 2 to 1	`[0,4,1,inf]`	`(4,1)`, `(1,2)`
`(1, 2)`	relax 1 to `1+2 = 3`; relax 3 to `1+5 = 6`	`[0,3,1,6]`	`(3,1)`, `(6,3)`
`(3, 1)`	relax 3 to `3+1 = 4`	`[0,3,1,4]`	`(4,3)`
`(4, 1)`	stale (`4 > dist[1] = 3`), skip		
`(4, 3)`	nothing to relax		

pop `(d, u)`	what happens	`dist` after (0,1,2,3)	heap gets
`(6, 3)`	stale, skip		

Final distances `[0,3,1,4]`. The `if (d > dist[u]) continue;` line discards outdated heap entries instead of searching the heap to update them. **Why negative edges break it:** with `0->1 (2)`, `0->2 (3)`, `2->1 (-2)`, vertex 1 is popped first with distance 2 and treated as final, but the true shortest distance is `3 - 2 = 1`; the argument "nothing farther away can make it shorter" no longer holds.

```
vector<long long> dijkstra(int src, const vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;   // (dist,
↪   node)
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;                      // stale entry
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

SOLVED EXAMPLE 9.8 M

Bellman-Ford & Cheapest Flights Within K Stops

Idea: relax **all edges** `V-1` times. Round `i` finds shortest paths using at most `i` edges, which is exactly what "at most K stops" needs: run `K+1` rounds, using a copy of the previous round so one round adds only one edge.

```
int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int K) {
    const int INF = 1e9;
    vector<int> dist(n, INF);
    dist[src] = 0;
    for (int i = 0; i <= K; i++) {
        vector<int> next = dist;                          // use only previous round's values
        for (auto& f : flights) {
            int u = f[0], v = f[1], w = f[2];
            if (dist[u] != INF && dist[u] + w < next[v]) next[v] = dist[u] + w;
        }
        dist = next;
    }
    return dist[dst] == INF ? -1 : dist[dst];
}
```

Trace with `n = 3`, flights `0->1 (100)`, `1->2 (100)`, `0->2 (500)`, `src = 0`, `dst = 2`. With `K = 1` we run 2 rounds. Round 1 (paths of at most 1 flight): `dist = [0,100,500]`. Round 2 (at most 2 flights): `2` improves via `1`, to `100 + 100 = 200`. Answer `200`. With `K = 0` only round 1 runs, so the answer is `500`. If we updated `dist` in place, round 1 could already use the freshly improved `dist[1]` to reach 2 and cheat the stop limit; the copy `next` prevents that.

For plain Bellman-Ford, run `V-1` rounds in place; if round `V` still relaxes any edge, there is a **negative cycle** reachable from the source.

SOLVED EXAMPLE 9.9 M

Floyd-Warshall

Idea (DP): `dist[i][j]` = best path from `i` to `j` allowing intermediate vertices `0..k`. Try each `k` as a stepping stone.

```
void floydWarshall(vector<vector<long long>>& d) {       // d[i][j] = INF if no edge, d[i][i] =
↪  0
    int n = d.size();
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < LLONG_MAX / 2 && d[k][j] < LLONG_MAX / 2)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

The **loop order `k` outermost** is essential. After the iteration for `k` finishes, `d[i][j]` is the best path from `i` to `j` that uses only vertices `0..k` as stepping stones. The update uses `d[i][k]` and `d[k][j]`, which at that moment still mean "using only `0..k-1`", exactly what the recurrence needs. Example: edges `0->1 (3)`, `1->2 (4)`, `0->2 (10)`: at `k = 1` we get `d[0][2] = min(10, 3 + 4) = 7`. After the loops, a vertex with `d[i][i] < 0` lies on a negative cycle. Cost: three nested loops, $\mathcal{O}(V^3)$ time and $\mathcal{O}(V^2)$ space, which is why it is used for `V` up to a few hundred.

Practice Set 9.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Shortest Paths* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Chef and Reversing (0-1 BFS) M	Edges as-is cost 0, reversed edges cost 1; deque-based 0-1 BFS (push front for 0, back for 1).
Shortest Distance from All Buildings H	BFS from every building, summing distances into a grid; pick the reachable empty cell minimizing the sum.
Shortest Cycle in an Undirected Graph H	BFS from each vertex; a non-tree edge `(u, v)` gives a cycle of length `dist[u] + dist[v] + 1`; take the min.
Swim in Rising Water H	Dijkstra where path cost is the **max** cell value along the path; or binary search + BFS.
Johnson's Algorithm H	Add a virtual source, run Bellman-Ford to get potentials `h`, reweight `w' = w + h[u] - h[v]` (non-negative), then Dijkstra from every vertex.

9.5 Pattern: Minimum Spanning Tree

PATTERN CARD

SPOT IT WHEN	'Connect all nodes with minimum total cost'.
KNOW FIRST	Union-Find; sorting edges.
BREAKS WHEN	Directed graphs (needs arborescence algorithms) or when you need shortest PATHS not minimum total weight.
COMMON VARIATIONS	Kruskal (sort edges + DSU), Prim (heap).

An **MST** connects all vertices with `V-1` edges and minimum total weight. Two classic algorithms:

- **Kruskal:** sort edges; add an edge if it connects two different components (use DSU).
- **Prim:** grow a tree from one vertex, always adding the cheapest edge leaving the tree (heap).

SOLVED EXAMPLE 9.10 M

Kruskal's and Prim's Algorithm

First, the DSU structure (explained fully in 9.6):

```
struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) { iota(parent.begin(), parent.end(), 0); }
    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }   // path
    ↪ compression
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;                       // already connected
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a; sz[a] += sz[b];
        return true;
```

```
    }
};
```

Kruskal:

```
// edges: (weight, u, v)
long long kruskal(int n, vector<tuple<int,int,int>>& edges) {
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long total = 0;
    int used = 0;
    for (auto [w, u, v] : edges)
        if (dsu.unite(u, v)) { total += w; used++; }
    return used == n - 1 ? total : -1;                  // -1: graph not connected
}
```

Prim:

```
long long prim(int n, const vector<vector<pair<int,int>>>& adj) {
    vector<bool> inTree(n, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;   // (weight, node)
    pq.push({0, 0});
    long long total = 0;
    int taken = 0;
    while (!pq.empty() && taken < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (inTree[u]) continue;
        inTree[u] = true; total += w; taken++;
        for (auto [v, wt] : adj[u]) if (!inTree[v]) pq.push({wt, v});
    }
    return taken == n ? total : -1;
}
```

Trace on 4 vertices with edges `0-1 (1)`, `2-3 (1)`, `1-2 (2)`, `0-2 (3)`. *Kruskal:* sorted edges are `0-1 (1)`, `2-3 (1)`, `1-2 (2)`, `0-2 (3)`. Take `0-1` (different components), take `2-3`, take `1-2` (it joins `{0,1}` with `{2,3}`), skip `0-2` (already connected). Total `1 + 1 + 2 = 4` using `V - 1 = 3` edges. *Prim from vertex 0:* take 0 (the heap gets `(1,1)` and `(3,2)`); pop `(1,1)`: take 1, total 1 (the heap gets `(2,2)`); pop `(2,2)`: take 2, total 3 (the heap gets `(1,3)`); pop `(1,3)`: take 3, total 4. Same answer. Both rely on the **cut property**: for any split of the vertices into two groups, the cheapest edge crossing the split belongs to some MST, so taking it greedily never hurts. Kruskal costs $\mathcal{O}(E \log E)$ (the sort); Prim with a heap costs $\mathcal{O}((V + E) \log V)$.

Connecting Cities With Minimum Cost is exactly this MST. **Optimize Water Distribution in a Village** adds a virtual node 0 connected to each house `i` with edge weight `wells[i]`, then runs MST.

9.6 Pattern: Union-Find (Disjoint Set Union)

PATTERN CARD

SPOT IT WHEN	Connectivity as edges arrive, grouping, 'are a and b connected?'.
KNOW FIRST	Parent array; path compression; union by rank/size.
BREAKS WHEN	Edges are removed over time (offline reverse, or other structures).
COMMON VARIATIONS	Components count, redundant edge, accounts merge, Kruskal engine.

Purpose: maintain groups under "merge" and "are these in the same group?" queries in nearly $\mathcal{O}(1)$ each. Two optimizations: **path compression** and **union by size**, shown in the DSU above.

Trace with 5 elements (`parent = [0,1,2,3,4]`). `unite(0,1)`: roots 0 and 1 have equal size, so `parent[1] = 0`, `sz[0] = 2`. `unite(2,3)`: `parent[3] = 2`, `sz[2] = 2`. `unite(1,3)`: `find(1) = 0`, `find(3) = 2`, equal sizes, so `parent[2] = 0`, `sz[0] = 4`; now `parent = [0,0,0,2,4]`. A later `find(3)` follows `3 -> 2 -> 0` and **compresses** the path by setting `parent[3] = 0`, so the next `find(3)` takes one step. Union by size hangs the smaller tree under the larger, which keeps every tree shallow (height at most `log2 n`); together with path compression the amortized cost per operation is the inverse Ackermann function `alpha(n)`, below 5 for any input you will meet. `unite` returns `false` when both elements already share a root: that is exactly the "this edge closes a cycle" signal used by Kruskal and by Redundant Connection.

SOLVED EXAMPLE 9.11 M

Redundant Connection

Problem: a tree with one extra edge added; find the edge that creates a cycle.

Idea: add edges one by one; the first edge whose endpoints are **already connected** closes a cycle.

```
vector<int> findRedundantConnection(vector<vector<int>>& edges) {
    DSU dsu(edges.size() + 1);
    for (auto& e : edges)
        if (!dsu.unite(e[0], e[1])) return e;
    return {};
}
```

DFS vs. DSU (concept): DFS finds components of a *static* graph in $\mathcal{O}(V+E)$. DSU shines when edges **arrive over time** or when you only need connectivity answers, not paths.

Practice Set 9.6

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Union-Find* (*Disjoint Set Union*) idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Disjoint Set Union - Introduction E	Implement `find` and `unite` as above and test a few unions.
Regions Cut by Slashes M	Split each cell into 4 triangles; unite according to `/`, `\` or blank; count components.
Sentence Similarity II M	Unite words from similar pairs (map words to ids); compare words pairwise by `find`.
Satisfiability of Equality Equations M	First unite all `==` pairs; then check every `!=` pair is in different sets.
Smallest String With Swaps M	Unite swap-connected indices; within each component sort the letters and place them back in sorted index order.
Most Stones Removed with Same Row or Column M	Unite stones sharing a row or column (treat rows/columns as nodes); answer = stones - components.
Minimum Swaps to Make Two Arrays Identical M	Think of values as nodes and mismatches as edges; components of size `s` need `s-1` swaps.
Redundant Connection II H	Directed variant: handle the case of a node with two parents first, then cycle detection with DSU.

Problem	Level 2 hint (approach)
Minimize Malware Spread H	DSU components with sizes; count infected nodes per component; remove an infected node that is the only one in its component with the largest size.
Graph Connectivity With Threshold H	For each divisor `d > threshold`, unite all multiples of `d` (sieve-style); answer queries with `find`.

9.7 Pattern: SCC, Articulation Points & Euler Circuits

PATTERN CARD

SPOT IT WHEN	'Strongly connected', 'critical node/bridge', 'Eulerian path'.
KNOW FIRST	DFS timestamps (tin/low); transpose graph.
BREAKS WHEN	Using undirected-graph rules (low-link) on directed graphs and vice versa.
COMMON VARIATIONS	Kosaraju/Tarjan, articulation points, bridges, Hierholzer.

SOLVED EXAMPLE 9.12 H

Strongly Connected Components (Kosaraju)

An **SCC** is a maximal set of vertices where every vertex can reach every other.

Algorithm:

1. DFS on the original graph; push vertices to a stack in **finishing order**.
2. Reverse all edges.
3. Pop vertices from the stack; DFS on the **reversed** graph. Each DFS tree is one SCC.

```
void dfs1(int u, const vector<vector<int>>& g, vector<bool>& vis, vector<int>& order) {
    vis[u] = true;
    for (int v : g[u]) if (!vis[v]) dfs1(v, g, vis, order);
    order.push_back(u);
}
void dfs2(int u, const vector<vector<int>>& rg, vector<bool>& vis, vector<int>& comp) {
    vis[u] = true; comp.push_back(u);
    for (int v : rg[u]) if (!vis[v]) dfs2(v, rg, vis, comp);
}
vector<vector<int>> kosaraju(int n, const vector<vector<int>>& g) {
    vector<int> order;
    vector<bool> vis(n, false);
    for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i, g, vis, order);
    vector<vector<int>> rg(n);
    for (int u = 0; u < n; u++) for (int v : g[u]) rg[v].push_back(u);
    fill(vis.begin(), vis.end(), false);
    vector<vector<int>> sccs;
    for (int i = n - 1; i >= 0; i--) {
        int u = order[i];
        if (!vis[u]) { vector<int> comp; dfs2(u, rg, vis, comp); sccs.push_back(comp); }
    }
    return sccs;
}
```

Trace on edges `0->1, 1->2, 2->0, 1->3, 3->4, 4->3` (two SCCs: `{0,1,2}` and `{3,4}`). *Pass 1*, DFS from 0: `0 -> 1 -> 2` (2's edge back to 0 is already visited), so `2` finishes first; then from 1 go `3 -> 4`: `4` finishes,

3 finishes, 1 finishes, 0 finishes. Finish order [2,4,3,1,0]. Reverse every edge: 1->0, 2->1, 0->2, 3->1, 4->3, 3->4. *Pass 2*, take vertices from the END of the finish order. Start at 0: the DFS on the reversed graph reaches 2 and 1, giving the component {0,2,1}; it cannot reach 3 or 4, because the only edge between the groups, 3->1, points from {3,4} into {0,1,2}. The next unvisited vertex from the end is 3, which reaches 4: component {3,4}. **Why it works:** the vertex that finishes last in pass 1 belongs to a "source" component (nothing outside can reach into it); in the reversed graph no edge leaves that component towards unvisited vertices, so the second DFS collects exactly one SCC. Then the next-latest finisher is a source of what remains, and so on.

SOLVED EXAMPLE 9.13 H

Articulation Points

An **articulation point** is a vertex whose removal disconnects the graph.

Idea: DFS with discovery time tin[u] and low[u] (the earliest discovery time reachable from u's subtree using at most one back edge). A non-root u is an articulation point if some child v has low[v] >= tin[u] (the child's subtree cannot bypass u). The root is one if it has 2 or more DFS children.

```
void apDFS(int u, int parent, const vector<vector<int>>& adj, vector<int>& tin, vector<int>&
↪  low,
           vector<bool>& isAP, int& timer) {
    tin[u] = low[u] = ++timer;
    int children = 0;
    for (int v : adj[u]) {
        if (v == parent) continue;
        if (tin[v]) low[u] = min(low[u], tin[v]);                // back edge
        else {
            apDFS(v, u, adj, tin, low, isAP, timer);
            low[u] = min(low[u], low[v]);
            if (parent != -1 && low[v] >= tin[u]) isAP[u] = true;
            children++;
        }
    }
    if (parent == -1 && children > 1) isAP[u] = true;
}
vector<int> articulationPoints(const vector<vector<int>>& adj) {
    int n = adj.size(), timer = 0;
    vector<int> tin(n, 0), low(n, 0);
    vector<bool> isAP(n, false);
    for (int i = 0; i < n; i++) if (!tin[i]) apDFS(i, -1, adj, tin, low, isAP, timer);
    vector<int> res;
    for (int i = 0; i < n; i++) if (isAP[i]) res.push_back(i);
    return res;
}
```

Trace on edges 0-1, 1-2, 2-3, 3-1 (a tail 0 hanging off the triangle 1-2-3). DFS from 0: tin[0] = 1; go to 1 (tin[1] = 2), then 2 (tin[2] = 3), then 3 (tin[3] = 4). Vertex 3 sees its neighbor 1, already visited and not its parent: a back edge, so low[3] = min(4, tin[1] = 2) = 2. Back in 2: low[2] = 2; test low[3] >= tin[2]? 2 >= 3 is false. Back in 1: low[1] = 2; test for child 2: low[2] = 2 >= tin[1] = 2 is true, so **1 is an articulation point**: the subtree below it cannot reach anything above it without passing through 1 (removing 1 cuts vertex 0 off from the triangle). The root 0 has only one DFS child, so it is not an articulation point. In words: low[v] >= tin[u] means "the subtree of v cannot climb above u".

Bridges (edges whose removal disconnects the graph) use the same DFS with the stricter test low[v] > tin[u] ("cannot even reach u itself by another route"). Here the edge 0-1 is a bridge (low[1] = 2 > tin[0] = 1) and the triangle edges are not.

Euler Circuits: the Rule

- **Undirected graph:** an Euler circuit exists iff every vertex has **even degree** and all edges are in one connected component.
- **Directed graph:** exists iff every vertex has **in-degree == out-degree** and all edges are in one strongly connected component.

An Euler **path** (uses every edge once but need not return to the start) exists in a connected undirected graph iff exactly 0 or 2 vertices have odd degree; with 2 it must start at one of them and end at the other. **Hierholzer's algorithm** builds the circuit or path itself: walk unused edges until you get stuck (you are back at the start of a closed loop), then splice in further loops from vertices on the route (Reconstruct Itinerary in 9.9 is the classic use).

Practice Set 9.7

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *SCC, Articulation Points & Euler Circuits* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Euler Circuit in an Undirected Graph M	Check degrees and connectivity (ignoring isolated vertices) using the rule above.
Euler Circuit in a Directed Graph M	Check in-degree equals out-degree for all vertices, plus connectivity.
Doctor Strange M	Reduce the statement to a graph property (components, cycles, or Euler/degree condition) and apply the matching tool.

9.8 Pattern: Max Flow & Bipartite Matching

PATTERN CARD

SPOT IT WHEN	'Maximum assignments / throughput / minimum cut'.
KNOW FIRST	BFS for augmenting paths; residual graph.
BREAKS WHEN	Capacities are huge and graph dense (use Dinic); problem has no flow modelling.
COMMON VARIATIONS	Edmonds-Karp, Dinic, bipartite matching as flow.

Max flow: how much "stuff" can be pushed from a source `s` to a sink `t` through edges with capacities. **Ford-Fulkerson** repeatedly finds an augmenting path and pushes flow along it. **Edmonds-Karp** chooses the shortest augmenting path using BFS, guaranteeing $\mathcal{O}(V \cdot E^2)$.

SOLVED EXAMPLE 9.14 H

Edmonds-Karp (Max Flow)

```
int maxFlow(vector<vector<int>> cap, int s, int t) {          // cap = residual capacity matrix
↪  (copy)
    int n = cap.size(), flow = 0;
    while (true) {
        vector<int> parent(n, -1);
```

```
        parent[s] = s;
        queue<int> q; q.push(s);
        while (!q.empty() && parent[t] == -1) {              // BFS for an augmenting path
            int u = q.front(); q.pop();
            for (int v = 0; v < n; v++)
                if (parent[v] == -1 && cap[u][v] > 0) { parent[v] = u; q.push(v); }
        }
        if (parent[t] == -1) break;                          // no more augmenting paths
        int push = INT_MAX;
        for (int v = t; v != s; v = parent[v]) push = min(push, cap[parent[v]][v]);
        for (int v = t; v != s; v = parent[v]) {
            cap[parent[v]][v] -= push;
            cap[v][parent[v]] += push;                       // reverse edge allows undoing
        }
        flow += push;
    }
    return flow;
}
```

Trace. Source `0`, sink `3`; capacities `0->1: 3`, `0->2: 2`, `1->3: 2`, `2->3: 3`, `1->2: 1`. BFS finds the shortest augmenting path `0->1->3`; its bottleneck is `min(3, 2) = 2`, so push 2 (flow 2): residual `0->1` becomes 1, `1->3` becomes 0, and the reverse edges `1->0` and `3->1` gain 2. Next path `0->2->3`, bottleneck `min(2, 3) = 2` (flow 4). Next path `0->1->2->3`, bottleneck `min(1, 1, 1) = 1` (flow 5). Now both edges out of the source are used up, no augmenting path remains, and the answer is **5**. **Why reverse edges?** Pushing flow along `u->v` also creates residual capacity `v->u`; a later path may use it to cancel part of an earlier decision. That is how the algorithm repairs a greedy choice instead of getting stuck.

SOLVED EXAMPLE 9.15 H

Maximum Bipartite Matching

Reduce to flow: source connects to every left node (capacity 1), left to right edges (capacity 1), every right node to sink (capacity 1). The max flow equals the maximum matching. A direct "Kuhn's algorithm" is shorter:

```
bool tryMatch(int u, const vector<vector<int>>& adj, vector<bool>& seen, vector<int>& matchR) {
    for (int v : adj[u]) {
        if (seen[v]) continue;
        seen[v] = true;
        if (matchR[v] == -1 || tryMatch(matchR[v], adj, seen, matchR)) {   // free, or can
        ↪ re-route
            matchR[v] = u;
            return true;
        }
    }
    return false;
}
int maxMatching(int L, int R, const vector<vector<int>>& adj) {     // adj[u] = right nodes for
↪ left u
    vector<int> matchR(R, -1);
    int result = 0;
    for (int u = 0; u < L; u++) {
        vector<bool> seen(R, false);
        if (tryMatch(u, adj, seen, matchR)) result++;
    }
```

```
    return result;
}
```

Trace. Left nodes `0,1,2`, right nodes `0,1,2`, with `adj[0] = {0,1}`, `adj[1] = {0}`, `adj[2] = {1,2}`. `u = 0`: right 0 is free, so match `0-0`. `u = 1`: its only choice, right 0, is held by left 0, so we ask left 0 to move: `tryMatch(0)` skips the already-seen right 0, finds right 1 free and takes it; now left 0 holds right 1 and left 1 takes right 0. `u = 2`: right 1 is held by left 0, so we ask left 0 to move again, but all its options are already `seen` or cannot be re-routed, so that attempt fails; the next choice, right 2, is free: match `2-2`. The matching has size 3. The `seen` array is reset for every left node; it stops the search from walking around the same alternating path twice. Cost: $\mathcal{O}(V \cdot E)$.

9.9 Advanced / Miscellaneous Graph Problems

Problem	Level 2 hint (approach)
Castle Run M	Grid shortest path with obstacles or limited moves; BFS with `(cell, extra state)` visited.
Reconstruct Itinerary H	Euler **path** with lexicographic order: Hierholzer's algorithm with neighbors in a min-heap/multiset; output reversed post-order.
Largest Color Value in a Directed Graph H	Topological order with `dp[node][color]` = max count of that color on a path ending at node; a cycle means `-1`.
Cracking the Safe H	De Bruijn sequence: DFS on edges (strings of length `n-1` as nodes), visit each edge once, Hierholzer-style.
Bricks Falling When Hit H	Reverse time: remove all hit bricks, then add them back with DSU, counting how many bricks newly connect to the top.
Remove Max Number of Edges to Keep Graph Fully Traversable H	Two DSUs (Alice, Bob). Add type-3 edges first, then types 1 and 2; count edges used; check both connected.

CHAPTER COVERAGE

all 61 Graph problems appear above.

CHAPTER 9 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 9.1 BFS/DFS, 9.3 Number of Islands.
2. **Variations:** Ex 9.4 Rotting Oranges (multi-source), Problem 9B.
3. **Combined:** Ex 9.6 Topological Sort, 9.7 Dijkstra, 9.11 Redundant Connection.
4. **Independent recognition:** Problem 9A (graph or not?).
5. **Challenge:** Ex 9.5 Word Ladder, 9.12 SCC, 9.13 Articulation Points, 9.14 Max Flow.

Compare and Contrast

	BFS	DFS
Structure	queue	stack / recursion
Finds	shortest path in unweighted graphs	any path, components, cycles, ordering
Memory	$\mathcal{O}(\text{width})$	$\mathcal{O}(\text{depth})$

	Dijkstra	Bellman-Ford
Edge weights	non-negative only	negatives allowed (detects negative cycles)
Time	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(VE)$
Use for	most shortest-path tasks	negatives, 'at most K edges'

Why It Works (Correctness Corner)

BFS distance by induction on levels. All nodes at distance 0 (the source) are labelled correctly. If every node at distance `d` is labelled `d` when it is first dequeued, then their unvisited neighbours have distance exactly `d + 1` and are labelled so when first discovered. Hence the first label a node gets is its shortest distance.

Dijkstra. With non-negative weights, the node with the smallest tentative distance in the heap has a final distance: any other route to it passes through a node at least as far away and can only add non-negative weight.

Debug Lab: Marking visited too late

```
void dfs(vector<vector<char>>& g, int r, int c) {
    if (r < 0 || c < 0 || r >= (int)g.size() || c >= (int)g[0].size() || g[r][c] != '1') return;
    dfs(g, r + 1, c);  dfs(g, r - 1, c);
    dfs(g, r, c + 1);  dfs(g, r, c - 1);
    g[r][c] = '0';                                  // <- too late
}
```

Your task: What happens on the grid `[['1','1']]`?

ANSWER

Cell A recurses into its neighbour B; B recurses straight back into A, which is still '1' because it is only cleared at the very end: infinite recursion (stack overflow). Mark the cell visited *before* recursing.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 9A E

Is There a Path?

Statement. An undirected graph has `n` nodes (0..n-1) and a list of edges. Decide whether there is a path from `src` to `dst`.

Example. `n = 3, edges = [[0,1],[1,2],[2,0]], src = 0, dst = 2` → `true`

Constraints. $1 \le n \le 2 \times 10^5$; edges up to 2×10^5.

Expected complexity. $\mathcal{O}(E \log n)$ time with path compression alone ($\mathcal{O}(E\,\alpha(n))$ if you also union by size, as in Example 9.10), $\mathcal{O}(n)$ space.

Prerequisites. 9.6 Union-Find

HINT 1 which idea is relevant?

Do you need the path itself, or only whether one exists?

HINT 2 outline of the approach

Connectivity questions on a fixed edge list are the specialty of one structure.

HINT 3 the key step

Union every edge's endpoints; answer `find(src) == find(dst)`.

SOLUTION and editorial (try first, then compare)

```cpp
struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }   // path compression
    void unite(int a, int b) { p[find(a)] = find(b); }
};
bool validPath(int n, const vector<vector<int>>& edges, int src, int dst) {
    DSU d(n);
    for (const auto& e : edges) d.unite(e[0], e[1]);
    return d.find(src) == d.find(dst);
}
```

BFS/DFS also work in $\mathcal{O}(V + E)$, but union-find has the shortest code and shines when edges arrive one by one. Path compression alone already keeps `find` cheap ($\mathcal{O}(\log n)$ amortized); the `DSU` of Example 9.10 adds union by size, which brings it down to nearly constant.

PROBLEM 9B M

Shortest Path in a Binary Matrix

Statement. In an `n x n` binary grid, find the length (number of cells) of the shortest clear path from the top-left to the bottom-right cell, moving in 8 directions through cells that are 0. Return -1 if none exists.

Example. `[[0,1],[1,0]]` → `2`

Constraints. $1 \leq n \leq 100$.

Expected complexity. $\mathcal{O}(n^2)$ time and space.

Prerequisites. 9.2 Grid BFS

HINT 1 which idea is relevant?

All moves cost the same. What does 'shortest' suggest for the algorithm?

HINT 2 outline of the approach

BFS gives shortest paths when every edge has equal weight.

HINT 3 the key step

Mark a cell visited when you push it, store the path length in the queue, and loop over all 8 neighbours.

SOLUTION and editorial (try first, then compare)

```cpp
int shortestPathBinaryMatrix(vector<vector<int>> g) {
    int n = g.size();
    if (g[0][0] || g[n - 1][n - 1]) return -1;
    queue<array<int, 3>> q;                              // {row, col, path length so far}
    q.push({0, 0, 1});
    g[0][0] = 1;                                         // mark visited when pushed
    while (!q.empty()) {
        auto [r, c, d] = q.front(); q.pop();
        if (r == n - 1 && c == n - 1) return d;
        for (int dr = -1; dr <= 1; dr++)
            for (int dc = -1; dc <= 1; dc++) {
                int nr = r + dr, nc = c + dc;
                if (nr < 0 || nc < 0 || nr >= n || nc >= n || g[nr][nc]) continue;
                g[nr][nc] = 1;
                q.push({nr, nc, d + 1});
            }
    }
    return -1;
}
```

Marking at push time guarantees each cell is queued once. Check the start and end cells for blocked values first; a 1x1 grid with `0` has answer 1.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T9.1 Several people start a rumor at the same time in a grid; each hour it spreads to the four neighbours. How many hours until everyone knows?

PATTERN

Multi-source BFS.

T9.2 Tasks have prerequisites; can all be completed?

> **PATTERN**
> Topological sort: finished count == n means no cycle.

Mixed Practice

Combine 9.4 with a stop limit: 'cheapest flight with at most K stops'. Why can plain Dijkstra with a visited array give a wrong answer here, and what does Bellman-Ford's 'K rounds' do differently?

Revision Sheet

Key takeaways

- Mark visited when you push, not when you pop.
- Equal weights → BFS; non-negative weights → Dijkstra; negatives → Bellman-Ford.
- Topological order exists iff the graph is acyclic.
- Union-find answers connectivity almost in $\mathcal{O}(1)$.

Pattern-recognition clues

- 'Minimum steps', unweighted → BFS.
- 'Prerequisites' → topological sort.
- 'Connect all, cheapest' → MST.
- 'Merge groups' → DSU.

Common mistakes

- Missing visited marking → infinite loop.
- Dijkstra with negative edges.
- Forgetting both directions in undirected graphs.
- Using `int` for distances that overflow.

Complexity summary

Algorithm	Time
BFS / DFS	$\mathcal{O}(V + E)$
Dijkstra (heap)	$\mathcal{O}((V + E) \log V)$
Bellman-Ford	$\mathcal{O}(VE)$
Floyd-Warshall	$\mathcal{O}(V^3)$
Kruskal	$\mathcal{O}(E \log E)$
DSU op	~$\mathcal{O}(1)$ amortized

Recall questions (answer aloud, then check the chapter)

1. Why mark visited when pushing?
2. When is Dijkstra wrong?
3. How does Kahn's algorithm detect a cycle?
4. What two optimizations make union-find fast?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Without notes, model a new story problem (e.g. 'minimum moves for a knight') as a graph, state nodes/edges, choose the algorithm, and code it.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 10

HashMap & Heap

10

START HERE

'How many stretches of days had a net change of exactly zero?' Nested loops check every stretch in $\mathcal{O}(n^2)$. A hash map of running totals counts them in one pass. Hashing trades memory for time and shows up in most medium problems.

ANALOGY

A hash map is a coat-check desk: hand over the ticket (key), get the coat (value) back in one step. A heap is a pile where the top item is always the one you want next.

AFTER THIS CHAPTER YOU CAN

- ✓ Combine prefix sums with a hash map to count subarrays.
- ✓ Use frequency arrays/maps for anagram and window problems.
- ✓ Keep the k best items with a heap of size k.
- ✓ Maintain a running median with two heaps.

Picture it

Prefix sums + hash map: count subarrays with sum 3 in [1, 2, -3, 3]

```
prefix P: 0  1  3  0  3          map starts {0:1}
i=1 P=1: need P-3=-2 -> 0 found ; map {0:1, 1:1}
i=2 P=3: need 0      -> 1 found (subarray [1,2])         ; map {0:1, 1:1, 3:1}
i=3 P=0: need -3     -> 0 found ; map {0:2, 1:1, 3:1}
i=4 P=3: need 0      -> 2 found ([3] and [1,2,-3,3])    ; total = 3
```

10.0 Concept

Hash map (`unordered_map`): store key-value pairs with $\mathcal{O}(1)$ average lookup. It trades memory for speed, and is the most common way to turn an $\mathcal{O}(n^2)$ solution into $\mathcal{O}(n)$.

Heap (`priority_queue`): always gives you the largest (or smallest) element in $\mathcal{O}(\log\ n)$. It is the right tool when you repeatedly need "the best so far."

Pattern	Recognition clue
HashMap + prefix sum	"number of subarrays with sum / equal 0s and 1s / divisible by k"
Frequency counting	anagrams, duplicates, "same characters"
Sliding window + map	"longest/shortest substring with condition"
Top-K with heap	"k largest/smallest/closest/most frequent"

Pattern	Recognition clue
Two heaps	running median
Design with map + array	$\mathcal{O}(1)$ insert/delete/random

10.1 Pattern: HashMap + Prefix Sum for Subarrays

PATTERN CARD

SPOT IT WHEN	'Number of subarrays with sum / balance equal to K'.
KNOW FIRST	Prefix sums (3.4).
BREAKS WHEN	Sliding window would work only for non-negative values; here negatives force the map.
COMMON VARIATIONS	Sum = K, divisible by K, equal 0s and 1s (map 0 → -1), longest such subarray (store first index).

Key identity: `sum(i+1..j) = pre[j] - pre[i]`. So the number of subarrays ending at `j` with sum `k` equals the number of earlier prefixes equal to `pre[j] - k`. Store how many times each prefix sum has occurred.

SOLVED EXAMPLE 10.1 — M

Number of Subarrays with Sum Exactly K

```cpp
int subarraySum(vector<int>& a, int k) {
    unordered_map<int,int> seen;          // prefix sum -> how many times
    seen[0] = 1;                          // empty prefix
    int pre = 0, count = 0;
    for (int x : a) {
        pre += x;
        if (seen.count(pre - k)) count += seen[pre - k];
        seen[pre]++;
    }
    return count;
}
```

Why not sliding window? Negative numbers break the "grow/shrink" logic: adding an element can make the sum smaller, so you can no longer tell which end to move. The hash map handles them. (If the values can be large, declare `pre` as `long long`; the sum of 10^5 values near 10^9 overflows `int`.)

SOLVED EXAMPLE 10.2 — M

Subarray with Equal Number of 0s and 1s

Trick: replace every `0` by `-1`. Now "equal 0s and 1s" means "sum is 0." We look for a repeated prefix sum, and the longest such subarray comes from the **first** occurrence.

```cpp
int longestEqual01(vector<int>& a) {
    unordered_map<int,int> firstIdx;      // prefix sum -> earliest index
    firstIdx[0] = -1;
    int pre = 0, best = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        pre += (a[i] == 0) ? -1 : 1;
        if (firstIdx.count(pre)) best = max(best, i - firstIdx[pre]);
```

```
        else firstIdx[pre] = i;
    }
    return best;
}
```

Trace with `a = [1,0,0,1,1,0]` (mapped to `+1,-1,-1,+1,+1,-1`). Running sums: `1, 0, -1, 0, 1, 0`. With `firstIdx = {0: -1}`: at `i=0` sum 1 is new (store 0); at `i=1` sum 0 was seen at -1, so length `1 - (-1) = 2`; at `i=2` sum -1 is new; at `i=3` sum 0 again, length `3 - (-1) = 4`; at `i=4` sum 1 was first seen at 0, length 4; at `i=5` sum 0 again, length `5 - (-1) = 6`. The whole array has three 0s and three 1s, so the answer 6 is right. We store only the **first** index and never overwrite it, because an earlier start means a longer subarray.

For **0s, 1s and 2s**, track the pair `(count1 - count0, count2 - count1)` as the key (use a `map<pair<int, int>, int>`).

SOLVED EXAMPLE 10.3 M

Subarray Sums Divisible by K

Two prefixes with the same remainder mod `k` have a difference divisible by `k`. Count pairs of equal remainders.

```
int subarraysDivByK(vector<int>& a, int k) {
    vector<int> cnt(k, 0);
    cnt[0] = 1;
    int pre = 0, ans = 0;
    for (int x : a) {
        pre = ((pre + x) % k + k) % k;       // keep remainder non-negative
        ans += cnt[pre]++;
    }
    return ans;
}
```

Trace with `a = [4,5,0,-2,-3,1]`, `k = 5`. Remainders of the running sum: after 4 it is 4; after 5 it is 9, remainder 4; after 0 it is 4; after -2 it is 2; after -3 the sum is -1, and `((-1 % 5) + 5) % 5 = 4` (in C++ `-1 % 5` is `-1`, which is why we add `k` before taking the remainder again); after 1 the sum is 0. Each time the remainder was already seen `c` times we gain `c` new subarrays: `0 + 1 + 2 + 0 + 3 + 1 = 7` (the starting `cnt[0] = 1` is the empty prefix and contributes the last `+1`). This needs `k > 0`.

Practice Set 10.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *HashMap + Prefix Sum for Subarrays* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Count Pairs Whose Sum Is Divisible by K E	Count remainders; pair `r` with `k - r` (special-case `0` and `k/2`).
Substring with Equal Number of 0s, 1s and 2s M	Key = `(c1 - c0, c2 - c1)` in a map; count equal keys.
Length of Largest Subarray with Contiguous Elements M	A subarray with distinct elements is contiguous iff `max - min == length - 1`; check with a loop that tracks min/max.
Length of Largest Subarray with Contiguous Elements II M	Same, but with duplicates allowed: use a set to reject duplicates in the inner loop.

10.2 Pattern: Frequency Counting & Anagrams

PATTERN CARD

SPOT IT WHEN	'Anagram', 'same letters', 'frequency equal'.
KNOW FIRST	Counting array of 26 / map.
BREAKS WHEN	Unicode or huge alphabets (use map, not array[26]).
COMMON VARIATIONS	Sorted key, count-array key, sliding frequency compare.

Idea: two strings are anagrams iff they have the same character counts. Use `int freq[26]` for lowercase letters, or `unordered_map` otherwise. A sorted string or a count signature can serve as a **canonical key**.

SOLVED EXAMPLE 10.4 — M

Group Anagrams

```
vector<vector<string>> groupAnagrams(vector<string>& words) {
    unordered_map<string, vector<string>> groups;
    for (auto& w : words) {
        string key = w;
        sort(key.begin(), key.end());            // all anagrams share the same sorted form
        groups[key].push_back(w);
    }
    vector<vector<string>> res;
    for (auto& [key, g] : groups) res.push_back(g);
    return res;
}
```

SOLVED EXAMPLE 10.5 — M

Longest Consecutive Sequence

Problem: longest run of consecutive integers (order in the array does not matter), in $\mathcal{O}(n)$.

Insight: put everything in a set. Only start counting from a number `x` where `x - 1` is **not** in the set (the start of a run). Every element is then visited at most twice overall.

```
int longestConsecutive(vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;            // not the start of a run
        int len = 1;
        while (s.count(x + len)) len++;
        best = max(best, len);
    }
    return best;
}
```

SOLVED EXAMPLE 10.6 M

Find All Anagrams in a String

Approach: a fixed-size sliding window of length `|p|`, maintaining a count array and comparing with `p`'s counts.

```
vector<int> findAnagrams(string s, string p) {
    vector<int> res;
    if (s.size() < p.size()) return res;
    vector<int> need(26, 0), have(26, 0);
    for (char c : p) need[c - 'a']++;
    for (int i = 0; i < (int)s.size(); i++) {
        have[s[i] - 'a']++;
        if (i >= (int)p.size()) have[s[i - p.size()] - 'a']--;     // slide the window
        if (have == need) res.push_back(i - (int)p.size() + 1);
    }
    return res;
}
```

Trace with `s = "cbaebabacd"`, `p = "abc"` (`need = a1 b1 c1`). The window of length 3 is `cba` at `i = 2`: counts equal, so record start `0`. It slides to `bae`, `aeb`, `eba`, `bab`, `aba` (none match), then `bac` (starts at 6, a match, found when `i = 8`), then `acd` (no match). Answer `[0, 6]`. Comparing two 26-element vectors costs 26 steps, so the total is $\mathcal{O}(26 \cdot n)$, which is $\mathcal{O}(n)$.

Practice Set 10.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Frequency Counting & Anagrams* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Anagram Palindrome Check E	A permutation is a palindrome iff at most one character has an odd count.
Find Anagram Mappings E	Map each value in `B` to its index; look up each element of `A`.
Isomorphic Strings E	Two maps (`s -> t`, `t -> s`) must stay consistent.
X of a Kind in a Deck of Cards E	Count frequencies; answer is true iff `gcd` of all counts is at least 2.
K-Anagrams M	Two strings are k-anagrams if after matching counts, the number of unmatched characters of one is at most `k`.
Rabbits in a Forest M	For answer `x`, groups of size `x + 1`: `ceil(count / (x+1)) * (x+1)` per distinct answer.
Array of Doubled Pairs M	Sort by absolute value; greedily match each `x` with `2x`, decrementing counts.

10.3 Pattern: Sliding Window with HashMap

PATTERN CARD

SPOT IT WHEN	Longest/shortest substring with a distinct-characters or coverage condition.
KNOW FIRST	Variable-size sliding window.
BREAKS WHEN	Condition cannot be restored by shrinking from the left.
COMMON VARIATIONS	Longest without repeats, at most K distinct, minimum window cover.

Use the template from Chapter 3 (grow `r`, shrink `l` while invalid), keeping a map of counts as the window state.

SOLVED EXAMPLE 10.7 — M

Longest Substring Without Repeating Characters

```
int lengthOfLongestSubstring(const string& s) {
    vector<int> lastSeen(256, -1);              // last index of each character
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (lastSeen[(unsigned char)s[r]] >= l) l = lastSeen[(unsigned char)s[r]] + 1;   //
        ↪  jump past duplicate
        lastSeen[(unsigned char)s[r]] = r;
        best = max(best, r - l + 1);
    }
    return best;
}
```

SOLVED EXAMPLE 10.8 — H

Smallest Window Containing All Characters of Another String

Idea: expand `r` until the window contains every needed character (tracked by a `missing` counter), then shrink `l` as far as possible while still valid; record the smallest valid window.

```
string minWindow(const string& s, const string& t) {
    if (t.empty() || s.size() < t.size()) return "";   // nothing to cover / impossible
    vector<int> need(256, 0);                          // index by unsigned char: safe for any byte
    for (unsigned char c : t) need[c]++;
    int missing = t.size(), l = 0, bestLen = INT_MAX, bestStart = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (need[(unsigned char)s[r]]-- > 0) missing--;       // this char was still needed
        while (missing == 0) {                        // window valid: try shrinking
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; bestStart = l; }
            if (++need[(unsigned char)s[l]] > 0) missing++;   // we are removing a needed char
            l++;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
}
```

Trace with `s = "ADOBECODEBANC"`, `t = "ABC"`. `need` starts as A1 B1 C1, `missing = 3`. Grow `r` until `missing = 0`: the window `ADOBEC` (indices 0..5) is the first valid one, length 6. Shrink from the left: removing `A` makes `missing = 1`, so stop shrinking and grow again. At `r = 10` (the second `A`) the window is valid again and

shrinks down to `CODEBA` (length 6, not better than 6). At `r = 12` (the last `C`) it is valid again and shrinks down to `BANC` (indices 9..12), length 4, the minimum. The trick is that `need[c]` goes **negative** for surplus characters, so `++need[s[l]] > 0` is true only when we remove a character the window genuinely still required. The `l` pointer only moves right, so the whole loop is O(|s| + |t|).

Practice Set 10.3

BEFORE OPENING THE HINTS
try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Sliding Window with HashMap* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Same Frequency After One Removal M	Count frequencies, then count frequencies of frequencies; check the few valid shapes (all 1s, one char with freq 1, one char with freq+1).
Rearrange String K Distance Apart M	Max-heap by frequency + a cooldown queue of size `k`.
Smallest Subarray with All Occurrences of the Most Frequent Element M	Track `first[x]`, `last[x]`, `count[x]`; among max-frequency values pick min `last - first + 1`.

10.4 Pattern: Top-K & Heap Basics

PATTERN CARD

SPOT IT WHEN	'K largest / smallest / closest / most frequent'.
KNOW FIRST	priority_queue; min-heap vs max-heap.
BREAKS WHEN	k is close to n (sorting may be simpler), or you need the exact order of ALL elements.
COMMON VARIATIONS	Heap of size k, quickselect, bucket sort by frequency, merge k lists.

C++ heap facts: `priority_queue<int>` is a **max**-heap. For a min-heap: `priority_queue<int, vector<int>, greater<int>>`.

Top-K rule of thumb: to find the **K largest**, keep a **min**-heap of size K. When it grows past K, pop the smallest. At the end the heap holds the K largest. Cost $\mathcal{O}(n \log K)$.

SOLVED EXAMPLE 10.9 M

K Closest Points to Origin

Keep a **max**-heap of size `k` ordered by distance. If a new point is closer than the farthest in the heap, replace it.

```
vector<vector<int>> kClosest(vector<vector<int>>& pts, int k) {
    priority_queue<pair<int, int>> pq;                    // (squared distance, index), max-heap
    for (int i = 0; i < (int)pts.size(); i++) {
        int d = pts[i][0] * pts[i][0] + pts[i][1] * pts[i][1];
        pq.push({d, i});
        if ((int)pq.size() > k) pq.pop();                  // drop the farthest
    }
    vector<vector<int>> res;
    while (!pq.empty()) { res.push_back(pts[pq.top().second]); pq.pop(); }
```

```
    return res;
}
```

Note: squared distances fit in `int` only while coordinates stay below about 20,000 in absolute value; use `long long` for larger ranges. The heap is ordered by `(distance, index)` pairs, so ties are broken by index and the result is one valid set of `k` closest points (the order of the output is not meaningful).

SOLVED EXAMPLE 10.10 M

Insert Delete GetRandom O(1)

Idea: a `vector` gives $\mathcal{O}(1)$ random access, a hash map gives $\mathcal{O}(1)$ lookup of an element's index. To delete in $\mathcal{O}(1)$, **swap the element with the last one**, then pop.

```
class RandomizedSet {
    vector<int> vals;
    unordered_map<int,int> idx;                    // value -> position in vals
public:
    bool insert(int v) {
        if (idx.count(v)) return false;
        idx[v] = vals.size();
        vals.push_back(v);
        return true;
    }
    bool remove(int v) {
        auto it = idx.find(v);
        if (it == idx.end()) return false;
        int pos = it->second, last = vals.back();
        vals[pos] = last; idx[last] = pos;          // move last into the hole
        vals.pop_back(); idx.erase(v);
        return true;
    }
    int getRandom() { return vals[rand() % vals.size()]; }
};
```

With duplicates allowed (Insert Delete GetRandom $\mathcal{O}(1)$ Duplicates Allowed [H]): map each value to an `unordered_set<int>` of positions; remove one position by swapping with the last element and updating the position sets of both values.

Practice Set 10.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Top-K & Heap Basics* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Longest Consecutive 1's (After One Flip) E	Sliding window allowing at most one zero (equivalently, track `left` run and `right` run around each zero).

NOTE

the sheet files this row under *HashMap and Heap*, but it is really a sliding-window problem (see 10.3); no heap is needed. It is kept here so the chapter matches the sheet one-to-one. The Level 1 sentence above

does not apply to it.

10.5 Pattern: Two Heaps & Median Techniques

PATTERN CARD

SPOT IT WHEN	'Running median', 'balance two halves'.
KNOW FIRST	Heap basics.
BREAKS WHEN	Deletions from the middle (need multiset or lazy deletion).
COMMON VARIATIONS	Two heaps, sliding-window median, IPO-style selection.

SOLVED EXAMPLE 10.11 H

Median from Data Stream

Idea: keep the smaller half in a **max**-heap `lo` and the larger half in a **min**-heap `hi`, with sizes differing by at most 1. The median is the top of `lo` (odd count) or the mean of both tops (even count).

```
class MedianFinder {
    priority_queue<int> lo;                                  // max-heap: lower half
    priority_queue<int, vector<int>, greater<int>> hi;       // min-heap: upper half
public:
    void addNum(int x) {
        lo.push(x);
        hi.push(lo.top()); lo.pop();                         // balance across halves
        if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
    }
    double findMedian() {
        if (lo.size() > hi.size()) return lo.top();
        return (lo.top() + hi.top()) / 2.0;
    }
};
```

Trace adding `5, 2, 8`. Add 5: push to `lo`, move its top to `hi`, then `hi` is bigger than `lo`, so move it back: `lo = {5}`, `hi = {}`; median 5. Add 2: `lo = {5,2}`, move top 5 to `hi`: `lo = {2}`, `hi = {5}` (equal sizes); median `(2+5)/2 = 3.5`. Add 8: `lo = {2,8}`, move top 8 to `hi`: `lo = {2}`, `hi = {5,8}`; `hi` is bigger, so move 5 back: `lo = {2,5}`, `hi = {8}`; median 5 (top of the larger `lo`). Moving through `hi` first guarantees every element of `lo` is at most every element of `hi`. `findMedian` must only be called after at least one `addNum`.

Problem	Level 2 hint (approach)

Practice Set 10.5

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Two Heaps & Median Techniques* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Kth Smallest Prime Fraction H	Min-heap of `(arr[i]/arr[j])` starting with `i = 0` for each `j`; pop `k` times and push `(i+1, j)`. Or binary search on the fraction value.
Employee Free Time H	Merge all intervals (sort by start or heap); gaps between merged intervals are the free times.
Minimum Number of Refueling Stops H	Max-heap of fuel amounts of passed stations; when you cannot reach the next, refuel from the largest passed station.

10.6 Pattern: Miscellaneous HashMap Applications

PATTERN CARD

SPOT IT WHEN	$\mathcal{O}(1)$ average insert/delete/lookup with extra requirements.
KNOW FIRST	Hash functions; unordered_map complexity.
BREAKS WHEN	Adversarial inputs can degrade unordered_map (use custom hash) or you need ordered output (use map).
COMMON VARIATIONS	Random-set design, brick wall (count of aligned edges), group-by key.

SOLVED EXAMPLE 10.12 M

Brick Wall

Idea: the line crossing the fewest bricks passes through the most **gaps**. Count how many rows have a gap at each x-position (cumulative brick widths excluding the final edge). Answer = `rows - maxGapCount`.

```
int leastBricks(vector<vector<int>>& wall) {
    unordered_map<long long,int> gaps;
    int best = 0;
    for (auto& row : wall) {
        long long x = 0;
        for (int i = 0; i + 1 < (int)row.size(); i++) {   // skip the right edge
            x += row[i];
            best = max(best, ++gaps[x]);
        }
    }
    return (int)wall.size() - best;
}
```

Practice Set 10.6

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Miscellaneous HashMap Applications* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Island Perimeter E	For every land cell add 4, subtract 2 for each land neighbor to the right/down.
Mode of Frequencies E	Count values, then count how many values share each frequency; pick the most common frequency.
Pairs of Coinciding Points M	Hash map of points `(x, y)`, count pairs `c * (c - 1) / 2`; for a "same row/column but not both" variant, subtract duplicates.
Morning Assembly M	Usually mapping/frequency on heights/positions; read the exact rule and use `unordered_map` counts.
Smallest Number Whose Digit Product Equals a Given Number M	Divide by digits 9 down to 2 greedily; collect digits, sort ascending.
Line Reflection M	A reflection line exists at `(minX + maxX) / 2`; check each point's mirror `(minX + maxX - x, y)` is in the set.
Grid Illumination H	Maps for row, column, diagonal and anti-diagonal lamp counts + a set of lamps; for each query, check and then turn off the 3x3 neighborhood.
The Skyline Problem H	Sweep line: events (start, +height) and (end, -height); a `multiset` (or max-heap with lazy deletion) gives the current max height; report changes.

CHAPTER COVERAGE

all 39 HashMap and Heap problems appear above.

CHAPTER 10 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 10.4 Group Anagrams, 10.5 Longest Consecutive, Problem 10A.
2. **Variations:** Ex 10.1-10.3 prefix + map family.
3. **Combined:** Ex 10.6 Anagrams in String, 10.7 Longest Substring.
4. **Independent recognition:** Problem 10B (frequency + heap).
5. **Challenge:** Ex 10.8 Min Window, 10.11 Median from Data Stream.

Compare and Contrast

	Sorting	Heap of size k
Time (top k of n)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log k)$
Space	$\mathcal{O}(1)$-$\mathcal{O}(n)$	$\mathcal{O}(k)$
Best when	k close to n, or you need full order	k small, data streaming

	`unordered_map`	`map`
Lookup	$\mathcal{O}(1)$ average	$\mathcal{O}(\log n)$
Ordered iteration	no	yes
Pitfall	adversarial collisions	slower

Why It Works (Correctness Corner)

Prefix + map counting. Let `P[i]` be the sum of the first `i` elements. A subarray `(j, i]` has sum `K` iff `P[i] - P[j] = K`, i.e. `P[j] = P[i] - K`. For each `i`, the number of valid starts is the number of earlier prefixes equal to `P[i] - K`, which the map stores. Insert `P[i]` *after* querying (and start with `P[0] = 0`) so a subarray is never empty.

Debug Lab: Inserting before looking up

```
vector<int> twoSum(vector<int>& a, int target) {
    unordered_map<int,int> seen;
    for (int i = 0; i < (int)a.size(); i++) {
        seen[a[i]] = i;
        auto it = seen.find(target - a[i]);
        if (it != seen.end()) return {it->second, i};
    }
    return {};
}
```

Your task: What does `twoSum({3, 2}, 6)` return?

ANSWER

`{0, 0}`: the element pairs with itself, because `a[0]` was inserted before the lookup. Look up the complement first, then insert the current element.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 10A E

Two Sum

Statement. Return the indices of two different elements whose sum equals `target`. Exactly one solution exists.

Example. `[2,7,11,15], target = 9` → `[0,1]`

Constraints. $2 \leq n \leq 10^5$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(n)$ space.

Prerequisites. 1.6 Containers; Practice 3.2 contrast (sorted version)

HINT 1 which idea is relevant?

For each element, which single value would complete the pair?

HINT 2 outline of the approach

How can you answer 'have I seen that value, and where?' in $\mathcal{O}(1)$?

HINT 3 the key step

Store value → index in a hash map; look up `target - a[i]` before inserting `a[i]`.

SOLUTION and editorial (try first, then compare)

```cpp
vector<int> twoSum(const vector<int>& a, int target) {
    unordered_map<int, int> seen;                          // value -> index where we saw it
    for (int i = 0; i < (int)a.size(); i++) {
        auto it = seen.find(target - a[i]);                // look for the partner BEFORE
        ↪ inserting a[i]
        if (it != seen.end()) return {it->second, i};
        seen[a[i]] = i;
    }
    return {};
}
```

The brute force is $\mathcal{O}(n^2)$. The map turns the inner loop into one lookup. If the array were sorted, two pointers would use $\mathcal{O}(1)$ space.

PROBLEM 10B M

Top K Frequent Elements

Statement. Return the `k` most frequent values of an array (any order). The answer is guaranteed to be unique.

Example. `[1,1,1,2,2,3], k = 2` → `[1,2]`

Constraints. $1 \leq n \leq 10^5$; $1 \leq k \leq$ number of distinct values.

Expected complexity. Better than $\mathcal{O}(n \log n)$ when k is small: $\mathcal{O}(n \log k)$.

Prerequisites. 10.4 Top-K & Heap Basics

HINT 1 which idea is relevant?

First count. What structure then picks the best k?

HINT 2 outline of the approach

Keep a *min*-heap of size k keyed by frequency.

HINT 3 the key step

Push (freq, value); when the heap exceeds k, pop the smallest.

SOLUTION and editorial (try first, then compare)

```cpp
vector<int> topKFrequent(const vector<int>& a, int k) {
    unordered_map<int, int> cnt;
    for (int x : a) cnt[x]++;
    using P = pair<int, int>;                          // (frequency, value)
    priority_queue<P, vector<P>, greater<P>> pq;       // min-heap: smallest frequency on top
    for (const auto& [v, f] : cnt) {
        pq.push({f, v});
        if ((int)pq.size() > k) pq.pop();              // keep only the k most frequent
    }
    vector<int> out;
    while (!pq.empty()) { out.push_back(pq.top().second); pq.pop(); }
    return out;
}
```

The heap holds the k best seen so far, with the weakest on top ready to be evicted. Using a max-heap would force you to store everything. Bucket sort by frequency gives $\mathcal{O}(n)$ as an alternative.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T10.1 Count the periods of consecutive trading days whose net profit is exactly zero.

PATTERN

Prefix sum + hash map.

T10.2 Print the 3 most frequent words in a text.

PATTERN

Frequency map + size-3 heap.

Mixed Practice

Combine 3.3 and 10.3: 'longest substring with at most K distinct characters'. Which structure tracks the window's distinct count, and when do you shrink?

Revision Sheet

Key takeaways

- Hash map + prefix sum counts subarrays.
- Heap of size k beats a full sort for small k.

- Query before insert when pairing elements.
- `P[0] = 0` must be in the map.

Pattern-recognition clues

- 'Number of subarrays with sum K' → prefix + map.
- 'Anagram' → counts.
- 'k largest / most frequent' → heap.
- 'Median of a stream' → two heaps.

Common mistakes

- Forgetting the base prefix `P[0] = 0`.
- Negative modulo results in 'divisible by K'.
- Using `unordered_map` with a pair key without a hash.

Complexity summary

Task	Time	Space
Prefix + map	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Top-k with heap	$\mathcal{O}(n \log k)$	$\mathcal{O}(k)$
Two heaps median	$\mathcal{O}(\log n)$ per insert	$\mathcal{O}(n)$

Recall questions (answer aloud, then check the chapter)

1. Why is `P[0] = 0` needed?
2. When is sorting better than a heap?
3. Why query before insert in Two Sum?
4. How do two heaps give the median?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Solve 'subarray sum equals K' with negatives from scratch, explaining why a sliding window fails, then test all-negative and K = 0 inputs.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

PART III

Problem-Solving Paradigms

11 Greedy

12 Dynamic Programming

13 String Matching Algorithms

14 Backtracking

15 Bit Manipulation

16 Tries

CHAPTER 11

Greedy

11

START HERE

You have one meeting room and 100 requests. Which requests should you accept to fit the most meetings? Picking the *shortest* meetings sounds right, picking the *earliest-starting* sounds right, and both are wrong. Greedy algorithms are fast and short, but only if you can prove the local choice is safe.

ANALOGY

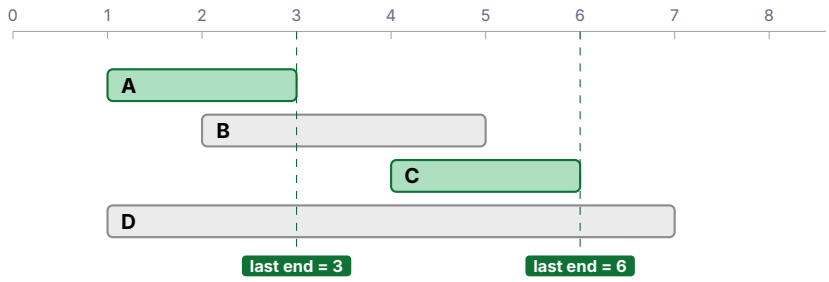
Packing a suitcase by always taking the item with the best value per kilo works for sand (fractional) but not for rigid boxes (0/1). Greedy works exactly when the pieces can be exchanged without loss.

AFTER THIS CHAPTER YOU CAN

- ✓ Decide whether a greedy choice is safe using an exchange argument.
- ✓ Sort intervals correctly for selection vs merging problems.
- ✓ Use a heap to repeatedly pick the best remaining item.
- ✓ Recognise when greedy fails and DP is needed.

PICTURE IT

Interval selection: sort by END time



keep: first interval

skip: starts at 2 < last end 3

keep: starts at 4 ≥ 3

skip: starts at 1 < last end 6

sorted by end: A(3) B(5) C(6) D(7). Answer: **2** intervals kept (A, C).

11.0 Concept

A **greedy** algorithm builds a solution step by step, always taking the choice that looks best *right now*, and never reconsiders. It is fast and simple, but only correct when a **greedy-choice property** holds.

How to Prove (or at Least Believe) a Greedy Works

Exchange argument: take any optimal solution that does not make your greedy choice; show you can swap in the greedy choice without making the solution worse.

Typical greedy setups:

Pattern	Typical first move
Interval scheduling	sort by **end time**
Deadline/profit scheduling	sort by profit descending, place at latest free slot
Pairing	sort and pair extremes (or neighbors)
Merge smallest	min-heap, repeatedly combine the two smallest (Huffman, ropes)
Ratio-based	sort by value/weight (fractional knapsack)

> **WARNING**
> If a greedy choice fails on a tiny counterexample, switch to DP (Chapter 12). Coin change with arbitrary coins is the classic case where greedy fails.

11.1 Pattern: Interval & Scheduling Greedy

PATTERN CARD

SPOT IT WHEN	Choose the maximum number of non-overlapping intervals, or minimum rooms/platforms.
KNOW FIRST	Sorting by a key.
BREAKS WHEN	You pick by START time instead of END time for 'maximum activities' (fails).
COMMON VARIATIONS	Sort by end (select), sort by start (merge), sweep events.

SOLVED EXAMPLE 11.1 — E

Activity Selection / Maximum Meetings in One Room

Problem: choose the maximum number of non-overlapping activities.

Greedy choice: always pick the activity that **finishes earliest** (leaves the most room for later ones).

```
int maxActivities(vector<pair<int,int>>& acts) {              // (start, end)
    sort(acts.begin(), acts.end(), [](auto& a, auto& b) { return a.second < b.second; });
    int count = 0, lastEnd = -1;
    for (auto& [s, e] : acts) {
        if (s > lastEnd) { count++; lastEnd = e; }           // use s >= lastEnd if touching is
        ↪ allowed
    }
    return count;
}
```

Maximum Meetings in One Room is the same problem with meeting indices carried along.

Trace with `(start, end)` = `(1,3) (2,5) (4,6) (1,7)`. Sorted by end: `(1,3) (2,5) (4,6) (1,7)` (already sorted). `(1,3)`: `1 > -1`, keep, `lastEnd = 3`. `(2,5)`: `2 > 3` is false, skip. `(4,6)`: `4 > 3`, keep, `lastEnd = 6`. `(1,7)`: `1 > 6` is false, skip. Answer 2. **Why sort by end and not by start or length?** The activity that ends first blocks the least of the future (see the exchange argument in this chapter's Toolkit); a long early-starting activity (`(1,7)`) or a short one in the middle can block two others.

SOLVED EXAMPLE 11.2 M

Minimum Platforms

Idea: sort arrivals and departures separately; sweep through time with two pointers, increasing platforms on an arrival and decreasing on a departure; the peak is the answer.

```
int minPlatforms(vector<int> arr, vector<int> dep) {
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());
    int n = arr.size(), i = 0, j = 0, cur = 0, best = 0;
    while (i < n) {
        if (arr[i] <= dep[j]) { cur++; i++; best = max(best, cur); }
        else { cur--; j++; }
    }
    return best;
}
```

Trace with `arr = [900, 940, 950, 1100, 1500, 1800]` and `dep = [910, 1200, 1120, 1130, 1900, 2000]`. Sorted departures: `910, 1120, 1130, 1200, 1900, 2000`. Sweep: 900 arrives (`900 <= 910`, platforms 1); 940 arrives? `940 <= 910` is false, so a train departs (0) and then `940 <= 1120` arrives (1); 950 arrives (2); 1100 arrives (3, the peak); $1500 > 1120$ so one departs (2), $1500 > 1130$ (1), $1500 > 1200$ (0), then 1500 arrives (1); 1800 arrives (2). The answer is **3**. Because we test `arr[i] <= dep[j]`, a train arriving at the same minute another leaves needs its own platform; use `<` if the problem lets the platform be reused at the same instant. The code assumes `arr` and `dep` have the same length and every train departs no earlier than it arrives.

SOLVED EXAMPLE 11.3 M

Gas Station

Problem: circular route; `gas[i]` is fuel at station `i`, `cost[i]` the fuel to reach `i+1`. Find a starting station to complete the circuit.

Insight: if total gas `>=` total cost, an answer exists. Scan once; whenever the running tank drops below zero, no station in the failed stretch can be the start, so restart from the **next** station.

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < (int)gas.size(); i++) {
        int diff = gas[i] - cost[i];
        total += diff; tank += diff;
        if (tank < 0) { start = i + 1; tank = 0; }
    }
    return total >= 0 ? start : -1;
}
```

Trace with `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]`. The differences `gas - cost` are `-2, -2, -2, +3, +3`. At `i = 0` the tank is $-2 < 0$, so `start = 1`, tank reset; same at `i = 1` (`start = 2`) and `i = 2` (`start = 3`). At `i = 3` the tank is 3, at `i = 4` it is 6. Total is 0 (≥ 0), so the answer is station 3. **Why it is correct:** (1) if the total is negative, no start can work. (2) If starting at `s` first fails at `i`, then every station between `s` and `i` also fails: each of them would have arrived at `i` with at most the fuel `s` had, and `s` did not make it. So it is safe to jump straight to `i + 1`. (3) The candidate that survives to the end collects a non-negative surplus; the earlier stretches only lose in total what the surplus can repay when total ≥ 0, so the trip wraps around. Both arrays must have the same length.

SOLVED EXAMPLE 11.4 M

Job Sequencing Problem

Problem: each job takes 1 unit of time and has a deadline and a profit. Maximize total profit.

Greedy: sort by profit descending; put each job in the **latest free slot** on or before its deadline.

```cpp
pair<int,int> jobSequencing(vector<array<int,3>> jobs) {      // {id, deadline, profit}
    sort(jobs.begin(), jobs.end(), [](auto& a, auto& b) { return a[2] > b[2]; });
    int maxD = 0;
    for (auto& j : jobs) maxD = max(maxD, j[1]);
    vector<bool> used(maxD + 1, false);
    int done = 0, profit = 0;
    for (auto& j : jobs) {
        for (int d = j[1]; d >= 1; d--)
            if (!used[d]) { used[d] = true; done++; profit += j[2]; break; }
    }
    return {done, profit};
}
```

Trace with jobs `(id, deadline, profit)`: `(a,4,20) (b,1,10) (c,1,40) (d,1,30)`. Sorted by profit: `c(40) d(30) a(20) b(10)`. `c`: latest free slot at or before 1 is slot 1, take it. `d`: slot 1 is taken and there is nothing earlier, skip. `a`: slot 4 is free, take it. `b`: skip. Result: 2 jobs, profit 60. The slot search can cost $\mathcal{O}(\text{maxDeadline})$ per job; a disjoint-set "next free slot" structure (Section 9.6) makes it nearly $\mathcal{O}(1)$, which is what T11.2 below alludes to. The array `used` has `maxDeadline + 1` entries, so this version suits deadlines up to about 10^6.

Practice Set 11.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Interval & Scheduling Greedy* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Maximum Trains for Which Stoppage Can Be Provided M	Activity selection per platform: sort by departure, track last departure per platform.

11.2 Pattern: Classic Greedy Algorithms

PATTERN CARD

SPOT IT WHEN	Known textbook greedy: fractional knapsack, Huffman, coin change on canonical coins.
KNOW FIRST	Priority queue; proof by exchange argument.
BREAKS WHEN	0/1 knapsack or non-canonical coins → DP.
COMMON VARIATIONS	Fractional knapsack, Huffman/ropes, job sequencing.

SOLVED EXAMPLE 11.5 E

Fractional Knapsack

Greedy: items can be split, so take the highest **value-per-weight** first.

```
double fractionalKnapsack(vector<pair<int,int>> items, int W) {    // (value, weight)
    sort(items.begin(), items.end(), [](auto& a, auto& b) {
        return (double)a.first / a.second > (double)b.first / b.second;
    });
    double total = 0;
    for (auto& [v, w] : items) {
        if (W >= w) { total += v; W -= w; }
        else { total += (double)v * W / w; break; }              // take a fraction and stop
    }
    return total;
}
```

Trace with `(value, weight)` items `(60,10) (100,20) (120,30)` and `W = 50`. Ratios are 6, 5, 4. Take all of item 1 (`total = 60`, `W = 40`). Take all of item 2 (`total = 160`, `W = 20`). Item 3 does not fit: take `20/30` of it, `120 * 20 / 30 = 80`, `total = 240`. The sort comparator divides by the weight, so every weight must be positive. **Why fractional works:** because we may cut items, any unit of capacity used by a lower-ratio item can be swapped for a higher-ratio one without loss. That swap is impossible for indivisible items, which is why 0/1 knapsack needs DP (Section 12.2).

SOLVED EXAMPLE 11.6 E/M

Minimum Cost of Ropes (and Huffman Coding)

Problem: connect ropes; joining two costs the sum of their lengths. Minimize total cost.

Greedy: always join the two **shortest** ropes (min-heap). This is also the core of **Huffman coding**: repeatedly merge the two least frequent symbols.

```
long long minCostRopes(vector<int>& ropes) {
    priority_queue<long long, vector<long long>, greater<>> pq(ropes.begin(), ropes.end());
    long long cost = 0;
    while (pq.size() > 1) {
        long long a = pq.top(); pq.pop();
        long long b = pq.top(); pq.pop();
        cost += a + b;
        pq.push(a + b);
    }
    return cost;
}
```

Trace with ropes `[4, 3, 2, 6]`. Join `2 + 3 = 5` (cost 5), heap `{4, 5, 6}`. Join `4 + 5 = 9` (cost 14), heap `{6, 9}`. Join `6 + 9 = 15` (cost 29). Total 29. Joining the large ropes late means they are counted fewer times: a rope's length is paid once per join it takes part in, exactly like a symbol's frequency being multiplied by its depth in a Huffman tree. With zero or one rope the loop never runs and the cost is 0.

SOLVED EXAMPLE 11.7 E

Minimum Number of Coins (Greedy Coin Change)

Works for the coin system used in the code (1, 2, 5, 10, 20, 50, 100, 200, 500, 2000); we compared it against a DP for every amount up to 20,000 and the coin counts matched. It does **not** work for every system: with `{1, 3, 4}` and amount 6 greedy gives `4 + 1 + 1` (3 coins) but `3 + 3` needs only 2 (that case needs DP, Example 12.5). Example: `2897 -> 2000, 500, 200, 100, 50, 20, 20, 5, 2` (9 coins). `amount` must be non-negative.

```
vector<int> minCoins(int amount) {
    int coins[] = {2000, 500, 200, 100, 50, 20, 10, 5, 2, 1};
    vector<int> used;
    for (int c : coins)
        while (amount >= c) { amount -= c; used.push_back(c); }
    return used;
}
```

Practice Set 11.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Classic Greedy Algorithms* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Huffman Coding M	Build a tree with a min-heap of nodes by frequency; left edge = `0`, right edge = `1`; read codes by DFS.
Water Connection Problem M	Graph of pipes (each house has at most one in/out pipe); find chains: start at nodes with out-edge but no in-edge, follow to the end, report the min diameter on the way.

11.3 Pattern: Array Greedy & Exchange Arguments

PATTERN CARD

SPOT IT WHEN	'Rearrange to minimize / maximize', 'assign', 'pair up'.
KNOW FIRST	Sorting; exchange argument.
BREAKS WHEN	No clean exchange argument can be made: brute-force small cases to test your greedy.
COMMON VARIATIONS	Sort + pair, heap-based re-schedule, gas station.

SOLVED EXAMPLE 11.8 E

Chocolate Distribution Problem

Problem: distribute one packet to each of `m` students minimizing the difference between the largest and smallest packet given.

Insight: after sorting, the best choice is always `m` **consecutive** packets.

```cpp
int minDifference(vector<int> packets, int m) {
    if (m <= 0 || m > (int)packets.size()) return -1;      // no valid distribution
    sort(packets.begin(), packets.end());
    int best = INT_MAX;
    for (int i = 0; i + m - 1 < (int)packets.size(); i++)
        best = min(best, packets[i + m - 1] - packets[i]);
    return best;
}
```

Trace with `packets = [7, 3, 2, 4, 9, 12, 56]`, `m = 3`. Sorted: `2 3 4 7 9 12 56`. Windows of 3: `(2,3,4)` gives 2, `(3,4,7)` gives 4, `(4,7,9)` gives 5, `(7,9,12)` gives 5, `(9,12,56)` gives 47. Best is 2. **Why consecutive?** Whatever `m` packets you choose, with smallest `x` and largest `y`, the `m` consecutive sorted packets starting at `x` have a largest value no greater than `y`, so they are at least as good. The function returns `-1` when `m < 1` or `m` is larger than the number of packets (no valid distribution).

SOLVED EXAMPLE 11.9 M

Rearrange Characters So No Two Adjacent Are Same

Greedy: always place the most frequent remaining character that is not the one just placed. A max-heap provides it.

```cpp
string rearrange(const string& s) {
    vector<int> cnt(26, 0);
    for (char c : s) cnt[c - 'a']++;
    priority_queue<pair<int,char>> pq;
    for (int i = 0; i < 26; i++) if (cnt[i]) pq.push({cnt[i], 'a' + i});
    string res;
    pair<int,char> prev = {0, '#'};
    while (!pq.empty()) {
        auto cur = pq.top(); pq.pop();
        res += cur.second;
        if (prev.first > 0) pq.push(prev);                // earlier char is allowed again
        prev = {cur.first - 1, cur.second};
    }
    return res.size() == s.size() ? res : "";             // empty = impossible
}
```

Trace with `"aab"`: heap `{(2,a), (1,b)}`. Pop `(2,a)`: result `a`, nothing to restore, hold back `(1,a)`. Pop `(1,b)`: result `ab`, put `(1,a)` back, hold back `(0,b)` (exhausted). Pop `(1,a)`: result `aba`. Size matches, so the answer is `aba`. For `"aaab"` the heap runs dry after `aba`, one `a` is stuck in `prev`, the result is shorter than the input, and the function returns `""` (impossible: a letter occurs more than `(n + 1) / 2` times). Holding the just-used letter out of the heap for one step is what prevents two equal neighbours. An empty input also returns `""`, so check `s.empty()` first if you must tell the cases apart.

Practice Set 11.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Array Greedy & Exchange Arguments* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Smallest Subset With Sum Greater Than All Other Elements E	Sort descending; take elements until your sum exceeds the remainder.
Maximize Array Sum After K Negations E	Repeatedly negate the smallest element (heap), or sort and flip negatives first; if `k` remains odd flip the smallest absolute value.
Find Smallest Number With Given Digit Count and Digit Sum E	Reserve 1 for the leading digit, so distribute `sum - 1` from the rightmost digit leftwards with `min(9, remaining)` each; the leading digit gets what is left plus the reserved 1. Impossible if `sum < 1` or `sum > 9 * digits`.
Maximum Product Subset of an Array M	Multiply all non-zero numbers; if the count of negatives is odd, drop the negative with the smallest absolute value; handle zeros/single element.
Maximize Sum of arr[i] * i via Rotation M	Compute `R0`, then `R(k) = R(k-1) + sum - n * arr[n-k]`.
Maximum Sum of Absolute Differences of an Array M	Sort, then arrange alternately small/large (zigzag from the middle outward).
Maximize Sum of Consecutive Differences in a Circular Array M	Sort; alternate smallest and largest around the circle. The best total is `2 * (sum of the largest floor(n/2) values - sum of the smallest floor(n/2) values)`; for odd `n` the median cancels out. Check with `[1,2,3,4,5]`: `2 * ((4+5) - (1+2)) = 12`.
Minimum Sum of Absolute Differences of Pairs of Two Arrays M	Sort both arrays and pair index to index.

11.4 Pattern: Resource Allocation & Scheduling

PATTERN CARD

SPOT IT WHEN	Schedule or allocate limited resources.
KNOW FIRST	Sorting plus heap.
BREAKS WHEN	Resources interact in non-greedy ways (DP/flow).
COMMON VARIATIONS	Task scheduler, meeting rooms, platforms.

> **READING NOTE**
> Sections 11.4 and 11.5 list sheet problems with approach hints only; there are no separate solved examples here. Use the closest solved example in 11.1 to 11.3 as your template (sort + sweep, sort + pair, or heap).

Problem	Level 2 hint (approach)
Shortest Job First (SJF) CPU Scheduling E	Sort by burst time; waiting time of a job is the sum of all earlier burst times.
Buy Maximum Stocks if Only i Stocks Can Be Bought on Day i M	Sort days by price ascending; buy `min(i, budget / price)` from each.
Find Min & Max Amount to Buy All N Candies M	Sort; for the minimum buy the cheapest and take `k` free from the most expensive end; for the maximum do the reverse.
Minimize Cash Flow Among a Set of Friends M	Compute each net balance; repeatedly settle the max creditor with the max debtor (two heaps). This greedy always finishes in at most `n - 1` transactions, which is what such problems usually ask for; note that finding the true minimum number of transactions is a harder (subset-sum style) problem.

Problem	Level 2 hint (approach)
Minimum Cost to Cut a Board Into Squares M	Sort horizontal and vertical cut costs descending; always take the larger cut next and multiply by the number of pieces in the opposite direction.
Check if It's Possible to Survive on an Island M	Count Sundays when the shop is closed; simulate with greedy buying; check the food needed is at most the food you can buy.
LRU Page Replacement Algorithm M	Simulate with the LRU cache from Chapter 6 and count page faults.

11.5 SPOJ Classics & Advanced Greedy

Problem	Level 2 hint (approach)
DEFKIN: Defense of a Kingdom M	Sort the tower coordinates (separately for x and y), include the borders (`0` and `W + 1`, `0` and `H + 1`); take the largest gap in each dimension; answer `(maxGapX - 1) * (maxGapY - 1)` (check the exact statement).
DIEHARD: Die Hard M	Greedy simulation or small DP: alternate between the three environments to maximize survival time.
GERGOVIA: Wine Trading in Gergovia M	Running prefix sum of net demand; answer is the sum of absolute prefix values.
Picking Up Chicks M	Chicks are listed from the back to the front. Compute whether each chick can reach the barn in time. Scan from the chick nearest the barn backwards, keeping `slow` = number of chicks seen that cannot make it; every chick that can make it must overtake all `slow` chicks ahead of it, costing `slow` swaps. Stop after `K` such chicks, or report impossible if there are fewer (the statement is not in the sheet; check it).
CHOCOLA: Chocolate M	Same as Minimum Cost to Cut a Board Into Squares.
ARRANGE: Arranging Amplifiers M	Compare `x^y` versus `y^x` to decide ordering: sort ascending, place 1s first, and special-case values like `2` and `3` (or `2` and `4`) where the comparison flips or ties.
K Centers Problem H	Farthest-first traversal: pick any center, then repeatedly choose the point farthest from the current centers. This is a 2-approximation; the exact version uses binary search on the radius.

CHAPTER COVERAGE

all 35 Greedy problems appear above.

CHAPTER 11 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 11.1 Activity Selection, 11.5 Fractional Knapsack, 11.7 Coin Change (canonical).
2. **Variations:** Ex 11.2 Min Platforms, Problem 11B.
3. **Combined:** Ex 11.6 Min Cost of Ropes (greedy + heap), 11.9 Rearrange Characters.
4. **Independent recognition:** Problem 11A (assignment disguised as a story).
5. **Challenge:** Ex 11.3 Gas Station, 11.4 Job Sequencing.

Compare and Contrast

	Greedy	Dynamic programming
Idea	commit to one locally best choice	try all choices, keep best answers to subproblems
Needs	greedy-choice property + optimal substructure	optimal substructure + overlapping subproblems
Speed	usually $\mathcal{O}(n \log n)$	polynomial but larger
Failure example	coins {1,3,4}, amount 6: greedy 4+1+1, optimum 3+3	handles it

Test: if you cannot find an exchange argument, check the greedy against brute force on small inputs.

Why It Works (Correctness Corner)

Exchange argument for Activity Selection. Let `g` be the activity that ends earliest. Take any optimal schedule `S`; swap its first activity for `g`. Since `g` ends no later, it still does not overlap the rest, so the new schedule is feasible and the same size, hence also optimal. So some optimal schedule contains `g`; remove `g` and the conflicting activities and repeat by induction.

Debug Lab: Sorting by start time

```
// maximum number of non-overlapping intervals
sort(iv.begin(), iv.end());           // sorts by start time
int kept = 0, lastEnd = INT_MIN;
for (auto& v : iv)
    if (v[0] >= lastEnd) { kept++; lastEnd = v[1]; }
```

Your task: Give an input where this fails.

ANSWER

`[[1,10],[2,3],[4,5]]`: it keeps `[1,10]` and rejects both others (answer 1) while the optimum is `[2,3],[4,5]` (answer 2). To maximise the count, sort by **end** time.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 11A E

Assign Cookies

Statement. Child `i` is content with a cookie of size at least `greed[i]`; cookie `j` has size `cookies[j]`. Each child gets at most one cookie and each cookie goes to at most one child. Maximise the number of content children.

Example. `greed = [1,2,3], cookies = [1,1]` → `1`

Constraints. $0 \leq \text{sizes} \leq 3 \times 10^4$.

Expected complexity. $\mathcal{O}(n \log n + m \log m)$.

Prerequisites. 11.3 Array Greedy & Exchange Arguments

HINT 1 which idea is relevant?

Is giving a big cookie to a not-so-greedy child ever useful?

HINT 2 outline of the approach

Sort both lists.

HINT 3 the key step

Walk cookies from smallest; give each to the least greedy unsatisfied child if it fits.

SOLUTION and editorial (try first, then compare)

```cpp
int findContentChildren(vector<int> greed, vector<int> cookies) {
    sort(greed.begin(), greed.end());
    sort(cookies.begin(), cookies.end());
    int child = 0;
    for (int c : cookies)                                  // smallest cookie first
        if (child < (int)greed.size() && greed[child] <= c) child++;   // feed the least
        ↪ greedy child
    return child;
}
```

Exchange argument: if a larger cookie satisfies a child that a smaller one could, swapping keeps everyone satisfied but saves the larger cookie. So always use the smallest cookie that works.

PROBLEM 11B M

Non-overlapping Intervals

Statement. Given intervals, return the minimum number of intervals to remove so that the rest do not overlap. Intervals that only touch (`[1,2]`, `[2,3]`) do not overlap.

Example. `[[1,2],[2,3],[3,4],[1,3]]` → `1`

Constraints. $0 \leq \text{n} \leq 10^5$.

Expected complexity. $\mathcal{O}(n \log n)$.

Prerequisites. 11.1 Interval & Scheduling Greedy

HINT 1 which idea is relevant?

Minimum removals = total - maximum kept. What problem is 'maximum kept'?

HINT 2 outline of the approach

That is activity selection. Which sort key makes the greedy safe?

HINT 3 the key step

Sort by end time; keep an interval if its start $\geq$ the last kept end.

SOLUTION and editorial (try first, then compare)

```cpp
int eraseOverlapIntervals(vector<vector<int>> iv) {
    sort(iv.begin(), iv.end(), [](const auto& a, const auto& b) { return a[1] < b[1]; });
    int kept = 0, lastEnd = INT_MIN;
    for (const auto& v : iv)
        if (v[0] >= lastEnd) { kept++; lastEnd = v[1]; } // touching intervals
        ↪  ([1,2],[2,3]) do not overlap
    return (int)iv.size() - kept;
}
```

Keeping the interval with the earliest end leaves the most room (exchange argument). Note the `>=` so touching intervals are both kept.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T11.1 Count the minimum number of conference rooms required for a list of (start, end) meetings.

PATTERN

Sort + min-heap of end times (or sweep line), as in Min Platforms.

T11.2 Tasks each take one slot and pay a profit if finished by their deadline; maximise profit.

PATTERN

Job sequencing: sort by profit, place in latest free slot (DSU/heap).

Mixed Practice

Compare Example 11.5 and Section 12.2: fractional knapsack is greedy, 0/1 knapsack is DP. State exactly which property of 'fractional' makes the exchange argument work.

Revision Sheet

Key takeaways

- Greedy needs a proof; try an exchange argument.
- Selection: sort by end. Merging: sort by start.
- Heap = 'best remaining' repeatedly.
- When greedy fails, the problem is usually DP.

Pattern-recognition clues

- 'Maximum number of non-overlapping' $\rightarrow$ sort by end.
- 'Minimum cost to merge' $\rightarrow$ min-heap.
- 'Assign / pair up' $\rightarrow$ sort both sides.

Common mistakes

- Sorting by the wrong key.

- `>` vs `>=` on touching intervals.
- Assuming greedy works for 0/1 or arbitrary coins.

Complexity summary

Task	Time
Interval selection	$\mathcal{O}(n \log n)$
Heap-based greedy	$\mathcal{O}(n \log n)$
Fractional knapsack	$\mathcal{O}(n \log n)$

Recall questions (answer aloud, then check the chapter)

1. What is an exchange argument?
2. Which key do you sort by to maximise the number of intervals?
3. Why does greedy fail for coins {1,3,4}?
4. When do you need a heap in a greedy?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Invent a counterexample to a *wrong* greedy for each of: activity selection (sort by start), coin change (non-canonical coins). Then explain the correct choice.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 12

Dynamic Programming

12

START HERE

How many ways can you climb 50 stairs taking 1 or 2 steps? The naive recursion makes over 10^{10} calls; remembering answers makes 50. Dynamic programming is 'recursion that remembers', and the real skill is defining what the table cell *means*.

ANALOGY

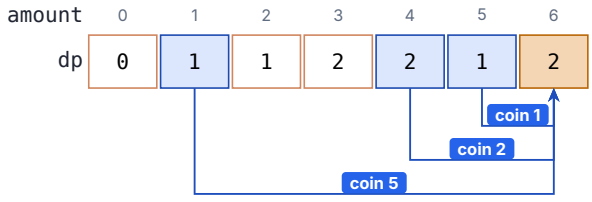
Filling a spreadsheet: each cell is computed once from cells you have already filled in. The hard part is choosing the rows and columns (the **state**) so every cell depends only on earlier ones.

AFTER THIS CHAPTER YOU CAN

- ✓ Define a DP state in words and write its recurrence.
- ✓ Convert recursion + memoization into tabulation and optimise space.
- ✓ Recognise knapsack, LIS, grid, string-alignment and interval DP.
- ✓ Explain the loop-order rules (0/1 vs unbounded; combinations vs permutations).

PICTURE IT

Coin change table: coins {1, 2, 5}, min coins for each amount



$$dp[6] = 1 + \min(dp[5], dp[4], dp[1])$$
$$= 1 + \min(1, 2, 1) = 2 \quad (5 + 1)$$

state: $dp[x]$ = fewest coins that make **exactly** x (coins $\{1, 2, 5\}$)

12.0 Concept

Dynamic Programming (**DP**) solves a problem by breaking it into **overlapping subproblems**, solving each once, and storing the answer. It applies when a problem has:

1. **Optimal substructure:** the best answer is built from best answers to smaller pieces.
2. **Overlapping subproblems:** the same smaller pieces are needed again and again.

Warm-up: Fibonacci Three Ways

```cpp
// 1. Plain recursion: about 1.6^n calls (O(2^n) is a safe upper bound), recomputes everything
int fib1(int n) { return n < 2 ? n : fib1(n - 1) + fib1(n - 2); }

// 2. Memoization (top-down): O(n), cache each answer
int fib2(int n, vector<int>& memo) {
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib2(n - 1, memo) + fib2(n - 2, memo);
}

// 3. Tabulation (bottom-up) + space optimization: O(n) time, O(1) space
//    (int overflows after fib(46); use long long for larger n)
int fib3(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) { int c = a + b; a = b; b = c; }
    return a;
}
```

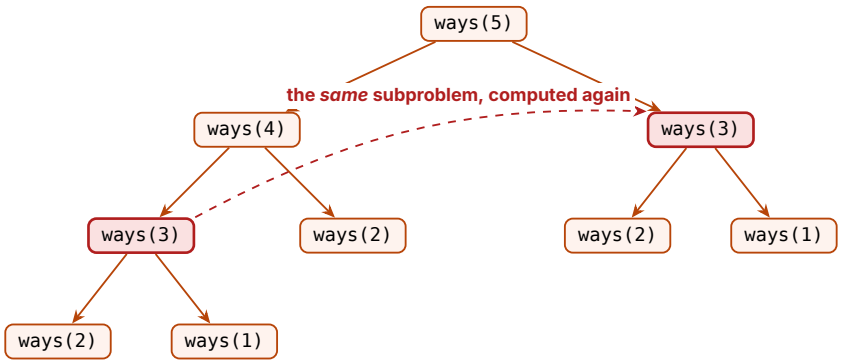
Worked Derivation: Climbing Stairs, from Brute Force to O(1) Space

Before the recipe, watch the whole journey once on the simplest problem. **Problem:** you climb `n` stairs taking 1 or 2 steps at a time. How many different ways are there?

Intuition. Look at the *last* move. Either you arrived from stair `n-1` with a 1-step, or from stair `n-2` with a 2-step. So `ways(n) = ways(n-1) + ways(n-2)`. The smallest cases are known: there is one way to stand at the bottom (`ways(0) = 1`) and one way to reach stair 1 (`ways(1) = 1`).

Brute force. Turn that sentence straight into recursion. Draw the calls for `n = 5`:

PICTURE IT



`ways(3)` and `ways(2)` appear several times. This repeated work is what "overlapping subproblems" means, and it makes the call count grow like `1.6^n`.

Optimization 1: memoization. Keep a notebook (`memo`). The first time you compute `ways(k)` write it down; every later request reads it back. Each value is now computed once: $\mathcal{O}(n)$.

Optimization 2: tabulation. Instead of asking for big values and recursing downwards, fill a table from the smallest value upwards, so each cell only reads cells that are already filled.

Optimization 3: space. `dp[i]` reads only `dp[i-1]` and `dp[i-2]`, so two variables replace the whole table.

```cpp
// Version 1: brute-force recursion, about 1.6^n calls
long long climb1(int n) { return n <= 1 ? 1 : climb1(n - 1) + climb1(n - 2); }

// Version 2: memoization: remember each answer the first time it is computed
long long climb2(int n, vector<long long>& memo) {      // memo has n + 1 entries, all -1
    if (n <= 1) return 1;
    if (memo[n] != -1) return memo[n];
    return memo[n] = climb2(n - 1, memo) + climb2(n - 2, memo);
}

// Version 3: tabulation: fill the table from small n upward
long long climb3(int n) {
    vector<long long> dp(n + 2, 1);                      // dp[0] = dp[1] = 1
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// Version 4: only the last two values are ever read, so keep just two
long long climb4(int n) {
    long long prev = 1, cur = 1;                         // ways(0), ways(1)
    for (int i = 2; i <= n; i++) { long long next = prev + cur; prev = cur; cur = next; }
    return cur;
}
```

The table for the first values reads `n: 0 1 2 3 4 5` and `ways: 1 1 2 3 5 8`. (We ran all four versions against each other for `n = 0..25`, and `climb4(50) = 20365011074`.) Every DP problem in this chapter follows the same path: **state and recurrence from the last choice, then a table, then (if possible) fewer rows.**

How to Find the State

The state is "the smallest set of facts that lets you finish the problem from here". Ask three questions:

1. **How far through the input am I?** An index `i` (stones, houses, days, characters). This alone is enough for Climbing Stairs, Frog Jump and House Robber.
2. **What resource is left?** A capacity, amount or number of moves left. Knapsack uses `(i, capacity)`; Coin Change uses `amount`; Egg Drop uses `(eggs, moves)`.
3. **What did my last choice limit?** A flag such as "am I holding a stock?", "was the previous bit 1?", "which colour was the previous house?". The stock problems use `(day, holding)`.

If two different histories that reach the same state can be finished in exactly the same way, the state is complete; if not, add the missing fact. Write the meaning as one sentence (for example "`dp[i][j]` = LCS length of the first `i` letters of `a` and the first `j` letters of `b`"). If you cannot say it in a sentence, the state is not ready.

The 5-Step DP Recipe

1. **State:** what do I need to know to describe a subproblem? (`dp[i]`, `dp[i][j]`...)
2. **Meaning:** write one sentence: *"$dp[i]$ = the best answer for the first i elements."*
3. **Transition:** how does `dp[i]` depend on smaller states?
4. **Base cases:** the smallest subproblems with known answers.
5. **Order and answer:** fill order so dependencies are ready; where is the final answer?

Then **optimize space** if `dp[i]` only needs the previous row/few values.

Pattern Map

Pattern	Recognition clue
1D DP	"ways to reach step i", "min cost to i", no extra constraint
Knapsack	pick items under a capacity/target; "number of ways", "min coins"
LIS	increasing subsequence, nesting, chains
Stock	buy/sell with transaction rules (state machine)
Grid DP	paths through a matrix
Interval DP	split a sequence at every point, `dp[l][r]`
LCS family	two strings/sequences compared
Palindrome DP	substrings, `dp[i][j]` = is `s[i..j]` a palindrome
Game DP	two players, optimal play

12.1 Pattern: 1D DP Foundations

PATTERN CARD

SPOT IT WHEN	'Count ways' or 'min cost' where the answer for n uses answers for smaller n.
KNOW FIRST	Recursion with overlapping subproblems.
BREAKS WHEN	State is too large (exponential) or the problem needs greedy proof.
COMMON VARIATIONS	Fibonacci-like, house robber, climbing stairs, jump game.

SOLVED EXAMPLE 12.1 E

Frog Jump / Paint House

Frog Jump: stones with heights `h`; from stone `i` jump to `i+1` or `i+2` paying `|h[i] - h[j]|`. Minimize the cost to reach the last stone.

- State: `dp[i]` = min cost to reach stone `i`.
- Transition: `dp[i] = min(dp[i-1] + |h[i]-h[i-1]|, dp[i-2] + |h[i]-h[i-2]|)`.

```cpp
int frogJump(const vector<int>& h) {
    int n = h.size();
    if (n <= 1) return 0;
    int prev2 = 0, prev1 = abs(h[1] - h[0]);
    for (int i = 2; i < n; i++) {
        int cur = min(prev1 + abs(h[i] - h[i - 1]), prev2 + abs(h[i] - h[i - 2]));
        prev2 = prev1; prev1 = cur;                    // O(1) space
    }
    return prev1;
}
```

Trace with heights `h = [10, 30, 40, 20]`. `dp[0] = 0`; `dp[1] = |30-10| = 20`; `dp[2] = min(dp[1] + |40-30|, dp[0] + |40-10|) = min(30, 30) = 30`; `dp[3] = min(dp[2] + |20-40|, dp[1] + |20-30|) = min(50, 30) = 30`. The answer is **30** (jump 0, then 1, then 3). The code keeps only `prev2 = dp[i-2]` and `prev1 = dp[i-1]`; with one stone (or none) the cost is 0.

Paint House: `cost[i][c]` = cost of painting house `i` with color `c`; adjacent houses must differ. `dp[i][c] = cost[i][c] + min(dp[i-1][other colors])`.

```cpp
int minCostPaintHouse(vector<array<int,3>>& cost) {
    if (cost.empty()) return 0;
    int a = cost[0][0], b = cost[0][1], c = cost[0][2];
    for (int i = 1; i < (int)cost.size(); i++) {
        int na = cost[i][0] + min(b, c);
        int nb = cost[i][1] + min(a, c);
        int nc = cost[i][2] + min(a, b);
        a = na; b = nb; c = nc;
    }
    return min({a, b, c});
}
```

Trace of Paint House with `cost = [[17,2,17],[16,16,5],[14,3,19]]`. Row 0: `(a,b,c) = (17, 2, 17)`. Row 1: `a = 16 + min(2,17) = 18`, `b = 16 + min(17,17) = 33`, `c = 5 + min(17,2) = 7`. Row 2: `a = 14 + min(33,7) = 21`, `b = 3 + min(18,7) = 10`, `c = 19 + min(18,33) = 37`. Answer `min(21, 10, 37) = 10`. Each new row needs only the previous row's three numbers.

SOLVED EXAMPLE 12.2 M

Count Binary Strings Without Consecutive 1s

Let `end0[i]` and `end1[i]` count valid strings of length `i` ending with 0 or 1. A `0` can follow anything; a `1` can follow only a `0`.

```cpp
long long countStrings(int n) {
    if (n <= 0) return 1;                          // only the empty string
    long long end0 = 1, end1 = 1;                  // length 1: "0" and "1"
    for (int i = 2; i <= n; i++) {
        long long n0 = end0 + end1;
        long long n1 = end0;
        end0 = n0; end1 = n1;
    }
    return end0 + end1;                            // it is Fibonacci!
}
```

Trace. Length 1: `"0"`, `"1"` give `end0 = 1`, `end1 = 1` (2 strings). Length 2: `end0 = 1 + 1 = 2` (`00`, `10`), `end1 = end0_old = 1` (`01`): total 3. Length 3: `end0 = 2 + 1 = 3`, `end1 = 2`: total 5 (`000`, `001`, `010`, `100`, `101`). The totals 2, 3, 5 are Fibonacci numbers. For `n <= 0` the function returns 1 (only the empty string); `long long` is exact up to about `n = 90`.

The same two-state idea solves **Ways to Construct Buildings (No Two Adjacent)** (answer is `f(n)^2` where `f` counts one side of the street) and **Paint Fence**.

SOLVED EXAMPLE 12.3 M

Jump Game II

Greedy view: minimum jumps to reach the end. Track the farthest index reachable in the current jump; when you pass the end of the current range, take another jump.

```cpp
int jump(vector<int>& a) {
    int jumps = 0, curEnd = 0, farthest = 0;
    for (int i = 0; i + 1 < (int)a.size(); i++) {
        farthest = max(farthest, i + a[i]);
        if (i == curEnd) { jumps++; curEnd = farthest; }
    }
```

```
    return jumps;
}
```

Trace with `a = [2, 3, 1, 1, 4]`. At `i = 0`: `farthest = 2`; `i == curEnd (0)`, so take jump 1 and the new range ends at 2. At `i = 1`: `farthest = 4`. At `i = 2`: `farthest = 4`; `i == curEnd (2)`, so take jump 2 and the range now ends at 4, which is the last index. The answer is **2**. Each jump takes you to the farthest point any stone in the current range could reach, which is why greedy is safe here. The code assumes the last index is reachable; on an array like `[0, 1]` it would report a meaningless count.

SOLVED EXAMPLE 12.4 M

Ugly Number II

Idea: every ugly number is an earlier ugly number times 2, 3, or 5. Maintain three pointers into the growing list.

```
int nthUglyNumber(int n) {
    vector<int> ugly(n);
    ugly[0] = 1;
    int i2 = 0, i3 = 0, i5 = 0;
    for (int i = 1; i < n; i++) {
        int nxt = min({ugly[i2] * 2, ugly[i3] * 3, ugly[i5] * 5});
        ugly[i] = nxt;
        if (nxt == ugly[i2] * 2) i2++;                  // use if, not else-if: skip duplicates
        if (nxt == ugly[i3] * 3) i3++;
        if (nxt == ugly[i5] * 5) i5++;
    }
    return ugly[n - 1];
}
```

Trace. `ugly = [1]`, pointers `i2 = i3 = i5 = 0`. Candidates `(2, 3, 5)`: take 2, `i2 = 1`. Candidates `(4, 3, 5)`: take 3, `i3 = 1`. Candidates `(4, 6, 5)`: take 4, `i2 = 2`. Candidates `(6, 6, 5)`: take 5, `i5 = 1`. Candidates `(6, 6, 10)`: take 6, and **both** `i2` and `i3` advance (that is why the three `if`s are not `else if`: `2*3` and `3*2` are the same number and must be counted once). Sequence: `1, 2, 3, 4, 5, 6, 8, 9, 10, 12`. Requires `n >= 1`; `int` is safe for the usual `n <= 1690`.

Super Ugly Number generalizes this with `k` primes and `k` pointers.

Practice Set 12.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *1D DP Foundations* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Paint Fence M	`same[i] = diff[i-1]`, `diff[i] = (same[i-1] + diff[i-1]) * (k - 1)`; answer is the sum.
Paint House II M	Same as Paint House with `k` colors; keep the smallest and second smallest of the previous row for $\mathcal{O}(nk)$.
Ways to Construct Buildings (No Two Adjacent) M	Count one-side arrangements with the binary strings idea, then square it.

12.2 Pattern: Knapsack Family

PATTERN CARD

SPOT IT WHEN	'Pick items with capacity / target sum', 'can we make the sum'.
KNOW FIRST	1D/2D DP table; loop order.
BREAKS WHEN	Capacity is huge (1e9): use meet-in-the-middle or greedy if allowed.
COMMON VARIATIONS	0/1 (descend j), unbounded (ascend j), coin ways (coins outer), subset sum.

SOLVED EXAMPLE 12.5 — M

Coin Change, Minimum Coins

State: `dp[a]` = fewest coins to make amount `a`. **Transition:** try each coin `c`: `dp[a] = min(dp[a - c] + 1)`.

```
int coinChange(vector<int>& coins, int amount) {
    const int INF = 1e9;
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a && dp[a - c] != INF) dp[a] = min(dp[a], dp[a - c] + 1);
    return dp[amount] == INF ? -1 : dp[amount];
}
```

Why the `INF` test? Without `dp[a - c] != INF`, `INF + 1` could overflow and an unreachable amount could look reachable. For `min`, the two loops can be swapped; the answer is the same. **Trace** `dp[3]` with coins `{1, 2, 5}`: coin 1 gives `dp[2] + 1 = 2`, coin 2 gives `dp[1] + 1 = 2`, coin 5 is skipped (`5 > 3`), so `dp[3] = 2`. Cost $\mathcal{O}$(amount * number of coins).

SOLVED EXAMPLE 12.6 — M

Coin Change II, Number of Ways

Important ordering: to count **combinations** (not permutations), loop over coins in the **outer** loop and amounts inside.

```
int change(int amount, vector<int>& coins) {
    vector<long long> dp(amount + 1, 0);
    dp[0] = 1;
    for (int c : coins)                                  // coins outside => combinations
        for (int a = c; a <= amount; a++)
            dp[a] += dp[a - c];
    return (int)dp[amount];
}
```

Trace with coins `{1, 2, 5}`, amount 5, `dp = [1,0,0,0,0,0]` (index = amount). After coin 1: `[1,1,1,1,1,1]`. After coin 2 (`a` from 2 up): `dp[2] += dp[0]`, `dp[3] += dp[1]`, `dp[4] += dp[2]`, `dp[5] += dp[3]` gives `[1,1,2,2,3,3]`. After coin 5: `dp[5] += dp[0]` gives 4. The four ways are `5`, `2+2+1`, `2+1+1+1`, `1+1+1+1+1`. **Why combinations?** Each coin value is processed once, in a fixed order, so `1+2` and `2+1` can never both be generated. If you put the amount loop outside and the coins inside, the same table counts *ordered* sequences instead: for amount 3 and coins `{1, 2}` that gives 3 (`1+1+1`, `1+2`, `2+1`) rather than 2.

SOLVED EXAMPLE 12.7 M

0/1 Knapsack (and Unbounded Knapsack)

0/1 Knapsack: each item used at most once. Iterate capacity **downwards** so an item is not reused within the same pass.

```
int knapsack01(const vector<int>& wt, const vector<int>& val, int W) {
    vector<int> dp(W + 1, 0);                          // dp[w] = best value with capacity w
    for (int i = 0; i < (int)wt.size(); i++)
        for (int w = W; w >= wt[i]; w--)                // downwards: 0/1
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Unbounded Knapsack: each item can be used many times; iterate capacity **upwards**.

```
int knapsackUnbounded(const vector<int>& wt, const vector<int>& val, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)wt.size(); i++)
        for (int w = wt[i]; w <= W; w++)                // upwards: reuse allowed
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Trace with `wt = [1, 3, 4]`, `val = [15, 20, 30]`, `W = 4`; `dp[w]` for `w = 0..4`. Start `[0,0,0,0,0]`. Item 1 (1, 15), `w` from 4 down to 1: `[0,15,15,15,15]`. Item 2 (3, 20): `w = 4`: `max(15, dp[1] + 20 = 35) = 35`; `w = 3`: `max(15, dp[0] + 20 = 20) = 20`; now `[0,15,15,20,35]`. Item 3 (4, 30): `w = 4`: `max(35, dp[0] + 30 = 30) = 35`. The answer is **35** (items 1 and 2). Now run item 1 with `w` going **up**: `dp[2] = dp[1] + 15 = 30`, `dp[3] = 45`, `dp[4] = 60`: the same item was used four times, which is exactly the unbounded answer (60 for these items). Both functions need positive weights.

MEMORY AID

0/1 means loop **down**; unbounded means loop **up**.

Practice Set 12.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Knapsack Family* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Tri Tiling M	Tiling a `3 x n` board with dominoes: `f(n) = 0` for odd `n`; `f(0) = 1`, `f(2) = 3`, and `f(n) = 4 f(n-2) - f(n-4)` for even `n >= 4` (so `f(4) = 11`); or use a state DP over partial profiles.

12.3 Pattern: Longest Increasing Subsequence (LIS)

PATTERN CARD

SPOT IT WHEN	'Longest increasing / non-decreasing subsequence', nesting (envelopes).
KNOW FIRST	Binary search (5.1).
BREAKS WHEN	Subsequence must be contiguous (that is a window/DP-on-i problem).
COMMON VARIATIONS	$\mathcal{O}(n^2)$ DP, $\mathcal{O}(n \log n)$ tails array, count LIS, Russian doll.

SOLVED EXAMPLE 12.8 M/H

LIS in O(n^2) and O(n log n)

$\mathcal{O}(n^2)$: `dp[i]` = length of LIS ending at `i`; `dp[i] = 1 + max(dp[j])` over `j < i` with `a[j] < a[i]`.

```
int lisQuadratic(const vector<int>& a) {
    int n = a.size(), best = 0;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
        best = max(best, dp[i]);
    }
    return best;
}
```

$\mathcal{O}(n \log n)$ **(patience sorting):** maintain `tails[k]` = smallest possible tail of an increasing subsequence of length `k+1`. For each number, binary search its position: replace the first tail `>= x` (or append).

```
int lisFast(const vector<int>& a) {
    vector<int> tails;
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);   // use upper_bound for
        ↪ non-decreasing
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();                                        // length only; tails is NOT the
    ↪ LIS itself
}
```

Trace with `a = [10, 9, 2, 5, 3, 7, 101, 18]`. *Quadratic:* `dp = [1, 1, 1, 2, 2, 3, 4, 4]` (for example `dp[5]` for the value 7 is `1 + max(dp[3], dp[4]) = 3`, from 2,5,7 or 2,3,7). Answer 4. *Fast:* `tails` becomes `[10]`, `[9]` (9 replaces 10: a smaller tail is always better), `[2]`, `[2,5]`, `[2,3]`, `[2,3,7]`, `[2,3,7,101]`, `[2,3,7,18]`. Length 4. **Why `tails` stays sorted:** a subsequence of length `k+1` always has a bigger tail than the best subsequence of length `k` (drop its last element), so `tails` is strictly increasing and binary search is valid. **`tails` is not the subsequence itself:** for `[4, 5, 1]` it ends as `[1, 5]`, but `1` comes after `5` in the input; only the length (2) is meaningful.

SOLVED EXAMPLE 12.9 H

Russian Doll Envelopes

Reduction to LIS: sort envelopes by width **ascending**, and for equal widths by height **descending** (so two envelopes of equal width cannot both appear in an increasing run). Then LIS on heights.

```
int maxEnvelopes(vector<vector<int>>& env) {
    sort(env.begin(), env.end(), [](auto& a, auto& b) {
        return a[0] == b[0] ? a[1] > b[1] : a[0] < b[0];
    });
    vector<int> tails;
    for (auto& e : env) {
        auto it = lower_bound(tails.begin(), tails.end(), e[1]);
        if (it == tails.end()) tails.push_back(e[1]); else *it = e[1];
    }
    return tails.size();
}
```

Trace with envelopes `(5,4) (6,4) (6,7) (2,3)`. After sorting (width ascending, equal widths by height descending): `(2,3) (5,4) (6,7) (6,4)`. Heights: `3, 4, 7, 4`. LIS with `lower_bound` (strictly increasing): `[3]`, `[3,4]`, `[3,4,7]`, then 4 replaces 4: length **3** (`(2,3) -> (5,4) -> (6,7)`). **Why descending heights for equal widths?** With the single pair `(6,4), (6,7)` an ascending order would give heights `4, 7` and wrongly report 2 nested envelopes; descending gives `7, 4`, whose longest strictly increasing run is 1.

Practice Set 12.3 (LIS and LIS-style DP)

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Longest Increasing Subsequence* (*LIS*) idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Minimum Number of Increasing Subsequences (Patience Sorting) M	The answer equals the length of the longest **non-increasing** subsequence (Dilworth).
Building Bridges M	Sort by one coordinate; LIS (non-decreasing) on the other.
Box Stacking M	Generate all 3 rotations of each box, sort by base area descending, LIS on strictly smaller bases with heights summed.
Weighted Job Scheduling M	Sort by end time; `dp[i] = max(dp[i-1], profit[i] + dp[last non-overlapping job])`, found by binary search.
Longest Bitonic Subsequence M	`lis[i]` from the left plus `lds[i]` from the right minus 1; take the max.

12.4 Pattern: Stock Trading (State Machine DP)

PATTERN CARD

SPOT IT WHEN	Buy/sell with limits, cooldown or fee.
KNOW FIRST	State machine thinking: states = holding / not holding.
BREAKS WHEN	Transaction count too large (then greedy: sum of positive differences).
COMMON VARIATIONS	Single, unlimited, cooldown, fee, at most k.

Idea: at each day you are in a **state** (holding or not holding). Transitions are buy/sell/rest. Track the best profit in each state.

SOLVED EXAMPLE 12.10 — E

Stocks, Single Transaction

```cpp
int maxProfitOne(const vector<int>& p) {
    int minPrice = INT_MAX, best = 0;
    for (int x : p) {
        minPrice = min(minPrice, x);
        best = max(best, x - minPrice);
    }
    return best;
}
```

Trace with prices `[7, 1, 5, 3, 6, 4]`: `minPrice` goes `7, 1, 1, 1, 1, 1` and `best` goes `0, 0, 4, 4, 5, 5`. Answer **5** (buy at 1, sell at 6). Taking the minimum *before* the current day guarantees you never sell before you buy.

SOLVED EXAMPLE 12.11 — E

Stocks, Unlimited Transactions

Capture every upward step.

```cpp
int maxProfitMany(const vector<int>& p) {
    int profit = 0;
    for (int i = 1; i < (int)p.size(); i++) profit += max(0, p[i] - p[i - 1]);
    return profit;
}
```

Trace with `[7, 1, 5, 3, 6, 4]`: the upward steps are `1 -> 5` (+4) and `3 -> 6` (+3), so profit **7**. Adding up every rise equals buying at every valley and selling at every peak, and no strategy can do better because it can never earn more than the sum of the rises.

SOLVED EXAMPLE 12.12 — M

Stocks with Cooldown and Transaction Fee

Cooldown: after selling, you cannot buy the next day. States: `hold`, `sold` (just sold today), `rest` (free, not holding).

```cpp
int maxProfitCooldown(const vector<int>& p) {
    int hold = INT_MIN, sold = 0, rest = 0;
```

```
    for (int x : p) {
        int prevSold = sold;
        sold = hold + x;                    // sell today
        hold = max(hold, rest - x);         // keep holding, or buy from rest
        rest = max(rest, prevSold);         // rest after a sale or keep resting
    }
    return max(sold, rest);
}
```

Trace of cooldown with prices `[1, 2, 3, 0, 2]` (`hold` = best balance while holding, `sold` = balance after selling today, `rest` = balance while free):

day (price)	hold	sold	rest
1 (1)	-1	impossible	0
2 (2)	-1	1	0
3 (3)	-1	2	1
4 (0)	1	-1	2
5 (2)	1	3	2

The answer is `max(sold, rest) = 3` (buy at 1, sell at 2, cool down, buy at 0, sell at 2). Buying always starts from `rest`, never from the day we sold: that is the cooldown.

Transaction Fee: pay the fee when selling. The function needs at least one price (an empty list returns 0).

```
int maxProfitFee(const vector<int>& p, int fee) {
    if (p.empty()) return 0;
    int hold = -p[0], cash = 0;
    for (int i = 1; i < (int)p.size(); i++) {
        cash = max(cash, hold + p[i] - fee);
        hold = max(hold, cash - p[i]);
    }
    return cash;
}
```

Trace of the fee version with prices `[1, 3, 2, 8, 4, 9]`, `fee = 2`. Start `hold = -1`, `cash = 0`. Price 3: `cash = max(0, -1 + 3 - 2) = 0`. Price 2: unchanged. Price 8: `cash = max(0, -1 + 8 - 2) = 5`. Price 4: `cash = 5`, `hold = max(-1, 5 - 4) = 1`. Price 9: `cash = max(5, 1 + 9 - 2) = 8`. Answer **8**.

SOLVED EXAMPLE 12.13 H

Stocks with At Most K Transactions (covers III and IV)

`buy[j]` = best balance after the j-th buy; `sell[j]` = best balance after the j-th sell. For **Stock III** set `k = 2`.

```
int maxProfitK(int k, const vector<int>& p) {
    int n = p.size();
    if (k >= n / 2) return maxProfitMany(p);            // effectively unlimited
    vector<int> buy(k + 1, INT_MIN), sell(k + 1, 0);
    for (int price : p) {
        for (int j = 1; j <= k; j++) {
            buy[j]  = max(buy[j],  sell[j - 1] - price);
            sell[j] = max(sell[j], buy[j] + price);
        }
    }
```

```
    return sell[k];
}
```

Trace with `k = 2` and prices `[3, 3, 5, 0, 0, 3, 1, 4]` (the shortcut `k >= n/2` is not taken, because `2 < 4`):

price	buy[1]	buy[2]	sell[1]	sell[2]
3	-3	-3	0	0
3	-3	-3	0	0
5	-3	-3	2	2
0	0	2	2	2
0	0	2	2	2
3	0	2	3	5
1	0	2	3	5
4	0	2	4	6

Answer `sell[2] = 6` (buy 0 sell 3, buy 1 sell 4, so 3 + 3). `buy[j]` is the best balance right after the `j`-th purchase and already contains the profit of the earlier transactions. If `k >= n/2`, you cannot make more than `n/2` profitable transactions, so the answer equals the unlimited version.

12.5 Pattern: Grid & Path DP

PATTERN CARD

SPOT IT WHEN	Move through a grid with right/down moves; counting/min-cost paths.
KNOW FIRST	2D DP indexing.
BREAKS WHEN	Moves in four directions with cycles (shortest-path algorithm instead).
COMMON VARIATIONS	Unique paths, min path sum, obstacles, largest square.

SOLVED EXAMPLE 12.14 — E

Minimum Cost Path

Move right, down, or diagonally down-right. `dp[i][j] = cost[i][j] + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])`.

```
int minCostPath(const vector<vector<int>>& cost) {
    if (cost.empty() || cost[0].empty()) return 0;
    int R = cost.size(), C = cost[0].size();
    vector<vector<int>> dp(R, vector<int>(C, 0));
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            int best = (i == 0 && j == 0) ? 0 : INT_MAX;
            if (i > 0) best = min(best, dp[i - 1][j]);
            if (j > 0) best = min(best, dp[i][j - 1]);
            if (i > 0 && j > 0) best = min(best, dp[i - 1][j - 1]);
            dp[i][j] = cost[i][j] + best;
        }
    }
    return dp[R - 1][C - 1];
```

```
}
```

Trace with `cost = [[1,2,3],[4,8,2],[1,5,3]]`. Row 0: `1, 3, 6`. Row 1: `4 + 1 = 5`; `8 + min(3, 5, 1) = 9`; `2 + min(6, 9, 3) = 5`. Row 2: `1 + 5 = 6`; `5 + min(9, 6, 5) = 10`; `3 + min(5, 10, 9) = 8`. Answer **8** along `1 -> 2 -> 2 -> 3`. An empty grid returns 0.

SOLVED EXAMPLE 12.15 M

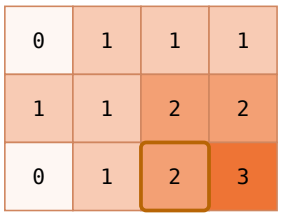
Max Size Square Sub-Matrix with All 1s

`dp[i][j]` = side of the largest all-ones square whose **bottom-right corner** is `(i, j)`. If the cell is 1: `1 + min(up, left, up-left)`.

```
int maxSquare(const vector<vector<int>>& m) {
    if (m.empty() || m[0].empty()) return 0;
    int R = m.size(), C = m[0].size(), best = 0;
    vector<vector<int>> dp(R, vector<int>(C, 0));
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) {
            if (m[i][j] == 0) continue;
           dp[i][j] = (i == 0 || j == 0) ? 1 : 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
            best = max(best, dp[i][j]);
        }
    return best;
}
```

Trace with the matrix `[[0,1,1,1],[1,1,1,1],[0,1,1,1]]`. The `dp` table (side of the largest square ending at each cell as its bottom-right corner) is

PICTURE IT



dp table: side of the largest square of 1s whose bottom-right corner is that cell

the largest value, 3, is bottom-right

For the bottom-right cell the up, left and up-left neighbours are all 2, so `1 + min(2, 2, 2) = 3`: a 3 x 3 block of ones. A cell in the first row or column can only be a 1 x 1 square. An empty matrix returns 0.

Problem	Level 2 hint (approach)

Practice Set 12.5

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Grid & Path DP* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Knight's Probability in a Chessboard M	`dp[step][r][c]` = probability of being at `(r, c)` after `step` moves; divide by 8 each step.
Cherry Pickup H	Think of two people walking down together simultaneously: state `(step, r1, r2)` since `c = step - r`.
Cherry Pickup II H	Two robots move down row by row: `dp[row][c1][c2]`, each of the 3 moves for both (9 transitions).
Dungeon Game H	Work **backward** from the princess: `dp[i][j] = max(1, min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j])`.

12.6 Pattern: Partition & Interval DP

PATTERN CARD

SPOT IT WHEN	'Split a sequence/expression optimally', 'cost to merge ranges'.
KNOW FIRST	DP on length; dp[i][j] = best for range [i..j].
BREAKS WHEN	n > ~500 ($\mathcal{O}(n^3)$ too slow) unless there is a known optimization.
COMMON VARIATIONS	Matrix chain, burst balloons, Catalan, palindrome partition cost.

Idea: `dp[l][r]` = answer for the segment `l..r`. Try every split point `k` between them; solve the two sides; combine. Always fill by **increasing length**.

SOLVED EXAMPLE 12.16 M

Matrix Chain Multiplication

Dimensions array `d` where matrix `i` has size `d[i-1] x d[i]`. Minimize scalar multiplications to multiply the chain.

```
int matrixChain(const vector<int>& d) {
    if (d.size() < 2) return 0;                          // no matrices
    int n = d.size() - 1;                                // number of matrices
    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++)
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] + d[i - 1] * d[k] * d[j]);
        }
    }
```

```
    return dp[1][n];
}
```

Trace with `d = [10, 30, 5, 60]`, meaning matrices `A1: 10x30`, `A2: 30x5`, `A3: 5x60`. `dp[1][2] = 10*30*5 = 1500`; `dp[2][3] = 30*5*60 = 9000`. For the full chain, split at `k = 1`: `dp[1][1] + dp[2][3] + 10*30*60 = 9000 + 18000 = 27000`; split at `k = 2`: `dp[1][2] + dp[3][3] + 10*5*60 = 1500 + 3000 = 4500`. The minimum is **4500**, meaning `(A1 A2) A3`. Lengths are filled in increasing order so that both halves are known when a range is processed; fewer than two numbers in `d` means no matrices (result 0).

SOLVED EXAMPLE 12.17 H

Burst Balloons

Key trick: think about the **last** balloon to burst in a range, not the first. If `k` is last in `(l, r)`, then its neighbors are the boundaries `l` and `r`, and the two sides are independent.

```
int maxCoins(vector<int>& nums) {
    vector<int> v = {1};
    v.insert(v.end(), nums.begin(), nums.end());
    v.push_back(1);
    int n = v.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));           // dp[l][r]: open interval (l, r)
    for (int len = 2; len < n; len++) {
        for (int l = 0; l + len < n; l++) {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
                dp[l][r] = max(dp[l][r], dp[l][k] + dp[k][r] + v[l] * v[k] * v[r]);
        }
    }
    return dp[0][n - 1];
}
```

Trace with `nums = [3, 1, 5]`, so `v = [1, 3, 1, 5, 1]`. Single-balloon ranges: `dp[0][2] = 1*3*1 = 3`, `dp[1][3] = 3*1*5 = 15`, `dp[2][4] = 1*5*1 = 5`. Longer ranges: `dp[0][3] = max(k=1: 0 + 15 + 1*3*5 = 30, k=2: 3 + 0 + 1*1*5 = 8) = 30`; `dp[1][4] = max(k=2: 5 + 3*1*1 = 8, k=3: 15 + 3*5*1 = 30) = 30`. Finally `dp[0][4]`: `k=1: 30 + 3 = 33`, `k=2: 3 + 5 + 1 = 9`, `k=3: 30 + 5 = 35`. Answer **35**: burst 1, then 3, then 5 (`3*1*5 + 1*3*5 + 1*5*1`). **Why "last"?** If `k` is the last balloon left in `(l, r)`, its neighbours when it bursts are exactly `l` and `r`, so the left and right parts no longer influence each other.

SOLVED EXAMPLE 12.18 M

Catalan Numbers: Total Number of BSTs

Choose root `i`; left subtree has `i-1` nodes, right has `n-i`. `C(n) = sum C(i) * C(n-1-i)`.

```
long long numBSTs(int n) {
    vector<long long> dp(max(n, 1) + 1, 0);              // room for dp[0] and dp[1] even when
↪   n = 0
    dp[0] = dp[1] = 1;
    for (int nodes = 2; nodes <= n; nodes++)
        for (int left = 0; left < nodes; left++)
            dp[nodes] += dp[left] * dp[nodes - 1 - left];
    return dp[n];
}
```

Trace. `dp[0] = dp[1] = 1`. `dp[2] = dp[0]*dp[1] + dp[1]*dp[0] = 2`. `dp[3] = dp[0]*dp[2] + dp[1]*dp[1] + dp[2]*dp[0] = 2 + 1 + 2 = 5`: with root 1 the right side has 2 nodes (2 shapes), with root 2 each side has one node (1 shape), with root 3 the left has 2 nodes (2 shapes). The code also works for `n = 0` (answer 1, the empty tree). `long long` is exact up to `n = 35`; `C(36)` overflows.

Applications of Catalan numbers (all give `C(n)`): valid parenthesizations with `n` pairs, ways to triangulate an `(n+2)`-gon, full binary trees with `n+1` leaves, Dyck paths, ways to parenthesize a product.

SOLVED EXAMPLE 12.19 E

Binomial Coefficient

Pascal's identity: `C(n, k) = C(n-1, k-1) + C(n-1, k)`.

```
long long binomial(int n, int k) {
    vector<long long> row(k + 1, 0);
    row[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = min(i, k); j > 0; j--) row[j] += row[j - 1];
    return row[k];
}
```

Trace of `binomial(4, 2)`: the row starts `[1, 0, 0]`. After `i = 1`: `[1, 1, 0]`; after `i = 2`: `[1, 2, 1]`; after `i = 3`: `[1, 3, 3]`; after `i = 4`: `[1, 4, 6]`. The answer is `row[2] = 6`. The inner loop runs **downwards** for the same reason as in 0/1 knapsack: `row[j]` must add the *old* `row[j-1]`, not the one just updated. The result overflows `long long` for large `n` (about `n > 66` in the middle of the row).

Problem	Level 2 hint (approach)

Practice Set 12.6

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Partition & Interval DP* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Boolean Parenthesization M	`dp[i][j][T/F]` = ways the sub-expression evaluates to true/false; combine around each operator.
Minimum and Maximum Value of an Expression M	Interval DP storing both `min` and `max` for each segment; combine around each operator.
Minimum Score Triangulation of a Polygon M	`dp[i][j] = min over k (dp[i][k] + dp[k][j] + v[i]*v[k]*v[j])`.
Highway Billboard Problem M	`dp[x]` = best revenue up to mile `x`: skip it, or place a billboard here and add `dp[x - gap - 1]`.
Optimal BST H	`dp[i][j] = sum(freq[i..j]) + min over root r (dp[i][r-1] + dp[r+1][j])`.

12.7 Pattern: String DP, LCS Family

PATTERN CARD

SPOT IT WHEN	Two strings: align, compare, transform.
KNOW FIRST	2D DP over prefixes.
BREAKS WHEN	Strings very long (needs $\mathcal{O}(n)$ space optimization or Hirschberg).
COMMON VARIATIONS	LCS, edit distance, longest common substring, distinct subsequences.

SOLVED EXAMPLE 12.20 M

Longest Common Subsequence

`dp[i][j]` = LCS length of the first `i` chars of `a` and the first `j` chars of `b`. If `a[i-1] == b[j-1]`: `1 + dp[i-1][j-1]`; else `max(dp[i-1][j], dp[i][j-1])`.

```
int lcs(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i - 1] == b[j - 1]) ? 1 + dp[i - 1][j - 1]
                                              : max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}
```

Trace with `a = "abcde", b = "ace"` (rows are prefixes of `a`, columns prefixes of `b`):

PICTURE IT

	""	a	c	e
""	0	0	0	0
a	0	1	1	1
ab	0	1	1	1
abc	0	1	2	2
abcd	0	1	2	2
abcde	0	1	2	3

rows: prefixes of a = "abcde"
columns: prefixes of b = "ace"

bottom-right cell: LCS length **3** ("ace")

`dp[3][2] = 2` because `c == c`, so it is `1 + dp[2][1]`. Where letters differ the cell copies the larger of the cell above and the cell to the left. Answer 3 (`ace`). To *recover* a common subsequence, start at `dp[n][m]`: on a match step diagonally and keep the letter; otherwise move towards the larger neighbour.

SOLVED EXAMPLE 12.21 M

Edit Distance

Minimum insert/delete/replace operations to turn `a` into `b`.

```
int editDistance(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i;              // delete all
    for (int j = 0; j <= m; j++) dp[0][j] = j;              // insert all
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1];
            else dp[i][j] = 1 + min({dp[i - 1][j - 1],      // replace
                                     dp[i - 1][j],          // delete
                                     dp[i][j - 1]});        // insert
        }
    return dp[n][m];
}
```

Trace with `a = "horse"`, `b = "ros"` (rows "", h, ho, hor, hors, horse; columns "", r, o, s):

PICTURE IT

	""	r	o	s
""	0	1	2	3
h	1	1	2	3
ho	2	2	1	2
hor	3	2	2	2
hors	4	3	3	2
horse	5	4	4	3

rows: "" h ho hor hors horse
columns: "" r o s

bottom-right cell: edit distance **3**
(horse → ros)

The first row and column are the cost of building one string from nothing (all insertions or all deletions).

Example cell `dp[2][2]` (`o` vs `o`) is a match, so it copies the diagonal `dp[1][1] = 1`. Cell `dp[1][1]` (`h` vs `r`) is `1 + min(replace 0, delete 1, insert 1) = 1`. Answer **3** (`horse -> rorse -> rose -> ros`: replace `h`, delete `r`, delete `e`).

SOLVED EXAMPLE 12.22 M

Longest Common Substring

Substrings must be **contiguous**: reset to 0 on mismatch and track the global maximum.

```
int longestCommonSubstring(const string& a, const string& b) {
    int n = a.size(), m = b.size(), best = 0;
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                best = max(best, dp[i][j]);
            }
    return best;
}
```

Trace with `a = "xabcy"` and `b = "abcz"`: matching cells only extend a diagonal, so `dp[i][j]` is the length of the common *suffix* of `a[0..i)` and `b[0..j)`. The diagonal `a-b-c` reaches 3 and the answer is 3 (`"abc"`). A mismatch resets the cell to 0, which is the difference from LCS (where a mismatch keeps the best of the neighbours).

SOLVED EXAMPLE 12.23 H

Distinct Subsequences

Count the ways `t` appears as a subsequence of `s`. `dp[i][j]` = ways using `s[0..i)` to form `t[0..j)`. Always you can skip `s[i-1]`; if it matches `t[j-1]` you can also use it.

```
long long numDistinct(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = 1;              // empty t: one way
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            dp[i][j] = dp[i - 1][j];                        // skip s[i-1]
            if (s[i - 1] == t[j - 1]) dp[i][j] += dp[i - 1][j - 1];
        }
    return dp[n][m];
}
```

Trace with `s = "aab"`, `t = "ab"` (the two `a`s are different positions). Column 0 is all 1. Row `a`: `dp[1][1] = dp[0][1] + dp[0][0] = 1` (match), `dp[1][2] = 0`. Row `aa`: `dp[2][1] = dp[1][1] + dp[1][0] = 2`, `dp[2][2] = 0`. Row `aab`: `dp[3][1] = 2` (the `b` cannot extend `t[0] = 'a'`), `dp[3][2] = dp[2][2] + dp[2][1] = 0 + 2 = 2`. Answer **2** (use either `a`, then the `b`). Counts can grow large; this version needs the answer to fit in `long long`.

Practice Set 12.7 (all variations on LCS)

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *String DP, LCS Family* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
LCS of Three Strings M	Add a third dimension: `dp[i][j][k]`.
Shortest Common Supersequence M	Length = `n + m - LCS(a, b)`.
Longest Repeating Subsequence M	LCS of the string with itself, but require `i != j` for a match.
Number of Sequences of Type $a^i\ b^j\ c^k$ M	Track counts: `a = 2a + 1` on `'a'`, `b = 2b + a` on `'b'`, `c = 2c + b` on `'c'`.

12.8 Pattern: Palindromes in Strings

PATTERN CARD

SPOT IT WHEN	Palindromic substring/subsequence questions.
KNOW FIRST	dp[i][j] over substrings; expand-around-center.
BREAKS WHEN	Only need the longest palindromic substring in $\mathcal{O}(n)$: Manacher.
COMMON VARIATIONS	Expand around centre, LPS via LCS(s, reverse s), min cuts.

SOLVED EXAMPLE 12.24 M

Longest Palindromic Subsequence

Trick: LPS of `s` = LCS of `s` and `reverse(s)`.

```
int longestPalindromicSubsequence(const string& s) {
    string r(s.rbegin(), s.rend());
    return lcs(s, r);                                    // lcs from Example 12.20
}
```

Trace with `s = "bbbab"`. The reverse is `"babbb"`; their LCS is `"bbbb"`, so the longest palindromic subsequence has length **4**. Idea: a palindromic subsequence reads the same forwards and backwards, which makes it a common subsequence of `s` and its reverse; and the longest common subsequence of the two can always be arranged symmetrically. This reuses `lcs` from Example 12.20, so keep both functions in the same file when you compile.

SOLVED EXAMPLE 12.25 M

Palindromic Substrings (Count)

Expand around centers: every palindrome has a center (a character, or between two characters): `2n - 1` centers.

```
int countSubstrings(const string& s) {
    int n = s.size(), count = 0;
    auto expand = [&](int l, int r) {
```

```
        while (l >= 0 && r < n && s[l] == s[r]) { count++; l--; r++; }
    };
    for (int c = 0; c < n; c++) { expand(c, c); expand(c, c + 1); }
    return count;
}
```

Trace with `s = "aaa"`. Centre 0: `expand(0,0)` counts `a` (1); `expand(0,1)` counts `aa` (2). Centre 1: `expand(1,1)` counts `a` (3) and then `aaa` (4); `expand(1,2)` counts `aa` (5). Centre 2: `expand(2,2)` counts `a` (6); `expand(2,3)` stops. Total **6** (`a, a, a, aa, aa, aaa`). Each centre can expand $\mathcal{O}(n)$ steps, so the total is $\mathcal{O}(n^2)$ with $\mathcal{O}(1)$ extra space.

SOLVED EXAMPLE 12.26 H

Palindrome Partitioning, Minimum Cuts

Precompute `isPal[i][j]`, then `cuts[i]` = min cuts for the prefix ending at `i`.

```
int minCut(const string& s) {
    int n = s.size();
    if (n == 0) return 0;
    vector<vector<bool>> isPal(n, vector<bool>(n, false));
    for (int len = 1; len <= n; len++)
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            isPal[i][j] = (s[i] == s[j]) && (len <= 2 || isPal[i + 1][j - 1]);
        }
    vector<int> cuts(n);
    for (int i = 0; i < n; i++) {
        if (isPal[0][i]) { cuts[i] = 0; continue; }
        cuts[i] = i;                                    // worst case: cut everywhere
        for (int j = 1; j <= i; j++)
            if (isPal[j][i]) cuts[i] = min(cuts[i], cuts[j - 1] + 1);
    }
    return cuts[n - 1];
}
```

Trace with `s = "aab"`. `cuts[0] = 0` (`a`), `cuts[1] = 0` (`aa` is a palindrome). For `i = 2`: `aab` is not a palindrome, so start from `cuts[2] = 2`; `j = 1` (`ab`) is not a palindrome; `j = 2` (`b`) is, so `cuts[2] = cuts[1] + 1 = 1`. Answer **1** (`aa | b`). `isPal[i][j]` is filled by increasing length so that the inner substring `isPal[i+1][j-1]` is known first; an empty string needs 0 cuts. Time and space are $\mathcal{O}(n^2)$.

Problem	Level 2 hint (approach)

Practice Set 12.8

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Palindromes in Strings* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Count All Palindromic Subsequences H	`dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]`, plus `dp[i+1][j-1] + 1` when `s[i] == s[j]`.
Count Distinct Palindromic Subsequences H	Interval DP with next/previous occurrence of `s[i]` inside `(i, j)` to avoid double counting.
Say No to Palindrome H	Any string of 3+ chars with no palindromic substring of length 2 or 3 is a repeated pattern; DP on the last two chosen characters.

12.9 Pattern: String DP, Pattern Matching

PATTERN CARD

SPOT IT WHEN	Matching a string against wildcards/regex.
KNOW FIRST	2D DP; meaning of '*' and '?'.
BREAKS WHEN	Complex regex features beyond the stated subset.
COMMON VARIATIONS	Wildcard, regex '.' and '*', interleaving strings.

SOLVED EXAMPLE 12.27 M

Word Break

`dp[i]` = can the prefix of length `i` be segmented? True if some `j < i` has `dp[j]` and `s[j..i)` is a dictionary word.

```
bool wordBreak(const string& s, vector<string>& dict) {
    unordered_set<string> words(dict.begin(), dict.end());
    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j < i; j++)
            if (dp[j] && words.count(s.substr(j, i - j))) { dp[i] = true; break; }
    return dp[n];
}
```

Trace with `s = "leetcode"` and `dict = {"leet", "code"}`. `dp[0] = true`. For `i = 1..3` no prefix is a word, so `false`. `dp[4] = true` (`j = 0`, `"leet"`). `dp[5..7]` are `false` (for `j = 4` the pieces `"c"`, `"co"`, `"cod"` are not words). `dp[8] = true` (`j = 4`, `"code"`). Answer `true`. The substring creation makes this $\mathcal{O}(n^2 \cdot L)$ with `L` the longest word.

SOLVED EXAMPLE 12.28 H

Wildcard Pattern Matching

? matches any one char; * matches any sequence (including empty).

```
bool isMatch(const string& s, const string& p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 1; j <= m; j++)
        if (p[j - 1] == '*') dp[0][j] = dp[0][j - 1];      // leading stars match empty
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (p[j - 1] == '*') dp[i][j] = dp[i][j - 1] || dp[i - 1][j];   // empty or consume
            ↪ one
            else if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) dp[i][j] = dp[i - 1][j - 1];
        }
    return dp[n][m];
}
```

Trace with `s = "ab"`, `p = "*b"`. Row for the empty text: `dp[0][0] = true`, `dp[0][1] = true` (a leading * matches nothing), `dp[0][2] = false`. Text `a`: `dp[1][1] = dp[1][0] || dp[0][1] = true` (* may swallow `a`), `dp[1][2] = false` (`b != a`). Text `ab`: `dp[2][1] = dp[2][0] || dp[1][1] = true`, `dp[2][2]`: `p[1] == s[1]` so it copies the diagonal `dp[1][1] = true`. Answer `true`.

Practice Set 12.9

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *String DP, Pattern Matching* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Regular Expression Matching H	* applies to the previous char: `dp[i][j] = dp[i][j-2] || (match(s[i-1], p[j-2]) && dp[i-1][j])`.
Scramble String H	`dp[len][i][j]` = can `s1[i..]` scramble to `s2[j..]` of length `len`; try every split with and without swapping.
Super Ugly Number M	Generalize Example 12.4 to `k` pointers over `k` primes.

12.10 Pattern: Game Theory DP & Miscellaneous

PATTERN CARD

SPOT IT WHEN	Two players, optimal play, 'maximize your score difference'.
KNOW FIRST	Minimax; difference DP.
BREAKS WHEN	Game has hidden information or cycles (not a simple DP).
COMMON VARIATIONS	Optimal strategy for coins, egg drop, counting arrangements.

SOLVED EXAMPLE 12.29 M

Optimal Strategy for a Game

Problem: two players alternately take a coin from either end of a row. Both play optimally. Find the maximum value the first player can secure.

Trick: let `dp[i][j]` = (current player's score) - (opponent's score) on coins `i..j`. Taking `a[i]` gives `a[i] - dp[i+1][j]` (the opponent then plays on the rest as the current player).

```
int optimalStrategy(const vector<int>& a) {
    int n = a.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) dp[i][i] = a[i];
    for (int len = 2; len <= n; len++)
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = max(a[i] - dp[i + 1][j], a[j] - dp[i][j - 1]);
        }
    int total = accumulate(a.begin(), a.end(), 0);
    return (total + dp[0][n - 1]) / 2;                    // first player's actual score
}
```

Trace with `a = [8, 15, 3, 7]`. Differences (`dp[i][j]` = current player's lead on `a[i..j]`): singles `8, 15, 3, 7`. Pairs: `(0,1) = max(8-15, 15-8) = 7`, `(1,2) = max(15-3, 3-15) = 12`, `(2,3) = max(3-7, 7-3) = 4`. Triples: `(0,2) = max(8 - 12, 3 - 7) = -4`, `(1,3) = max(15 - 4, 7 - 12) = 11`. Whole row: `(0,3) = max(8 - 11, 7 - (-4)) = 11`. The total is 33, so the first player gets `(33 + 11) / 2 = 22` and the second 11. Taking `a[i]` gives `a[i] - dp[i+1][j]` because after our move the opponent becomes the "current player" on the rest and their lead is subtracted from ours.

SOLVED EXAMPLE 12.30 H

Egg Dropping Puzzle

Reverse the question: with `e` eggs and `m` moves, what is the maximum number of floors we can distinguish? `f(e, m) = f(e-1, m-1) + f(e, m-1) + 1` (egg breaks: check below; survives: check above). Find the smallest `m` with `f(e, m) >= floors`.

```
int eggDrop(int eggs, int floors) {
    if (floors <= 0) return 0;
    if (eggs <= 0) return -1;                          // cannot test anything without eggs
    vector<int> f(eggs + 1, 0);                        // f[e] for the current number of moves
    int moves = 0;
    while (f[eggs] < floors) {
        moves++;
        for (int e = eggs; e >= 1; e--) f[e] = f[e] + f[e - 1] + 1;
    }
    return moves;                                      // eggDrop(2, 100) = 14
}
```

Trace with `eggs = 2`, `floors = 6`. `f = [0,0,0]` (`f[e]` = floors we can settle with `e` eggs and the current number of moves). Move 1: `f[2] = 0 + 0 + 1 = 1`, `f[1] = 1`. Move 2: `f[2] = 1 + 1 + 1 = 3`, `f[1] = 2`. Move 3: `f[2] = 3 + 2 + 1 = 6 >= 6`. Answer **3**. The loop over `e` goes downwards so that `f[e-1]` still holds the value from the previous move (as in 0/1 knapsack). The function returns 0 when there are no floors and -1 when there are floors but no eggs.

SOLVED EXAMPLE 12.31 M

Friends Pairing Problem

Person `n` stays single (`f(n-1)`) or pairs with one of `n-1` others (`(n-1) * f(n-2)`).

```
long long friendsPairing(int n) {
    long long a = 1, b = 1;                                  // f(0) = f(1) = 1
    for (int i = 2; i <= n; i++) { long long c = b + (i - 1) * a; a = b; b = c; }
    return b;
}
```

Trace. `f(0) = f(1) = 1`, `f(2) = 1 + 1*1 = 2` (`{1}{2}` or `{1,2}`), `f(3) = f(2) + 2*f(1) = 4` (`{1}{2}{3}`, `{1,2}{3}`, `{1,3}{2}`, `{2,3}{1}`), `f(4) = f(3) + 3*f(2) = 10`. `long long` is exact up to `n = 31` and overflows at `n = 32`, so use a modulus if the problem asks for one.

Practice Set 12.10

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Game Theory DP & Miscellaneous* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Max Sum Subarray with At Least K Elements M	Kadane's `best[i]` (best ending at `i`) plus a prefix-sum window of exactly `k`: `ans = max(window_sum(k) + max(0, best[i-k]))`.
Number of Digit One H	Digit DP: for each digit position, count by `high`, `cur`, `low` parts of the number.

CHAPTER COVERAGE

all 66 Dynamic Programming problems appear above.

CHAPTER 12 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 12.1 Frog Jump, 12.5 Coin Change, Problem 12A.
2. **Variations:** Ex 12.2 No Consecutive 1s, 12.6 Coin Change II, Problem 12B.
3. **Combined:** Ex 12.8 LIS, 12.10-12.13 Stocks, 12.20 LCS.
4. **Independent recognition:** Practice Sets 12.1 and 12.5 without the section names.
5. **Challenge:** Ex 12.17 Burst Balloons, 12.23 Distinct Subsequences, 12.28 Wildcard Matching.

Compare and Contrast

	Memoization (top-down)	Tabulation (bottom-up)
Style	recursion + cache	loops filling a table
Computes	only reachable states	all states
Risk	deep recursion / stack overflow	need the right loop order
Space tricks	harder	easy rolling arrays

	Greedy	DP
Choices	commit to one	consider all and compare

Why It Works (Correctness Corner)

Define, justify, order. (1) *State:* say in a sentence what `dp[i]` means (e.g. 'best loot from houses 0..i'). (2) *Recurrence:* argue every optimal solution for the state is built from optimal solutions of smaller states (optimal substructure). (3) *Order:* each cell must depend only on cells already computed.

Why 0/1 knapsack loops capacity downward. With a single 1D array, `dp[j]` should use `dp[j - w]` from the *previous item's* row. Iterating `j` from high to low guarantees `dp[j - w]` has not yet been overwritten by the current item; iterating upward would reuse the item many times (unbounded knapsack).

Debug Lab: 0/1 knapsack that reuses items

```
vector<int> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int j = wt[i]; j <= W; j++)          // <- direction
        dp[j] = max(dp[j], dp[j - wt[i]] + val[i]);
```

Your task: With one item (weight 1, value 5) and capacity 3 the code returns 15. Why?

ANSWER

Ascending `j` lets `dp[j - wt]` already include the current item, so the item is taken three times. Loop `j` from `W` down to `wt[i]` for 0/1 knapsack.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 12A E

House Robber

Statement. Houses lie in a row with `a[i]` money each. You cannot rob two adjacent houses. Return the maximum you can rob.

Example. `[2,7,9,3,1]` → `12`

Constraints. $0 \le n \le 100$; $0 \le a[i] \le 400$.

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 12.1 1D DP Foundations

HINT 1 which idea is relevant?

At each house you choose: rob or skip. What do you need to know about the previous house?

HINT 2 outline of the approach

Let `take` / `skip` be the best total if the previous house was / was not robbed.

HINT 3 the key step

`newTake = skip + a[i]`; `skip = max(skip, take)`; answer `max(take, skip)`.

SOLUTION and editorial (try first, then compare)

```cpp
int rob(const vector<int>& a) {
    int take = 0, skip = 0;          // best total if we DID / DID NOT rob the previous house
    for (int x : a) {
        int newTake = skip + x;      // robbing house i forces us to have skipped i-1
        skip = max(skip, take);
        take = newTake;
    }
    return max(take, skip);
}
```

State: best total for the prefix with a flag for the last house. Only the previous state is needed, so two variables replace the table.

PROBLEM 12B M

Unique Paths with Obstacles

Statement. A robot starts at the top-left of an `m x n` grid and moves only right or down to reach the bottom-right. Cells with `1` are obstacles. Count the paths (fits in 32-bit).

Example. `[[0,0,0],[0,1,0],[0,0,0]]` → `2`

Constraints. $1 \le m, n \le 100$.

Expected complexity. $\mathcal{O}(mn)$ time, $\mathcal{O}(n)$ space.

Prerequisites. 12.5 Grid & Path DP

HINT 1 which idea is relevant?

Every path enters a cell from above or from the left. What does that say about the count for a cell?

HINT 2 outline of the approach

`ways(i,j) = ways(i-1,j) + ways(i,j-1)`, but 0 for obstacles.

HINT 3 the key step

Use one row `dp[j]`: before adding, `dp[j]` still holds the value from the row above.

SOLUTION and editorial (try first, then compare)

```cpp
int uniquePathsWithObstacles(const vector<vector<int>>& g) {
    int m = g.size(), n = g[0].size();
    vector<long long> dp(n, 0);      // dp[j] = ways to reach (current row, j)
    dp[0] = (g[0][0] == 0);
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++) {
            if (g[i][j] == 1) dp[j] = 0;            // blocked cell: no way in
           else if (j > 0) dp[j] += dp[j - 1];     // from above (old dp[j]) + from the left
        }
    return (int)dp[n - 1];
}
```

Rolling one row works because `dp[j]` (old) is 'from above' and `dp[j-1]` (new) is 'from the left'. Handle the start cell being blocked: `dp[0] = (g[0][0] == 0)`.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T12.1 Count the ways to climb n steps taking 1, 2 or 3 steps at a time.

PATTERN

1D DP: ways[i] = ways[i-1] + ways[i-2] + ways[i-3].

T12.2 Fewest single-character edits to turn one word into another.

PATTERN

2D string DP: edit distance.

Mixed Practice

Combine 5.1 and 12.3: LIS in $\mathcal{O}(n \log n)$ uses a `tails` array and binary search. Explain what `tails[k]` stores and why it stays sorted.

Revision Sheet

Key takeaways

- Write the state in words first.
- Recurrence = last choice; base cases = smallest inputs.
- 0/1: capacity downward; unbounded: upward.
- Combinations: coins outer; permutations: target outer.

Pattern-recognition clues

- 'Count ways', 'min cost' with overlapping subproblems → DP.
- 'Pick items with capacity' → knapsack.
- 'Two strings' → 2D prefix DP.

- 'Split at every point' → interval DP.

Common mistakes

- Vague state definitions.
- Wrong loop direction/order.
- Uninitialised base cases or `INT_MAX + 1` overflow.

Complexity summary

Family	Time	Space (optimised)
1D	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Knapsack	$\mathcal{O}(nW)$	$\mathcal{O}(W)$
LCS / edit distance	$\mathcal{O}(mn)$	$\mathcal{O}(\min(m, n))$
Interval DP	$\mathcal{O}(n^3)$	$\mathcal{O}(n^2)$

Recall questions (answer aloud, then check the chapter)

1. What does your `dp[i]` mean, in one sentence?
2. Why does 0/1 knapsack iterate capacity downward?
3. Memoization vs tabulation: one advantage of each?
4. How do you count combinations rather than permutations?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Pick a DP problem you have never seen. Before coding, write: state meaning, recurrence, base cases, order, answer cell, complexity. Then code and test with n = 0, 1 and a hand-made example.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 13

String Matching Algorithms

13

START HERE

Search a 3 GB genome for a 30-letter sequence. Comparing at every position is slow in the worst case. KMP never moves backwards in the text, giving $\mathcal{O}(n+m)$: the trick is to reuse what you have already matched.

ANALOGY

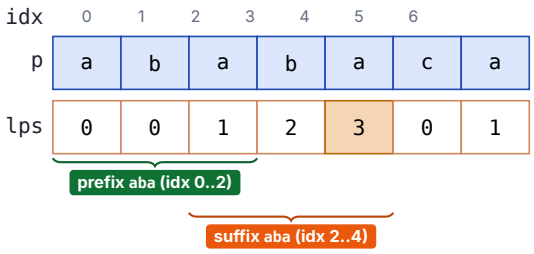
When reading 'abababc' looking for 'ababc', after matching 'abab' and failing on the next letter you do not restart; you notice 'ab' is both a prefix and a suffix of what you matched and resume from there.

AFTER THIS CHAPTER YOU CAN

- ✓ Explain and build the prefix function (lps).
- ✓ Search with KMP, Z-algorithm or a rolling hash.
- ✓ Use lps for periodic strings and shortest palindrome tricks.
- ✓ Choose between the algorithms and `std::string::find`.

PICTURE IT

The lps (prefix function) of ababaca



`lps[4]=3`: the prefix `"aba"` is also the suffix of `"ababa"`.

Mismatch after matching 5 chars? Fall back to $k = \texttt{lps}[4] = 3$, not to 0.

13.0 Concept

Problem: find where a pattern `p` (length m) occurs in a text `s` (length n). Brute force compares at every start: $\mathcal{O}(n \cdot m)$. The algorithms below get $\mathcal{O}(n+m)$ by **never re-examining** characters unnecessarily.

Algorithm	Core idea	Time
KMP	precompute the longest proper prefix that is also a suffix (LPS)	$\mathcal{O}(n+m)$
Z-algorithm	for each position, length of match with the string's prefix	$\mathcal{O}(n+m)$
Rabin-Karp	compare rolling hash values, verify on collision	$\mathcal{O}(n+m)$ average

Algorithm	Core idea	Time
Manacher	longest palindromic substring using mirror symmetry	$\mathcal{O}(n)$

13.1 KMP Algorithm

PATTERN CARD

SPOT IT WHEN	Find a pattern in a text in $\mathcal{O}(n+m)$.
KNOW FIRST	Prefix function (lps) meaning.
BREAKS WHEN	Pattern is tiny and text small (std::string::find is enough).
COMMON VARIATIONS	Search, shortest palindrome, repeated-substring test.

LPS array: `lps[i]` = length of the longest proper prefix of `p[0..i]` that is also a suffix of it. When a mismatch occurs after matching `j` characters, we do not restart: we fall back to `lps[j-1]`, because that prefix is already matched.

Example for `p = "ababaca"`: `lps = [0, 0, 1, 2, 3, 0, 1]`.

Building `lps` by hand for `p = "ababaca"`. `len` is the length of the border we are currently extending. `i = 1` (`b`): `b != p[0] = a` and `len = 0`, so `lps[1] = 0`. `i = 2` (`a`): equals `p[0]`, so `len = 1`, `lps[2] = 1`. `i = 3` (`b`): equals `p[1]`, `len = 2`, `lps[3] = 2`. `i = 4` (`a`): equals `p[2]`, `len = 3`, `lps[4] = 3`. `i = 5` (`c`): `c != p[3] = b`; instead of giving up, fall back: `len = lps[2] = 1` (`c != p[1] = b`), then `len = lps[0] = 0` (`c != p[0]`), so `lps[5] = 0`. `i = 6` (`a`): equals `p[0]`, `len = 1`, `lps[6] = 1`. The fallback `len = lps[len-1]` asks "what is the next-longest border of the border I had?", and it is the same move KMP makes inside the text.

SOLVED EXAMPLE 13.1 — M

KMP Search

```cpp
vector<int> buildLPS(const string& p) {
    int m = p.size();
    vector<int> lps(m, 0);
    for (int i = 1, len = 0; i < m;) {
        if (p[i] == p[len]) lps[i++] = ++len;
        else if (len > 0)   len = lps[len - 1];            // fall back, don't advance i
        else                lps[i++] = 0;
    }
    return lps;
}

vector<int> kmpSearch(const string& s, const string& p) {
    vector<int> lps = buildLPS(p), matches;
    int n = s.size(), m = p.size();
    if (m == 0) return matches;                                  // empty pattern: nothing to
    ↪ search
    for (int i = 0, j = 0; i < n;) {
        if (s[i] == p[j]) { i++; j++; }
        if (j == m) { matches.push_back(i - m); j = lps[j - 1]; }   // keep searching
        else if (i < n && s[i] != p[j]) {
            if (j > 0) j = lps[j - 1];
            else i++;
        }
```

```
    }
    return matches;                                          // starting indices
}
```

Trace of the search with `s = "abcababcab"`, `p = "abcab"` (`lps = [0,0,0,1,2]`). The first five characters match, so `i = 5`, `j = 5 = m`: record start `5 - 5 = 0` and set `j = lps[4] = 2` (the `ab` at the end of the match may begin the next one; we keep `i`). Now `s[5] = a` but `p[2] = c`: mismatch, so `j = lps[1] = 0` (the text pointer does not move). `s[5] = a = p[0]`, so `i = 6`, `j = 1`; then `b`, `c`, `a`, `b` match, `i = 10`, `j = 5`: record start `10 - 5 = 5`. Result `[0, 5]`. The text index `i` never decreases. The code assumes a non-empty pattern: with `p = ""` it returns no matches (an empty pattern has no sensible "first position"), and `std::string::find` returns 0 for it.

SOLVED EXAMPLE 13.2 H

Shortest Palindrome (via KMP)

Problem: add characters only at the **front** to make `s` a palindrome; find the shortest result.

Insight: we need the longest palindromic **prefix** of `s`. Build `t = s + "#" + reverse(s)`; the last value of `lps(t)` is the length of that prefix. Then prepend the reverse of the remaining suffix.

```
string shortestPalindrome(const string& s) {
    string rev(s.rbegin(), s.rend());
    string t = s + "#" + rev;
    vector<int> lps = buildLPS(t);
    int palPrefix = lps.back();
    return rev.substr(0, s.size() - palPrefix) + s;
}
```

Trace with `s = "aacecaaa"`. Its reverse is `"aaacecaa"`, so `t = "aacecaaa#aaacecaa"`. The last `lps` value is 7: the prefix `"aacecaa"` of `s` equals the suffix of `t` (the end of the reversed copy), meaning `"aacecaa"` is a palindrome that starts at the front of `s`. Only the last character `a` is left over, so we put its mirror in front: `rev.substr(0, 8 - 7) + s = "a" + "aacecaaa" = "aaacecaaa"`. The `#` stops a border from running across the join; it must be a character that does not occur in `s`. For an empty string the answer is the empty string.

13.2 Z Algorithm

PATTERN CARD

SPOT IT WHEN	Same as KMP but you want 'match length at each position'.
KNOW FIRST	Z-array meaning.
BREAKS WHEN	Forgetting the separator between pattern and text (matches cross the border).
COMMON VARIATIONS	Pattern+'#'+text, string periods.

`Z[i]` = length of the longest substring starting at `i` that equals a prefix of the string. To search `p` in `s`, build `p + "$" + s`; every position where `Z == |p|` is a match.

SOLVED EXAMPLE 13.3 M

Z Algorithm

```
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);           // reuse info from the current Z-box
        ↪ [l, r)
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

vector<int> zSearch(const string& s, const string& p) {
    string t = p + "$" + s;
    vector<int> z = zFunction(t), res;
    for (int i = p.size() + 1; i < (int)t.size(); i++)
        if (z[i] == (int)p.size()) res.push_back(i - p.size() - 1);
    return res;
}
```

Trace of the Z-array for `s = "aabxaab"`: `z = [0, 1, 0, 0, 3, 1, 0]` (by convention `z[0] = 0`). At `i = 1` the match with the prefix is `a`, length 1, and the Z-box becomes `[1, 2)`. At `i = 4` the substring `aab` equals the prefix, length 3, box `[4, 7)`. At `i = 5` we are inside the box, so we start from `min(r - i, z[i - l]) = min(2, z[1] = 1) = 1` without comparing anything, then try to extend: `s[1] = a` against `s[6] = b` fails, so `z[5] = 1`. **Why linear:** the box's right end `r` only moves right, and every comparison either extends `r` or ends the `while` loop, so there are $\mathcal{O}(n)$ comparisons in total. **Search trace:** `p = "aab"`, `s = "xaabaab"`: `t = "aab$xaabaab"`, `z = [0,1,0,0,0,3,1,0,3,1,0]`; the values equal to 3 are at `i = 5` and `i = 8`, giving starts `5 - 3 - 1 = 1` and `8 - 3 - 1 = 4`. The `$` must not occur in either string, otherwise a match could run across the border. Like `kmpSearch`, `zSearch` expects a non-empty pattern.

13.3 Rabin-Karp Algorithm

PATTERN CARD

SPOT IT WHEN	Many patterns / 'any substring equals' / 2D matching.
KNOW FIRST	Rolling hash; modular arithmetic.
BREAKS WHEN	Adversarial anti-hash tests (use a random base / two mods).
COMMON VARIATIONS	Single pattern, multiple patterns, longest duplicate substring.

Idea: treat each window as a number in base `B` modulo a big prime `M`. Sliding the window updates the hash in $\mathcal{O}(1)$: remove the leading character, shift, add the new one. If hashes match, confirm by direct comparison (guards against collisions).

SOLVED EXAMPLE 13.4 M

Rabin-Karp

```
vector<int> rabinKarp(const string& s, const string& p) {
    const long long B = 257, M = 1000000007;
    int n = s.size(), m = p.size();
    vector<int> res;
    if (n < m) return res;
    long long hp = 0, hs = 0, power = 1;                    // power = B^(m-1) mod M
    for (int i = 0; i < m; i++) {
        hp = (hp * B + (unsigned char)p[i]) % M;
        hs = (hs * B + (unsigned char)s[i]) % M;
        if (i > 0) power = power * B % M;
    }
    for (int i = 0; i + m <= n; i++) {
        if (hp == hs && s.compare(i, m, p) == 0) res.push_back(i);
        if (i + m < n) {
            hs = (hs - (unsigned char)s[i] * power % M + M) % M;          // drop s[i]
            hs = (hs * B + (unsigned char)s[i + m]) % M;                  // add s[i+m]
        }
    }
    return res;
}
```

How the rolling update works. Write the window as a number in base `B`: for a window `s[i..i+m)` the hash is `s[i]*B^(m-1) + s[i+1]*B^(m-2) + ... + s[i+m-1]`. Toy version with base 10 and letters `a=1, b=2, c=3, d=4`: `"abc"` hashes to `123`. To slide to `"bcd"`, drop the leading `a` (subtract `1 * 100`, where `100 = B^(m-1)` is the variable `power`), shift everything left (`* 10`) and add the new letter: `(123 - 100) * 10 + 4 = 234`. The real code does the same arithmetic modulo `M` (the `+ M` keeps the value non-negative) and converts each byte through `unsigned char`, so letters above 127 behave too. Two different strings can share a hash, so every hash match is confirmed with `s.compare`; a bad hash can only slow the code down (for example, when every window collides), never make it wrong. With the fixed base 257 an adversary can build colliding inputs; real systems pick a random base or two moduli.

Rabin-Karp shines for **multiple pattern** search and for substring-equality checks via hashing.

13.4 Manacher's Algorithm

PATTERN CARD

SPOT IT WHEN	'Longest palindromic substring' in $\mathcal{O}(n)$.
KNOW FIRST	Palindrome mirror property.
BREAKS WHEN	Skipping the separator trick for even-length palindromes.
COMMON VARIATIONS	Manacher odd/even unified, count palindromic substrings.

Goal: the longest palindromic substring in $\mathcal{O}(n)$. **Idea:** insert `#` between characters (so all palindromes are odd length), keep the palindrome with the rightmost boundary `[l, r]`, and for position `i` use its **mirror** to start with a known lower bound for the radius.

SOLVED EXAMPLE 13.5 H

Manacher's Algorithm

```
string longestPalindrome(const string& s) {
    string t = "#";
    for (char c : s) { t += c; t += '#'; }
    int n = t.size();
    vector<int> rad(n, 0);                              // palindrome radius at each center in t
    int center = 0, right = 0, bestC = 0, bestR = 0;
    for (int i = 0; i < n; i++) {
        if (i < right) rad[i] = min(right - i, rad[2 * center - i]);   // mirror
        while (i - rad[i] - 1 >= 0 && i + rad[i] + 1 < n && t[i - rad[i] - 1] == t[i + rad[i] +
        ↪  1])
            rad[i]++;
        if (i + rad[i] > right) { center = i; right = i + rad[i]; }
        if (rad[i] > bestR) { bestR = rad[i]; bestC = i; }
    }
    int start = (bestC - bestR) / 2;                        // map back to the original string
    return s.substr(start, bestR);
}
```

Trace with `s = "babad"`. After inserting separators `t = "#b#a#b#a#d#"` (indices 0..10). At `i = 3` (the `a`): the neighbours `#`/`#`, then `b`/`b`, then `#`/`#` match, so `rad[3] = 3`, which is the palindrome `bab` of length 3 in `s`; the box is now centre 3, right end 6. At `i = 4` (`#`): the mirror position is `2 * 3 - 4 = 2` with `rad = 0`, so we start at 0 and the first comparison (`a` against `b`) fails. At `i = 5` (the second `b`): the mirror is index 1 with radius 1, capped by `right - i = 1`, so we start at 1 and extend to 3 (`aba`). The best radius is 3, first found at centre 3 (the comparison is strict), so the answer is `s.substr((3 - 3) / 2, 3) = "bab"`. **Why separators?** Without `#`, even-length palindromes such as `abba` have no single centre; with them every palindrome is odd-length in `t`, and the radius in `t` equals the palindrome's length in `s`. **Why** $\mathcal{O}(n)$**:** `right` only moves forward, and the `while` loop runs only when it extends `right`, because mirrored positions inside the box start with a guaranteed lower bound. An empty input returns an empty string.

Practice Set 13

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Manacher's Algorithm* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Chef and Secret Password M	Read the statement: it is typically a pattern/period problem (the statement is not in the sheet; check it). Compute the LPS (or Z) array: the smallest period is `n - lps[n-1]`, and the string is an exact repetition only if that period divides `n`. This row uses KMP, not Manacher, so the Level 1 sentence above does not apply to it.

CHAPTER COVERAGE

all 6 String Matching problems appear above (KMP, Z, Rabin-Karp, Shortest Palindrome, Manacher, and the practice item).

CHAPTER 13 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 13.1 KMP, Problem 13A.
2. **Variations:** Problem 13B (periodic strings), Ex 13.3 Z-algorithm.
3. **Combined:** Ex 13.2 Shortest Palindrome.
4. **Independent recognition:** Practice (find all anagram windows with hashing).
5. **Challenge:** Ex 13.5 Manacher.

Compare and Contrast

	KMP	Z-algorithm	Rabin-Karp
Core	prefix function	match length at each position	rolling hash
Worst time	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$	$\mathcal{O}(nm)$ worst, $\mathcal{O}(n+m)$ expected
Strength	streaming text	easy to reason about	many patterns, 2D
Weakness	subtle fallback	needs separator	hash collisions

Why It Works (Correctness Corner)

KMP amortised bound. `k` (matched length) increases by at most 1 per text character, and each fallback `k = lps[k-1]` strictly decreases `k`. So total fallbacks $\leq$ total increments $\leq$ n, giving $\mathcal{O}(n)$. The same argument bounds building `lps` by $\mathcal{O}(m)$. **Invariant:** after processing `text[i]`, `k` is the length of the longest prefix of the pattern that is a suffix of `text[0..i]`.

Debug Lab: Building lps from index 0

```cpp
vector<int> buildLps(const string& p) {
    vector<int> lps(p.size(), 0);
    for (int i = 0, k = 0; i < (int)p.size(); i++) {   // <- starts at 0
        while (k > 0 && p[i] != p[k]) k = lps[k - 1];
        if (p[i] == p[k]) k++;
        lps[i] = k;
    }
    return lps;
}
```

Your task: What does it return for "abab"? What should it return?

ANSWER

For `"abab"` it returns `[1, 2, 3, 4]` (we ran it): at `i = 0` it compares `p[0]` with itself, sets `k = 1` and `lps[0] = 1`, and every later character then matches "itself" too. The correct table is `[0, 0, 1, 2]`. `lps[0]` is always 0 (a proper prefix must be shorter), so start with `i = 1`.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 13A M

Find the First Occurrence

Statement. Return the index of the first occurrence of `pat` in `text`, or -1. An empty pattern returns 0.

Example. `text = "hello", pat = "ll"` → 2

Constraints. |text|, |pat| $\leq 10^6$.

Expected complexity. $\mathcal{O}(n + m)$ time.

Prerequisites. 13.1 KMP

HINT 1 which idea is relevant?

Brute force restarts at every index. What information from a failed attempt could be reused?

HINT 2 outline of the approach

The `lps` table says how far to fall back inside the pattern.

HINT 3 the key step

Build `lps`; scan text keeping `k`; on mismatch `k = lps[k-1]`; when `k == m` you found a match.

SOLUTION and editorial (try first, then compare)

```
vector<int> buildLps(const string& p) {      // lps[i] = length of the longest proper
↪  prefix of p[0..i] that is also its suffix
    vector<int> lps(p.size(), 0);
    for (int i = 1, k = 0; i < (int)p.size(); i++) {
        while (k > 0 && p[i] != p[k]) k = lps[k - 1];   // fall back, never move i backwards
        if (p[i] == p[k]) k++;
        lps[i] = k;
    }
    return lps;
}
int strStr(const string& text, const string& pat) {
    if (pat.empty()) return 0;
    vector<int> lps = buildLps(pat);
    for (int i = 0, k = 0; i < (int)text.size(); i++) {
        while (k > 0 && text[i] != pat[k]) k = lps[k - 1];
        if (text[i] == pat[k]) k++;
        if (k == (int)pat.size()) return i - k + 1;      // full match ends at i
    }
    return -1;
}
```

The text index never moves backwards. The invariant ('k is the longest pattern prefix ending at i') is what makes the fallback correct.

PROBLEM 13B E

Repeated Substring Pattern

Statement. Decide whether a string can be built by repeating one of its proper substrings two or more times.

Example. `"abab"` → `true`; `"aba"` → `false`

Constraints. $1 \leq n \leq 10^5$.

Expected complexity. $\mathcal{O}(n)$ time.

Prerequisites. 13.1 KMP, Problem 13A

HINT 1 which idea is relevant?

If `s` is made of copies of a block, the string has a long border (prefix = suffix). What would the border be?

HINT 2 outline of the approach

Let `b = lps[n-1]`. The candidate block length is `n - b`.

HINT 3 the key step

Answer is true iff `b > 0` and `n % (n - b) == 0`.

SOLUTION and editorial (try first, then compare)

```cpp
bool repeatedSubstringPattern(const string& s) {
    int n = s.size();
    if (n < 2) return false;
    int border = buildLps(s)[n - 1];     // longest proper prefix that is also a suffix
    int period = n - border;             // smallest candidate repeating unit
    return border > 0 && n % period == 0;
}
```

A string with period `p` has border length `n - p`; conversely the smallest period is `n - lps[n-1]`. The string is a repetition only if that period divides `n`.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T13.1 Does a 30-letter DNA read occur anywhere in a 10^6-letter genome?

PATTERN

KMP / Z / rolling hash.

T13.2 Find the longest prefix of a word that is also its suffix (not the whole word).

PATTERN

`lps[n-1]` directly.

Mixed Practice

Combine 13.1 and 12.8: the shortest palindrome by prepending characters uses lps on `s + '#' + reverse(s)`. Explain what the last lps value means.

Revision Sheet

Key takeaways

- lps[i] = longest proper prefix of p[0..i] that is also a suffix.
- Never move the text pointer backward.
- Use a separator in Z-based search.
- Hashing: pick random base/two mods.

Pattern-recognition clues

- 'Find pattern in text' → KMP/Z.
- 'Period', 'repeated' → lps.
- 'Many patterns / substrings equal' → hashing.

Common mistakes

- Starting lps at i = 0.
- Forgetting `k = lps[k-1]` after a full match when finding all matches.
- Weak hash with a small modulus.

Complexity summary

Algorithm	Time	Space
KMP	$\mathcal{O}(n+m)$	$\mathcal{O}(m)$
Z	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$
Rabin-Karp	$\mathcal{O}(n+m)$ expected	$\mathcal{O}(1)$
Manacher	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Recall questions (answer aloud, then check the chapter)

1. Define lps in one sentence.
2. Why is KMP linear?
3. Why do we need a separator in Z-search?
4. What does `n - lps[n-1]` represent?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Implement `lps` from memory, hand-run it on 'aabaaab', then use it to solve 13A and 13B. Test with an empty pattern and a pattern longer than the text.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 14

Backtracking

14

START HERE

List every way to place 8 queens on a chessboard so none attack each other. There are 92. Brute force looks at billions of boards; backtracking abandons a branch the moment it breaks a rule. The pattern: choose, explore, un-choose.

ANALOGY

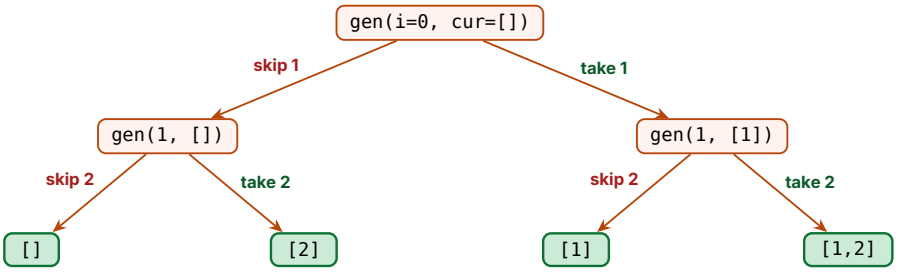
Exploring a maze with chalk: mark your path, walk forward, and when you hit a dead end rub out the chalk and return to the last junction.

AFTER THIS CHAPTER YOU CAN

- ✓ Write the choose / explore / undo template and state the invariant.
- ✓ Prune a search with feasibility checks.
- ✓ Avoid duplicates by sorting and skipping.
- ✓ Estimate when backtracking is too slow (n beyond ~20-25).

PICTURE IT

Decision tree for subsets of [1, 2]



Each leaf is one subset; "undo" = `pop_back` when returning from a take branch.

14.0 Concept

Backtracking explores all possibilities by building a solution one choice at a time, and **undoing** a choice (backtracking) when it leads to a dead end. It is DFS over the tree of choices, with **pruning** to cut hopeless branches.

The Universal Template

```
void solve(state) {
    if (isComplete(state)) { record(state); return; }      // base case
    for (choice : choicesAt(state)) {
        if (!isValid(choice)) continue;                    // prune
        apply(choice);                                     // choose
        solve(nextState);                                  // explore
        undo(choice);                                      // un-choose (backtrack)
    }
}
```

Complexity: typically exponential (`2^n` for subsets, `n!` for permutations); pruning is what makes it practical.

Pattern	Recognition clue
Generation	permutations, combinations, subsets
Grid path search	maze, paths with obstacles
Constraint satisfaction	N-Queens, Sudoku, coloring
String partitioning	palindrome partitions, word break (all solutions)

14.1 Pattern: Combinatorial Generation

PATTERN CARD

SPOT IT WHEN	'All subsets / permutations / combinations / partitions'.
KNOW FIRST	Recursion with state; undo step.
BREAKS WHEN	n larger than ~20-25 (exponential blow-up); then DP or math.
COMMON VARIATIONS	Include/exclude, swap-based permutation, choose-from-index, duplicates via sort + skip.

SOLVED EXAMPLE 14.1 — M

Print All Permutations of a String

Approach: fix a position, try every unused character there, recurse. A `used` array tracks what has been placed.

```
void permute(string& s, string& cur, vector<bool>& used, vector<string>& out) {
    if (cur.size() == s.size()) { out.push_back(cur); return; }
    for (int i = 0; i < (int)s.size(); i++) {
        if (used[i]) continue;
        // For duplicate characters: sort s first and add
        // if (i > 0 && s[i] == s[i-1] && !used[i-1]) continue;
        used[i] = true; cur.push_back(s[i]);               // choose
        permute(s, cur, used, out);                        // explore
        cur.pop_back(); used[i] = false;                   // un-choose
    }
}
```

How to call it, and a trace. Prepare `string cur; vector<bool> used(s.size(), false); vector<string> out;` and call `permute(s, cur, used, out)`. For `s = "ab"`: choose `a` (position 0), then the only unused letter `b`: `cur = "ab"` is complete, record it; undo `b`, undo `a`. Choose `b`, then `a`: record `"ba"`.

Two letters give 2 results, `n` distinct letters give `n!`. **Duplicates:** for `"aab"` first sort the string and enable the commented rule: it says "do not start a branch with the same letter as the previous *unused* twin", so the result is `aab, aba, baa` (3) instead of six strings with repeats.

SOLVED EXAMPLE 14.2 M

Combination Sum

Problem: find all combinations of candidates summing to `target`; each candidate may be reused.

Idea: at each step pick a candidate index `>= start` (avoids duplicate combos in different orders). Reuse is allowed, so recurse with the **same** index.

```
void comb(vector<int>& c, int start, int remain, vector<int>& cur, vector<vector<int>>& out) {
    if (remain == 0) { out.push_back(cur); return; }
    for (int i = start; i < (int)c.size(); i++) {
        if (c[i] > remain) continue;                        // (break if c is sorted)
        cur.push_back(c[i]);
        comb(c, i, remain - c[i], cur, out);                // i, not i+1: reuse allowed
        cur.pop_back();
    }
}
```

Trace with `c = [2, 3, 6, 7]`, `target = 7`. Start `[]`, remain 7. Take 2 (`[2]`, remain 5), take 2 again (`[2,2]`, remain 3), take 2 again would leave remain 1 and nothing fits, so try 3 instead: `[2,2,3]`, remain 0: record. Undo back to `[2]`, try 3 (`[2,3]`, remain 2): nothing fits; try 6 and 7: too big. Back at `[]`: 3, 6 fail; 7: `[7]`, remain 0: record. Result `[[2,2,3],[7]]`. Starting each loop at `start = i` keeps candidates in non-decreasing order, so `[2,3,2]` is never produced as a second copy of `[2,2,3]`. The candidates must be positive; a candidate `0` would recurse forever.

SOLVED EXAMPLE 14.3 M

Find the K-th Permutation Sequence

No need to generate all. With `n` digits there are `(n-1)!` permutations for each leading digit. Use this to pick digits directly (factorial number system).

```
string getPermutation(int n, int k) {
    vector<int> fact(n, 1);
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    string digits, res;
    for (int i = 1; i <= n; i++) digits += char('0' + i);
    k--;                                                    // zero-based
    for (int i = n; i >= 1; i--) {
        int idx = k / fact[i - 1];
        res += digits[idx];
        digits.erase(digits.begin() + idx);
        k %= fact[i - 1];
    }
    return res;
}
```

Trace with `n = 3`, `k = 3`. `fact = [1, 1, 2]`, `k = 2` (zero-based), `digits = "123"`. First digit: `idx = 2 / fact[2] = 1`, so `'2'`; `digits = "13"`, `k = 2 % 2 = 0`. Second digit: `idx = 0 / 1 = 0`, so `'1'`; `digits = "3"`. Last digit `'3'`. The result `"213"` is the third permutation in order (`123, 132, 213`). Each leading digit owns a block of `(n-1)!` permutations, and `k / (n-1)!` says which block we are in. The code uses digits `1..n` with `n <= 9`, `1 <= k <= n!`, and an `int` factorial (valid up to `12!`).

Practice Set 14.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Combinatorial Generation* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Find Maximum Number Possible by At-Most K Swaps M	Recursion: for position `i`, find the max digit to the right; try swapping with every occurrence; recurse with `k - 1`; undo.

14.2 Pattern: Grid / Path Backtracking

PATTERN CARD

SPOT IT WHEN	Paths in a grid/maze; 'find all' or 'does a path exist'.
KNOW FIRST	DFS on grid, visited and un-visit.
BREAKS WHEN	Only need the shortest path (BFS is better than backtracking).
COMMON VARIATIONS	Rat in a maze, word search, all paths.

SOLVED EXAMPLE 14.4 M

Rat in a Maze

Problem: find all paths from `(0,0)` to `(n-1,n-1)` moving D, L, R, U through cells equal to 1, never revisiting a cell.

```
void ratPaths(vector<vector<int>>& m, int r, int c, string& path, vector<string>& out) {
    int n = m.size();
    if (n == 0) return;                                   // empty maze: no path
    if (r == n - 1 && c == n - 1) { out.push_back(path); return; }
    m[r][c] = 0;                                          // mark visited
    int dr[] = {1, 0, 0, -1}, dc[] = {0, -1, 1, 0};
    char mv[] = {'D', 'L', 'R', 'U'};
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr >= 0 && nc >= 0 && nr < n && nc < n && m[nr][nc] == 1) {
            path.push_back(mv[d]);
            ratPaths(m, nr, nc, path, out);
            path.pop_back();
        }
    }
    m[r][c] = 1;                                          // un-mark: BACKTRACK
}
```

Trace with the 2 x 2 maze `[[1,1],[0,1]]`. From `(0,0)`: down is blocked (0), left and up are outside, right is open: path `R`, now at `(0,1)`. From there down `(1,1)` is open: path `RD`, which is the destination, so record `RD`. Un-mark cells while returning, so another path may use them. Result `["RD"]`. Directions are tried in the order `D, L, R, U`, so results come out in alphabetical order. Call the function only when the start cell is open (`m[0][0] == 1`); an empty maze returns nothing. Because a cell is blocked only while it is on the current path, the number of paths can be exponential: backtracking, not BFS, is the right tool for "list all paths".

Practice Set 14.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Grid / Path Backtracking* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Print All Possible Paths in an M x N Matrix M	Moves right/down only: recurse, appending the cell, and pop on return.
Find if a Path of More Than K Length Exists from a Source M	DFS with a `visited` set and remaining length; if you reach `>= K` return true; backtrack visited.
Find Shortest Safe Route in a Path with Landmines M	Mark cells adjacent to landmines as unsafe, then BFS/DFS from every cell of the first column to the last.
Longest Possible Route in a Matrix with Hurdles H	DFS with backtracking (visited reset), tracking the max path length to the destination.

14.3 Pattern: Constraint Satisfaction

PATTERN CARD

SPOT IT WHEN	Place things under constraints: queens, Sudoku, coloring.
KNOW FIRST	Pruning with quick feasibility tests.
BREAKS WHEN	No pruning: search space explodes.
COMMON VARIATIONS	N-Queens (column + diagonal sets), Sudoku, m-coloring.

SOLVED EXAMPLE 14.5 H

N-Queens

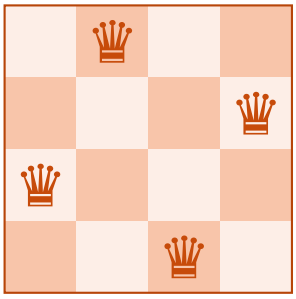
Idea: place one queen per row. A column or diagonal conflicts when already occupied. Diagonals are identified by `r - c` and `r + c`.

```
void solveQueens(int r, int n, vector<int>& colOf, vector<bool>& col, vector<bool>& d1,
↪   vector<bool>& d2,
                 vector<vector<string>>& out) {
    if (r == n) {
        vector<string> board(n, string(n, '.'));
        for (int i = 0; i < n; i++) board[i][colOf[i]] = 'Q';
        out.push_back(board);
        return;
    }
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r - c + n - 1] || d2[r + c]) continue;     // attacked
        col[c] = d1[r - c + n - 1] = d2[r + c] = true;              // place
        colOf[r] = c;
        solveQueens(r + 1, n, colOf, col, d1, d2, out);
        col[c] = d1[r - c + n - 1] = d2[r + c] = false;             // remove
    }
}
```

```
vector<vector<string>> nQueens(int n) {
    vector<vector<string>> out;
    if (n <= 0) return out;                                  // nothing to place
    vector<int> colOf(n);
    vector<bool> col(n, false), d1(2 * n - 1, false), d2(2 * n - 1, false);
    solveQueens(0, n, colOf, col, d1, d2, out);
    return out;
}
```

Trace for `n = 4`. Row 0, try column 0: queen at `(0,0)`. Row 1: column 0 is taken, column 1 shares a diagonal (`r - c` equal), column 2 is free: `(1,2)`. Row 2: column 0 and column 2 are taken, `(2,1)` shares the `/` diagonal of `(1,2)` (`r + c = 3`), and `(2,3)` shares its `\` diagonal (`r - c = -1`), so every column is attacked: a dead end. Undo `(1,2)` and try `(1,3)`; that leads to `(2,1)` and then row 3 has no free square, so it also fails. Undo `(0,0)`. Row 0, column 1: then `(1,3)`, `(2,0)`, `(3,2)` all fit, giving `colOf = [1, 3, 0, 2]` and the board

PICTURE IT



```
. Q . .
. . . Q
Q . . .
. . Q .
```

one queen per row; no two share a column or a diagonal

The other solution is its mirror image, so `nQueens(4)` has 2 answers (`n = 6`: 4, `n = 8`: 92). Cells `(r, c)` on one `\` diagonal share `r - c`, cells on one `/` diagonal share `r + c`; adding `n - 1` shifts `r - c` to a non-negative index. For `n <= 0` the function returns no boards.

SOLVED EXAMPLE 14.6 H

Sudoku Solver

Find an empty cell; try digits 1-9 that do not violate row/column/box rules; recurse; undo on failure.

```
bool canPlace(vector<vector<char>>& b, int r, int c, char d) {
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == d || b[i][c] == d) return false;
        if (b[3 * (r / 3) + i / 3][3 * (c / 3) + i % 3] == d) return false;   // 3x3 box
    }
    return true;
}
bool solveSudoku(vector<vector<char>>& b) {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++) {
            if (b[r][c] != '.') continue;
            for (char d = '1'; d <= '9'; d++) {
                if (!canPlace(b, r, c, d)) continue;
                b[r][c] = d;
                if (solveSudoku(b)) return true;
                b[r][c] = '.';                               // backtrack
            }
            return false;                                    // no digit fits: dead end
```

```
        }
    return true;                                        // no empty cell: solved
}
```

How it works. `solveSudoku` finds the *first* empty cell, tries each digit that breaks no rule, and recurses. If the recursion fails, the digit is erased (`'.'`) and the next one is tried; if no digit fits the cell, the function returns `false`, which makes the *previous* cell try its next digit. That "fail upward" is the backtracking. `canPlace` scans the row, the column and the 3 x 3 box in one loop (`i / 3` and `i % 3` walk the box). Without any cleverness the search is still very fast on ordinary puzzles because most digits are rejected immediately; picking the most constrained cell first would make it faster. The board must start valid, with `'.'` for blanks.

SOLVED EXAMPLE 14.7 M

M-Coloring Problem

Color vertices with at most `m` colors so no two adjacent vertices share one.

```
bool colorGraph(int v, int n, int m, const vector<vector<int>>& adj, vector<int>& color) {
    if (v == n) return true;
    for (int c = 1; c <= m; c++) {
        bool ok = true;
        for (int u : adj[v]) if (color[u] == c) { ok = false; break; }
        if (!ok) continue;
        color[v] = c;
        if (colorGraph(v + 1, n, m, adj, color)) return true;
        color[v] = 0;                                   // backtrack
    }
    return false;
}
```

Trace on a triangle (vertices 0, 1, 2 all adjacent) with `m = 3`. Vertex 0 gets colour 1. Vertex 1: colour 1 clashes with vertex 0, colour 2 is fine. Vertex 2: colour 1 clashes with vertex 0 and colour 2 clashes with vertex 1, but colour 3 is free: success with colours `1, 2, 3`. With `m = 2` vertex 2 has no colour, so vertex 1 tries a larger colour (none), vertex 0 tries colour 2 (symmetric, also fails) and the function returns `false`: a triangle needs three colours. Vertices are `0..n-1`, `adj` is symmetric and `color` starts as all zeros.

Practice Set 14.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Constraint Satisfaction* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Tug of War M	Choose `n/2` elements (include/exclude recursion) minimizing the difference between the subset sum and the rest.
The Knight's Tour Problem H	Try the 8 knight moves; mark visited with step numbers; backtrack. Warnsdorff's rule (move to the square with the fewest onward moves) speeds it up greatly.
Partition of a Set into K Subsets with Equal Sum H	Target = `total / k`; assign each element to a bucket that still has room; skip identical bucket states to prune.

14.4 Pattern: String Backtracking

PATTERN CARD

SPOT IT WHEN	Build strings/partitions with rules.
KNOW FIRST	Substring slicing; palindrome check.
BREAKS WHEN	Recomputing palindrome checks repeatedly (precompute with DP).
COMMON VARIATIONS	Partitioning, parentheses generation, restore IP.

SOLVED EXAMPLE 14.8 M

Subset Sum Problem

Include or exclude each element.

```
bool subsetSum(const vector<int>& a, int i, int target) {
    if (target == 0) return true;
    if (i == (int)a.size() || target < 0) return false;
    return subsetSum(a, i + 1, target - a[i]) || subsetSum(a, i + 1, target);   // take or skip
}
```

(For larger inputs convert this to the DP table of Chapter 12: `dp[s]` is reachable using 0/1 knapsack style updates.)

Trace with `a = [3, 34, 4, 12, 5, 2]`, `target = 9`. Take 3 (need 6), take 34 (need -28, dead end); skip 34, take 4 (need 2), take 12 (dead end), skip 12, take 5 (need -3, dead end), skip 5, take 2 (need 0): **true** (`3 + 4 + 2`). The two prunes are `target == 0` (success) and `target < 0` (failure, valid only for non-negative numbers). In the worst case all `2^n` choices are explored.

SOLVED EXAMPLE 14.9 M

Print All Palindromic Partitions / Word Break Using Backtracking

At each position try every prefix; if it is valid (palindrome / dictionary word) keep it and recurse on the rest.

```
bool isPal(const string& s, int l, int r) {
    while (l < r) if (s[l++] != s[r--]) return false;
    return true;
}
void palPartitions(const string& s, int start, vector<string>& cur, vector<vector<string>>&
↪  out) {
    if (start == (int)s.size()) { out.push_back(cur); return; }
    for (int end = start; end < (int)s.size(); end++) {
        if (!isPal(s, start, end)) continue;                  // prune
        cur.push_back(s.substr(start, end - start + 1));
        palPartitions(s, end + 1, cur, out);
        cur.pop_back();
    }
}
```

Trace with `s = "aab"`. `start = 0`: prefix `a` is a palindrome, recurse from 1: prefix `a`, recurse from 2: prefix `b`, recurse from 3 (the end): record `[a, a, b]`. Undo back to `start = 1`: prefix `ab` is not a palindrome. Back to `start = 0`: prefix `aa` is a palindrome, recurse from 2: `b`, record `[aa, b]`; prefix `aab` is not. Result `[[a,a,b],[aa,b]]`. Calling `isPal` again and again costs $\mathcal{O}(n)$ per check; precomputing the table `isPal[i][j]` of Example 12.26 makes each check $\mathcal{O}(1)$.

Word Break (all sentences): same shape, replace `isPal` with `dict.count(prefix)` and collect words into a sentence.

Practice Set 14.4

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *String Backtracking* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Remove Invalid Parentheses H	Count the minimum extra `(` and `)` first; DFS removing characters, skipping duplicates at the same level; or BFS level by level until a valid string appears.

CHAPTER COVERAGE

all 19 Backtracking problems appear above.

CHAPTER 14 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 14.1 Permutations, Problem 14A Subsets.
2. **Variations:** Ex 14.2 Combination Sum, 14.3 K-th Permutation, Problem 14B.
3. **Combined:** Ex 14.4 Rat in a Maze, 14.9 Palindromic Partitions.
4. **Independent recognition:** Practice Sets 14.1 and 14.4.
5. **Challenge:** Ex 14.5 N-Queens, 14.6 Sudoku.

Compare and Contrast

	Backtracking	Dynamic programming
Question	'list / find ALL / does ANY exist'	'how many / best value'
Repeated subproblems	ignored (re-explored)	cached
Complexity	exponential	polynomial in the state space

If you only need a *count* or *optimum* and the state repeats, prefer DP; if you must output every solution, backtrack.

Why It Works (Correctness Corner)

Invariant of a correct backtracker: when the recursive call returns, the shared state (current path, visited marks, counters) is *exactly* what it was before the call. Every `push`/`mark` must be paired with an `undo` on the same code path. If this holds, each branch starts from a clean state and the search enumerates each valid configuration exactly once.

Debug Lab: Forgetting to undo

```
void gen(const vector<int>& a, int i, vector<int>& cur, vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(cur); return; }
    gen(a, i + 1, cur, out);
    cur.push_back(a[i]);
    gen(a, i + 1, cur, out);        // <- missing cur.pop_back()
}
```

Your task: For `{1, 2}` it returns subsets containing extra elements. Why?

ANSWER

For `{1, 2}` the buggy code outputs `{}`, `{2}`, `{2, 1}`, `{2, 1, 2}` (we ran it) instead of `{}`, `{2}`, `{1}`, `{1, 2}`. After the 'take' branch returns, `a[i]` stays in `cur`, so later branches (and the other subsets) inherit it. Add `cur.pop_back()` after the second recursive call.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 14A

Subsets

M

Statement. Return all subsets of a list of distinct integers (order of subsets and elements does not matter).

Example. `[1,2]` → `[[],[1],[2],[1,2]]`

Constraints. $0 \leq n \leq 15$.

Expected complexity. $\mathcal{O}(n \cdot 2^n)$ time (output size).

Prerequisites. 14.1 Combinatorial Generation

HINT 1 which idea is relevant?

Each element has two states. What are they?

HINT 2 outline of the approach

For index `i`: either skip it or take it.

HINT 3 the key step

Recurse with `i+1` after skipping; push, recurse, then pop to take.

SOLUTION and editorial (try first, then compare)

```
void gen(const vector<int>& a, int i, vector<int>& cur, vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(cur); return; }   // decided on every element
    gen(a, i + 1, cur, out);              // choice 1: skip a[i]
    cur.push_back(a[i]);                  // choice 2: take a[i]
    gen(a, i + 1, cur, out);
    cur.pop_back();                       // undo the choice (backtrack)
}
vector<vector<int>> subsets(const vector<int>& a) {
    vector<vector<int>> out;
    vector<int> cur;
    gen(a, 0, cur, out);
    return out;
}
```

The recursion tree has depth n and 2^n leaves; each leaf copies a path of up to n elements. The pop restores the invariant described above.

PROBLEM 14B

Generate Parentheses

M

Statement. Given `n` pairs, generate all well-formed parentheses strings.

Example. `n = 3` → `["((()))","(()())","(())()","()(())","()()()"]`

Constraints. $1 \leq n \leq 8$.

Expected complexity. Output-sensitive: the n-th Catalan number of strings.

Prerequisites. 14.4 String Backtracking, 7.2 Brackets

HINT 1 which idea is relevant?

Which strings are valid? What rules limit the next character?

HINT 2 outline of the approach

Track how many `(` and `)` you have placed.

HINT 3 the key step

You may place `(` if `open < n`; you may place `)` if `close < open`.

SOLUTION and editorial (try first, then compare)

```
void build(int n, int open, int close, string& cur, vector<string>& out) {
    if ((int)cur.size() == 2 * n) { out.push_back(cur); return; }
    if (open < n) {                        // we may open while we still have openers left
        cur.push_back('('); build(n, open + 1, close, cur, out); cur.pop_back();
    }
    if (close < open) {                    // we may close only if something is open
        cur.push_back(')'); build(n, open, close + 1, cur, out); cur.pop_back();
    }
}
vector<string> generateParenthesis(int n) {
    vector<string> out;
    string cur;
    build(n, 0, 0, cur, out);
    return out;
}
```

Pruning by the two rules means only valid prefixes are explored, so there are no dead ends: the number of leaves equals the Catalan number (1430 for n = 8).

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T14.1 Insert +, - or nothing between the digits of '123' so the expression equals a target; list all ways.

PATTERN

Backtracking with a running value.

T14.2 Print all letter combinations a phone number could spell on a keypad.

PATTERN

Choose-one-per-position backtracking.

Mixed Practice

Compare 14.4 'palindromic partitions' with 12.8 palindromes: how would DP precomputation of `isPal[i][j]` speed up the backtracking?

Revision Sheet

Key takeaways

- Choose, explore, undo.
- Prune early with simple feasibility tests.

- Sort + skip equal neighbours to remove duplicates.
- Exponential is fine only for small n.

Pattern-recognition clues

- 'Generate all', 'print all' → backtracking.
- n $\leq$ 20 and 'any/all arrangements' → backtracking or bitmask.
- 'Place under constraints' → prune with sets.

Common mistakes

- Missing the undo step.
- Pushing a reference that later changes (copy when saving).
- Forgetting the base case.

Complexity summary

Problem	Time
Subsets	$\mathcal{O}(n\,2^n)$
Permutations	$\mathcal{O}(n\,n!)$
N-Queens	exponential, pruned

Recall questions (answer aloud, then check the chapter)

1. What are the three steps of the template?
2. What invariant must hold after a recursive call?
3. How do you avoid duplicate subsets from repeated values?
4. When is DP better than backtracking?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Write the generic template from memory and instantiate it for subsets, permutations and parentheses. Test `n = 0`, `n = 1` and a duplicate-element input.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 15

Bit Manipulation

15

START HERE

Every number in a list of 10^6 sensor ids appears twice except one. Sorting works; a hash map works; but XOR-ing them all together gives the odd one in one pass and $\mathcal{O}(1)$ memory. Bits let you compute with the binary form directly.

ANALOGY

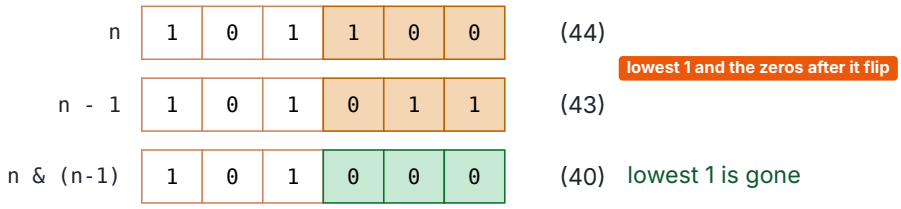
A row of light switches: set, clear, toggle or test any switch, and treat a whole row as a tiny set of up to 32 or 64 items.

AFTER THIS CHAPTER YOU CAN

- ✓ Use &, |, ^, ~, shifts to test, set, clear and toggle bits.
- ✓ Apply XOR properties to cancel pairs.
- ✓ Enumerate subsets with bitmasks.
- ✓ Avoid signed-shift and precedence traps.

PICTURE IT

n & (n - 1) clears the lowest set bit



Repeat until $n = 0$ to count the set bits.

15.0 Concept

Numbers are stored as bits. Bit operations are the fastest operations a CPU has and unlock elegant $\mathcal{O}(1)$-space tricks.

Operator	Meaning	Example
`a & b`	AND	`6 & 3 = 2`
`a | b`	OR	`6 | 3 = 7`
`a ^ b`	XOR (different bits give 1)	`6 ^ 3 = 5`

Operator	Meaning	Example
`~a`	NOT (flip all bits)	`~0 = -1`
`a << k`	shift left (multiply by `2^k`)	`1 << 3 = 8`
`a >> k`	shift right (divide by `2^k`)	`16 >> 2 = 4`

Essential identities:

- `x ^ x = 0`, `x ^ 0 = x`, XOR is commutative and associative. So XOR-ing a list cancels everything that appears twice.
- Check bit `i`: `(x >> i) & 1`. Set: `x | (1 << i)`. Clear: `x & ~(1 << i)`. Toggle: `x ^ (1 << i)`.
- `x & (x - 1)` clears the **lowest set bit**.
- `x & -x` isolates the **lowest set bit**.
- Built-ins: `__builtin_popcount(x)` counts set bits; `__builtin_clz(x)` leading zeros; `__builtin_ctz(x)` trailing zeros.

`__builtin_clz(0)` and `__builtin_ctz(0)` are **undefined**, so test `x != 0` first (`__builtin_popcount(0)` is fine and equals 0). For 64-bit values use the `ll` versions (`__builtin_popcountll`).

Reading Binary and Negative Numbers (read this if bits are new to you)

Place values. In binary each position is worth a power of two, counting from the right: `... 32 16 8 4 2 1`. So `44 = 32 + 8 + 4 = 101100` in binary, and `10 = 8 + 2 = 1010`.

Test, set, clear, toggle with `x = 10 = 1010`.

Operation	Expression	Result
test bit 1	`(10 >> 1) & 1`	`1`
set bit 0	`10 | (1 << 0)`	`1011 = 11`
clear bit 1	`10 & ~(1 << 1)`	`1000 = 8`
toggle bit 3	`10 ^ (1 << 3)`	`0010 = 2`

Negative numbers (two's complement). `-x` is stored as `~x + 1`: flip every bit, then add 1. With 8 bits, `12 = 00001100`, `~12 = 11110011`, and `-12 = 11110100`. Adding 1 turns the trailing run of 1s into 0s and carries into the lowest 0, so below and including the lowest set bit of `x` the pattern is unchanged, and above it every bit is flipped. That is why `12 & -12 = 00000100 = 4` isolates the lowest set bit. The one number with no positive partner is `INT_MIN`: `-INT_MIN` overflows `int`, which is why the two-singles code below does its arithmetic in `unsigned`.

Precedence trap. Comparison operators (`==`, `<`) bind **tighter** than `&`, `^` and `|`, so `n & m == 0` means `n & (m == 0)`. Always write `(n & m) == 0`. **Shift traps.** Shifting by the width of the type or more is undefined (`1 << 32` on an `int`), and `1 << 31` on an `int` reaches the sign bit; write `1u << 31` or `1LL << 40`. Right-shifting a negative `int` is implementation-defined before C++20, so convert to `unsigned` first when you care about the bits.

15.1 Pattern: Bit Basics

PATTERN CARD

SPOT IT WHEN	Count/test/set bits, powers of two.
KNOW FIRST	Binary representation; two's complement.

BREAKS WHEN	Signed shifts: right-shifting negative numbers is implementation-defined in C++ before C++20; prefer unsigned.
COMMON VARIATIONS	popcount, n & (n-1), lowest set bit, bit masks.

SOLVED EXAMPLE 15.1 E

Count Set Bits, Power of Two, Bits to Flip, Hamming Distance

```
int countSetBits(unsigned int x) {                    // Brian Kernighan: loops once per set
↪  bit
    int c = 0;
    while (x) { x &= (x - 1); c++; }
    return c;
}

bool isPowerOfTwo(long long x) { return x > 0 && (x & (x - 1)) == 0; }   // exactly one bit set

int bitsToFlip(int a, int b) { return countSetBits(a ^ b); }             // differing bits

int hammingDistance(int x, int y) { return countSetBits(x ^ y); }        // same idea
```

SOLVED EXAMPLE 15.2 E

Reverse Bits

Trace with 4 bits instead of 32 (same loop, `i < 4`): `n = 1011`. Step 1: `res = 0 << 1 | 1 = 1`, `n = 101`. Step 2: `res = 10 | 1 = 11`, `n = 10`. Step 3: `res = 110 | 0 = 110`, `n = 1`. Step 4: `res = 1100 | 1 = 1101`. So `1011` becomes `1101`. The loop always runs 32 times, so leading zeros of `n` become trailing zeros of the result.

```
uint32_t reverseBits(uint32_t n) {
    uint32_t res = 0;
    for (int i = 0; i < 32; i++) {
        res = (res << 1) | (n & 1);                   // append lowest bit of n
        n >>= 1;
    }
    return res;
}
```

15.2 Pattern: XOR Tricks

PATTERN CARD

SPOT IT WHEN	'Everything appears twice except…', pairs cancel.
KNOW FIRST	XOR properties: a^a=0, a^0=a.
BREAKS WHEN	Elements appear 3 times (XOR alone fails): use bit counting mod 3.
COMMON VARIATIONS	Single number, two singles (split by a set bit), missing number.

SOLVED EXAMPLE 15.3 M

Find the Two Non-Repeating Elements

Problem: every number appears twice except two, `x` and `y`. Find them in $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Approach:

1. XOR everything: the pairs cancel, leaving `x ^ y`.
2. `x != y`, so some bit is set in `x ^ y`; pick the lowest one (`d & -d`; the code writes it as `xr & (0u - xr)` in `unsigned` arithmetic, because `-xr` overflows `int` when `xr` is `INT_MIN`).
3. Split the numbers into two groups by that bit. Each group has exactly one of `x`, `y`; XOR each group.

```
pair<int,int> twoSingles(const vector<int>& a) {
    unsigned xr = 0;
    for (int v : a) xr ^= (unsigned)v;                    // xr = x ^ y (unsigned: no overflow
    ↪  worries)
    unsigned bit = xr & (0u - xr);                        // lowest set bit: a bit where x and y
    ↪  differ
    int x = 0, y = 0;
    for (int v : a) {
        if ((unsigned)v & bit) x ^= v; else y ^= v;
    }
    return {x, y};
}
```

Trace with `a = [1, 2, 1, 3, 2, 5]`. XOR of everything: `1^2^1^3^2^5 = 3^5 = 6 = 110`. The lowest set bit is `010 = 2`. Numbers with that bit set: `2, 3, 2` (XOR = 3); numbers without it: `1, 1, 5` (XOR = 5). The two singles are `{3, 5}`. Each group still contains pairs, so the pairs cancel inside each group; the two singles land in different groups because they differ at that bit.

SOLVED EXAMPLE 15.4 M

Single Number II (every element appears three times except one)

Idea: count each bit position modulo 3. The bits that remain are the single number.

```
int singleNumber(const vector<int>& a) {
    unsigned res = 0;                                     // unsigned: shifting into bit 31 is
    ↪  well defined
    for (int bit = 0; bit < 32; bit++) {
        int sum = 0;
        for (int v : a) sum += ((unsigned)v >> bit) & 1u;
        if (sum % 3) res |= (1u << bit);
    }
    return (int)res;
}
```

Trace with `a = [2, 2, 3, 2]`. Bit 0: the values contribute `0, 0, 1, 0`, sum 1, `1 % 3 != 0`, so bit 0 is set. Bit 1: `1, 1, 1, 1`, sum 4, `4 % 3 = 1`, so bit 1 is set. Result `11 = 3`. A number that appears three times adds 0 or 3 to every bit count, so after `% 3` only the single number's bits remain. The code works in `unsigned` so that bit 31 (the sign bit) is handled without signed-overflow questions.

Practice Set 15.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *XOR Tricks* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Minimum XOR Value Pair M	Sort the array; the minimum XOR is always between **adjacent** elements.
Maximum XOR of Two Numbers in a Range [L, R] M	Let `p` = highest bit where `L` and `R` differ; the answer is `(1 << (p + 1)) - 1`. (If `L == R` it is `0`.)

15.3 Pattern: Advanced Bit Manipulation

PATTERN CARD

SPOT IT WHEN	Subsets of up to ~20 items, optimizing over masks, division without '/'.
KNOW FIRST	Shifts, masks, subset enumeration.
BREAKS WHEN	More than ~25 items (2^n too large).
COMMON VARIATIONS	Bitmask subsets, submask enumeration, bitwise OR of ranges.

SOLVED EXAMPLE 15.5 M

Divide Two Integers (without `/`)

Idea: subtract the largest `divisor * 2^k` that fits, repeatedly. Use `long long` to avoid overflow.

```
int divide(int dividend, int divisor) {
    if (dividend == INT_MIN && divisor == -1) return INT_MAX;       // overflow case
    long long a = llabs((long long)dividend), b = llabs((long long)divisor), q = 0;
    while (a >= b) {
        long long t = b, mult = 1;
        while (a >= (t << 1)) { t <<= 1; mult <<= 1; }             // largest doubling that fits
        a -= t; q += mult;
    }
    return ((dividend < 0) ^ (divisor < 0)) ? -q : q;
}
```

Trace of `divide(10, 3)`. `a = 10`, `b = 3`: the largest `3 * 2^k` not above 10 is `6` (`mult = 2`), so `a = 4`, `q = 2`. Now `3` fits once: `a = 1`, `q = 3`. The loop ends: `3`. For `divide(-7, 2)`: work with `7` and `2`: `4` fits (`mult = 2`), `a = 3`; `2` fits (`mult = 1`), `a = 1`; `q = 3`, and the signs differ, so the result is `-3` (division truncates towards zero). `divide(INT_MIN, -1)` would be `2^31`, which does not fit, so it is special-cased. The divisor must not be 0 (that is undefined division, and the loop would not stop).

SOLVED EXAMPLE 15.6 M/H

Bitwise ORs of Subarrays and Subsequences via Bitmask

Bitwise ORs of Subarrays: the set of distinct ORs of subarrays ending at `i` is tiny (at most 32 values, since each OR can only gain bits). Maintain it as we scan.

```
int subarrayBitwiseORs(const vector<int>& a) {
    unordered_set<int> all, cur;
    for (int x : a) {
        unordered_set<int> next = {x};
        for (int y : cur) next.insert(x | y);
        cur = next;
        all.insert(cur.begin(), cur.end());
    }
    return all.size();
}
```

Trace with `a = [1, 1, 2]`. `x = 1`: `cur = {1}`. `x = 1`: `cur = {1}` (`1 | 1 = 1`). `x = 2`: `cur = {2, 2|1 = 3}`. All distinct values seen: `{1, 2, 3}`, so the answer is 3. `cur` never holds more than about 32 values because every OR only adds bits, and a chain of ORs can grow at most 31 times; hence the whole scan is $\mathcal{O}(32\,n)$.

Subsequences of a string using bitmasks: a string of length `n` has `2^n` subsequences; mask `m` from `0` to `2^n - 1` says which positions to keep.

```
vector<string> allSubsequences(const string& s) {
    int n = s.size();
    vector<string> res;
    for (int mask = 0; mask < (1 << n); mask++) {
        string t;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) t += s[i];
        res.push_back(t);
    }
    return res;
}
```

Trace with `s = "abc"` (`n = 3`, masks `0..7`): `000 -> ""`, `001 -> "a"`, `010 -> "b"`, `011 -> "ab"`, `100 -> "c"`, `101 -> "ac"`, `110 -> "bc"`, `111 -> "abc"`. Bit `i` of the mask says "keep character `i`". Use this only while `n` is small (about 20 or fewer): there are `2^n` subsequences, and `1 << n` is undefined once `n >= 31`.

Practice Set 15.3

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Advanced Bit Manipulation* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Copy Set Bits in a Range M	Build a mask of bits `l..r` (counted from 1 at the right): `mask = ((1u << (r - l + 1)) - 1) << (l - 1)`; `y |= x & mask`. Use `unsigned` or `long long` so a 32-bit range does not overflow the shift.
Calculate Square Without Using `*`, `/` or `pow()` M	`n^2 = sum of n shifted by each set bit of n`; or use `(n-1)^2 = n^2 - 2n + 1` recursively with `n << 1`.

CHAPTER COVERAGE

all 14 Bit Manipulation problems appear above.

CHAPTER 15 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 15.1 Bit Basics, Problem 15A.
2. **Variations:** Ex 15.3 Two Non-Repeating, 15.4 Single Number II.
3. **Combined:** Ex 15.5 Divide Without `/`, 15.6 Bitmask subsets.
4. **Independent recognition:** Problem 15B (a DP hiding in bits).
5. **Challenge:** Ex 15.6 Bitwise ORs of Subarrays.

Compare and Contrast

	XOR trick	Sum trick
Overflow	none	possible
Handles 'pairs cancel'	yes	needs full sum formula
Fails	elements appear 3 times (use bit counting mod 3)	duplicates of unknown multiplicity

Why It Works (Correctness Corner)

Why XOR finds the unique element. XOR is commutative and associative, `x ^ x = 0` and `x ^ 0 = x`. Reordering the XOR so equal values sit next to each other cancels every pair and leaves the single value. **Why `n & (n-1)` clears the lowest set bit:** subtracting 1 flips the lowest set bit to 0 and all lower zeros to 1; AND-ing leaves everything above unchanged and zeros the rest.

Debug Lab: Operator precedence

```cpp
bool isPowerOfTwo(int n) {
    return n & (n - 1) == 0;
}
```

Your task: What does `isPowerOfTwo(8)` return?

ANSWER

In C++ `==` binds tighter than `&`, so it parses as `n & ((n-1) == 0)`, which is `8 & 0 = 0` (false). Parenthesize: `n > 0 && (n & (n - 1)) == 0`. Note the extra `n > 0` check for zero and negatives.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 15A E

Single Number

Statement. Every element appears exactly twice except one. Find the single one using $\mathcal{O}(1)$ extra space.

Example. `[4,1,2,1,2]` → `4`

Constraints. $1 \le n \le 3 \times 10^4$ (n odd).

Expected complexity. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

Prerequisites. 15.2 XOR Tricks

HINT 1 which idea is relevant?

What operation undoes itself when applied twice?

HINT 2 outline of the approach

XOR is commutative and associative.

HINT 3 the key step

XOR every element; pairs cancel.

SOLUTION and editorial (try first, then compare)

```
int singleNumber(const vector<int>& a) {
    int x = 0;
    for (int v : a) x ^= v;          // pairs cancel (v ^ v = 0), and 0 ^ s = s
    return x;
}
```

`a ^ a = 0` and `a ^ 0 = a`, so the order does not matter and only the unpaired value survives. Works for negative numbers too.

PROBLEM 15B M

Counting Bits

Statement. For every `i` in `0..n` output the number of 1 bits in `i`.

Example. `n = 5` → `[0,1,1,2,1,2]`

Constraints. $0 \le n \le 10^5$.

Expected complexity. $\mathcal{O}(n)$ total, not $\mathcal{O}(n \log n)$.

Prerequisites. 15.1 Bit Basics, 12.1 1D DP

HINT 1 which idea is relevant?

Calling popcount for each number costs $\mathcal{O}(\log n)$ each. Can you reuse smaller answers?

HINT 2 outline of the approach

How is `i` related to `i >> 1`?

HINT 3 the key step

`bits[i] = bits[i >> 1] + (i & 1)`.

SOLUTION and editorial (try first, then compare)

```cpp
vector<int> countBits(int n) {
    vector<int> ans(n + 1, 0);
    for (int i = 1; i <= n; i++)
        ans[i] = ans[i >> 1] + (i & 1);   // i = (i/2 shifted left) + lowest bit
    return ans;
}
```

`i` is `i/2` shifted left plus its lowest bit, so its popcount is the popcount of `i >> 1` plus that bit. A small DP expressed in bits.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T15.1 Every sensor id is reported twice except one faulty sensor; find it.

PATTERN
XOR of all ids.

T15.2 Try every subset of 15 items to find the sum closest to T.

PATTERN
Bitmask enumeration `for (mask = 0; mask < (1<<n); mask++)`.

Mixed Practice

Combine 15.2 and 16.2 (later): maximum XOR of two numbers uses a binary trie. Why does going greedily bit by bit from the top find the maximum?

Revision Sheet

Key takeaways

- `x & (x-1)` clears the lowest set bit; `x & -x` isolates it.
- XOR cancels pairs.
- Parenthesize bit operations.
- Use unsigned types for shifts.

Pattern-recognition clues

- 'Appears twice except…' → XOR.
- 'Subsets of ≤ 20 items' → bitmask.
- 'Power of two', 'set bits' → bit tricks.

Common mistakes

- Precedence of `==` vs `&`.
- `1 << 31` overflow in `int` (use `1LL <<` or `1u <<`).
- Right shift of negatives.

Complexity summary

Trick	Time
popcount via loop	$\mathcal{O}(\text{bits})$
XOR scan	$\mathcal{O}(n)$
Subset enumeration	$\mathcal{O}(2^n\, n)$

Recall questions (answer aloud, then check the chapter)

1. Why does XOR find the single number?
2. What does `x & (x - 1)` do?
3. Why parenthesize `(n & m) == 0`?
4. How many subsets does a mask loop enumerate?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	Without notes: write functions for isPowerOfTwo, popcount (two ways), singleNumber, and a subset-enumeration loop. Test with 0, negative numbers and 2^{31}.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

CHAPTER 16

Tries

16

START HERE

Type ‘ca’ and your phone suggests ‘cat’, ‘car’, ‘cart’. Searching the whole dictionary each time is too slow; a trie stores words by shared prefixes so a lookup costs the length of the word, independent of the dictionary size.

ANALOGY

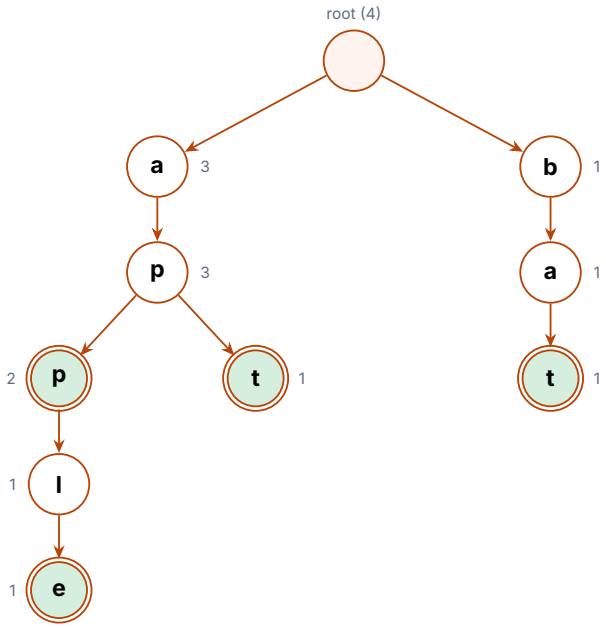
A dictionary’s thumb-index: first letter, then second letter, and so on. Every word starting ‘ca’ lives under the same branch.

AFTER THIS CHAPTER YOU CAN

- ✓ Build and query a trie.
- ✓ Add counters/flags to answer prefix-count questions.
- ✓ Use a binary trie for maximum-XOR problems.
- ✓ Compare a trie with a hash set.

PICTURE IT

A trie holding apple, app, apt, bat (pass counts)



double circle = a word ends here
(number = how many words pass through)

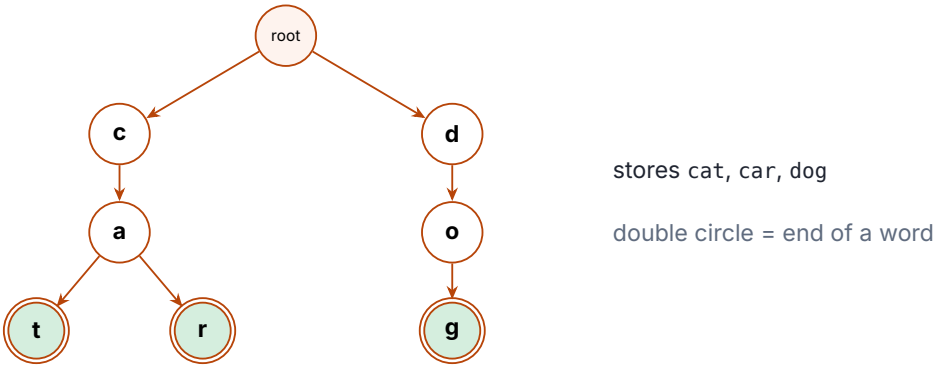
`countPrefix("ap")` = 3
`countPrefix("app")` = 2
`countPrefix("")` = 4 (root)

words: apple, app, apt, bat

16.0 Concept

A **trie** (prefix tree) stores strings character by character along a path from the root. Words with a common prefix share the path. Insert, search, and prefix-search all take $\mathcal{O}(L)$ time for a word of length L, independent of how many words are stored.

PICTURE IT



Pattern	Recognition clue
Prefix queries	autocomplete, “starts with”, prefix counting
Dictionary of words	many lookups against a fixed set of strings
XOR maximization	binary trie over bits

16.1 Pattern: Trie Fundamentals

PATTERN CARD

SPOT IT WHEN	‘Prefix’, ‘autocomplete’, ‘dictionary of words’.
KNOW FIRST	Tree of children arrays; index = char - ‘a’.
BREAKS WHEN	Memory: 26-array nodes for huge dictionaries (use map children or compressed trie).
COMMON VARIATIONS	Insert/search/startsWith, word counts per node, delete.

SOLVED EXAMPLE 16.1 M

Implement Trie (Prefix Tree)

```
struct TrieNode {
    TrieNode* next[26] = {};
    bool isEnd = false;
    int passCount = 0;                                  // how many words pass through (useful
    ↪  later)
};

class Trie {
    TrieNode* root = new TrieNode();
    static void freeNode(TrieNode* n) {                  // free every node, children first
```

```
        if (!n) return;
        for (TrieNode* c : n->next) freeNode(c);
        delete n;
    }
public:
    Trie() = default;
    Trie(const Trie&) = delete;                          // the raw pointer must have one owner
    Trie& operator=(const Trie&) = delete;
    ~Trie() { freeNode(root); }
    void insert(const string& w) {
        TrieNode* cur = root;
        for (char ch : w) {
            int i = ch - 'a';
            if (!cur->next[i]) cur->next[i] = new TrieNode();
            cur = cur->next[i];
            cur->passCount++;
        }
        cur->isEnd = true;
    }
    bool search(const string& w) {
        TrieNode* n = find(w);
        return n && n->isEnd;
    }
    bool startsWith(const string& pre) { return find(pre) != nullptr; }
    TrieNode* find(const string& s) {
        TrieNode* cur = root;
        for (char ch : s) {
            cur = cur->next[ch - 'a'];
            if (!cur) return nullptr;
        }
        return cur;
    }
};
```

Trace inserting `cat`, then `car`. The root has no children. `cat`: create `c` (below the root), then `a` (below `c`), then `t` (below `a`); mark `t` as a word end; the pass counters on `c`, `a`, `t` are 1. `car`: `c` and `a` already exist, so the counters on `c` and `a` become 2; there is no `r` below `a`, so create it and mark it as a word end. `search("ca")` walks `c`, `a`, finds a node but it is not an end: `false`; `startsWith("ca")` is `true`; `search("cat")` is `true`. A word costs one step per letter, no matter how many words are stored.

Memory and input. Each node is created with `new`, so the trie must give the memory back: this version has a destructor that frees the whole tree recursively and forbids copying (a copy would free the same nodes twice). The code assumes lowercase letters `a` to `z` (`ch - 'a'` indexes the child array); for other alphabets use `unordered_map<char, TrieNode*>` children.

SOLVED EXAMPLE 16.2 M

Shortest Unique Prefix for Every Word

Idea: with `passCount` stored per node, the unique prefix of a word ends at the first character whose node has `passCount == 1` (only this word passes through it).

```
vector<string> shortestUniquePrefix(const vector<string>& words) {
    Trie t;
    for (auto& w : words) t.insert(w);
    vector<string> res;
```

```
    for (auto& w : words) {
        TrieNode* cur = t.find("");                    // root
        string pre;
        for (char ch : w) {
            cur = cur->next[ch - 'a'];
            pre += ch;
            if (cur->passCount == 1) break;
        }
        res.push_back(pre);
    }
    return res;
}
```

Trace with `["zebra", "dog", "duck", "dove"]`. Pass counts: `z:1`; `d:3`, `o:2`, `g:1`; `d:3`, `u:1`; `d:3`, `o:2`, `v:1`. The first node with count 1 gives the prefixes `z`, `dog`, `du`, `dov`. (If a word is a prefix of another word, for example `app` and `apple`, no node on the path of `app` has count 1, so the function returns the whole word, which is not unique; the problem normally rules that case out.)

Practice Set 16.1

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Trie Fundamentals* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
No Prefix Set Problem M	Insert words one at a time; fail if you pass through a word-end node, or if you finish a word at a node that has children.

16.2 Pattern: Advanced Trie Applications

PATTERN CARD

SPOT IT WHEN	'Max XOR pair', 'shortest unique prefix', 'suggestions'.
KNOW FIRST	Trie plus a stored counter/flag.
BREAKS WHEN	Not a prefix problem (use hashing).
COMMON VARIATIONS	Binary trie for XOR, counter per node, DFS collect results.

SOLVED EXAMPLE 16.3 M

Maximum XOR of Two Numbers in an Array

Idea: store numbers as 31-bit binary strings in a trie. For each number, walk from the highest bit, always preferring the child with the **opposite** bit (that makes the XOR bit 1).

```
struct BitTrie {
    struct Node { Node* ch[2] = {nullptr, nullptr}; };
    Node* root = new Node();
    BitTrie() = default;
```

```
    BitTrie(const BitTrie&) = delete;                       // one owner for the raw pointers
    BitTrie& operator=(const BitTrie&) = delete;
    ~BitTrie() { freeNode(root); }
    static void freeNode(Node* n) {
        if (!n) return;
        freeNode(n->ch[0]); freeNode(n->ch[1]);
        delete n;
    }
    void insert(int x) {                                    // x >= 0 (31 bits: bit 30 down to 0)
        Node* cur = root;
        for (int b = 30; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (!cur->ch[bit]) cur->ch[bit] = new Node();
            cur = cur->ch[bit];
        }
    }
    int maxXorWith(int x) {                                 // trie must be non-empty
        Node* cur = root;
        int res = 0;
        for (int b = 30; b >= 0; b--) {
            int want = 1 - ((x >> b) & 1);                  // opposite bit gives a 1 in the XOR
            if (cur->ch[want]) { res |= (1 << b); cur = cur->ch[want]; }
            else cur = cur->ch[1 - want];
        }
        return res;
    }
};

int findMaximumXOR(const vector<int>& a) {                  // all values >= 0
    if (a.empty()) return 0;
    BitTrie t;
    for (int x : a) t.insert(x);
    int best = 0;
    for (int x : a) best = max(best, t.maxXorWith(x));
    return best;
}
```

Trace with the numbers `[3, 10, 5, 25, 2, 8]` (5 bits shown). Take `x = 5 = 00101`. Bit 4: we want a 1 here to make the XOR bit 1; `25 = 11001` has a 1, so go that way and set the result bit. Bit 3: `x` has 0, we want 1; `25` has 1, go that way. Bit 2: `x` has 1, we want 0; `25` has 0, go. Bit 1: `x` has 0, want 1; `25` has 0 and nothing else is left on this path, so take the 0. Bit 0: `x` has 1, want 0; `25` has 1, so take 1. Result `11100 = 28 = 5 ^ 25`, the maximum for the array. **Why greedy from the top is right:** the bit at position `b` is worth `2^b`, which is more than all lower bits together (`2^b - 1`), so securing a 1 at the highest bit is never worse than any combination of lower bits. The numbers must be non-negative (31 bits are used); the function returns 0 for an empty array, and nodes are freed by the destructor.

SOLVED EXAMPLE 16.4 M

Search Suggestions System

Problem: after each typed character, suggest up to 3 lexicographically smallest products with that prefix.

Idea (simple and fast enough): sort the products; for each prefix, `lower_bound` finds the first match; take up to three following entries that share the prefix.

```cpp
vector<vector<string>> suggestedProducts(vector<string>& products, const string& word) {
    sort(products.begin(), products.end());
    vector<vector<string>> res;
    string prefix;
    for (char c : word) {
        prefix += c;
        auto it = lower_bound(products.begin(), products.end(), prefix);
        vector<string> sug;
        for (int k = 0; k < 3 && it != products.end() && it->compare(0, prefix.size(), prefix)
        ↪   == 0; k++, ++it)
            sug.push_back(*it);
        res.push_back(sug);
    }
    return res;
}
```

(Trie variant: store up to 3 smallest words at every node while inserting in sorted order.)

Trace with `products = [mobile, mouse, moneypot, monitor, mousepad]`, `word = "mouse"`. Sorted: `mobile, moneypot, monitor, mouse, mousepad`. Prefix `m` and `mo`: the first three entries (`mobile, moneypot, monitor`). Prefix `mou`, `mous`, `mouse`: `lower_bound` lands on `mouse`, the next `mousepad` also matches, and the list ends: `[mouse, mousepad]`. The function sorts its argument, so pass a copy if the order matters, and the products are assumed distinct (duplicates would appear twice in a suggestion list).

Practice Set 16.2

BEFORE OPENING THE HINTS

try each problem for 15 to 20 minutes. **Level 1 (concept):** every row below falls to the *Advanced Trie Applications* idea; re-read this section's Pattern Card. **Level 2 (approach):** the hint column. **Level 3 (key step):** the closest solved example above.

Problem	Level 2 hint (approach)
Kth Smallest in Lexicographical Order H	Treat numbers `1..n` as a 10-ary trie; count how many numbers lie under a prefix (`steps`), and skip the subtree if `k` exceeds the count; otherwise descend.

CHAPTER COVERAGE

all 6 Tries problems appear above.

CHAPTER 16 TOOLKIT

Practice, Review and Mastery

Problem Ladder

1. **Foundation:** Ex 16.1 Implement Trie.
2. **Variations:** Problem 16A (prefix counts).
3. **Combined:** Ex 16.2 Shortest Unique Prefix, Problem 16B.
4. **Independent recognition:** Ex 16.4 Search Suggestions.
5. **Challenge:** Ex 16.3 Maximum XOR.

Compare and Contrast

	Hash set	Trie
Exact word lookup	$\mathcal{O}(L)$ average	$\mathcal{O}(L)$
Prefix query	scan everything	$\mathcal{O}(L)$
Ordered / autocomplete	no	yes (DFS under node)
Memory	compact	can be large

Why It Works (Correctness Corner)

Trie invariant. The path from the root to a node spells exactly one prefix; two words share a node iff they share that prefix. Hence `countPrefix(p)` is the `pass` counter at the node reached by walking `p`, and the answer for the *empty* prefix is the root's counter: it must also be maintained.

Debug Lab: search() that confuses prefix with word

```
bool search(const string& w) {
    int cur = 0;
    for (char ch : w) {
        cur = t[cur].next[ch - 'a'];
        if (cur == -1) return false;
    }
    return true;                    // <- bug
}
```

Your task: After inserting "apple", `search("app")` returns true. Why?

ANSWER

Reaching a node only proves the *prefix* exists. A word is stored only if its last node is marked `isEnd`. Return `t[cur].isEnd`.

Self-Contained Problems (with staged hints)

Try each for at least 15 minutes. Open hints one at a time.

PROBLEM 16A M

Prefix Counter

Statement. Support two operations: `insert(word)` and `countPrefix(p)`, the number of inserted words that start with `p` (the empty prefix matches every word).

Example. insert apple, app, apt, bat: `countPrefix("ap")` $\to 3$, `countPrefix("app")` $\to 2$, `countPrefix("")` $\to 4$

Constraints. Lowercase letters; total length $\le 10^5$.

Expected complexity. $\mathcal{O}(L)$ per operation.

Prerequisites. 16.1 Trie Fundamentals

HINT 1 which idea is relevant?

What would you have to store at a trie node to answer a prefix count without scanning below it?

HINT 2 outline of the approach

Count how many inserted words pass through each node.

HINT 3 the key step

Increment `pass` on every node along the path (and on the root for the empty prefix).

SOLUTION and editorial (try first, then compare)

```
struct PrefixCounter {
    struct Node { int next[26]; int pass = 0; Node() { fill(next, next + 26, -1); } };
    vector<Node> t;
    PrefixCounter() : t(1) {}                          // node 0 is the root
    void insert(const string& w) {
        int cur = 0;
       t[0].pass++;                                    // the empty prefix matches every word
        for (char ch : w) {
            int c = ch - 'a';
           if (t[cur].next[c] == -1) { t[cur].next[c] = (int)t.size(); t.emplace_back(); }
            cur = t[cur].next[c];
            t[cur].pass++;                             // one more word passes through
↪   this node
        }
    }
    int countPrefix(const string& p) const {
        int cur = 0;
        for (char ch : p) {
            cur = t[cur].next[ch - 'a'];
            if (cur == -1) return 0;
        }
        return t[cur].pass;
    }
};
```

Without `t[0].pass++` the empty prefix would return 0 instead of the number of words. Always test the empty prefix.

PROBLEM 16B

Replace Words

M

Statement. Given a dictionary of root words and a sentence, replace every word by the shortest root that is a prefix of it (if any).

Example. roots `[cat,bat,rat]`, "the cattle was rattled by the battery" → "the cat was rat by the bat"

Constraints. Sentence length $\leq 10^6$.

Expected complexity. $\mathcal{O}(\text{total dictionary length} + \text{sentence length})$.

Prerequisites. 16.1, 16.2

HINT 1 which idea is relevant?

You need the *shortest* root that prefixes a word. What does walking down a trie visit first?

HINT 2 outline of the approach

Mark nodes where a root ends.

HINT 3 the key step

Walk the word in the trie; stop at the first node marked as an end.

SOLUTION and editorial (try first, then compare)

```cpp
string replaceWords(const vector<string>& roots, const string& sentence) {
    vector<array<int, 26>> nxt(1);
    nxt[0].fill(-1);
    vector<bool> isEnd(1, false);
    for (const string& r : roots) {                    // build the trie of roots
        int cur = 0;
        for (char ch : r) {
            int c = ch - 'a';
            if (nxt[cur][c] == -1) {
                nxt[cur][c] = (int)nxt.size();
                nxt.emplace_back(); nxt.back().fill(-1);
                isEnd.push_back(false);
            }
            cur = nxt[cur][c];
        }
        isEnd[cur] = true;
    }
    istringstream in(sentence);
    string word, result;
    while (in >> word) {
        int cur = 0, cut = -1;                         // cut = length of the shortest
        ↪ root found
        for (int i = 0; i < (int)word.size(); i++) {
            cur = nxt[cur][word[i] - 'a'];
            if (cur == -1) break;
           if (isEnd[cur]) { cut = i + 1; break; }     // the first hit is the shortest root
        }
        if (!result.empty()) result += ' ';
        result += (cut == -1) ? word : word.substr(0, cut);
    }
    return result;
}
```

The first end-marker encountered is the shortest root, because trie depth equals prefix length. If the walk falls off the trie, keep the word unchanged.

Transfer Check (what pattern is hiding here?)

Decide the technique *before* opening the answer.

T16.1 Autocomplete: suggest up to 3 words for what the user has typed so far.

> **PATTERN**
> Trie + DFS (or sorted list + binary search).

T16.2 From a list of integers choose two with maximum XOR.

> **PATTERN**
> Binary trie, choose the opposite bit at each level when possible.

Mixed Practice

Combine 10.2 and 16.1: group words by anagram. Would a trie or a hash map keyed by sorted letters be simpler? Why?

Revision Sheet

Key takeaways

- Trie lookups cost $\mathcal{O}(\text{length})$, not $\mathcal{O}(\text{dictionary})$.
- Store counters/flags at nodes.
- `isEnd` distinguishes words from prefixes.
- Binary trie = trie over bits.

Pattern-recognition clues

- 'Prefix', 'autocomplete' → trie.
- 'Maximum XOR' → binary trie.
- 'Shortest unique prefix' → counter per node.

Common mistakes

- Search returns true for a prefix.
- Not freeing/limiting memory with 26-array nodes.
- Forgetting the empty prefix.

Complexity summary

Operation	Time
insert / search / startsWith	$\mathcal{O}(L)$
Space	$\mathcal{O}(\text{total characters x alphabet})$

Recall questions (answer aloud, then check the chapter)

1. What does a node represent?
2. How is a word different from a prefix in a trie?
3. Why does a trie beat a hash set for prefix queries?
4. How is a binary trie used for XOR?

Mastery Checkpoint

Tick every box **without notes or hints** before moving on (see Appendix C).

Ability	Evidence	Done
Explain	each pattern in two sentences	☐
Recognize	all Transfer Check items named correctly	☐
Derive	brute force to optimized idea, with the reason it is correct	☐
Code	From scratch: implement insert/search/startsWith, then add `countPrefix`. Test empty string, repeated inserts and a prefix that is not a word.	☐
Test	planted bug in the Debug Lab found; five edge cases listed	☐
Analyze	time and space complexity stated with reasons	☐

Appendices

A C++ STL Quick Reference

B Pattern Recognition Cheat Sheet

C Mastery-Based Study Plan

D Interview Checklist

E Sheet Coverage Index

F Mistake and Learning Log

G Spaced Revision and Self-Tests

H Quality Assurance Report

I C++ Implementation Lab

J Companion Solutions

APPENDIX A

C++ STL Quick Reference

A.1 Containers at a Glance

Container	Header	Key operations	Complexity
`vector<T>`	`<vector>`	`push_back`, `pop_back`, `[]`, `size`, `back`, `insert`, `erase`, `resize`, `assign`	push/pop $\mathcal{O}(1)$ amortized, insert/erase $\mathcal{O}(n)$
`string`	`<string>`	`+=`, `substr`, `find`, `compare`, `push_back`, `pop_back`	find $\mathcal{O}(n \cdot m)$ worst
`array<T,N>`	`<array>`	fixed-size; `[]`, `size`, `fill`	$\mathcal{O}(1)$
`deque<T>`	`<deque>`	`push_front/back`, `pop_front/back`, `[]`	$\mathcal{O}(1)$
`list<T>`	`<list>`	`push_front/back`, `splice`, `insert(it)`, `erase(it)`	$\mathcal{O}(1)$ at iterator
`stack<T>`	`<stack>`	`push`, `pop`, `top`, `empty`	$\mathcal{O}(1)$
`queue<T>`	`<queue>`	`push`, `pop`, `front`, `back`	$\mathcal{O}(1)$
`priority_queue<T>`	`<queue>`	`push`, `pop`, `top` (max-heap by default)	$\mathcal{O}(\log n)$
`set<T>`	`<set>`	`insert`, `erase`, `find`, `count`, `lower_bound`, `upper_bound`	$\mathcal{O}(\log n)$
`multiset<T>`	`<set>`	same; duplicates allowed (`erase(find(x))` removes one)	$\mathcal{O}(\log n)$
`map<K,V>`	`<map>`	`[]`, `insert`, `erase`, `find`, `count`, `lower_bound`	$\mathcal{O}(\log n)$
`unordered_set<T>`	`<unordered_set>`	`insert`, `erase`, `find`, `count`	$\mathcal{O}(1)$ average
`unordered_map<K,V>`	`<unordered_map>`	`[]`, `insert`, `erase`, `find`, `count`	$\mathcal{O}(1)$ average
`pair<A,B>`	`<utility>`	`first`, `second`, `make_pair`	$\mathcal{O}(1)$
`tuple<...>`	`<tuple>`	`get<i>`, `make_tuple`, structured bindings	$\mathcal{O}(1)$
`bitset<N>`	`<bitset>`	`set`, `reset`, `flip`, `count`, `test`	$\mathcal{O}(N/64)$

Function	Purpose

A.2 Algorithms (<algorithm>, <numeric>)

Function	Purpose
`sort(b, e)` / `sort(b, e, cmp)`	sort a range, $\mathcal{O}(n \log n)$
`stable_sort`	sort preserving the order of equal elements
`reverse(b, e)`	reverse a range
`rotate(b, mid, e)`	rotate so `mid` becomes the first element
`max_element(b, e)`, `min_element(b, e)`	iterator to max/min
`*max_element(...)`	the value itself
`accumulate(b, e, init)`	sum (use `0LL` for long long)
`count(b, e, x)`, `count_if`	counting
`find(b, e, x)`	first occurrence (iterator)
`lower_bound(b, e, x)`	first element `>= x` (sorted range)
`upper_bound(b, e, x)`	first element `> x` (sorted range)
`binary_search(b, e, x)`	existence check
`unique(b, e)`	remove consecutive duplicates (then `erase`)
`next_permutation(b, e)`	next lexicographic permutation (`false` when wrapped)
`nth_element(b, nth, e)`	partial sort: `nth` element in sorted position, $\mathcal{O}(n)$ average
`swap(a, b)`	swap two values
`min({a,b,c})`, `max({a,b,c})`	multi-argument min/max
`iota(b, e, start)`	fill with `start, start+1, ...`
`fill(b, e, x)`	set all to `x`
`gcd(a, b)`, `lcm(a, b)`	`<numeric>`, C++17
`partial_sum(b, e, out)`	prefix sums
`__builtin_popcount(x)`	number of set bits (a GCC/Clang built-in that needs no header; `__builtin_popcountll` for 64-bit values)

A.3 Idioms You Will Use Constantly

Each snippet below is independent: names such as `a`, `n`, `mp`, `x`, `key` are assumed to be declared already, and each snippet compiled in its own scope.

```
// Sort descending
sort(a.begin(), a.end(), greater<int>());

// Remove duplicates
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());

// Min-heap
priority_queue<int, vector<int>, greater<int>> pq;
```

```cpp
// Heap of pairs ordered by first
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

// Heap with a custom comparator
auto cmp = [](const Node* a, const Node* b) { return a->val > b->val; };
priority_queue<Node*, vector<Node*>, decltype(cmp)> pq(cmp);

// Iterate a map
for (auto& [key, value] : mp) { /* ... */ }

// 2D vector
vector<vector<int>> dp(n, vector<int>(m, 0));

// Fill a vector with a value
vector<int> v(n, -1);

// Check if a key exists
if (mp.count(key)) { /* ... */ }

// Index of an element found via lower_bound
int pos = lower_bound(a.begin(), a.end(), x) - a.begin();

// Max of a vector
int mx = *max_element(a.begin(), a.end());

// Character <-> index
int idx = ch - 'a';
char ch = 'a' + idx;

// Infinity constants
const int INF = 1e9;
const long long LINF = 1e18;
```

A.4 Complexity Cheat Sheet of Common Operations

Operation	Cost
`vector` index / back	$\mathcal{O}(1)$
`vector` insert in middle	$\mathcal{O}(n)$
`string` concatenation `s += c`	$\mathcal{O}(1)$ amortized
`s = s + t` (new string)	O(|s| + |t|)
`substr(i, len)`	$\mathcal{O}(\text{len})$
`map` / `set` operations	$\mathcal{O}(\log n)$
`unordered_map` operations	$\mathcal{O}(1)$ average, $\mathcal{O}(n)$ worst
`sort`	$\mathcal{O}(n \log n)$
`priority_queue` push/pop	$\mathcal{O}(\log n)$
`priority_queue` build from vector	$\mathcal{O}(n)$

Choosing `INF`. With `const int INF = 1e9`, the sum `INF + INF = 2e9` still fits in a 32-bit `int` (maximum about 2.147e9), but `INF + INF + INF` does not. Always test `dist[u] != INF` before adding an edge weight to it, or use `LINF` with `long long` when several distances are summed.

APPENDIX B

Pattern Recognition Cheat Sheet

B

Read the problem statement, find the clue in the left column, and reach for the pattern on the right.

Clue in the problem	Pattern	Chapter
Sorted array, find a pair or triplet	Two pointers	3
Contiguous subarray, max sum	Kadane	3
Longest/shortest subarray or substring with a condition	Sliding window	3, 10
Many range sum queries / range updates	Prefix sum / difference array	3
"More than n/2" or "n/k" times	Boyer-Moore / Misra-Gries	3
Values in 1..n, find missing/duplicate	Cyclic placement / index as hash	3
Rotate, partition, segregate in place	Reversal trick / Dutch flag	3
Search on a sorted or rotated array	Binary search	5
"Minimize the maximum" or "smallest value such that"	Binary search on the answer	5
Count pairs `i < j` with a relation	Merge sort counting	5
Middle / cycle in a list	Fast and slow pointers	6
Reverse part of a list	Pointer reversal	6
$\mathcal{O}(1)$ get/put with eviction	Hash map + doubly linked list	6
Next greater / smaller element	Monotonic stack	7
Balanced brackets	Stack / counter	7
Sliding window maximum	Monotonic deque	7
Level by level / zigzag / views	BFS on tree	8
Max path / diameter in a tree	Postorder DFS with a global answer	8
"Construct tree from traversals"	Recursion with index map	8
Ancestor relationships	LCA	8
Shortest path, unweighted	BFS	9
Shortest path, weights ≥ 0	Dijkstra	9
Shortest path with at most K edges or negative weights	Bellman-Ford	9
Prerequisites / ordering	Topological sort	9
Connectivity as edges arrive, group merging	Union-Find	9
Connect all nodes cheaply	MST (Kruskal/Prim)	9
Grid regions / flood	DFS/BFS on grid (multi-source BFS for spreading)	9
Count subarrays with a sum condition	Prefix sum + hash map	10
Anagram / permutation / same letters	Frequency array / sorted key	10
K largest / smallest / closest	Heap of size K	10
Running median	Two heaps	10
Choose intervals / schedule	Greedy, sort by end time	11

Clue in the problem	Pattern	Chapter
Merge the smallest repeatedly	Min-heap greedy	11
Count ways / min cost with overlapping subproblems	DP	12
Pick items with a capacity	Knapsack DP	12
Increasing subsequence	LIS ($\mathcal{O}(n \log n)$)	12
Two strings, align or compare	LCS / edit distance DP	12
Split a sequence at every point	Interval DP	12
Two players, optimal play	Game DP / minimax difference	12
Find a pattern in a text	KMP / Z / Rabin-Karp	13
Generate all arrangements or solve a puzzle	Backtracking	14
Pairs cancelling, "appears once"	XOR	15
Subsets of up to ~20 items	Bitmask	15
Prefix queries over many words	Trie	16
Maximum XOR	Binary trie	16

When Stuck, Ask:

1. **What is repeated or recomputed?** (hashing, prefix sums, DP)
2. **Would sorting help?** (two pointers, greedy, binary search)
3. **Is there a monotonic yes/no?** (binary search on the answer)
4. **Is there a structure to shrink?** (recursion, divide and conquer)
5. **Can I solve a smaller version and extend?** (DP, induction)
6. **Can I model this as a graph?** (states as nodes, moves as edges)

B.2 Look-alike Patterns: How to Tell Them Apart

Often confused	Decide by asking	Pick
Sliding window vs prefix sums	Are all values non-negative and does shrinking restore validity?	window if yes; prefix sums + hash map otherwise
Two pointers vs hash map	Is the data sorted (or may it be sorted)? Is $\mathcal{O}(1)$ space required?	two pointers if sorted; hash map if order must be kept
BFS vs DFS	Shortest path in an unweighted graph or level-by-level order?	BFS; use DFS for components, cycles, ordering, backtracking
Dijkstra vs Bellman-Ford	Any negative edge, or a limit on the number of edges?	Dijkstra if all weights ≥ 0; otherwise Bellman-Ford
Dijkstra vs BFS	Are all edge weights equal?	BFS if equal; Dijkstra otherwise
Greedy vs DP	Can you prove a local choice is safe (exchange argument)? Does a counterexample exist?	greedy if provable; DP if choices interact
Sorting vs heap	Need the full order, or only the best k / the current best?	sort for full order; heap for top-k or streaming
Memoization vs tabulation	Are only some states reachable? Is recursion depth a risk?	memoization for sparse states; tabulation for dense states and rolling arrays
Backtracking vs DP	List all solutions, or only count / optimize with repeated states?	backtracking to list; DP to count or optimize

Often confused	Decide by asking	Pick
Binary search on index vs on answer	Searching positions in data, or the smallest/largest feasible value?	index search for data; answer search with a monotone check
KMP vs Z vs hashing	One pattern, guaranteed linear time, or many patterns / substring equality?	KMP or Z for one pattern; hashing for many or equality tests
Union-Find vs DFS components	Do edges arrive over time, or is the graph fixed?	DSU for incremental connectivity; DFS for a fixed graph
Kruskal vs Prim	Sparse (edge list) or dense (adjacency)?	Kruskal for sparse; Prim for dense
Stack vs queue vs deque	Most recent unmatched, oldest first, or both ends with a moving window?	stack, queue, deque respectively
Trie vs hash set	Do you need prefix queries?	trie for prefixes; hash set for exact membership

Does the pattern fail here? Before committing, run the card's *Breaks when* line from the chapter against the problem's constraints (negatives? unsorted? huge n? cycles?).

APPENDIX C

Mastery-Based Study Plan

A calendar says *when* you studied; mastery says *what you can do*. Progress by passing **gates**, not by finishing days. Go faster on topics you pass quickly and slower on topics you do not.

The Mastery Gate (applies to every chapter)

You pass a chapter's gate when, **without notes or hints**, you can do all six:

#	Ability	Evidence
1	Explain	tell the core idea of each pattern in two sentences
2	Recognize	name the pattern for each Transfer Check problem within 2 minutes
3	Derive	go from brute force to the optimized idea and say why it is correct
4	Code	write the chapter's mastery task from a blank file in the time you set
5	Test	list edge cases and find the planted bug in the Debug Lab
6	Analyze	state time and space complexity with reasons

Suggested thresholds: solve ladder tiers 1 to 3 alone; tiers 4 and 5 with at most Level-2 hints; then pass the Mastery Checkpoint. If you miss, do not move on: log the cause (Appendix F), revisit the pattern card, redo the closest solved example from scratch, then retry the gate two days later.

Stages and Indicative Pace

Pace is a guide, not a rule. A reader who passes gates quickly may finish early; one who needs more time loses nothing by taking it.

Stage	Chapters	Gate to pass before moving on	Indicative time
0. Foundations	1-2	write input/sort/dedupe program; estimate complexity of 5 code snippets	3-5 days
1. Arrays and search	3-5	Gates for 3, 4, 5	2-4 weeks
2. Linked structures	6-7	Gates for 6, 7; reverse a list and implement monotonic stack from blank; Test 1 (Appendix G)	1-2 weeks
3. Trees and graphs	8-9	Gates for 8, 9	3-5 weeks
4. Hashing and heaps	10	Gate for 10; prefix-map and top-k from blank; Test 2 (Appendix G)	1 week
5. Paradigms	11-14	Gates for 11-14	4-7 weeks

Stage	Chapters	Gate to pass before moving on	Indicative time
6. Specialist tools	15-16	Gates for 15-16; Test 3 (Appendix G)	1 week
7. Integration	all	Appendix G capstone with fresh problems under timed conditions	ongoing

Weekly Rhythm

- **4 days new material:** opener, pattern cards, solved examples (rungs 1-4), practice with hint levels.
- **1 day toolkit:** Debug Lab, Problems nA/nB, Transfer Check, gate attempt.
- **1 day revision:** follow the schedule in Appendix G; re-solve three earlier problems from scratch.
- **1 day rest.**

Fast-Track Reference: the Original 69-Day Pace

If you prefer a fixed calendar, the first edition's pace is still a workable fast track, provided you also pass the gates.

Topic	Days	Chapter
Array & String	1-10	3
Matrix	10-11	4
Sorting and Searching	12-14	5
Linked List	14-16	6
Stacks and Queue	16-21	7
Tree	21-28	8
Graph	28-39	9
HashMap and Heap	39-45	10
Greedy	45-50	11
Dynamic Programming	50-61	12
String Matching	61-62	13
Backtracking	62-66	14
Bit Manipulation	66-68	15
Tries	68-69	16

Track status for every problem as: **Solved alone / Needed hint level 1-3 / Needed the solution**.

APPENDIX D

Interview Checklist

Before You Code

- ☐ Restate the problem in your own words.
- ☐ Ask about constraints: size of `n`, value ranges, duplicates, negative numbers, sorted or not.
- ☐ Ask about edge cases: empty input, one element, all equal, maximum size.
- ☐ Walk through an example by hand.
- ☐ State the brute force and its complexity first.
- ☐ Identify the pattern (use Appendix B) and explain *why* it fits.

While Coding

- ☐ Use meaningful names (`left`, `right`, `windowSum`), not `a`, `b`, `c`.
- ☐ Watch for **integer overflow** (`long long`), off-by-one errors, and empty containers.
- ☐ Keep functions small; extract helpers.
- ☐ Narrate what you are doing.

After Coding

- ☐ Dry-run with your example and one edge case.
- ☐ State **time and space complexity**.
- ☐ Mention possible optimizations or trade-offs.
- ☐ Discuss follow-ups (streaming input, sorted input, duplicates, huge constraints).

Common Bug Checklist

Bug	Where it hides
Off-by-one	loop bounds, binary search `lo <= hi` vs `lo < hi`
Overflow	sums, products, `mid = (lo + hi) / 2`, factorials
Uninitialized	DP base cases, `dist` arrays, counters
Null dereference	linked lists, trees (`root->left` on null)
Modifying while iterating	erasing in a loop

Bug	Where it hides
Forgetting to backtrack	recursion with shared state (undo after the call)
Wrong loop order	0/1 knapsack (descending), Floyd-Warshall (`k` outermost), coin combinations (coins outside)
Copying big containers	missing `&` in function parameters

APPENDIX E

Sheet Coverage Index (all 425 problems)

This index lists **every one of the 425 problems** of the problem sheet, in the sheet's own order and topic groups, and says where this book deals with it and how thoroughly. It was built by matching names by script and then correcting every match by hand; the 53 names that differ from the book's headings (combined or abbreviated headings) were mapped manually.

How to read the "How it is covered" column

- **Worked solution:** a solved example with explanation and code that was compiled and run in the tests described in Appendix H.
- **Worked inside a combined example:** the same, but the example solves several sheet problems together (for example *Frog Jump / Paint House*).
- **Companion solution in Appendix J:** the chapter gives only a hint or a note; Appendix J gives full, tested code.
- **Short note inside an example or section:** one paragraph or bullet explaining the idea. There is no separate code, so treat it as a hint.
- **Hint only:** one line in a practice table. It names the technique and the key step. **The book contains no code for these and nothing was run for them.** Solve them yourself, then compare with the hint.
- ⚠**:** a competitive-programming style problem whose exact statement was not available while writing this book. The hint describes the likely technique. Read the statement on your sheet or on the judge and confirm that the hint fits before relying on it.

The *Level* column is the sheet's own label.

Array & String (60 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
1	Rotate Array	Easy	Example 3.1	Worked solution	yes
2	Squares of a Sorted Array	Easy	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
3	Reverse Vowels of a String	Easy	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
4	Next Greater Element III	Medium	Example 3.3	Worked inside a combined example	yes
5	Maximum Swap	Medium	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
6	Wiggle Sort	Medium	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
7	Push Dominoes	Medium	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
8	Minimum Domino Rotations For Equal Row	Medium	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
9	Multiply Strings	Medium	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
10	Two Sum	Easy	Example 3.6	Worked inside a combined example	yes
11	Segregate 0s and 1s	Easy	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
12	Segregate 0s, 1s and 2s (Dutch National Flag)	Easy	Example 3.2	Worked solution	yes
13	Sort Array By Parity	Easy	Practice Set 3.1 (hint)	Hint only (one line in a practice table)	no
14	Valid Palindrome II	Easy	Practice Set 3.2 (hint)	Hint only (one line in a practice table)	no
15	Pair with Given Difference	Medium	Example 3.6 (note)	Short note inside an example or section (no separate code)	no
16	Boats to Save People	Medium	Practice Set 3.2 (hint)	Hint only (one line in a practice table)	no
17	Container With Most Water	Medium	Example 3.4	Worked solution	yes
18	Trapping Rain Water	Medium	Example 3.5	Worked solution	yes
19	Maximum Subarray (Kadane's Algorithm)	Easy	Example 3.7	Worked solution	yes
20	Number of Subarrays with Bounded Maximum	Medium	Example 3.9	Worked solution	yes
21	Maximum Product Subarray	Medium	Example 3.8	Worked solution	yes
22	K-Concatenation Maximum Sum	Medium	Practice Set 3.3 (hint)	Hint only (one line in a practice table)	no
23	Max Sum of Two Non-Overlapping Subarrays	Hard	Practice Set 3.3 (hint)	Hint only (one line in a practice table)	no
24	Smallest Range Covering Elements from K Lists	Hard	Practice Set 3.3 (hint)	Hint only (one line in a practice table)	no
25	Product of Array Except Self	Medium	Example 3.10	Worked solution	yes
26	Range Addition	Medium	Example 3.11	Worked solution	yes
27	Max Range Query	Medium	Practice Set 3.4 (hint)	Hint only (one line in a practice table)	no
28	Napoleon Cake	Medium	Practice Set 3.4 (hint)	Hint only (one line in a practice table) ⚠	no
29	Consecutive Numbers Sum	Hard	Practice Set 3.4 (hint)	Hint only (one line in a practice table)	no
30	Sum of Subsequence Widths	Hard	Practice Set 3.4 (hint)	Hint only (one line in a practice table)	no
31	Majority Element	Easy	Example 3.12	Worked inside a combined example	yes
32	Largest Number At Least Twice the Others	Easy	Practice Set 3.5 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
33	Check Whether One String Is a Rotation of Another	Easy	Practice Set 3.5 (hint)	Hint only (one line in a practice table)	no
34	Majority Element II	Medium	Practice Set 3.5 (hint)	Hint only (one line in a practice table)	no
35	Max Chunks To Make Sorted	Medium	Example 3.14	Worked solution	yes
36	Minimum Swaps to Sort an Array (Permutation Cycles)	Medium	Example 3.16	Worked solution	yes
37	Max Chunks To Make Sorted II	Hard	Example 3.15	Worked solution	yes
38	Majority Element (General — more than n/k times)	Hard	Example 3.13	Worked inside a combined example	yes
39	Partition Labels	Medium	Example 3.18	Worked solution	yes
40	Maximize Distance to Closest Person	Medium	Practice Set 3.7 (hint)	Hint only (one line in a practice table)	no
41	Minimise the Maximum Difference Between Heights	Medium	Practice Set 3.7 (hint)	Hint only (one line in a practice table)	no
42	Long Pressed Name	Medium	Practice Set 3.7 (hint)	Hint only (one line in a practice table)	no
43	Buddy Nim	Medium	Practice Set 3.7 (hint)	Hint only (one line in a practice table) ⚠	no
44	Best Meeting Point	Hard	Practice Set 3.7 (hint)	Hint only (one line in a practice table)	no
45	Fibonacci Number	Easy	Practice Set 3.8 (hint)	Hint only (one line in a practice table)	no
46	Fast Exponentiation	Easy	Example 3.20	Worked inside a combined example	yes
47	Relatively Prime (Coprime Check)	Easy	Example 3.20	Worked inside a combined example	yes
48	Sieve of Eratosthenes	Easy	Example 3.19	Worked solution	yes
49	Segmented Sieve	Medium	Practice Set 3.8 (hint)	Hint only (one line in a practice table)	no
50	K-LCM I	Medium	Practice Set 3.8 (hint)	Hint only (one line in a practice table) ⚠	no
51	K-LCM II	Medium	Practice Set 3.8 (hint)	Hint only (one line in a practice table) ⚠	no
52	Meximization	Hard	Practice Set 3.8 (hint)	Hint only (one line in a practice table)	no
53	Bulb Switcher	Medium	Section 3.9 (hint table)	Hint only (one line in a practice table)	no
54	Kth Largest and Smallest Element in an Array	Medium	Section 3.9 (hint table)	Hint only (one line in a practice table)	no
55	Max Distance Between Two Elements ($a[i] \le a[j], i < j$)	Medium	Section 3.9 (hint table)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
56	Minimum Jump Required to Reach a Number	Medium	Section 3.9 (hint table)	Hint only (one line in a practice table)	no
57	Bulb Switcher III	Hard	Section 3.9 (hint table)	Hint only (one line in a practice table)	no
58	First Missing Positive	Hard	Example 3.17	Worked solution	yes
59	Orderly Queue	Hard	Section 3.9 (hint table)	Hint only (one line in a practice table) ⚠	no
60	Interesting Subarrays	Hard	Section 3.9 (hint table)	Hint only (one line in a practice table) ⚠	no

Matrix (10 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
61	Search an Element in a Matrix	Easy	Example 4.3	Worked solution	yes
62	Row with Maximum Number of 1's	Easy	Practice Set 4 (hint)	Hint only (one line in a practice table)	no
63	Spiral Traversal of a Matrix	Medium	Example 4.1	Worked solution	yes
64	Print Elements in Sorted Order (Row-Column Sorted Matrix)	Medium	Practice Set 4 (hint)	Hint only (one line in a practice table)	no
65	Common Elements in All Rows of a Matrix	Medium	Practice Set 4 (hint)	Hint only (one line in a practice table)	no
66	Rotate a Matrix by 90 Degrees	Medium	Example 4.2	Worked solution	yes
67	Median in a Row-Wise Sorted Matrix	Medium	Example 4.5	Worked solution	yes
68	Kth Smallest Element in a Row-Column Sorted Matrix	Medium	Example 4.4	Worked solution	yes
69	Find a Specific Pair in a Matrix	Hard	Practice Set 4 (hint)	Hint only (one line in a practice table)	no
70	Trapping Rain Water II	Hard	Practice Set 4 (hint)	Hint only (one line in a practice table)	no

Sorting and Searching (15 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
71	Binary Search	Easy	Example 5.1	Worked solution	yes
72	Search in Rotated Sorted Array	Medium	Example 5.2	Worked solution	yes
73	Search in Rotated Sorted Array II	Medium	Section 5.1 (note) + Appendix J.1	Note in the chapter, full solution in Appendix J	yes (Appendix J)

#	Problem	Level	Where in this book	How it is covered	Code run
74	Find Minimum in Rotated Sorted Array	Medium	Example 5.3	Worked solution	yes
75	Find Minimum in Rotated Sorted Array II	Medium	Section 5.1 (note) + Appendix J.2	Note in the chapter, full solution in Appendix J	yes (Appendix J)
76	Koko Eating Bananas	Medium	Example 5.4	Worked solution	yes
77	Smallest Divisor Given a Threshold	Medium	Practice Set 5.2 (hint)	Hint only (one line in a practice table)	no
78	Capacity to Ship Packages Within D Days	Medium	Example 5.5	Worked solution	yes
79	Painter's Partition Problem	Medium	Practice Set 5.2 (hint)	Hint only (one line in a practice table)	no
80	Split Array Largest Sum	Hard	Practice Set 5.2 (hint)	Hint only (one line in a practice table)	no
81	Median of Two Sorted Arrays	Hard	Example 5.6	Worked solution	yes
82	Counting Sort	Easy	Example 5.7	Worked solution	yes
83	Merge Sort	Medium	Example 5.8	Worked inside a combined example	yes
84	Sort an Array (Implement an $\mathcal{O}(n \log n)$ Sort)	Medium	Example 5.8	Worked inside a combined example	yes
85	Count Inversions (Using Merge Sort)	Medium	Example 5.8	Worked inside a combined example	yes

Linked List (19 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
86	Find the Middle Element of a Linked List	Easy	Example 6.3	Worked inside a combined example	yes
87	Reverse a Linked List	Easy	Example 6.1	Worked solution	yes
88	Remove Duplicates from a Sorted Linked List	Easy	Practice Set 6.1 (hint)	Hint only (one line in a practice table)	no
89	Split a Linked List into Two Parts	Easy	Practice Set 6.1 (hint)	Hint only (one line in a practice table)	no
90	Detect Loop in a Linked List (Floyd's Cycle Detection)	Easy	Example 6.3	Worked inside a combined example	yes
91	Check if a Linked List Is a Palindrome	Medium	Example 6.4	Worked solution	yes
92	Intersection Point of Two Linked Lists	Medium	Example 6.5	Worked solution	yes
93	Reverse a Linked List in Groups of Given Size	Medium	Example 6.2	Worked solution	yes
94	Rotate a Doubly Linked List in Groups of Given Size	Medium	Practice Set 6.1 (hint)	Hint only (one line in a practice table)	no
95	Delete Nodes with a Greater Value on the Right Side	Medium	Practice Set 6.1 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
96	Sort a Linked List of 0s, 1s and 2s	Easy	Practice Set 6.3 (hint)	Hint only (one line in a practice table)	no
97	Merge Sort for Linked Lists	Medium	Example 6.6	Worked inside a combined example	yes
98	Quicksort for Linked Lists	Medium	Practice Set 6.3 (hint)	Hint only (one line in a practice table)	no
99	Sort a 'K'-Sorted (Nearly Sorted) Doubly Linked List	Medium	Practice Set 6.3 (hint)	Hint only (one line in a practice table)	no
100	Split a Circular Linked List into Two Halves	Medium	Practice Set 6.3 (hint)	Hint only (one line in a practice table)	no
101	Add Two Numbers Represented by Linked Lists	Medium	Example 6.7	Worked solution	yes
102	Copy a Linked List with Random Pointers	Medium	Example 6.9	Worked solution	yes
103	LRU Cache	Medium	Example 6.8	Worked solution	yes
104	Flatten a Multilevel Linked List	Medium	Practice Set 6.4 (hint)	Hint only (one line in a practice table)	no

Stacks and Queue (28 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
105	Next Greater Element (Right Side)	Easy	Example 7.1	Worked solution	yes
106	Next Greater Element II (Circular Array)	Medium	Practice Set 7.1 (hint) + Appendix J.3	Hint in the chapter, full solution in Appendix J	yes (Appendix J)
107	Daily Temperatures	Medium	Example 7.2	Worked inside a combined example	yes
108	Stock Span Problem	Medium	Example 7.2 + Appendix J.4	Worked inside a combined example; stand-alone solution in Appendix J	yes (Appendix J)
109	Nearest Smaller to Left and Right	Medium	Practice Set 7.1 (hint)	Hint only (one line in a practice table)	no
110	Asteroid Collision	Medium	Practice Set 7.1 (hint)	Hint only (one line in a practice table)	no
111	Remove K Digits	Medium	Example 7.4	Worked solution	yes
112	Remove Duplicate Letters	Medium	Practice Set 7.1 (hint)	Hint only (one line in a practice table)	no
113	Min Cost Tree From Leaf Values	Medium	Practice Set 7.1 (hint)	Hint only (one line in a practice table)	no
114	Largest Rectangular Area in a Histogram	Hard	Example 7.3	Worked solution	yes
115	Maximum Size Rectangle of 1's in a Binary Matrix	Hard	Section 7.1 (note; reduces to Example 7.3)	Short note inside an example or section (no separate code)	no

#	Problem	Level	Where in this book	How it is covered	Code run
116	Valid Parentheses	Easy	Example 7.5	Worked solution	yes
117	Minimum Add to Make Parentheses Valid	Medium	Example 7.7	Worked inside a combined example	yes
118	Minimum Bracket Reversals to Balance an Expression	Medium	Example 7.7 + Appendix J.5	Worked inside a combined example; stand-alone solution in Appendix J	yes (Appendix J)
119	Count / Remove Duplicate Parentheses	Medium	Practice Set 7.2 (hint)	Hint only (one line in a practice table)	no
120	Length of Longest Valid Parentheses Substring	Hard	Example 7.6	Worked solution	yes
121	Min Stack	Medium	Example 7.8	Worked solution	yes
122	Validate Stack Sequences	Medium	Practice Set 7.3 (hint)	Hint only (one line in a practice table)	no
123	K Stacks in a Single Array	Medium	Practice Set 7.3 (hint)	Hint only (one line in a practice table)	no
124	K Queues in a Single Array	Medium	Practice Set 7.3 (hint)	Hint only (one line in a practice table)	no
125	Reverse First K Elements of a Queue	Medium	Practice Set 7.3 (hint)	Hint only (one line in a practice table)	no
126	Adapters (Stack Simulation)	Medium	Practice Set 7.3 (hint)	Hint only (one line in a practice table) ⚠	no
127	Max Frequency Stack	Hard	Practice Set 7.3 (hint)	Hint only (one line in a practice table)	no
128	First Negative Integer in Every Window of Size K	Easy	Example 7.10	Worked solution	yes
129	Backspace String Compare	Easy	Practice Set 7.4 (hint)	Hint only (one line in a practice table)	no
130	Maximum Sum of Smallest and Second-Smallest in a Window	Medium	Practice Set 7.4 (hint)	Hint only (one line in a practice table)	no
131	Car Fleet	Medium	Practice Set 7.4 (hint)	Hint only (one line in a practice table)	no
132	Print Binary Numbers from 1 to N Using a Queue	Easy	Example 7.11	Worked solution	yes

#	Problem	Level	Where in this book	How it is covered	Code run

Tree (47 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
133	Inorder Traversal	Easy	Example 8.1	Worked inside a combined example	yes
134	Preorder Traversal	Easy	Appendix J.6	Companion solution in Appendix J	yes (Appendix J)
135	Postorder Traversal	Easy	Appendix J.6	Companion solution in Appendix J	yes (Appendix J)
136	Level Order Traversal	Easy	Example 8.2	Worked inside a combined example	yes
137	Reverse Level Order Traversal	Easy	Example 8.2 (one-line change in a code comment)	Short note inside an example or section (no separate code)	no
138	Zigzag Level Order Traversal	Medium	Example 8.2 (comment) + Appendix J.7	Comment in the chapter, full solution in Appendix J	yes (Appendix J)
139	Right View	Easy	Example 8.3	Worked inside a combined example	yes
140	Left View	Easy	Example 8.3	Worked inside a combined example	yes
141	Top View	Medium	Example 8.3	Worked inside a combined example	yes
142	Bottom View	Medium	Example 8.3 (bullet) + Appendix J.7	Bullet in the chapter, full solution in Appendix J	yes (Appendix J)
143	Vertical Order Traversal	Medium	Example 8.3 (bullet)	Short note inside an example or section (no separate code)	no
144	Diagonal Traversal	Medium	Example 8.3 (bullet) + Appendix J.7	Bullet in the chapter, full solution in Appendix J	yes (Appendix J)
145	Boundary Traversal	Medium	Practice Set 8.2 (hint)	Hint only (one line in a practice table)	no
146	Closest BST Value	Easy	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
147	Convert BST to Greater Sum Tree	Medium	Example 8.6	Worked solution	yes
148	Delete a Node in a BST	Medium	Example 8.5	Worked solution	yes
149	Recover a Binary Search Tree	Medium	Example 8.7	Worked solution	yes
150	Check if Preorder Traversal Can Represent a BST	Medium	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
151	Construct BST from Preorder Traversal	Medium	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
152	Construct BST from Postorder Traversal	Medium	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
153	Convert a Sorted Doubly Linked List to a Balanced BST	Medium	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
154	Closest BST Value II	Hard	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
155	Merge Two BSTs	Hard	Practice Set 8.3 (hint)	Hint only (one line in a practice table)	no
156	LCA in a BST	Easy	Example 8.8	Worked inside a combined example	yes
157	LCA in a Binary Tree	Medium	Example 8.8	Worked inside a combined example	yes
158	LCA in $\mathcal{O}$(Height) Time, $\mathcal{O}(1)$ Extra Space	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
159	Construct Tree from Inorder and Preorder	Medium	Example 8.9	Worked solution	yes
160	Construct Tree from Inorder and Postorder	Medium	Example 8.9 (note)	Short note inside an example or section (no separate code)	no
161	Construct Tree from Preorder and Postorder	Medium	Practice Set 8.5 (hint)	Hint only (one line in a practice table)	no
162	Verify Inorder and Level-Order Belong to the Same BST	Medium	Practice Set 8.5 (hint)	Hint only (one line in a practice table)	no
163	Populate Next Right Pointers in Each Node	Medium	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
164	Sum Root to Leaf Numbers	Medium	Example 8.11	Worked solution	yes
165	Maximum Product of Split Binary Tree	Medium	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
166	Distribute Coins in a Binary Tree	Medium	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
167	Longest Zigzag Path in a Binary Tree	Medium	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
168	Binary Tree Coloring Game	Medium	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
169	Sum of Distances in Tree	Hard	Practice Set 8.6 (hint)	Hint only (one line in a practice table)	no
170	Maximum Path Sum in a Binary Tree	Hard	Example 8.10	Worked solution	yes
171	Inorder Successor in a BST	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
172	Flatten Binary Tree to Linked List	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
173	Convert Binary Tree to Circular Doubly Linked List	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
174	Clone a Binary Tree with Random Pointer	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
175	Count Complete Tree Nodes	Medium	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
176	Fenwick Tree (Binary Indexed Tree) — Introduction	Medium	Example 8.13	Worked solution	yes
177	Segment Tree — Introduction	Medium	Example 8.14	Worked solution	yes
178	Binary Tree Cameras	Hard	Practice Set 8.7 (hint)	Hint only (one line in a practice table)	no
179	Serialize and Deserialize a Binary Tree	Hard	Example 8.12	Worked solution	yes

Graph (61 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
180	BFS of Graph	Easy	Example 9.1	Worked inside a combined example	yes
181	DFS of Graph	Easy	Example 9.1	Worked inside a combined example	yes
182	Bipartite Graph Check	Medium	Example 9.2	Worked solution	yes
183	Evaluate Division	Medium	Practice Set 9.1 (hint)	Hint only (one line in a practice table)	no
184	Mother Vertex	Medium	Practice Set 9.1 (hint)	Hint only (one line in a practice table)	no
185	Number of Islands	Easy	Example 9.3	Worked solution	yes
186	Number of Enclaves	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
187	Number of Distinct Islands	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
188	0-1 Matrix (Multi-Source BFS Distances)	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
189	Rotting Oranges	Medium	Example 9.4	Worked inside a combined example	yes
190	Shortest Bridge	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
191	As Far from Land as Possible	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
192	Coloring a Border	Medium	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
193	Word Ladder	Hard	Example 9.5	Worked solution	yes
194	Sliding Puzzle	Hard	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
195	Bus Routes	Hard	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no
196	Number of Islands II	Hard	Practice Set 9.2 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
197	Topological Sorting (DFS-Based)	Medium	Section 9.3 (paragraph after Example 9.6)	Short note inside an example or section (no separate code)	no
198	Kahn's Algorithm (BFS Topological Sort)	Medium	Example 9.6	Worked solution	yes
199	Course Schedule II	Medium	Section 9.3 (note; the order Kahn's algorithm produces, Example 9.6)	Short note inside an example or section (no separate code)	no
200	Find Eventual Safe States	Medium	Practice Set 9.3 (hint)	Hint only (one line in a practice table)	no
201	Alien Dictionary	Hard	Practice Set 9.3 (hint)	Hint only (one line in a practice table)	no
202	Sort Items by Groups Respecting Dependencies	Hard	Practice Set 9.3 (hint)	Hint only (one line in a practice table)	no
203	Dijkstra's Algorithm	Medium	Example 9.7	Worked solution	yes
204	Bellman-Ford Algorithm	Medium	Example 9.8	Worked inside a combined example	yes
205	Floyd-Warshall Algorithm	Medium	Example 9.9	Worked solution	yes
206	Cheapest Flight Within K Stops	Medium	Example 9.8	Worked inside a combined example	yes
207	Chef and Reversing (0-1 BFS)	Medium	Practice Set 9.4 (hint)	Hint only (one line in a practice table) ⚠	no
208	Shortest Distance from All Buildings	Hard	Practice Set 9.4 (hint)	Hint only (one line in a practice table)	no
209	Shortest Cycle in an Undirected Graph	Hard	Practice Set 9.4 (hint)	Hint only (one line in a practice table)	no
210	Swim in Rising Water	Hard	Practice Set 9.4 (hint)	Hint only (one line in a practice table)	no
211	Johnson's Algorithm	Hard	Practice Set 9.4 (hint)	Hint only (one line in a practice table)	no
212	Prim's Algorithm	Medium	Example 9.10	Worked inside a combined example	yes
213	Kruskal's Algorithm	Medium	Example 9.10	Worked inside a combined example	yes
214	Connecting Cities With Minimum Cost	Medium	Section 9.5 (note) + Example 9.10	Worked inside a combined example	yes
215	Optimize Water Distribution in a Village	Medium	Section 9.5 (note) + Example 9.10	Short note inside an example or section (no separate code)	no
216	Disjoint Set Union — Introduction	Easy	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
217	DFS vs. DSU (Concept Comparison)	Easy	Section 9.6 (note)	Short note inside an example or section (no separate code)	no
218	Redundant Connection	Medium	Example 9.11	Worked solution	yes
219	Regions Cut by Slashes	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
220	Sentence Similarity II	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
221	Satisfiability of Equality Equations	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
222	Smallest String With Swaps	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
223	Most Stones Removed with Same Row or Column	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
224	Minimum Swaps to Make Two Arrays Identical	Medium	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
225	Redundant Connection II	Hard	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
226	Minimize Malware Spread	Hard	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
227	Graph Connectivity With Threshold	Hard	Practice Set 9.6 (hint)	Hint only (one line in a practice table)	no
228	Euler Circuit in an Undirected Graph	Medium	Practice Set 9.7 (hint)	Hint only (one line in a practice table)	no
229	Euler Circuit in a Directed Graph	Medium	Practice Set 9.7 (hint)	Hint only (one line in a practice table)	no
230	Doctor Strange	Medium	Practice Set 9.7 (hint)	Hint only (one line in a practice table) ⚠	no
231	Articulation Points	Hard	Example 9.13	Worked solution	yes
232	Strongly Connected Components (Kosaraju's Algorithm)	Hard	Example 9.12	Worked inside a combined example	yes
233	Ford-Fulkerson & Edmonds-Karp (Max Flow)	Hard	Example 9.14	Worked inside a combined example	yes
234	Maximum Bipartite Matching	Hard	Example 9.15	Worked solution	yes
235	Castle Run	Medium	Section 9.9 (hint table)	Hint only (one line in a practice table) ⚠	no
236	Reconstruct Itinerary	Hard	Section 9.9 (hint table)	Hint only (one line in a practice table)	no
237	Largest Color Value in a Directed Graph	Hard	Section 9.9 (hint table)	Hint only (one line in a practice table)	no
238	Cracking the Safe	Hard	Section 9.9 (hint table)	Hint only (one line in a practice table)	no
239	Bricks Falling When Hit	Hard	Section 9.9 (hint table)	Hint only (one line in a practice table)	no
240	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard	Section 9.9 (hint table)	Hint only (one line in a practice table)	no

HashMap and Heap (39 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
241	Count Pairs Whose Sum Is Divisible by K	Easy	Practice Set 10.1 (hint)	Hint only (one line in a practice table)	no
242	Number of Subarrays with Sum Exactly K	Medium	Example 10.1	Worked solution	yes
243	Subarray Sums Divisible by K	Medium	Example 10.3	Worked solution	yes
244	Subarray with Equal Number of 0s and 1s	Medium	Example 10.2	Worked solution	yes
245	Substring with Equal Number of 0s, 1s and 2s	Medium	Practice Set 10.1 (hint) + Appendix J.9	Hint in the chapter, full solution in Appendix J	yes (Appendix J)
246	Length of Largest Subarray with Contiguous Elements	Medium	Practice Set 10.1 (hint)	Hint only (one line in a practice table)	no
247	Length of Largest Subarray with Contiguous Elements II	Medium	Practice Set 10.1 (hint)	Hint only (one line in a practice table)	no
248	Anagram Palindrome Check	Easy	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
249	Find Anagram Mappings	Easy	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
250	Isomorphic Strings	Easy	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
251	X of a Kind in a Deck of Cards	Easy	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
252	Find All Anagrams in a String	Medium	Example 10.6	Worked solution	yes
253	Group Anagrams	Medium	Example 10.4	Worked solution	yes
254	K-Anagrams	Medium	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
255	Longest Consecutive Sequence	Medium	Example 10.5	Worked solution	yes
256	Rabbits in a Forest	Medium	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
257	Array of Doubled Pairs	Medium	Practice Set 10.2 (hint)	Hint only (one line in a practice table)	no
258	Longest Substring Without Repeating Characters	Medium	Example 10.7	Worked solution	yes
259	Same Frequency After One Removal	Medium	Practice Set 10.3 (hint)	Hint only (one line in a practice table)	no
260	Rearrange String K Distance Apart	Medium	Practice Set 10.3 (hint)	Hint only (one line in a practice table)	no
261	Smallest Subarray with All Occurrences of the Most Frequent Element	Medium	Practice Set 10.3 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
262	Smallest Window Containing All Characters of Another String	Hard	Example 10.8	Worked solution	yes
263	Longest Consecutive 1's (After One Flip)	Easy	Practice Set 10.4 (hint)	Hint only (one line in a practice table)	no
264	K Closest Points to Origin	Medium	Example 10.9	Worked solution	yes
265	Insert Delete GetRandom $\mathcal{O}(1)$	Medium	Example 10.10	Worked solution	yes
266	Insert Delete GetRandom $\mathcal{O}(1)$ — Duplicates Allowed	Hard	Example 10.10 (paragraph) + Appendix J.8	Paragraph in the chapter, full solution in Appendix J	yes (Appendix J)
267	Kth Smallest Prime Fraction	Hard	Practice Set 10.5 (hint)	Hint only (one line in a practice table)	no
268	Median from Data Stream	Hard	Example 10.11	Worked solution	yes
269	Employee Free Time	Hard	Practice Set 10.5 (hint)	Hint only (one line in a practice table)	no
270	Minimum Number of Refueling Stops	Hard	Practice Set 10.5 (hint)	Hint only (one line in a practice table)	no
271	Island Perimeter	Easy	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
272	Mode of Frequencies	Easy	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
273	Brick Wall	Medium	Example 10.12	Worked solution	yes
274	Pairs of Coinciding Points	Medium	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
275	Morning Assembly	Medium	Practice Set 10.6 (hint)	Hint only (one line in a practice table) ⚠	no
276	Smallest Number Whose Digit Product Equals a Given Number	Medium	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
277	Line Reflection	Medium	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
278	Grid Illumination	Hard	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no
279	The Skyline Problem	Hard	Practice Set 10.6 (hint)	Hint only (one line in a practice table)	no

Greedy (35 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
280	Activity Selection Problem	Easy	Example 11.1	Worked inside a combined example	yes
281	Maximum Meetings in One Room	Easy	Example 11.1	Worked inside a combined example	yes
282	Job Sequencing Problem	Medium	Example 11.4	Worked solution	yes

#	Problem	Level	Where in this book	How it is covered	Code run
283	Minimum Platforms Problem	Medium	Example 11.2	Worked solution	yes
284	Maximum Trains for Which Stoppage Can Be Provided	Medium	Practice Set 11.1 (hint)	Hint only (one line in a practice table)	no
285	Gas Station	Medium	Example 11.3	Worked solution	yes
286	Fractional Knapsack Problem	Easy	Example 11.5	Worked solution	yes
287	Minimum Number of Coins (Greedy Coin Change)	Easy	Example 11.7	Worked solution	yes
288	Minimum Cost of Ropes (Huffman-Style)	Easy	Example 11.6	Worked inside a combined example	yes
289	Huffman Coding	Medium	Practice Set 11.2 (hint); the merging idea is in Example 11.6	Hint only (one line in a practice table)	no
290	Water Connection Problem	Medium	Practice Set 11.2 (hint)	Hint only (one line in a practice table)	no
291	Chocolate Distribution Problem	Easy	Example 11.8	Worked solution	yes
292	Smallest Subset With Sum Greater Than All Other Elements	Easy	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
293	Maximize Array Sum After K Negations	Easy	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
294	Find Smallest Number With Given Digit Count and Digit Sum	Easy	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
295	Maximum Product Subset of an Array	Medium	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
296	Maximize Sum of arr[i] * i via Rotation	Medium	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
297	Maximum Sum of Absolute Differences of an Array	Medium	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
298	Maximize Sum of Consecutive Differences in a Circular Array	Medium	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
299	Minimum Sum of Absolute Differences of Pairs of Two Arrays	Medium	Practice Set 11.3 (hint)	Hint only (one line in a practice table)	no
300	Rearrange Characters in a String So No Two Adjacent Are Same	Medium	Example 11.9	Worked inside a combined example	yes
301	Shortest Job First (SJF) CPU Scheduling	Easy	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
302	Buy Maximum Stocks if Only i Stocks Can Be Bought on Day i	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
303	Find Min & Max Amount to Buy All N Candies	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
304	Minimize Cash Flow Among a Set of Friends	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
305	Minimum Cost to Cut a Board Into Squares	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no

#	Problem	Level	Where in this book	How it is covered	Code run
306	Check if It's Possible to Survive on an Island	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
307	LRU Page Replacement Algorithm	Medium	Section 11.4 (hint table)	Hint only (one line in a practice table) ⚠	no
308	DEFKIN — Defense of a Kingdom	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
309	DIEHARD — Die Hard	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
310	GERGOVIA — Wine Trading in Gergovia	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
311	Picking Up Chicks	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
312	CHOCOLA — Chocolate	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
313	ARRANGE — Arranging Amplifiers	Medium	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no
314	K Centers Problem	Hard	Section 11.5 (hint table)	Hint only (one line in a practice table) ⚠	no

Dynamic Programming (66 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
315	Paint House	Easy	Example 12.1	Worked inside a combined example	yes
316	Frog Jump	Easy	Example 12.1	Worked inside a combined example	yes
317	Paint Fence	Medium	Practice Set 12.1 (hint)	Hint only (one line in a practice table)	no
318	Paint House II	Medium	Practice Set 12.1 (hint)	Hint only (one line in a practice table)	no
319	Ways to Construct Buildings (No Two Adjacent)	Medium	Practice Set 12.1 (hint)	Hint only (one line in a practice table)	no
320	Count Binary Strings Without Consecutive 1s	Medium	Example 12.2	Worked solution	yes
321	Jump Game II	Medium	Example 12.3	Worked solution	yes
322	Ugly Number II	Medium	Example 12.4	Worked solution	yes
323	Coin Change — Minimum Coins	Medium	Example 12.5	Worked solution	yes
324	Coin Change II — Number of Ways	Medium	Example 12.6	Worked solution	yes
325	Unbounded Knapsack	Medium	Example 12.7	Worked inside a combined example	yes
326	Tri Tiling	Medium	Practice Set 12.2 (hint)	Hint only (one line in a practice table) ⚠	no

#	Problem	Level	Where in this book	How it is covered	Code run
327	Longest Increasing Subsequence — $\mathcal{O}(n^2)$	Medium	Example 12.8	Worked inside a combined example	yes
328	Minimum Number of Increasing Subsequences (Patience Sorting)	Medium	Practice Set 12.3 (hint)	Hint only (one line in a practice table)	no
329	Building Bridges	Medium	Practice Set 12.3 (hint)	Hint only (one line in a practice table)	no
330	Box Stacking	Medium	Practice Set 12.3 (hint)	Hint only (one line in a practice table)	no
331	Weighted Job Scheduling	Medium	Practice Set 12.3 (hint)	Hint only (one line in a practice table)	no
332	Longest Bitonic Subsequence	Medium	Practice Set 12.3 (hint)	Hint only (one line in a practice table)	no
333	Longest Increasing Subsequence — $\mathcal{O}(n \log n)$	Hard	Example 12.8	Worked inside a combined example	yes
334	Russian Doll Envelopes	Hard	Example 12.9	Worked solution	yes
335	Best Time to Buy and Sell Stock	Easy	Example 12.10	Worked solution	yes
336	Best Time to Buy and Sell Stock II	Easy	Example 12.11	Worked solution	yes
337	Best Time to Buy and Sell Stock with Transaction Fee	Medium	Example 12.12	Worked inside a combined example	yes
338	Best Time to Buy and Sell Stock with Cooldown	Medium	Example 12.12	Worked inside a combined example	yes
339	Best Time to Buy and Sell Stock III	Hard	Example 12.13	Worked inside a combined example	yes
340	Best Time to Buy and Sell Stock IV	Hard	Example 12.13	Worked inside a combined example	yes
341	Minimum Cost Path	Easy	Example 12.14	Worked solution	yes
342	Max Size Square Sub-Matrix with All 1s	Medium	Example 12.15	Worked solution	yes
343	Knight's Probability in a Chessboard	Medium	Practice Set 12.5 (hint)	Hint only (one line in a practice table)	no
344	Cherry Pickup	Hard	Practice Set 12.5 (hint)	Hint only (one line in a practice table)	no
345	Cherry Pickup II	Hard	Practice Set 12.5 (hint)	Hint only (one line in a practice table)	no
346	Dungeon Game	Hard	Practice Set 12.5 (hint)	Hint only (one line in a practice table)	no
347	Binomial Coefficient	Easy	Example 12.19	Worked solution	yes
348	Total Number of BSTs (Catalan Number)	Medium	Example 12.18	Worked inside a combined example	yes
349	Applications of Catalan Numbers	Medium	Example 12.18 (note)	Short note inside an example or section (no separate code)	no
350	Matrix Chain Multiplication	Medium	Example 12.16	Worked solution	yes
351	Boolean Parenthesization	Medium	Practice Set 12.6 (hint)	Hint only (one line in a practice table)	no
352	Minimum and Maximum Value of an Expression	Medium	Practice Set 12.6 (hint)	Hint only (one line in a practice table)	no

#	Problem	Level	Where in this book	How it is covered	Code run
353	Minimum Score Triangulation of a Polygon	Medium	Practice Set 12.6 (hint)	Hint only (one line in a practice table)	no
354	Highway Billboard Problem	Medium	Practice Set 12.6 (hint)	Hint only (one line in a practice table)	no
355	Optimal BST	Hard	Practice Set 12.6 (hint)	Hint only (one line in a practice table)	no
356	Burst Balloons	Hard	Example 12.17	Worked solution	yes
357	Longest Common Subsequence	Medium	Example 12.20	Worked solution	yes
358	Longest Common Substring	Medium	Example 12.22	Worked solution	yes
359	Edit Distance	Medium	Example 12.21	Worked solution	yes
360	LCS of Three Strings	Medium	Practice Set 12.7 (hint)	Hint only (one line in a practice table)	no
361	Shortest Common Supersequence	Medium	Practice Set 12.7 (hint)	Hint only (one line in a practice table)	no
362	Longest Repeating Subsequence	Medium	Practice Set 12.7 (hint)	Hint only (one line in a practice table)	no
363	Number of Sequences of Type $a^i\ b^j\ c^k$	Medium	Practice Set 12.7 (hint)	Hint only (one line in a practice table)	no
364	Distinct Subsequences	Hard	Example 12.23	Worked solution	yes
365	Longest Palindromic Subsequence	Medium	Example 12.24	Worked solution	yes
366	Palindromic Substrings — Count	Medium	Example 12.25	Worked solution	yes
367	Count All Palindromic Subsequences	Hard	Practice Set 12.8 (hint)	Hint only (one line in a practice table)	no
368	Count Distinct Palindromic Subsequences	Hard	Practice Set 12.8 (hint)	Hint only (one line in a practice table)	no
369	Palindrome Partitioning — Minimum Cuts	Hard	Example 12.26	Worked solution	yes
370	Say No to Palindrome	Hard	Practice Set 12.8 (hint)	Hint only (one line in a practice table) ⚠	no
371	Word Break Problem	Medium	Example 12.27	Worked solution	yes
372	Super Ugly Number	Medium	Practice Set 12.9 (hint)	Hint only (one line in a practice table)	no
373	Wildcard Pattern Matching	Hard	Example 12.28	Worked solution	yes
374	Regular Expression Matching	Hard	Practice Set 12.9 (hint)	Hint only (one line in a practice table)	no
375	Scramble String	Hard	Practice Set 12.9 (hint)	Hint only (one line in a practice table)	no
376	Optimal Strategy for a Game	Medium	Example 12.29	Worked solution	yes
377	Friends Pairing Problem	Medium	Example 12.31	Worked solution	yes
378	Egg Dropping Puzzle (2 Eggs, 100 Floors)	Hard	Example 12.30	Worked inside a combined example	yes
379	Max Sum Subarray with At Least K Elements	Medium	Practice Set 12.10 (hint)	Hint only (one line in a practice table)	no
380	Number of Digit One	Hard	Practice Set 12.10 (hint)	Hint only (one line in a practice table)	no

Bonus: String Matching Algorithms (6 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
381	KMP Algorithm	Medium	Example 13.1	Worked solution	yes
382	Z Algorithm	Medium	Example 13.3	Worked solution	yes
383	Rabin-Karp Algorithm	Medium	Example 13.4	Worked solution	yes
384	Chef and Secret Password	Medium	Practice Set 13 (hint)	Hint only (one line in a practice table) ⚠	no
385	Shortest Palindrome (via KMP)	Hard	Example 13.2	Worked solution	yes
386	Manacher's Algorithm	Hard	Example 13.5	Worked solution	yes

Backtracking (19 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
387	Print All Permutations of a String	Medium	Example 14.1	Worked solution	yes
388	Combination Sum	Medium	Example 14.2	Worked solution	yes
389	Find the K-th Permutation Sequence	Medium	Example 14.3	Worked solution	yes
390	Find Maximum Number Possible by At-Most K Swaps	Medium	Practice Set 14.1 (hint)	Hint only (one line in a practice table)	no
391	Rat in a Maze Problem	Medium	Example 14.4	Worked solution	yes
392	Print All Possible Paths in an M x N Matrix	Medium	Practice Set 14.2 (hint)	Hint only (one line in a practice table)	no
393	Find if a Path of More Than K Length Exists from a Source	Medium	Practice Set 14.2 (hint)	Hint only (one line in a practice table)	no
394	Find Shortest Safe Route in a Path with Landmines	Medium	Practice Set 14.2 (hint)	Hint only (one line in a practice table)	no
395	Longest Possible Route in a Matrix with Hurdles	Hard	Practice Set 14.2 (hint)	Hint only (one line in a practice table)	no
396	M-Coloring Problem	Medium	Example 14.7	Worked solution	yes
397	Tug of War	Medium	Practice Set 14.3 (hint)	Hint only (one line in a practice table)	no
398	N-Queens — Print All Solutions	Hard	Example 14.5	Worked solution	yes
399	Sudoku Solver	Hard	Example 14.6	Worked solution	yes
400	The Knight's Tour Problem	Hard	Practice Set 14.3 (hint)	Hint only (one line in a practice table)	no
401	Partition of a Set into K Subsets with Equal Sum	Hard	Practice Set 14.3 (hint)	Hint only (one line in a practice table)	no
402	Word Break Problem Using Backtracking	Medium	Example 14.9	Worked inside a combined example	yes

#	Problem	Level	Where in this book	How it is covered	Code run
403	Print All Palindromic Partitions of a String	Medium	Example 14.9	Worked inside a combined example	yes
404	Subset Sum Problem	Medium	Example 14.8	Worked solution	yes
405	Remove Invalid Parentheses	Hard	Practice Set 14.4 (hint)	Hint only (one line in a practice table)	no

Bit Manipulation (14 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
406	Count Set Bits in an Integer	Easy	Example 15.1	Worked inside a combined example	yes
407	Reverse Bits	Easy	Example 15.2	Worked solution	yes
408	Check if a Number Is a Power of Two	Easy	Example 15.1	Worked inside a combined example	yes
409	Count Bits to Flip to Convert A to B	Easy	Example 15.1	Worked inside a combined example	yes
410	Hamming Distance	Easy	Example 15.1	Worked inside a combined example	yes
411	Find the Two Non-Repeating Elements in an Array	Medium	Example 15.3	Worked inside a combined example	yes
412	Single Number II	Medium	Example 15.4	Worked inside a combined example	yes
413	Minimum XOR Value Pair	Medium	Practice Set 15.2 (hint)	Hint only (one line in a practice table)	no
414	Maximum XOR of Two Numbers in a Range [L, R]	Medium	Practice Set 15.2 (hint)	Hint only (one line in a practice table)	no
415	Copy Set Bits in a Range	Medium	Practice Set 15.3 (hint)	Hint only (one line in a practice table)	no
416	Calculate Square Without Using *, / or pow()	Medium	Practice Set 15.3 (hint)	Hint only (one line in a practice table)	no
417	Bitwise ORs of Subarrays	Medium	Example 15.6	Worked inside a combined example	yes
418	Divide Two Integers	Medium	Example 15.5	Worked inside a combined example	yes
419	Subsequence of a String Using Bit Manipulation	Hard	Example 15.6	Worked inside a combined example	yes

Tries (6 problems)

#	Problem	Level	Where in this book	How it is covered	Code run
420	Implement Trie (Prefix Tree)	Medium	Example 16.1	Worked solution	yes
421	No Prefix Set Problem	Medium	Practice Set 16.1 (hint)	Hint only (one line in a practice table)	no
422	Shortest Unique Prefix for Every Word	Medium	Example 16.2	Worked solution	yes
423	Search Suggestions System	Medium	Example 16.4	Worked solution	yes
424	Maximum XOR of Two Numbers in an Array	Medium	Example 16.3	Worked solution	yes
425	Kth Smallest in Lexicographical Order	Hard	Practice Set 16.2 (hint)	Hint only (one line in a practice table)	no

Summary: 177 with a worked solution, 12 with a companion solution in Appendix J, 10 with a short note, 226 with a hint only (total 425). ⚠ rows: 28.

APPENDIX F

Mistake and Learning Log

A mistake you write down is a mistake you do not repeat. Copy this table into a notebook or spreadsheet and fill one row **every time** you fail a problem, needed a hint, or found a bug.

Date	Problem	What went wrong (be specific)	Category	One-line insight / fix	Redo on
(today)	Longest ones, k zeros	returned 2 instead of 3	implementation	window length is `right - left + 1`	+1d, +3d, +7d

Categories: *concept* (did not understand the idea), *recognition* (did not see the pattern), *derivation* (could not get from brute force to optimal), *implementation* (right idea, wrong code), *edge case* (empty, one element, duplicates, overflow), *complexity* (too slow or wrong analysis).

Weekly ritual (15 minutes): count rows per category. The biggest category is next week's focus. If *recognition* dominates, do more Transfer Checks. If *implementation* dominates, go back up the scaffolding ladder and fill more blanks before coding blind. If *edge case* dominates, write the edge cases down *before* coding.

Keep the one-liners. Examples from this book: "move the shorter line", "query before insert", "mark visited when pushing", "sort by end time", "undo after the call".

APPENDIX G

Spaced Revision and Cumulative Self-Tests

G.1 Spaced Revision Schedule

Re-solve a problem **from a blank file** (not by rereading) at expanding intervals. Stop the schedule once you solve it quickly and cleanly three times in a row.

Review	When	Goal
1	next day	catch immediate forgetting
2	+3 days	confirm the idea, not the code, is remembered
3	+7 days	solve in under half your first time
4	+14 days	solve without hints
5	+30 days	solve a disguised variant

A practical daily budget: 30 minutes of review, 60 to 90 minutes of new material. Review problems you got wrong first, then older chapters in rotation.

G.2 Cumulative Self-Tests

Take each test **after** finishing the named chapters, closed-book, one problem at a time, 25 minutes each. For each problem write: pattern, brute force, optimized idea, complexity, then code. The pattern names are in the collapsed key; do not open it until you have decided.

Test 1: After Chapters 3 to 7

1. Longest subarray of positive numbers whose sum is at most `S`.
2. Rotate a linked list to the right by `k` places.
3. Split an array into at most `k` parts so the largest part sum is as small as possible.
4. For every element, find the distance to the previous greater element.
5. Does a matrix with sorted rows and columns contain `x`?
6. Remove duplicates from a sorted array in place.
7. Is a string of three bracket types balanced?
8. Merge `k` sorted linked lists.

KEY

1. Variable sliding window (3.3). 2. Close the list into a ring or find length, then cut at the right node (6.1, 6.2). 3. Binary search on the answer with a greedy `check` (5.2). 4. Monotonic stack (7.1). 5. Staircase search (4.2). 6. Read/write pointers (3.1). 7. Stack matching (7.2). 8. Min-heap of list heads, or divide and conquer merging (6.3, 10.4).

Test 2: After Chapters 8 to 10

1. Is a binary tree a mirror image of itself?
2. Minimum moves for a knight to reach a target square on a board.
3. Count subarrays whose sum is divisible by K.
4. Order courses given prerequisites, or report impossible.
5. Cheapest route in a road network with non-negative tolls.
6. Median of a stream of numbers.
7. Count groups of directly or indirectly connected cities.
8. List the nodes visible from the right side of a binary tree.

KEY

1. Recursion on a mirrored pair (8.1). 2. BFS on states (9.1, 9.2). 3. Prefix sums modulo K + hash map (10.1). 4. Topological sort (9.3). 5. Dijkstra (9.4). 6. Two heaps (10.5). 7. Union-Find or DFS components (9.6, 9.1). 8. Level-order BFS taking the last node of each level (8.2).

Test 3: After Chapters 11 to 16

1. Fewest coins to make an amount with an arbitrary coin set.
2. Maximum number of meetings that fit in one room.
3. Longest common subsequence of two strings.
4. Print all placements of N queens.
5. Find every occurrence of a pattern in a text of length 10^6.
6. Choose two numbers from a list with maximum XOR.
7. Find the element that appears once when all others appear three times.
8. Longest palindromic subsequence.

KEY

1. DP (12.2): greedy fails for arbitrary coins. 2. Greedy, sort by end time (11.1). 3. 2D string DP (12.7). 4. Backtracking with column/diagonal sets (14.3). 5. KMP or Z (13.1, 13.2). 6. Binary trie (16.2). 7. Bit counting modulo 3 (15.2). 8. DP, or LCS with the reversed string (12.8).

Capstone

Shuffle all 24 problems, hide the chapter labels, and solve six in a sitting. Score each as **A** (correct, independent), **B** (needed a Level-1 or Level-2 hint), **C** (needed more). Redo every B and C after 3 days.

APPENDIX H

Quality Assurance Report

This appendix states what was checked while preparing this edition, how, what passed, what was found and fixed, and what is **not** verified. Nothing is listed as verified unless a program or a script actually ran. Where the work was a human-style reading (one reviewer, no second pair of eyes), it is called a reading, not a test.

H.1 What Was Run

Check	How	Result
Every C++ block compiles	each fenced `cpp` block was extracted by script and compiled with `g++ -std=c++17 -O1 -g -Wall -Wextra -fsanitize=address,undefined -fno-sanitize-recover=undefined` inside a test program (blocks that need `ListNode` or `TreeNode` get the struct first; blocks that use a helper from earlier in the chapter get that helper)	256 C++ blocks in this book; 232 of them are part of a program that was compiled and run (see the next row for the rest)
Solved examples run on real inputs	one test program per chapter (`ch01`, `ch03` ... `ch16`) calls the printed functions with `CHECK` assertions: the worked example from the text, normal cases, edge cases (empty input, one element, all equal, duplicates, negatives, `INT_MIN`/`INT_MAX` where it matters, zero and one for counts) and boundary cases	all pass
Randomised comparison	most solutions were also compared with a slow, obviously correct version (brute force, `std::multiset`, `std::queue`, recursive traversal, a dynamic program written differently) on thousands of seeded random small inputs	all pass
Large inputs	only a handful of solutions were run on inputs of about 10^5 elements (inversion count by merge sort, Manacher, subarray ORs, `countBits`, a long `"aaa..."` palindrome test)	all pass. Large-input and deep-recursion testing was **not** done systematically; recursion on degenerate list-shaped trees was not tested
Self-contained Problems nA/nB (30)	compiled and run with the sanitizers on hand-made and random tests	all pass
Debug Labs (16)	each bug claim was **executed**: the buggy snippet was wrapped in a small program and the claimed wrong output, crash or hang was reproduced (crashes inside a forked child); the corrected version was then checked	all 16 reproduced as described

Check	How	Result
Appendix I (Implementation Lab) and Appendix J (companion solutions)	compiled and run; Appendix J functions were compared with brute force on random inputs, the tree functions of J.6 and J.7 against recursive versions and a hand-checked tree	all pass
Hand traces added in this edition	dry runs and tables (DP tables, KMP failure arrays, Dijkstra distances, trie nodes and so on) were worked by hand and then recomputed with a short script or the tested function where practical	the recomputed ones match; several traces were corrected during this check (see H.2). **Not every trace was recomputed.**
Sheet coverage	the sheet's 425 rows were matched by name against the book's headings and tables by script, then every match was read and corrected by hand; the 53 that the script could not match (combined headings, abbreviated names) were mapped manually	Appendix E lists all 425 rows with a location. Counts: 177 have a worked solution, 12 are solved in Appendix J, 10 have a short note, 226 have a hint only
Chapter, appendix and example cross-references	a script resolved every `Example x.y`, `Section x.y`, `Practice Set x.y` and `Appendix` reference against the headings of this book	every reference resolves
Markdown structure	code fences balanced, tables have a constant number of columns per table, heading levels do not skip	clean

The 24 C++ blocks that are not run exactly as printed are deliberate: the program skeleton, loop-body fragments and API tours (Chapter 1, Section 7.0, Appendix A), the Big-O one-liners, the fill-in-the-blank scaffolding exercise, the sliding-window and backtracking templates (they contain placeholders such as `state`), the adjacency-list and direction-array snippets, and five Debug Lab snippets that contain a bug on purpose (those are executed inside the wrappers described above, not as printed). The idioms block of Appendix A.3 was compiled idiom by idiom in a separate test. One more block, `freeTree` in Appendix I.2, was run in an equivalent one-line form.

H.2 Defects Found and Fixed

Running the code found real defects, mostly on boundary inputs. All are fixed in the printed solutions.

Where	Defect	Fix
Example 3.1 Rotate Array	`k %= n` divided by zero for an empty array	early return
Example 5.6 Median of Two Sorted Arrays	`int + int` could overflow before the division by 2	cast to `double` first
Example 8.5 Delete a Node in a BST	removed nodes were never freed (leak under the sanitizer)	`delete` the removed node
Example 10.8 Smallest Window	negative `char` used as an index; empty `t` not handled	index with `unsigned char`; guard
Examples 11.8, 12.1, 12.2, 12.12, 12.14 to 12.16, 12.26 and 14.4	empty or tiny input (`m = 0`, `n = 0`, empty vector) read or wrote out of bounds; `countStrings(0)` returned 2 instead of 1	guards and corrected base cases

Where	Defect	Fix
Example 12.18 numBSTs	wrote `dp[1]` on a vector of size 1 when `n = 0`	allocate `max(n,1) + 1`
Example 12.30 Egg Dropping	with zero eggs the loop never ended	guards for `eggs <= 0` and `floors <= 0`
Example 13.1 KMP	empty pattern read `lps[-1]`	early return
Example 13.4 Rabin-Karp	the hash used signed `char`, so it was wrong for non-ASCII bytes	cast to `unsigned char`
Example 14.5	`n = 0` gave an out-of-bounds access or a gigantic allocation	guards
Example 15.3 and Example 15.4	`1 << 31` and `xr & -xr` on `INT_MIN` are undefined behaviour for `int`	use `unsigned`
Examples 16.1 and 16.3	tries leaked every node	destructors, copying disabled
Appendix A.3	an idiom reused the name `idx` in the same scope, so it would not compile if the idioms were pasted together	renamed
Debug Labs 13 and 14	the printed answers were checked against a real run of the buggy code (for example the failure table for `"abab"` really is `[1, 2, 3, 4]`)	answers now state the output of the run

Content defects found by reading and fixed:

- Appendix E (first edition) sent *Topological Sort (DFS)*, *DFS vs DSU* and *Construct Tree from Inorder and Postorder* to examples that solve something else. The whole index was rebuilt from the sheet.
- The coin list in the greedy coin-change text left out 200 although the code had it; the claim "greedy works for this system" was re-verified by a dynamic program for every amount up to 20,000.
- Appendix C placed Test 2 and Test 3 before the material they test and referred to a "Test 1A" that does not exist.
- Several hints in Sections 11.4 and 11.5 (digit sum, circular differences, cash flow, DEFKIN, chicks) were corrected, and a note explains the sheet row *Longest Consecutive 1's*, which the sheet files under a different topic than its idea.
- The first edition claimed the book matched "450 problems"; the sheet lists 425. Counts in this book are produced by script.
- The first edition's QA appendix described a bug "found while testing" Problem 16A; that claim was reworded because it could not be reproduced as written.
- Several hand traces I wrote while expanding the DP, graph and string chapters were wrong on first writing (for example the minimum-window trace, an N-Queens dead end, an overflow figure in Friends Pairing). They were caught by recomputing and corrected before release.

H.3 What Was Not Verified (Limitations)

1. **Hint-only sheet problems are not solved or tested here.** 226 of the 425 sheet rows have only a one- or two-line hint in a practice table. Their code does not exist in this book, so nothing about them was executed. Rows marked ⚠ in Appendix E are competitive-programming problems whose full statements were not available while writing this book; their hints assume the statement printed in your sheet or on the judge. Check the statement first.
2. **Difficulty labels** (`[E]`, `[M]`, `[H]`) were compared by script with the sheet's labels for every sheet row that matched a tagged heading or practice row. No differences remain (the only two the script flagged, Preorder and Postorder Traversal, were false name matches and were corrected). Labels are opinions, not measurements, and many hint-table rows in Sections 3.9, 9.9, 11.4 and 11.5 carry no tag at all, so the sheet's *Level* column in Appendix E is the one to trust.

3. **Random tests are evidence, not proof.** A function that passes thousands of random small cases can still fail on a rare large case. Complexity claims were derived and checked by reading and by timing on large inputs, not formally proved. The "Why It Works" sections are teaching sketches, not rigorous proofs.
4. **Single reviewer.** The content audit (conceptual correctness, learning order, clarity, prerequisites, examples and constraints) was one careful pass by the author of this edition. No independent human reviewer or learner tried the book. The acceptance goal (a beginner can follow, derive and code) is a design target, not something measured on real learners.
5. **Platform.** Everything was compiled with GCC, C++17, on Linux. Other compilers were not tried. `<bits/stdc++.h>` is a GCC header.
6. **Links and external statements.** The book contains no web links that depend on being live. Statements of well-known problems are paraphrased from memory of the standard versions; they were not checked against the original sites.
7. **Debug Lab and fill-in code** is intentionally incomplete or wrong and is covered only by the wrappers described in H.1.

H.4 Reproduce the Checks

```
# compile and run one solution with the sanitizers
g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined -fno-sanitize-recover=undefined -o
↪  prog prog.cpp && ./prog

# a minimal test habit: put a CHECK macro and a few cases in main
#   #define CHECK(c) do { if (!(c)) { fprintf(stderr, "FAILED line %d\n", __LINE__); return 1;
↪  } } while (0)
#   CHECK(solve({}) == 0);          // empty
#   CHECK(solve({5}) == 5);         // one element
#   CHECK(solve(bruteForceInput()) == bruteForce(...));   // compare with a slow version
```

Write the slow, obviously correct version first, compare on a few thousand random small inputs, and read any sanitizer message as a bug until you have proven otherwise. That is the habit this appendix is meant to teach.

APPENDIX I

C++ Implementation Lab

Algorithms fail in C++ for mundane reasons. Each snippet below was compiled and tested.

I.1 Linked Lists: Creating, Changing the Head, Freeing

```
void pushFront(ListNode*& head, int v) {      // & so the CALLER'S head pointer changes
    ListNode* n = new ListNode(v);
    n->next = head;
    head = n;
}
void freeList(ListNode* p) {
    while (p) {
        ListNode* nxt = p->next;              // save next BEFORE deleting
        delete p;
        p = nxt;
    }
}
```

- Pass `ListNode*&` when the function may change which node is the head.
- Always save `next` **before** `delete` or before overwriting a link.
- Use `new` for nodes you build; every `new` needs a `delete` (or use a dummy node on the stack plus careful deletion).

I.2 Trees: Building from a Level-Order List

```
TreeNode* buildTree(const vector<int>& v) {   // level order, -1 means "no node"
    if (v.empty() || v[0] == -1) return nullptr;
    TreeNode* root = new TreeNode(v[0]);
    queue<TreeNode*> q;
    q.push(root);
    size_t i = 1;
    while (!q.empty() && i < v.size()) {
        TreeNode* cur = q.front(); q.pop();
        if (i < v.size() && v[i] != -1) { cur->left = new TreeNode(v[i]); q.push(cur->left); }
        i++;
       if (i < v.size() && v[i] != -1) { cur->right = new TreeNode(v[i]); q.push(cur->right); }
        i++;
    }
```

```
    return root;
}
```

Use this to turn test cases like `[1,2,3,-1,4]` into trees quickly. The value `-1` is the "no node" marker, so a real node cannot hold `-1`. Free the tree with a postorder recursion (children first, then the node):

```
void freeTree(TreeNode* root) {          // postorder: free the children before the node itself
    if (!root) return;
    freeTree(root->left);
    freeTree(root->right);
    delete root;
}
```

I.3 Graph Representations

```
int n = 5;
vector<vector<int>> adj(n);                          // unweighted graph
vector<vector<pair<int,int>>> wadj(n);               // weighted graph: (neighbour, weight)
void addUndirected(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }  // BOTH
↪  directions
void addWeightedDirected(int u, int v, int w) { wadj[u].push_back({v, w}); }
```

Representation	Space	Edge check	Best for
Adjacency list	$\mathcal{O}(V+E)$	$\mathcal{O}(\text{degree})$	almost everything
Adjacency matrix	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	dense graphs, Floyd-Warshall
Edge list	$\mathcal{O}(E)$	$\mathcal{O}(E)$	Kruskal, Bellman-Ford

Undirected graphs need **both** directions added. Grids need no explicit graph: use direction arrays `dr[] = {-1,1,0,0}`, `dc[] = {0,0,-1,1}`.

I.4 Comparators and Lambdas

```
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) { return a.second > b.second; };
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);   // min-heap on
↪  .second
```

- `priority_queue` is a **max**-heap by default; to get a min-heap, invert the comparison (or use `greater<>`).
- A comparator must be a *strict weak ordering*: never return true for equal elements.
- Capture with `[&]` inside functions; a lambda at namespace scope cannot capture.

```
long long factorial10() {
    function<long long(int)> fact = [&](int k) -> long long {   // a lambda cannot name itself,
↪  so wrap it
        return k <= 1 ? 1 : k * fact(k - 1);
    };
```

```
    return fact(10);
}
```

A lambda cannot call itself by name, so wrap it in `std::function` (or pass itself as a parameter).

I.5 Iterators and Erasing

```
void removeEvens(vector<int>& v) {
    for (auto it = v.begin(); it != v.end(); ) {
        if (*it % 2 == 0) it = v.erase(it);        // erase returns the next valid iterator
        else ++it;
    }
}
```

`erase` invalidates iterators at and after the erased position; use its return value. Never `push_back` on a `vector` while holding references or iterators into it.

I.6 References, Copies and Pointers: Quick Rules

Situation	Write	Why
Read a big container	`const vector<int>& a`	no copy
Modify the caller's container	`vector<int>& a`	no copy, changes visible
Loop over rows of a grid	`for (auto& row : g)`	`auto row` copies each row
Change the caller's pointer	`ListNode*& head`	pointer is passed by value otherwise
Optional "no node"	`nullptr` and check before `->`	null dereference is the #1 list/tree crash

I.7 Recursion Checklist

1. Base case first. 2. Each call moves strictly toward the base case. 3. State shared across calls (visited, path) is restored after the call. 4. Estimate depth: recursion on 10^5-node chains can overflow the stack; use an explicit stack or iterate. 5. Memoize when subproblems repeat.

I.8 Pitfall Table

Pitfall	Symptom	Fix
`v.size() - 1` on empty	huge loop / crash	`(int)v.size() - 1`
`int` overflow	wrong large answers	`long long`
`mid = (lo + hi) / 2`	overflow	`lo + (hi - lo) / 2`
`n & m == 0`	wrong result	parenthesize: `(n & m) == 0`

Pitfall	Symptom	Fix
`map[key]` on read	inserts a default	use `find` or `count`
Modify while iterating	skipped items / UB	use returned iterator
Uninitialised DP / dist	random answers	fill with sentinel (`INF`, `-1`)
Missing `&`	TLE from copies	pass by reference

APPENDIX J

Companion Solutions for Hint-Only Sheet Problems

Some sheet problems appear in the chapters only as a one-line hint or a short note. **J.1 to J.9** give a complete, tested solution for nine of them (J.6 covers two sheet rows and J.7 covers three, so twelve sheet rows in all). **J.10** is not a sheet row: it explains the two-stack queue that Section 7.3 prints, because the same idea (two stacks moving elements in reverse) is a common follow-up. Each entry follows the same shape: the problem, the idea, the code, a hand trace and the cost. Try the problem from the hint in its chapter first, then compare.

How these were checked. Every function below was compiled with `g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined` and compared with a slow, obviously correct reference (brute force, `std::multiset`, `std::queue`, recursive traversals) on thousands of random small inputs, including empty and single-element inputs. A passing random test is strong evidence, not a proof.

J.1 Search in Rotated Sorted Array II (duplicates allowed)

Where the hint is: Section 5.1 (variant of Example 5.2). **Problem:** a sorted array was rotated at an unknown point and may contain duplicates, for example `[2,5,6,0,0,1,2]`. Return an index of `target` or `-1`.

Idea. Example 5.2 decides which half is sorted by comparing `a[lo]` with `a[mid]`. With duplicates that comparison can be a tie. If `a[lo] == a[mid] == a[hi]` we cannot tell which side the rotation is on, but we can safely drop both ends: we already know `a[mid] != target`, and `a[lo]` and `a[hi]` equal `a[mid]`, so neither end is the target.

```
// Chapter 5, variant of 5.2: the array may contain DUPLICATES.  Returns an index of target, or -1.
int searchRotatedII(const vector<int>& a, int target) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] == a[mid] && a[mid] == a[hi]) {          // cannot tell which half is sorted: shrink both ends
            lo++; hi--;
        } else if (a[lo] <= a[mid]) {                      // left half [lo, mid] is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                           // right half [mid, hi] is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Trace. `a = [2,5,6,0,0,1,2]`, `target = 3`. `lo=0, hi=6, mid=3`: `a[3] = 0`. The left half is not sorted (`2 > 0`), so the right half `[0,0,1,2]` is; `0 < 3 <= 2` is false, so `hi = 2`. `mid = 1`: `a[1] = 5`; left half `[2,5]` is sorted, `2 <= 3 < 5`, so `hi = 0`. `mid = 0`: `a[0] = 2`, and `a[0] == a[0] == a[0]`, so `lo = 1, hi = -1`: stop, answer `-1`. Harder case `a = [1,0,1,1,1]`, `target = 0`: first `mid = 2` gives three equal values, so `lo = 1, hi = 3`; then `mid = 2`, the left part `[0,1]` is sorted and contains the target, `hi = 1`; `mid = 1` hits it: index 1.

Cost. $\mathcal{O}(\log n)$ typical, $\mathcal{O}(n)$ worst case (for instance `[1,1,1,1,1,2,1]` forces many shrink steps). That worst case is unavoidable for any comparison-based method.

J.2 Find Minimum in Rotated Sorted Array II (duplicates allowed)

Where the hint is: Section 5.1 (variant of Example 5.3).

Idea. Compare `a[mid]` with `a[hi]` (not with `a[lo]`). If `a[mid] > a[hi]` the minimum is strictly right of `mid`. If `a[mid] < a[hi]`, `mid` itself may be the minimum. If they are equal we cannot tell, but dropping `a[hi]` is safe because a copy of the same value stays at `mid`.

```
// Chapter 5, variant of 5.3: minimum of a rotated sorted array that may contain DUPLICATES.
// Worst case O(n) (e.g. all equal), typical O(log n).  Precondition: a is non-empty.
int findMinII(const vector<int>& a) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[hi])      lo = mid + 1;            // minimum is strictly to the right
        ↪ of mid
        else if (a[mid] < a[hi]) hi = mid;                // mid itself may be the minimum
        else                     hi--;                   // equal: a[hi] is not needed, drop it
        ↪ safely
    }
    return a[lo];
}
```

Trace. `a = [1,1,1,0,1]`: `lo=0, hi=4, mid=2`: `a[2] = a[4] = 1`, so `hi = 3`. `mid = 1`: `a[1] = 1 > a[3] = 0`, so `lo = 2`. `mid = 2`: `1 > 0`, `lo = 3`. Stop: `a[3] = 0`. The array must be non-empty.

Cost. $\mathcal{O}(\log n)$ typical, $\mathcal{O}(n)$ worst case (all values equal).

J.3 Next Greater Element in a Circular Array

Where the hint is: Practice Set 7.1, row *Next Greater Element II (Circular Array)* (a variant of Example 7.1).
Problem: for each element find the next strictly greater value when the array wraps around, or `-1`.

Idea. Run the monotonic-stack scan of Example 7.1 over the array **twice** (indices `2n-1` down to `0`, using `i % n`), but record answers only during the real pass (`i < n`). The first pass simply fills the stack with the elements that lie "to the right, after wrapping".

```
// Chapter 7, variant of 7.1: the array is CIRCULAR (after the last element we continue from
↪ the first).
// Walk the array twice from the right; index i % n makes it wrap.
vector<int> nextGreaterCircular(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;                                    // values, decreasing from bottom to top
```

```
    for (int i = 2 * n - 1; i >= 0; i--) {
        while (!st.empty() && st.top() <= a[i % n]) st.pop();
        if (i < n && !st.empty()) res[i] = st.top();       // only record answers during the
        ↪  "real" pass
        st.push(a[i % n]);
    }
    return res;
}
```

Trace. `a = [1,2,1]`, answer `[2,-1,2]`. Indices `5,4,3` (the pretend pass) push `1`, then pop it and push `2`, then push `1`: stack (top first) `1,2`. `i = 2` (value 1): pop `1` (not greater), top is `2`, so `res[2] = 2`. `i = 1` (value 2): pop `1` and `2`, stack empty, so `res[1] = -1`. `i = 0` (value 1): top is `2`, so `res[0] = 2`.

Cost. $\mathcal{O}(n)$ time (each element is pushed and popped at most twice), $\mathcal{O}(n)$ space.

J.4 Stock Span Problem

Where the hint is: Practice Set 7.1, row *Stock Span Problem* (and the title of Example 7.2). **Problem:** for each day, the number of consecutive days ending today with price at most today's price.

Idea. The span reaches back to the previous day with a **strictly greater** price. Keep a stack of indices of strictly decreasing prices; popping the days absorbed by today leaves exactly that previous greater day.

```
// Chapter 7, variant of 7.2: span[i] = number of consecutive days ending at i with price <=
↪  price[i].
vector<int> stockSpan(const vector<int>& p) {
    int n = p.size();
    vector<int> span(n);
    stack<int> st;                                    // indices of strictly decreasing prices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && p[st.top()] <= p[i]) st.pop();   // those days are absorbed into
        ↪  i's span
        span[i] = st.empty() ? i + 1 : i - st.top();
        st.push(i);
    }
    return span;
}
```

Trace. `p = [100,80,60,70,60,75,85]` gives `[1,1,1,2,1,4,6]`. Day 3 (`70`) pops day 2 (`60`) and stops at day 1 (`80`): span `3 - 1 = 2`. Day 5 (`75`) pops days 4 and 3 and stops at day 1: `5 - 1 = 4`. Day 6 (`85`) pops days 5 and 1 (`75`, `80`) and stops at day 0 (`100 > 85`): span `6 - 0 = 6`.

Cost. $\mathcal{O}(n)$ amortized.

J.5 Minimum Bracket Reversals to Balance an Expression

Where the hint is: Example 7.7. **Problem:** a string of `{` and `}` only; flip the fewest characters to make it balanced; `-1` if impossible.

Idea. An odd length can never balance. Cancel every matched `{}` pair while scanning; what remains is `}}}...{{{` (some closing braces followed by some opening braces). Flipping one `}` into `{` fixes two closers, so `close` unmatched closers need `ceil(close / 2)` flips and `open` unmatched openers need `ceil(open / 2)`.

```cpp
// Chapter 7: minimum reversals ('{' <-> '}') to balance a string of braces; -1 if impossible
↪  (odd length).
int minReversals(const string& s) {
    if (s.size() % 2) return -1;
    int open = 0, close = 0;                              // after cancelling matched pairs the
    ↪  string is "}}}...{{{"
    for (char c : s) {
        if (c == '{') open++;
        else if (open > 0) open--;                         // this '}' matches an earlier '{'
        else close++;                                      // unmatched '}'
    }
    return (close + 1) / 2 + (open + 1) / 2;               // ceil(close/2) + ceil(open/2)
}
```

Trace. `"}{"`: `close = 1`, `open = 1`, answer `1 + 1 = 2`. `"}}{{"`: `close = 2`, `open = 2`, answer `1 + 1 = 2`. `"{{{{"`: `open = 4`, answer 2. `"{{{"`: odd, `-1`.

Cost. $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space.

J.6 Preorder and Postorder Traversal (iterative)

Where the hint is: only the iterative *inorder* traversal (Example 8.1) is printed in the chapter; the sheet also lists preorder and postorder, which this entry supplies. **Problem:** return the preorder and postorder of a binary tree without recursion.

Idea. Preorder is node, left, right: pop a node, record it, push the **right** child first and then the left so the left is processed first. Postorder is left, right, node. The same loop with the children pushed in the opposite order records node, right, left; reversing that list gives left, right, node.

```cpp
// Chapter 8: iterative preorder and postorder with one stack (TreeNode as defined in 8.0).
vector<int> preorderIter(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    if (root) st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        res.push_back(n->val);                             // node, then left, then right
        if (n->right) st.push(n->right);                   // push right first so left is popped
        ↪  first
        if (n->left)  st.push(n->left);
    }
    return res;
}
vector<int> postorderIter(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    if (root) st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        res.push_back(n->val);                             // produces node, right, left ...
        if (n->left)  st.push(n->left);
        if (n->right) st.push(n->right);
    }
```

```
    reverse(res.begin(), res.end());                    // ... which reversed is left, right,
↪   node
    return res;
}
```

Trace. Tree 1 with children 2 (children 4, 5) and 3. Preorder: pop 1, push 3 then 2; pop 2, push 5 then 4; pop 4, pop 5, pop 3: `1 2 4 5 3`. Postorder: the loop records `1 3 2 5 4`; reversed, `4 5 2 3 1`.

Cost. $\mathcal{O}(n)$ time, $\mathcal{O}(h)$ space for the stack (`h` = height).

J.7 Zigzag Level Order, Bottom View and Diagonal Traversal

Where the hints are: a comment in Example 8.2 (zigzag) and the bullet list in Example 8.3 (bottom view, diagonal traversal); this entry gives the full code.

Idea. *Zigzag:* a normal BFS level by level; write the level's values from the left end on even levels and from the right end on odd levels. *Bottom view:* give each node a horizontal distance `hd` (root 0, left child `hd - 1`, right child `hd + 1`); a BFS reaches deeper nodes later, so overwriting `last[hd]` keeps the bottom-most node of every column. *Diagonal traversal:* nodes reachable from each other by right-child steps share a diagonal; walk each right spine and queue every left child as the start of the next diagonal.

```
// Chapter 8: zigzag level order, bottom view, diagonal traversal (TreeNode as defined in 8.0).
vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    bool leftToRight = true;
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level(sz);
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop();
            level[leftToRight ? i : sz - 1 - i] = n->val;  // write from the correct end
            if (n->left)  q.push(n->left);
            if (n->right) q.push(n->right);
        }
        res.push_back(level);
        leftToRight = !leftToRight;
    }
    return res;
}
vector<int> bottomView(TreeNode* root) {                  // horizontal distance hd: left child
↪   hd-1, right child hd+1
    map<int, int> last;                                    // hd -> value of the LAST node seen
↪   at that hd (level by level)
    queue<pair<TreeNode*, int>> q;
    if (root) q.push({root, 0});
    while (!q.empty()) {
        auto [n, hd] = q.front(); q.pop();
        last[hd] = n->val;                                 // a deeper node overwrites a
↪   shallower one
        if (n->left)  q.push({n->left, hd - 1});
        if (n->right) q.push({n->right, hd + 1});
```

```
    }
    vector<int> res;
    for (auto& kv : last) res.push_back(kv.second);        // left to right
    return res;
}
vector<int> diagonalTraversal(TreeNode* root) {            // nodes on the same "\" diagonal,
↪  top-right to bottom-left
    vector<int> res;
    queue<TreeNode*> q;
    if (root) q.push(root);
    while (!q.empty()) {
        TreeNode* n = q.front(); q.pop();
        for (; n; n = n->right) {                          // walk down the right spine: same
        ↪  diagonal
            res.push_back(n->val);
           if (n->left) q.push(n->left);                // a left child starts the next diagonal
        }
    }
    return res;
}
```

Trace on the perfect tree `1 (2 (4, 5), 3 (6, 7))`. Zigzag: `[[1], [3, 2], [4, 5, 6, 7]]`. Bottom view: columns `-2, -1, 0, 1, 2` end with `4, 2, 6, 3, 7` (column 0 is visited by `1`, then by `5`, then by `6`, and the last one wins). Diagonal: root diagonal `1, 3, 7`, then the queued left children `2` (giving `2, 5`) and `6` (giving `6`), then `4`: `1 3 7 2 5 6 4`.

Cost. $\mathcal{O}(n)$ for zigzag and diagonal, $\mathcal{O}(n \log n)$ for the bottom view because of the ordered `map` ($\mathcal{O}(n)$ with a plain array offset by the minimum `hd`).

J.8 Insert Delete GetRandom O(1), Duplicates Allowed

Where the hint is: the paragraph after Example 10.10. **Idea.** Keep a vector of all copies and, for each value, the **set of positions** where it occurs. To delete one copy, take any position of that value, move the last vector element into the hole and fix both position sets.

```
// Chapter 10, variant of 10.10: duplicates allowed.  insert returns true if v was not present
↪  before;
// remove deletes ONE copy.  getRandom picks uniformly among all stored copies.
class RandomizedCollection {
    vector<int> vals;                                      // all copies, in any order
    unordered_map<int, unordered_set<int>> pos;            // value -> set of indices where it
↪  occurs in vals
public:
    bool insert(int v) {
        bool first = pos[v].empty();
        pos[v].insert(vals.size());
        vals.push_back(v);
        return first;
    }
    bool remove(int v) {
        auto it = pos.find(v);
        if (it == pos.end() || it->second.empty()) return false;
        int idx = *it->second.begin();                     // some index holding v
```

```
        it->second.erase(it->second.begin());
        int last = vals.back(), lastIdx = (int)vals.size() - 1;
        if (idx != lastIdx) {                          // move the last element into the hole
            vals[idx] = last;
            pos[last].erase(lastIdx);
            pos[last].insert(idx);
        }
        vals.pop_back();
        return true;
    }
    int getRandom() const { return vals[rand() % vals.size()]; }   // precondition: not empty
    int size() const { return vals.size(); }
};
```

Trace. `insert(1)` returns `true`, `insert(1)` returns `false`, `insert(2)` returns `true`: `vals = [1,1,2]`. `remove(1)` chooses a position of a `1`, say 0, moves the last element `2` there (`vals = [2,1]`, `pos[2] = {0}`) and returns `true`. After `remove(1)` once more, `vals = [2]`; another `remove(1)` returns `false`. `getRandom` picks uniformly among stored copies, so a value stored twice is twice as likely.

Cost. Average $\mathcal{O}(1)$ per operation. (`rand() % size` is a teaching shortcut; use `<random>` for serious work.)

J.9 Substrings with Equal Numbers of 0s, 1s and 2s

Where the hint is: Practice Set 10.1. **Problem:** count substrings of a string over `0`, `1`, `2` that contain the same number of each digit.

Idea. Let `c0, c1, c2` be running counts. A substring is balanced exactly when the key `(c1 - c0, c2 - c1)` is the same at both ends, so count pairs of equal keys with the prefix-sum-plus-hash idea of Example 10.1 (keys are pairs, so use `map` or a custom hash).

```
// Chapter 10: count substrings of a string over {'0','1','2'} that contain equally many 0s, 1s
↪  and 2s.
// Two prefixes with the same key (c1-c0, c2-c1) enclose a balanced substring.
long long equal012(const string& s) {
    map<pair<int,int>, int> seen;
    seen[{0, 0}] = 1;                                  // the empty prefix
    int c[3] = {0, 0, 0};
    long long ans = 0;
    for (char ch : s) {
        c[ch - '0']++;
        pair<int,int> key = {c[1] - c[0], c[2] - c[1]};
        ans += seen[key]++;                            // pair with every earlier prefix that
↪  has the same key
    }
    return ans;
}
```

Trace. `"012012"` has five balanced substrings: `012`, `120`, `201`, `012` and the whole string. For `"012"`: the empty prefix has key `(0,0)`; after `0` the key is `(-1,0)`; after `1` it is `(0,-1)`; after `2` it is `(0,0)` again, which pairs with the empty prefix: one substring.

Cost. $\mathcal{O}(n \log n)$ with `map` ($\mathcal{O}(n)$ average with a hash).

J.10 Queue Using Two Stacks

Where it is: Section 7.3, as a worked example (the code is printed there). Reading the code beside this entry: the `in` stack takes pushes; when `out` is empty it receives all of `in` in reverse, which turns last-in-first-out into first-in-first-out. The refill happens **only** when `out` is empty, otherwise older elements would be buried under newer ones. It was tested against `std::queue` on random operation sequences.

AFTERWORD

Closing Words

DSA is not memorization; it is **pattern recognition built from deliberate practice**. If you worked through each chapter the way the loop in the Preface suggests (concept, pattern, solved example, practice), you now hold the tools behind the core patterns behind the problems on the sheet (many entries listed in Appendix E come with a hint only, so treat those as extra practice: Appendix E says which).

When you meet a new problem:

1. Say it aloud in plain words.
2. Name the brute force.
3. Ask "what am I wasting?" and pick the pattern that removes the waste.
4. Code clearly. Test small. State complexity.

Keep a notebook of the *one-line insight* for each pattern ("move the shorter line", "last balloon, not first", "mark when pushing", "sort by end time"). Those one-liners are what you will actually recall under interview pressure.

Good luck, and happy coding.